

?? File: JinjaLexer.g4

```
lexer grammar JinjaLexer;

// Jinja delimiters must come before HTML_TEXT to ensure they're matched first
JINJA_EXPR_START : '{{' -> pushMode(JINJA_MODE);
JINJA_STMT_START : '{%' -> pushMode(JINJA_MODE);

// Default mode - outside Jinja tags
HTML_TEXT : ( ~'{' | '~{' ~{{%#} )+ ;

// Jinja mode - inside {{ }} or {% %} mode
JINJA_MODE : JINJA_EXPR_END : '}}' -> popMode;
JINJA_STMT_END : '%}' -> popMode;

IF : 'if';
ELIF : 'elif';
ELSE : 'else';
ENDIF : 'endif';
FOR : 'for';
IN : 'in';
ENDFOR : 'endfor';

AND : 'and';
OR : 'or';
NOT : 'not';

TRUE : 'true';
FALSE : 'false';
NONE : 'none';

EQ : '==';
NE : '!=';
LT : '<';
GT : '>';
LE : '<=';
GE : '>=';

PLUS : '+';
MINUS : '-';
STAR : '*';
SLASH : '/';
MOD : '%';

DOT : '.';
LPAREN : '(';
RPAREN : ')';
LBRACK : '[';
RBRACK : ']';
COMMA : ',';

STRING : '"' (~['\\'] | '\\' .)* '"';
INT : [0-9]+;
FLOAT : [0-9]+ '.' [0-9]+;

IDENTIFIER : [_a-zA-Z] [_a-zA-Z0-9]*;

WS : [ \\t\\n]+ -> skip;
```

?? File: JinjaParser.g4

```
parser grammar JinjaParser;

options

// ===== // TEMPLATE ROOT // ===== template : body=templateBody EOF ;

// ===== // TEMPLATE BODY // ===== templateBody : element* ;

// ===== // ELEMENTS // ===== element : HTML_TEXT #htmlElement | expressionOutput #expressionElement |
//                                     ifStatement #ifElement | forStatement #forElement ;

// ===== // {{ expression }} // =====

expressionOutput : JINJA_EXPR_START expression=expr JINJA_EXPR_END ;

// ===== // {% if %} ... {% elif %} ... {% else %} ... {% endif %} // ===== ifStatement : JINJA_STMT_START IF
//                                     condition=expr JINJA_STMT_END thenBody=templateBody elifClause* elseClause? JINJA_STMT_START ENDIF JINJA_STMT_END ;

//                                     elifClause : JINJA_STMT_START ELIF condition=expr JINJA_STMT_END body=templateBody ;

//                                     elseClause : JINJA_STMT_START ELSE JINJA_STMT_END body=templateBody ;

// ===== // {% for %} ... {% endfor %} // ===== forStatement : JINJA_STMT_START FOR variable=IDENTIFIER
//                                     IN iterable=expr JINJA_STMT_END body=templateBody JINJA_STMT_START ENDFOR JINJA_STMT_END ;

// ===== // EXPRESSIONS (helper rules for operator precedence) // ===== expr : logicalOr ;

//                                     logicalOr : left=logicalOr op=OR right=logicalAnd #orExpr | logicalAnd #logicalAndExpr ;

//                                     logicalAnd : left=logicalAnd op=AND right=comparison #andExpr | comparison #comparisonExpr ;

//                                     comparison : left=comparison op=(EQ|NE|LT|GT|LE|GE) right=additive #comparisonOpExpr | additive #additiveExpr ;

//                                     additive : left=additive op=(PLUS|MINUS) right=multiplicative #addExpr | multiplicative #multiplicativeExpr ;

//                                     multiplicative : left=multiplicative op=(STAR|SLASH|MOD) right=unary #mulExpr | unary #unaryExpr ;

//                                     unary : op=NOT operand=unary #notExpr | op=MINUS operand=unary #unaryMinus | primary #primaryExpr ;

// ===== // PRIMARY EXPRESSIONS // ===== primary : LPAREN expression=expr RPAREN #parenExpr | literal
//                                     #literalExpr | identifier=IDENTIFIER trailers+=trailer* #identifierExpr ;
```

```
// ===== // LITERALS // ===== literal : STRING #stringLiteral | INT #intLiteral | FLOAT #floatLiteral | TRUE
#trueLiteral | FALSE #falseLiteral | NONE #noneLiteral ;

// ===== // TRAILERS (mirroring Python's atomTrailer) // ===== trailer : DOT attribute=IDENTIFIER
#attributeAccess | LBRACK index=expr RBRACK #indexAccess | LPAREN arguments=argList? RPAREN #callTrailer ;

// ===== // ARGUMENT LIST // ===== argList : expr (COMMA expr)* (COMMA)? ;
```

?? File: PythonLexer.g4

```
lexer grammar PythonLexer;

DEF : 'def'; RETURN : 'return';

IF : 'if'; ELIF : 'elif'; ELSE : 'else';

FOR : 'for'; IN : 'in'; WHILE : 'while';

TRUE : 'True'; FALSE : 'False'; NONE : 'None';

AND : 'and'; OR : 'or'; NOT : 'not';

EQ : '=='; NE : '!='; LE : '<='; GE : '>=';

ASSIGN : '=';

LT : '<'; GT : '>';

PLUS : '+'; MINUS : '-'; STAR : '*'; SLASH : '/'; MOD : '%';

LPAREN : '('; RPAREN : ')';

LBRACK : '['; RBRACK : ']';

LBRACE : '{'; RBRACE : '}';

COLON : ':'; COMMA : ','; DOT : '.'; AT : '@';

FLOAT_LITERAL : DIGIT+ '.' DIGIT+ ; INTEGER_LITERAL : DIGIT+ ;

STRING_LITERAL : '"' (ESC_SEQ | ~["'\r\n])* '"' | "'" (ESC_SEQ | ~["'\r\n])* "'";

IDENTIFIER : [_a-zA-Z] [_a-zA-Z0-9]* ;

NEWLINE : '\r'? '\n' | '\r' ;

COMMENT : '#' ~["'\r\n"]* -> skip ;

WS : [\t]+ -> skip ;

fragment DIGIT : [0-9]; fragment ESC_SEQ : '\\' . ;
```

?? File: PythonParser.g4

```
parser grammar PythonParser;

options

tokens { INDENT, DEDENT }

program : (statement | NEWLINE)* EOF ;

statement : RETURN expr (COMMA expr)* NEWLINE? #returnStatement | expr (ASSIGN expr)* NEWLINE? #assignOrExprStatement | decorator* NEWLINE? DEF IDENTIFIER
LPAREN paramList? RPAREN COLON suite #functionDef | IF expr COLON suite (ELIF expr COLON suite)* (ELSE COLON suite)? #ifStatement | WHILE expr COLON suite
#whileStatement | FOR IDENTIFIER IN expr COLON suite #forStatement ;

decorator : AT IDENTIFIER (DOT IDENTIFIER)* LPAREN argList? RPAREN ;

paramList : IDENTIFIER (COMMA IDENTIFIER)* ;

suite : simpleLine #singleSuite | NEWLINE INDENT statement+ DEDENT #indentedSuite ;

simpleLine : expr (ASSIGN expr)* NEWLINE?;
```

expr : expr OR expr #orExpr | expr AND expr #andExpr | NOT expr #notExpr | expr (EQ|NE|LT|GT|LE|GE) expr #comparisonExpr | expr (PLUS|MINUS) expr #addExpr | expr (STAR|SLASH|MOD) expr #mulExpr | MINUS expr #unaryMinus | atom #atomExpr ;

listItems : expr (COMMA expr)* (COMMA)? ;

atom : LPAREN expr RPAREN #parenExpr | LBRACK listItems? RBRACK #listExpr | LBRACE dictItems? RBRACE #dictExpr | STRING_LITERAL #stringLiteral | INTEGER_LITERAL #intLiteral | FLOAT_LITERAL #floatLiteral | TRUE #trueLiteral | FALSE #falseLiteral | NONE #noneLiteral | IDENTIFIER atomTrailer* #identifierAtom ;

atomTrailer : DOT IDENTIFIER #attributeAccess | LBRACK expr RBRACK #indexAccess | LPAREN argList? RPAREN #callTrailer ;

argList : argument (COMMA argument)* (COMMA)? ;

argument : expr #positionalArg | IDENTIFIER ASSIGN expr #keywordArg ;

dictItems : dictItem (COMMA dictItem)* (COMMA)? ;

dictItem : (STRING_LITERAL | IDENTIFIER) COLON expr ;

?? File: test.g4

parser grammar PythonParser;

options

//===== // Program //=====

program : programElement* EOF #Program ;

programElement : functionDefinition NEWLINE* #FunctionElement

```
| statement NEWLINE*
#StatementElement

| NEWLINE
#BlankLine
;
```

//===== // Statements //=====

statement : assignmentStatement #AssignmentStmt

```
| returnStatement
#ReturnStmt

| expressionStatement
#ExpressionStmt

| ifStatement
#IfStmt

| whileStatement
#WhileStmt

| forStatement
#ForStmt

| passStatement
#PassStmt
;
```

block : NEWLINE INDENT statement+ DEDENT #Block ;

//===== // Function Definition //=====

functionDefinition : routeDecorator* DEF IDENTIFIER LPAREN parameterList? RPAREN COLON block #FunctionDefinition ;

routeDecorator : AT IDENTIFIER DOT IDENTIFIER LPAREN STRING_LITERAL RPAREN NEWLINE #RouteDecorator ;

parameterList : parameter (COMMA parameter)* COMMA? #ParameterList ;

parameter : IDENTIFIER #Parameter ;

//===== // Assignment =====

assignmentStatement : assignmentTarget ASSIGN expression #AssignmentStatement ;

assignmentTarget : IDENTIFIER #VariableAssignment ;

//===== // Return =====

returnStatement : RETURN expression? #ReturnStatement ;

//===== // Expression Statement =====

expressionStatement : expression #ExpressionStatement ;

//===== // Pass =====

passStatement : IDENTIFIER #PassStatement ;

//===== // If Statement =====

ifStatement : IF expression COLON block elifClause* elseClause? #IfStatement ;

elifClause : ELIF expression COLON block #ElifClause ;

elseClause : ELSE COLON block #ElseClause ;

//===== // While Statement =====

whileStatement : WHILE expression COLON block #WhileStatement ;

//===== // For Statement =====

forStatement : FOR IDENTIFIER IN expression COLON block #ForStatement ;

//===== // Expression =====

expression : logicalOrExpression #Expression ;

//===== // Logical OR =====

logicalOrExpression : logicalOrExpression OR logicalAndExpression #OrExpression

```
| logicalAndExpression
| #LogicalAndRoot
| ;
```

//===== // Logical AND =====

logicalAndExpression : logicalAndExpression AND equalityExpression #AndExpression

```
| equalityExpression
| #EqualityRoot
| ;
```

//===== // Equality =====

equalityExpression : equalityExpression EQ comparisonExpression #EqualExpression

```
| equalityExpression
| NE
| comparisonExpression
| #NotEqualExpression

| comparisonExpression
| #ComparisonRoot
| ;
```

//===== // Comparison =====

comparisonExpression : comparisonExpression LT additiveExpression #LessThanExpression

```
| comparisonExpression
  LE
  additiveExpression
  #LessEqualExpression

| comparisonExpression
  GT
  additiveExpression
  #GreaterThanExpression

| comparisonExpression
  GE
  additiveExpression
  #GreaterEqualExpression

| additiveExpression
  #AdditiveRoot
;
```

?? File: Main.java

import test.CompilerPipelineTest; import java.io.IOException;

public class Main { public static void main(String[] args) throws IOException { CompilerPipelineTest pipelineTest = new CompilerPipelineTest();

```
String pythonFile = "test-data/python/compiler_full_test.py";
String templatesDirectory = "test-data/jinja";

pipelineTest.test(pythonFile, templatesDirectory);
}
```

}.

?? File: AbstractAstNode.java

package compiler.ast.common;

/**

- Base implementation for AST nodes. */ public abstract class AbstractAstNode implements AstNode {

private final String nodeName; private final int line;

protected AbstractAstNode(String nodeName, int line)

@Override public String getNodeName() { return nodeName; }

@Override public int getLine() { return line; }

@Override public String prettyPrint(String indent) { return indent + getNodeName() + " [line " + getLine() + "]; } }

.

?? File: AbstractExpression.java

package compiler.ast.common;

/**

- Base implementation for expression nodes. */ public abstract class AbstractExpression extends AbstractAstNode implements Expression {

protected AbstractExpression(String nodeName, int line) { super(nodeName, line); } }

.

?? File: Argument.java

```
package compiler.ast.common;
```

```
/**
```

- Base interface for function arguments. */ public interface Argument extends AstNode

```
.
```

?? File: AstNode.java

```
package compiler.ast.common;
```

```
/**
```

- Base interface for all AST nodes. */ public interface AstNode extends PrettyPrintable {

```
/**
```

- Node type name. */ String getNodeName();

```
/**
```

- Source line. */ int getLine();

```
/**
```

- Visitor entry. */ T accept(AstVisitor visitor);

```
}
```

```
.
```

?? File: AstVisitor.java

```
package compiler.ast.common;
```

```
import compiler.ast.python.; import compiler.ast.jinja.;
```

```
/**
```

- Generic visitor for the entire compiler AST.
- Supports both Python-specific and common expression nodes, plus
- Jinja-specific template nodes. */ public interface AstVisitor {

```
// Common expression nodes T visitIdentifier(Identifier node);
```

```
T visitIntegerLiteral(IntegerLiteral node);
```

```
T visitFloatLiteral(FloatLiteral node);
```

```
T visitStringLiteral(StringLiteral node);
```

```
T visitBooleanLiteral(BooleanLiteral node);
```

```
T visitNoneLiteral(NoneLiteral node);
```

```
T visitBinaryExpr(BinaryExpr node);
```

```
T visitUnaryExpr(UnaryExpr node);
```

```
T visitCallExpr(CallExpr node);
```

```
T visitAttributeAccess(AttributeAccess node);
```

```
T visitIndexAccess(IndexAccess node);
```

```
T visitPositionalArgument(PositionalArgument node);
```

```
T visitKeywordArgument(KeywordArgument node);
```

```
// Python-specific nodes T visitProgram(Program node);
```

```

T visitListExpr(ListExpr node);

T visitDictExpr(DictExpr node);

T visitDictEntry(DictEntry node);

T visitForStmt(ForStmt node);

T visitAssign(Assign node);

T visitExprStmt (ExprStmt node);

T visitElifClause(ElifClause node);

T visitReturnStmt(ReturnStmt node);

T visitFunctionParameter(FunctionParameter node);

T visitDecorator(Decorator node);

T visitIfStmt(IfStmt node);

T visitElseClause(ElseClause node);

T visitWhileStmt(WhileStmt node);

T visitFunctionDef(FunctionDef node);

// Jinja-specific nodes
T visitTemplate(Template node);

T visitHtmlText(HtmlText node);

T visitExpressionOutput(ExpressionOutput node);

T visitJinjaIfStmt(JinjaIfStmt node);

T visitJinjaElifClause(JinjaElifClause node);

T visitJinjaElseClause(JinjaElseClause node);

T visitJinjaForStmt(JinjaForStmt node); }

```

?? File: AttributeAccess.java

```

package compiler.ast.common;

import java.util.Objects;

/**
 *
 */
• obj.attribute */ public final class AttributeAccess extends AbstractExpression {

    private final Expression target;

    private final String attribute;

    public AttributeAccess( int line, Expression target, String attribute) {

        super("AttributeAccess", line);

        this.target = Objects.requireNonNull(target);
        this.attribute = Objects.requireNonNull(attribute);

    }

    public Expression getTarget() { return target; }

    public String getAttribute() { return attribute; }

    @Override public T accept(AstVisitor visitor) { return visitor.visitAttributeAccess(this); }

}

```

?? File: BinaryExpr.java

```
package compiler.ast.common;
```

```
import java.util.Objects;
```

```
/**
```

- Binary expression. */ public final class BinaryExpr extends AbstractExpression {

```
    private final Expression left;
```

```
    private final BinaryOperator operator;
```

```
    private final Expression right;
```

```
    public BinaryExpr( int line, Expression left, BinaryOperator operator, Expression right) {
```

```
        super("BinaryExpr", line);
```

```
        this.left = Objects.requireNonNull(left);
```

```
        this.operator = Objects.requireNonNull(operator);
```

```
        this.right = Objects.requireNonNull(right);
```

```
    }
```

```
    public Expression getLeft() { return left; }
```

```
    public BinaryOperator getOperator() { return operator; }
```

```
    public Expression getRight() { return right; }
```

```
    @Override public T accept(AstVisitor visitor) { return visitor.visitBinaryExpr(this); }
```

```
    @Override public String prettyPrint(String indent) {
```

```
        String child = indent + "  ";
```

```
        return indent + "BinaryExpr (" + operator + ") [line " + getLine() + "]\n"
            + left.prettyPrint(child) + "\n"
            + right.prettyPrint(child);
```

```
    }
```

```
    }
```

?? File: BinaryOperator.java

```
package compiler.ast.common;
```

```
/**
```

- Supported binary operators. */ public enum BinaryOperator {

```
    OR,
```

```
    AND,
```

```
    EQ, NE, LT, LE, GT, GE,
```

```
    ADD, SUBTRACT,
```

```
    MULTIPLY, DIVIDE, MODULO
```

```
    }
```


?? File: BooleanLiteral.java

```
package compiler.ast.common;
```

```
/**
```

- Boolean literal. */ public final class BooleanLiteral extends LiteralExpression {

```
    private final boolean value;
```

```
    public BooleanLiteral(int line, boolean value) {
```

```
        super("BooleanLiteral", line);

        this.value = value;
    }
```

```
    public boolean getValue() { return value; }
```

```
    @Override public T accept(AstVisitor visitor) { return visitor.visitBooleanLiteral(this); }
```

```
    @Override public String prettyPrint(String indent) {
```

```
        return indent +
            getNodeName() +
            " (" +
            value +
            ") [line " +
            getLine() +
            "]\n";
    }
```

```
}
```

```
.
```

?? File: CallExpr.java

```
package compiler.ast.common;
```

```
import java.util.List; import java.util.Objects;
```

```
public final class CallExpr extends AbstractExpression {
```

```

private final Expression callee;

private final List<Argument> arguments;

public CallExpr(
    int line,
    Expression callee,
    List<Argument> arguments) {

    super("CallExpr", line);

    this.callee = Objects.requireNonNull(callee);
    this.arguments = List.copyOf(arguments);
}

public Expression getCallee() {
    return callee;
}

public List<Argument> getArguments() {
    return arguments;
}

@Override
public <T> T accept(AstVisitor<T> visitor) {
    return visitor.visitCallExpr(this);
}

@Override
public String prettyPrint(String indent) {
    StringBuilder sb = new StringBuilder();
    sb.append(indent).append("CallExpr (line ").append(getLine()).append(")");
    sb.append("\n").append(indent).append("  Callee");
    sb.append("\n").append(callee.prettyPrint(indent + "  "));
    sb.append("\n").append(indent).append("  Arguments");
    if (arguments.isEmpty()) {
        sb.append(" []");
    } else {
        for (Argument arg : arguments) {
            sb.append("\n").append(arg.prettyPrint(indent + "  "));
        }
    }
    return sb.toString();
}

```

}

.

?? File: Expression.java

package compiler.ast.common;

/**

- Marker interface for expressions. */ public interface Expression extends AstNode

.

?? File: FloatLiteral.java

package compiler.ast.common;

/**

- Floating-point literal. */ public final class FloatLiteral extends LiteralExpression {

private final double value;

```
public FloatLiteral(int line, double value) {
```

```
    super("FloatLiteral", line);

    this.value = value;
```

```
}
```

```
public double getValue() { return value; }
```

```
@Override public T accept(AstVisitor visitor) { return visitor.visitFloatLiteral(this); }
```

```
@Override public String prettyPrint(String indent) {
```

```
    return indent +
        getNodeName() +
        " (" +
        value +
        ") [line " +
        getLine() +
        "];
```

```
}
```

```
}
```

```
.
```

?? File: Identifier.java

```
package compiler.ast.common;
```

```
import java.util.Objects;
```

```
/**
```

- Identifier expression. */ public final class Identifier extends AbstractExpression {

```
    private final String name;
```

```
    public Identifier(int line, String name) {
```

```
        super("Identifier", line);

        this.name = Objects.requireNonNull(name);
```

```
}
```

```
public String getName() { return name; }
```

```
@Override public T accept(AstVisitor visitor) { return visitor.visitIdentifier(this); }
```

```
@Override public String prettyPrint(String indent) {
```

```
    return indent +
        getNodeName() +
        " (" +
        name +
        ") [line " +
        getLine() +
        "];
```

```
}
```

```
}
```

```
.
```

?? File: IndexAccess.java

```
package compiler.ast.common;
```

```
import java.util.Objects;
```

```
/**
```

- target[index] */ public final class IndexAccess extends AbstractExpression {

```
    private final Expression target;
```

```
    private final Expression index;
```

```
    public IndexAccess( int line, Expression target, Expression index) {
```

```
        super("IndexAccess", line);
```

```
        this.target = Objects.requireNonNull(target);
```

```
        this.index = Objects.requireNonNull(index);
```

```
    }
```

```
    public Expression getTarget() { return target; }
```

```
    public Expression getIndex() { return index; }
```

```
    @Override public T accept(AstVisitor visitor) { return visitor.visitIndexAccess(this); }
```

```
    }
```

```
    .
```

?? File: IntegerLiteral.java

```
package compiler.ast.common;
```

```
/**
```

- Integer literal. */ public final class IntegerLiteral extends LiteralExpression {

```
    private final long value;
```

```
    public IntegerLiteral(int line, long value) {
```

```
        super("IntegerLiteral", line);
```

```
        this.value = value;
```

```
    }
```

```
    public long getValue() { return value; }
```

```
    @Override public T accept(AstVisitor visitor) { return visitor.visitIntegerLiteral(this); }
```

```
    @Override public String prettyPrint(String indent) {
```

```
        return indent +
            getNodeName() +
            " (" +
            value +
            ") [line " +
            getLine() +
            "];
```

```
    }
```

```
    }
```

```
    .
```

?? File: KeywordArgument.java

```
package compiler.ast.common;

import java.util.Objects;

public final class KeywordArgument extends AbstractAstNode implements Argument {
```

```
    private final String name;

    private final Expression value;

    public KeywordArgument(
        int line,
        String name,
        Expression value) {

        super("KeywordArgument", line);

        this.name = Objects.requireNonNull(name);
        this.value = Objects.requireNonNull(value);
    }

    public String getName() {
        return name;
    }

    public Expression getValue() {
        return value;
    }

    @Override
    public <T> T accept(AstVisitor<T> visitor) {
        return visitor.visitKeywordArgument(this);
    }

    @Override
    public String prettyPrint(String indent) {
        StringBuilder sb = new StringBuilder();
        sb.append(indent).append("KeywordArgument ").append(name).append(" [line ").append(getLine()).append("]");
        sb.append("\n").append(value.prettyPrint(indent + "  "));
        return sb.toString();
    }
}
```

}

.

?? File: LiteralExpression.java

```
package compiler.ast.common;
```

```
/**
```

- Base class for all literal expressions.

-

- Examples:

-

- Integer

-

- Float

-

- String

-

- Boolean

- - None
- - List
- - Dictionary */ public abstract class LiteralExpression extends AbstractExpression {

```
protected LiteralExpression( String nodeName, int line) {
```

```
    super( nodeName, line);
```

```
}
```

```
}
```

```
.
```

?? File: NoneLiteral.java

```
package compiler.ast.common;
```

```
/**
```

- None literal. */ public final class NoneLiteral extends LiteralExpression {

```
public NoneLiteral(int line) { super("NoneLiteral", line); }
```

```
@Override public T accept(AstVisitor visitor) { return visitor.visitNoneLiteral(this); }
```

```
@Override public String prettyPrint(String indent) {
```

```
    return indent +
           getNodeName() +
           " [line " +
           getLine() +
           " ]";
```

```
}
```

```
}
```

```
.
```

?? File: PositionalArgument.java

```
package compiler.ast.common;
```

```
import java.util.Objects;
```

```
public final class PositionalArgument extends AbstractAstNode implements Argument {
```

```

private final Expression value;

public PositionalArgument(int line, Expression value) {

    super("PositionalArgument", line);

    this.value = Objects.requireNonNull(value);
}

public Expression getValue() {
    return value;
}

@Override
public <T> T accept(AstVisitor<T> visitor) {
    return visitor.visitPositionalArgument(this);
}

@Override
public String prettyPrint(String indent) {
    StringBuilder sb = new StringBuilder();
    sb.append(indent).append("PositionalArgument (line ").append(getLine()).append(")");
    sb.append("\n").append(value.prettyPrint(indent + "  "));
    return sb.toString();
}

```

}

.

?? File: PrettyPrintable.java

package compiler.ast.common;

/**

- Represents an AST node that can print itself in a readable tree format. */ public interface PrettyPrintable {

String prettyPrint(String indent);

}

.

?? File: StringLiteral.java

package compiler.ast.common;

import java.util.Objects;

/**

- String literal. */ public final class StringLiteral extends LiteralExpression {

private final String value;

public StringLiteral(int line, String value) {

super("StringLiteral", line);

this.value = Objects.requireNonNull(value);

}

public String getValue() { return value; }

@Override public T accept(AstVisitor visitor) { return visitor.visitStringLiteral(this); }

@Override public String prettyPrint(String indent) {

```

        return indent +
            getNodeName() +
            " (" +
            value +
            ")" + [line] +
            getLine() +
            " ";
    }
}

```

?? File: UnaryExpr.java

```

package compiler.ast.common;

public class UnaryExpr extends AbstractExpression {

```

```

    private final UnaryOperator operator;
    private final Expression expr;

    public UnaryExpr(
        int line,
        UnaryOperator operator,
        Expression expr
    ) {
        super("UnaryExpr", line);
        this.operator = operator;
        this.expr = expr;
    }

    public UnaryOperator getOperator() {
        return operator;
    }

    public Expression getExpr() {
        return expr;
    }

    @Override
    public <T> T accept(AstVisitor<T> visitor) {
        return visitor.visitUnaryExpr(this);
    }
}

```

?? File: UnaryOperator.java

```

package compiler.ast.common;

public enum UnaryOperator { PLUS, MINUS, NOT }

```

?? File: AbstractTemplateElement.java

```

package compiler.ast.jinja;

import compiler.ast.common.AbstractAstNode;

/**

```

- Base class for all Jinja template element nodes

- (the Jinja-tree equivalent of AbstractStatement.java on the Python side). */ public abstract class AbstractTemplateElement extends AbstractAstNode implements TemplateElement {

```
protected AbstractTemplateElement(String nodeName, int line) { super(nodeName, line); }
```

```
}
```

```
.
```

?? File: ExpressionOutput.java

```
package compiler.ast.jinja;
```

```
import compiler.ast.common.AstVisitor; import compiler.ast.common.Expression;
```

```
import java.util.Objects;
```

```
/**
```

- A {{ expression }} output tag. Wraps the expression that should be
- evaluated and substituted into the rendered output. */ public final class ExpressionOutput extends AbstractTemplateElement {

```
private final Expression expression;
```

```
public ExpressionOutput(int line, Expression expression) {
```

```
    super("ExpressionOutput", line);

    this.expression = Objects.requireNonNull(expression);
```

```
}
```

```
public Expression getExpression() { return expression; }
```

```
@Override public T accept(AstVisitor visitor) { return visitor.visitExpressionOutput(this); }
```

```
@Override public String prettyPrint(String indent) { StringBuilder sb = new StringBuilder(); sb.append(indent).append("ExpressionOutput (line ").append(getLine()).append(")"); sb.append("\n").append(expression.prettyPrint(indent + " "); return sb.toString(); }
```

```
}
```

```
.
```

?? File: HtmlText.java

```
package compiler.ast.jinja;
```

```
import compiler.ast.common.AstVisitor;
```

```
import java.util.Objects;
```

```
/**
```

- Raw HTML/text content that sits between Jinja tags (the HTML_TEXT token
- from the Jinja lexer). A leaf node in the Jinja AST. */ public final class HtmlText extends AbstractTemplateElement {

```
private final String text;
```

```
public HtmlText(int line, String text) {
```

```
    super("HtmlText", line);

    this.text = Objects.requireNonNull(text);
```

```
}
```

```
public String getText() { return text; }
```

```
@Override public T accept(AstVisitor visitor) { return visitor.visitHtmlText(this); }
```

```
@Override public String prettyPrint(String indent) {
```

```

        String preview = text.replace("\n", "\\n").replace("\r", "");
        if (preview.length() > 40) {
            preview = preview.substring(0, 40) + "...";
        }

        return indent +
            getNodeName() +
            " (" +
            preview +
            "\") [line " +
            getLine() +
            " ]";
    }
}

```

?? File: JinjaElifClause.java

```

package compiler.ast.jinja;

import compiler.ast.common.AstVisitor; import compiler.ast.common.Expression;

import java.util.List; import java.util.Objects;

/**
 *
 * • {% elif condition %} body (the Jinja-tree equivalent of ElifClause.java). */ public final class JinjaElifClause extends AbstractTemplateElement {
    private final Expression condition;

    private final List body;

    public JinjaElifClause( int line, Expression condition, List body) {

        super("JinjaElifClause", line);

        this.condition =
            Objects.requireNonNull(condition);

        this.body =
            List.copyOf(body);
    }

    public Expression getCondition() { return condition; }

    public List getBody() { return body; }

    @Override public T accept(AstVisitor visitor) { return visitor.visitJinjaElifClause(this); }

    @Override public String prettyPrint(String indent) { StringBuilder sb = new StringBuilder(); sb.append(indent).append("JinjaElifClause (line ").append(getLine()).append(")");
    sb.append("\n").append(indent).append(" Condition"); sb.append("\n").append(condition.prettyPrint(indent + " ")); sb.append("\n").append(indent).append(" Body"); for
    (TemplateElement element : body) { sb.append("\n").append(element.prettyPrint(indent + " ")); } return sb.toString(); } }

```

?? File: JinjaElseClause.java

```

package compiler.ast.jinja;

import compiler.ast.common.AstVisitor;

import java.util.List; import java.util.Objects;

/**
 *
 * • {% else %} body (the Jinja-tree equivalent of ElseClause.java). */ public final class JinjaElseClause extends AbstractTemplateElement {

```

```
private final List body;
```

```
public JinjaElseClause( int line, List body) {
```

```
    super("JinjaElseClause", line);

    this.body = List.copyOf(
        Objects.requireNonNull(body));

```

```
}
```

```
public List getBody() { return body; }
```

```
@Override public T accept(AstVisitor visitor) { return visitor.visitJinjaElseClause(this); }
```

```
@Override public String prettyPrint(String indent) { StringBuilder sb = new StringBuilder(); sb.append(indent).append("JinjaElseClause (line ").append(getLine()).append(")");
sb.append("\n").append(indent).append(" Body"); for (TemplateElement element : body) { sb.append("\n").append(element.prettyPrint(indent + " ")); } return sb.toString(); }
```

?? File: JinjaForStmt.java

```
package compiler.ast.jinja;
```

```
import compiler.ast.common.AstVisitor; import compiler.ast.common.Expression; import compiler.ast.common.Identifier;
```

```
import java.util.List; import java.util.Objects;
```

```
/**
```

- {% for variable in iterable %} body {% endfor %}
- (the Jinja-tree equivalent of ForStmt.java on the Python side). */ public final class JinjaForStmt extends AbstractTemplateElement {

```
private final Identifier variable;
```

```
private final Expression iterable;
```

```
private final List body;
```

```
public JinjaForStmt( int line, Identifier variable, Expression iterable, List body) {
```

```
    super("JinjaForStmt", line);

    this.variable =
        Objects.requireNonNull(variable);

    this.iterable =
        Objects.requireNonNull(iterable);

    this.body =
        List.copyOf(body);

```

```
}
```

```
public Identifier getVariable() { return variable; }
```

```
public Expression getIterable() { return iterable; }
```

```
public List getBody() { return body; }
```

```
@Override public T accept(AstVisitor visitor) { return visitor.visitJinjaForStmt(this); }
```

```
@Override public String prettyPrint(String indent) { StringBuilder sb = new StringBuilder(); sb.append(indent).append("JinjaForStmt (line ").append(getLine()).append(")");
sb.append("\n").append(indent).append(" Variable"); sb.append("\n").append(variable.prettyPrint(indent + " ")); sb.append("\n").append(indent).append(" Iterable");
sb.append("\n").append(iterable.prettyPrint(indent + " ")); sb.append("\n").append(indent).append(" Body"); for (TemplateElement element : body) {
sb.append("\n").append(element.prettyPrint(indent + " ")); } return sb.toString(); }
```

```
}
```

?? File: JinjaIfStmt.java

```
package compiler.ast.jinja;

import compiler.ast.common.AstVisitor; import compiler.ast.common.Expression;

import java.util.List; import java.util.Objects;

/**
 *
 * • {% if condition %} thenBody {% elif ... %} ... {% else %} ... {% endif %}
 *
 * • (the Jinja-tree equivalent of IfStmt.java on the Python side). */ public final class JinjaIfStmt extends AbstractTemplateElement {

    private final Expression condition;

    private final List thenBody;

    private final List elifClauses;

    private final JinjaElseClause elseClause;

    public JinjaIfStmt( int line, Expression condition, List thenBody, List elifClauses, JinjaElseClause elseClause) {

        super("JinjaIfStmt", line);

        this.condition = Objects.requireNonNull(condition);
        this.thenBody = List.copyOf(
            Objects.requireNonNull(thenBody));
        this.elifClauses = List.copyOf(
            Objects.requireNonNull(elifClauses));
        this.elseClause = elseClause; // Can be null

    }

    public Expression getCondition() { return condition; }

    public List getThenBody() { return thenBody; }

    public List getElifClauses() { return elifClauses; }

    public JinjaElseClause getElseClause() { return elseClause; }

    @Override public T accept(AstVisitor visitor) { return visitor.visitJinjaIfStmt(this); }

    @Override public String prettyPrint(String indent) { StringBuilder sb = new StringBuilder(); sb.append(indent).append("JinjaIfStmt (line ").append(getLine()).append(")");
    sb.append("\n").append(indent).append(" Condition"); sb.append("\n").append(condition.prettyPrint(indent + " ")); sb.append("\n").append(indent).append(" ThenBody"); for
    (TemplateElement element : thenBody) { sb.append("\n").append(element.prettyPrint(indent + " ")); } for (JinjaElseClause elif : elifClauses) {
    sb.append("\n").append(elif.prettyPrint(indent + " ")); } if (elseClause != null) { sb.append("\n").append(elseClause.prettyPrint(indent + " ")); } return sb.toString(); } }
```

?? File: Template.java

```
package compiler.ast.jinja;

import compiler.ast.common.AbstractAstNode; import compiler.ast.common.AstVisitor;

import java.util.List; import java.util.Objects;

/**
 *
 * • Root of the Jinja AST (the Jinja-tree equivalent of Program.java on the
 *
 * • Python side). Holds the ordered sequence of top-level template elements:
 *
 * • raw HTML text, {{ expression }} outputs, {% if %} blocks and {% for %} blocks. */ public final class Template extends AbstractAstNode {

    private final List elements;

    public Template( int line, List elements) {
```

```

        super("Template", line);

        this.elements = List.copyOf(
            Objects.requireNonNull(elements));
    }

    public List getElements() { return elements; }

    @Override public T accept(AstVisitor visitor) { return visitor.visitTemplate(this); }

    @Override public String prettyPrint(String indent) {

        StringBuilder builder = new StringBuilder();

        builder.append(super.prettyPrint(indent));

        for (TemplateElement element : elements) {

            builder.append(System.lineSeparator())
                .append(element.prettyPrint(indent + "  "));

        }

        return builder.toString();
    }
}

```

?? File: TemplateElement.java

```

package compiler.ast.jinja;

import compiler.ast.common.AstNode;

/**

```

- Marker interface for Jinja template elements
- (the Jinja-tree equivalent of Statement.java on the Python side). */ public interface TemplateElement extends AstNode

?? File: AbstractStatement.java

```

package compiler.ast.python;

import compiler.ast.common.AbstractAstNode;

/**

```

- Base class for all statement nodes. */ public abstract class AbstractStatement extends AbstractAstNode implements Statement {

```

    protected AbstractStatement(String nodeName, int line) { super(nodeName, line); }

    }.

```

?? File: Assign.java

```

package compiler.ast.python;

import compiler.ast.common.AstVisitor; import compiler.ast.common.Expression;

import java.util.List; import java.util.Objects;

/**

```

- Assignment statement. */ public final class Assign extends AbstractStatement {

```

    private final List targets; private final Expression value;

```

```
public Assign( int line, List targets, Expression value) {
```

```
    super("Assign", line);

    this.targets = List.copyOf(
        Objects.requireNonNull(targets));

    this.value = Objects.requireNonNull(value);
```

```
}
```

```
public List getTargets() { return targets; }
```

```
public Expression getValue() { return value; }
```

```
@Override public T accept(AstVisitor visitor) { return visitor.visitAssign(this); }
```

```
@Override public String prettyPrint(String indent) {
```

```
    StringBuilder sb = new StringBuilder();

    sb.append(indent)
        .append("Assign [line ")
        .append(getLine())
        .append("]");

    sb.append("\n")
        .append(indent)
        .append("  Targets");

    for (Expression e : targets) {

        sb.append("\n")
            .append(e.prettyPrint(indent + "    "));
    }

    sb.append("\n")
        .append(indent)
        .append("  Value\n")
        .append(value.prettyPrint(indent + "    "));

    return sb.toString();
```

```
}
```

```
    }.
```

?? File: Decorator.java

```
package compiler.ast.python;
```

```
import compiler.ast.common.AstVisitor; import compiler.ast.common.AbstractAstNode; import compiler.ast.common.Argument; import compiler.ast.common.Expression;
```

```
import java.util.List; import java.util.Objects;
```

```
public final class Decorator extends AbstractAstNode {
```

```

private final Expression target;

private final List<Argument> arguments;

public Decorator(
    int line,
    Expression target,
    List<Argument> arguments) {

    super("Decorator", line);

    this.target = Objects.requireNonNull(target);
    this.arguments = List.copyOf(
        Objects.requireNonNull(arguments));
}

public Expression getTarget() {
    return target;
}

public List<Argument> getArguments() {
    return arguments;
}

@Override
public <T> T accept(AstVisitor<T> visitor) {
    return visitor.visitDecorator(this);
}

@Override
public String prettyPrint(String indent) {

    StringBuilder sb = new StringBuilder();

    sb.append(indent)
        .append("Decorator [line ")
        .append(getLine())
        .append("]");

    sb.append("\n")
        .append(target.prettyPrint(indent + "  "));

    for (Argument arg : arguments) {
        sb.append("\n")
            .append(arg.prettyPrint(indent + "  "));
    }

    return sb.toString();
}

```

}.

?? File: DictEntry.java

```
package compiler.ast.python;
```

```
import compiler.ast.common.AstVisitor; import compiler.ast.common.AbstractAstNode; import compiler.ast.common.Expression;
```

```
import java.util.Objects;
```

```
/**
```

- Single dictionary entry. */ public final class DictEntry extends AbstractAstNode {

```
    private final Expression key;
```

```
    private final Expression value;
```

```
public DictEntry( int line, Expression key, Expression value) {
```

```
    super("DictEntry", line);

    this.key = Objects.requireNonNull(key);
    this.value = Objects.requireNonNull(value);
```

```
}
```

```
public Expression getKey() { return key; }
```

```
public Expression getValue() { return value; }
```

```
@Override public T accept(AstVisitor visitor) { return visitor.visitDictEntry(this); }
```

```
@Override public String prettyPrint(String indent) {
```

```
    return indent + "DictEntry\n"
        + key.prettyPrint(indent + "  ")
        + "\n"
        + value.prettyPrint(indent + "  ");
```

```
}
```

```
    }.
```

?? File: DictExpr.java

```
package compiler.ast.python;
```

```
import compiler.ast.common.AstVisitor; import compiler.ast.common.LiteralExpression;
```

```
import java.util.List; import java.util.Objects;
```

```
/**
```

- Python dictionary literal. */ public final class DictExpr extends LiteralExpression {

```
    private final List entries;
```

```
public DictExpr( int line, List entries) {
```

```
    super("DictExpr", line);

    this.entries = List.copyOf(
        Objects.requireNonNull(entries)
    );
```

```
}
```

```
public List getEntries() { return entries; }
```

```
@Override public T accept(AstVisitor visitor) { return visitor.visitDictExpr(this); }
```

```
@Override public String prettyPrint(String indent) {
```



```

StringBuilder sb = new StringBuilder();

sb.append(indent)
    .append(getNodeName())
    .append(" [line ")
    .append(getLine())
    .append("]");

for (DictEntry entry : entries) {

    sb.append("\n")
        .append(entry.prettyPrint(indent + "  "));
}

return sb.toString();

```

```

}

```

```

}

```

?? File: ElifClause.java

```

package compiler.ast.python;

import compiler.ast.common.AstVisitor; import compiler.ast.common.Expression;

import java.util.List; import java.util.Objects;

public final class ElifClause extends AbstractStatement {

```

```

private final Expression condition;

private final List<Statement> body;

public ElifClause(
    int line,
    Expression condition,
    List<Statement> body) {

    super("ElifClause", line);

    this.condition =
        Objects.requireNonNull(condition);

    this.body =
        List.copyOf(body);
}

public Expression getCondition() {
    return condition;
}

public List<Statement> getBody() {
    return body;
}

@Override
public <T> T accept(AstVisitor<T> visitor) {
    return visitor.visitElifClause(this);
}

@Override
public String prettyPrint(String indent) {
    StringBuilder sb = new StringBuilder();
    sb.append(indent).append("ElifClause (line ").append(getLine()).append(")");
    sb.append("\n").append(indent).append("  Condition");
    sb.append("\n").append(indent).append(condition.prettyPrint(indent + "  "));
    sb.append("\n").append(indent).append("  Body");
    for (Statement stmt : body) {
        sb.append("\n").append(stmt.prettyPrint(indent + "  "));
    }
    return sb.toString();
}
}

```

}.

?? File: ElseClause.java

```

package compiler.ast.python;

import compiler.ast.common.AstVisitor;

import java.util.List; import java.util.Objects;

/**

```

- Else clause for if statements. */ public final class ElseClause extends AbstractStatement {

```
private final List body;
```

```
public ElseClause( int line, List body) {
```

```

    super("ElseClause", line);

    this.body = List.copyOf(
        Objects.requireNonNull(body));
}

```

```

}

public List getBody() { return body; }

@Override public T accept(AstVisitor visitor) { return visitor.visitElseClause(this); }

@Override public String prettyPrint(String indent) { StringBuilder sb = new StringBuilder(); sb.append(indent).append("ElseClause (line ").append(getLine()).append(")");
sb.append("\n").append(indent).append(" Body"); for (Statement stmt : body) { sb.append("\n").append(stmt.prettyPrint(indent + " ")); } return sb.toString(); } }

```

?? File: ExprStmt.java

```

package compiler.ast.python;

import compiler.ast.common.AstVisitor; import compiler.ast.common.Expression;

import java.util.Objects;

/**

```

- Expression statement.
-
- Example:
-
- print(x) */ public final class ExprStmt extends AbstractStatement {

```

private final Expression expression;

public ExprStmt( int line, Expression expression) {

```

```

    super("ExprStmt", line);

    this.expression =
        Objects.requireNonNull(expression);

```

```

}

public Expression getExpression() { return expression; }

@Override public T accept(AstVisitor visitor) { return visitor.visitExprStmt(this); }

@Override public String prettyPrint(String indent) {

```

```

    StringBuilder sb = new StringBuilder();

    sb.append(indent)
        .append("ExprStmt [line ")
        .append(getLine())
        .append("]");

    sb.append("\n")
        .append(expression.prettyPrint(indent + " "));

    return sb.toString();

```

```

}}.

```

?? File: ForStmt.java

```

package compiler.ast.python;

import compiler.ast.common.AstVisitor; import compiler.ast.common.Expression; import compiler.ast.common.Identifier;

import java.util.List; import java.util.Objects;

public final class ForStmt extends AbstractStatement {

```

```

private final Identifier variable;

private final Expression iterable;

private final List<Statement> body;

public ForStmt(
    int line,
    Identifier variable,
    Expression iterable,
    List<Statement> body) {

    super("ForStmt", line);

    this.variable =
        Objects.requireNonNull(variable);

    this.iterable =
        Objects.requireNonNull(iterable);

    this.body =
        List.copyOf(body);
}

public Identifier getVariable() {
    return variable;
}

public Expression getIterable() {
    return iterable;
}

public List<Statement> getBody() {
    return body;
}

@Override
public <T> T accept(AstVisitor<T> visitor) {

    return visitor.visitForStmt(this);
}

@Override
public String prettyPrint(String indent) {
    StringBuilder sb = new StringBuilder();
    sb.append(indent).append("ForStmt (line ").append(getLine()).append(")");
    sb.append("\n").append(indent).append("  Variable");
    sb.append("\n").append(indent).append(variable.prettyPrint(indent + "    "));
    sb.append("\n").append(indent).append("  Iterable");
    sb.append("\n").append(indent).append(iterable.prettyPrint(indent + "    "));
    sb.append("\n").append(indent).append("  Body");
    for (Statement stmt : body) {
        sb.append("\n").append(indent).append(stmt.prettyPrint(indent + "    "));
    }
    return sb.toString();
}

```

}.

?? File: FunctionDef.java

```

package compiler.ast.python;

import compiler.ast.common.AstVisitor;

import java.util.List; import java.util.Objects;

```

/**

- Python function definition. */ public final class FunctionDef extends AbstractStatement {

private final String name;

private final List parameters;

private final List decorators;

private final List body;

public FunctionDef(int line, String name, List parameters, List decorators, List body) {

```
super("FunctionDef", line);

this.name = Objects.requireNonNull(name);
this.parameters = List.copyOf(
    Objects.requireNonNull(parameters));
this.decorators = List.copyOf(
    Objects.requireNonNull(decorators));
this.body = List.copyOf(
    Objects.requireNonNull(body));
```

}

public String getName() { return name; }

public List getParameters() { return parameters; }

public List getDecorators() { return decorators; }

public List getBody() { return body; }

@Override public T accept(AstVisitor visitor) { return visitor.visitFunctionDef(this); }

@Override public String prettyPrint(String indent) {

```

StringBuilder sb = new StringBuilder();

sb.append(indent)
    .append("FunctionDef (")
    .append(name)
    .append(") [line ")
    .append(getLine())
    .append("]");

if (!decorators.isEmpty()) {

    sb.append("\n")
        .append(indent)
        .append("  Decorators");

    for (Decorator decorator : decorators) {
        sb.append("\n")
            .append(decorator.prettyPrint(indent + "    "));
    }
}

if (!parameters.isEmpty()) {

    sb.append("\n")
        .append(indent)
        .append("  Parameters");

    for (FunctionParameter parameter : parameters) {
        sb.append("\n")
            .append(parameter.prettyPrint(indent + "    "));
    }
}

sb.append("\n")
    .append(indent)
    .append("  Body");

for (Statement stmt : body) {
    sb.append("\n")
        .append(stmt.prettyPrint(indent + "    "));
}

return sb.toString();

```

}}.

?? File: FunctionParameter.java

```
package compiler.ast.python;
```

```
import compiler.ast.common.AstVisitor; import compiler.ast.common.AbstractAstNode;
```

```
import java.util.Objects;
```

```
/**
```

- Represents a function parameter. */ public final class FunctionParameter extends AbstractAstNode {

```
private final String name;
```

```
public FunctionParameter(int line, String name) {
```

```

    super("FunctionParameter", line);

    this.name = Objects.requireNonNull(name);

```

```
}
```

```
public String getName() { return name; }
```

```
@Override public T accept(AstVisitor visitor) { return visitor.visitFunctionParameter(this); }
```

```
@Override public String prettyPrint(String indent) {
```

```
    return indent +  
        "Parameter(" +  
        name +  
        ") [line " +  
        getLine() +  
        "];
```

```
}}.
```

?? File: IfStmt.java

```
package compiler.ast.python;
```

```
import compiler.ast.common.AstVisitor; import compiler.ast.common.Expression;
```

```
import java.util.List; import java.util.Objects;
```

```
public final class IfStmt extends AbstractStatement {
```

```

private final Expression condition;

private final List<Statement> thenBody;

private final List<ElifClause> elifClauses;

private final ElseClause elseClause;

public IfStmt(
    int line,
    Expression condition,
    List<Statement> thenBody,
    List<ElifClause> elifClauses,
    ElseClause elseClause) {

    super("IfStmt", line);

    this.condition = Objects.requireNonNull(condition);
    this.thenBody = List.copyOf(
        Objects.requireNonNull(thenBody));
    this.elifClauses = List.copyOf(
        Objects.requireNonNull(elifClauses));
    this.elseClause = elseClause; // Can be null
}

public Expression getCondition() {
    return condition;
}

public List<Statement> getThenBody() {
    return thenBody;
}

public List<ElifClause> getElifClauses() {
    return elifClauses;
}

public ElseClause getElseClause() {
    return elseClause;
}

@Override
public <T> T accept(AstVisitor<T> visitor) {
    return visitor.visitIfStmt(this);
}

@Override
public String prettyPrint(String indent) {
    StringBuilder sb = new StringBuilder();
    sb.append(indent).append("IfStmt (line ").append(getLine()).append(")");
    sb.append("\n").append(indent).append("  Condition");
    sb.append("\n").append(indent).append(condition.prettyPrint(indent + "  "));
    sb.append("\n").append(indent).append("  ThenBody");
    for (Statement stmt : thenBody) {
        sb.append("\n").append(stmt.prettyPrint(indent + "    "));
    }
    for (ElifClause elif : elifClauses) {
        sb.append("\n").append(elif.prettyPrint(indent + "  "));
    }
    if (elseClause != null) {
        sb.append("\n").append(elseClause.prettyPrint(indent + "  "));
    }
    return sb.toString();
}

```


}.
}

?? File: ListExpr.java

```
package compiler.ast.python;
```

```
import compiler.ast.common.AstVisitor; import compiler.ast.common.Expression; import compiler.ast.common.LiteralExpression;
```

```
import java.util.List; import java.util.Objects;
```

```
/**
```

- Python list literal. */ public final class ListExpr extends LiteralExpression {

```
    private final List elements;
```

```
    public ListExpr( int line, List elements) {
```

```
        super("ListExpr", line);

        this.elements = List.copyOf(
            Objects.requireNonNull(elements)
        );
    }
```

```
    public List getElements() { return elements; }

    @Override public T accept(AstVisitor visitor) { return visitor.visitListExpr(this); }
```

```
    @Override public String prettyPrint(String indent) {
```

```
        StringBuilder sb = new StringBuilder();

        sb.append(indent)
            .append(getNodeName())
            .append(" [line ")
            .append(getLine())
            .append("]");

        for (Expression expression : elements) {

            sb.append("\n")
                .append(expression.prettyPrint(indent + "  "));

        }

        return sb.toString();
    }
```

```
}
```

}.
}

?? File: Program.java

```
package compiler.ast.python;
```

```
import compiler.ast.common.AstVisitor; import compiler.ast.common.AbstractAstNode; import compiler.ast.common.Expression;
```

```
import java.util.List; import java.util.Objects;
```

```
/**
```

- Root of the Python AST. */ public final class Program extends AbstractAstNode {

```
    private final List statements;
```

```
    public Program( int line, List statements) {
```

```

        super("Program", line);

        this.statements = List.copyOf(
            Objects.requireNonNull(statements)
        );
    }

    public List getStatements() { return statements; }

    @Override public T accept(AstVisitor visitor) { return visitor.visitProgram(this); }

    @Override public String prettyPrint(String indent) {

        StringBuilder builder = new StringBuilder();

        builder.append(super.prettyPrint(indent));

        for (Statement stmt : statements) {

            builder.append(System.lineSeparator())
                .append(stmt.prettyPrint(indent + "  "));

        }

        return builder.toString();
    }

}

}.
```

?? File: ReturnStmt.java

```

package compiler.ast.python;

import compiler.ast.common.AstVisitor; import compiler.ast.common.Expression;

import java.util.Optional;

/**
 * return statement.
 */
public final class ReturnStmt extends AbstractStatement {

    private final Expression value;

    public ReturnStmt( int line, Expression value) {

        super("ReturnStmt", line);

        this.value = value;
    }

    public Optional getValue() {

        return Optional.ofNullable(value);
    }

    @Override public T accept(AstVisitor visitor) {

        return visitor.visitReturnStmt(this);
    }

    @Override public String prettyPrint(String indent) { if (value == null) { return indent + "ReturnStmt [line " + getLine() + "]\n"; } return indent + "ReturnStmt [line " + getLine() + "]\n"
        + value.prettyPrint(indent + "  "); }

}

}.
```

?? File: Statement.java

```
package compiler.ast.python;

import compiler.ast.common.AstNode;

/**
```

- Marker interface for statements. `*/ public interface Statement extends AstNode .`

?? File: WhileStmt.java

```
package compiler.ast.python;

import compiler.ast.common.AstVisitor; import compiler.ast.common.Expression;

import java.util.List; import java.util.Objects;

public final class WhileStmt extends AbstractStatement {
```

```
    private final Expression condition;

    private final List<Statement> body;

    public WhileStmt(
        int line,
        Expression condition,
        List<Statement> body) {

        super("WhileStmt", line);

        this.condition = Objects.requireNonNull(condition);
        this.body = List.copyOf(body);
    }

    public Expression getCondition() {
        return condition;
    }

    public List<Statement> getBody() {
        return body;
    }

    @Override
    public <T> T accept(AstVisitor<T> visitor) {
        return visitor.visitWhileStmt(this);
    }

    @Override
    public String prettyPrint(String indent) {
        StringBuilder sb = new StringBuilder();
        sb.append(indent).append("WhileStmt (line ").append(getLine()).append(")");
        sb.append("\n").append(indent).append("  Condition");
        sb.append("\n").append(indent).append(condition.prettyPrint(indent + "  "));
        sb.append("\n").append(indent).append("  Body");
        for (Statement stmt : body) {
            sb.append("\n").append(indent).append(stmt.prettyPrint(indent + "  "));
        }
        return sb.toString();
    }
}
```

```
}
```

```
.
```

?? File: LexicalException.java

```
package compiler.exception;
```

```
public class LexicalException extends RuntimeException {
```

```
    public LexicalException(String message) {  
        super(message);  
    }  
}
```

```
    }.
```

?? File: AstBuilder.java

```
package compiler.frontend.jinja;
```

```
import compiler.ast.jinja.; import compiler.ast.common.; import compiler.generated.jinja.JinjaParser; import compiler.generated.jinja.JinjaParserBaseVisitor; import  
org.antlr.v4.runtime.tree.TerminalNode;
```

```
import java.util.*; import java.util.stream.Collectors;
```

```
/**
```

- AstBuilder: converts ANTLR parse tree (compiler.generated.jinja.JinjaParser)
- into the project's Jinja AST nodes (compiler.ast.jinja.*), reusing the
- shared expression nodes from compiler.ast.common.* wherever possible
- (Identifier, literals, BinaryExpr, UnaryExpr, CallExpr, AttributeAccess,
- IndexAccess, PositionalArgument) so that the Python and Jinja trees stay
- interoperable.
-
- Mirrors compiler.frontend.python.AstBuilder in style and structure. */ public final class AstBuilder extends JinjaParserBaseVisitor {

```
/* ----- Public entry point ----- */
```

```
public Template build(JinjaParser.TemplateContext ctx) { List elements = ctx.templateBody().element().stream().map(this::toElement).collect(Collectors.toList()); return new  
Template(lineOf(ctx), elements); }
```

```
/* ----- Helpers ----- */
```

```
private int lineOf(org.antlr.v4.runtime.ParserRuleContext ctx) { return ctx == null ? -1 : ctx.getStart().getLine(); }
```

```
private int lineOf(TerminalNode node) { return node == null ? -1 : node.getSymbol().getLine(); }
```

```
private int lineOf(org.antlr.v4.runtime.Token token) { return token == null ? -1 : token.getLine(); }
```

```
private TemplateElement toElement(JinjaParser.ElementContext ctx) { Object o = visit(ctx); if (o == null) return null; if (!(o instanceof TemplateElement)) { throw new  
IllegalStateException("Expected TemplateElement but got " + o.getClass().getSimpleName() + " at line " + lineOf(ctx)); } return (TemplateElement) o; }
```

```
private List toElementList(JinjaParser.TemplateBodyContext ctx) { if (ctx == null) return Collections.emptyList(); return ctx.element().stream().map(this::toElement)  
.collect(Collectors.toList()); }
```

```
private Expression toExpression(JinjaParser.ExprContext ctx) { Object o = visit(ctx); if (o == null) return null; if (!(o instanceof Expression)) { throw new  
IllegalStateException("Expected Expression but got " + o.getClass().getSimpleName() + " at line " + lineOf(ctx)); } return (Expression) o; }
```

```
private List toArgumentList(JinjaParser.ArgListContext ctx) { if (ctx == null) return Collections.emptyList(); return ctx.expr().stream().map(e -> (Argument) new  
PositionalArgument(lineOf(e), toExpression(e))).collect(Collectors.toList()); }
```

```
private String unquote(String s) { if (s == null || s.length() < 2) return ""; String inner = s.substring(1, s.length() - 1); return inner.replace("\"\"", "").replace("'", "").replace("\n",  
"\n").replace("\t", "t"); }
```

```
private BinaryOperator mapComparisonOperator(String op) { switch (op) { case "==": return BinaryOperator.EQ; case "!=": return BinaryOperator.NE; case "<": return  
BinaryOperator.LT; case "<=": return BinaryOperator.LE; case ">": return BinaryOperator.GT; case ">=": return BinaryOperator.GE; default: throw new  
IllegalArgumentException("Unknown comparison operator: " + op); } }
```

```
/* ----- Template root / elements ----- */
```

```
@Override public Object visitTemplate(JinjaParser.TemplateContext ctx) { return build(ctx); }
```

```
@Override public Object visitHtmlElement(JinjaParser.HtmlElementContext ctx) { return new HtmlText(lineOf(ctx), ctx.HTML_TEXT().getText()); }
```

```
@Override public Object visitExpressionElement(JinjaParser.ExpressionElementContext ctx) { return visit(ctx.expressionOutput()); }
```

```

@Override public Object visitIfElement(JinjaParser.IfElementContext ctx) { return visit(ctx.ifStatement()); }

@Override public Object visitForElement(JinjaParser.ForElementContext ctx) { return visit(ctx.forStatement()); }

@Override public Object visitExpressionOutput(JinjaParser.ExpressionOutputContext ctx) { // JINJA_EXPR_START expression=expr JINJA_EXPR_END Expression expr =
toExpression(ctx.expression); return new ExpressionOutput(lineOf(ctx), expr); }

@Override public Object visitIfStatement(JinjaParser.IfStatementContext ctx) { // JINJA_STMT_START IF condition=expr JINJA_STMT_END // thenBody=templateBody //
elifClause* // elseClause? // JINJA_STMT_START ENDIF JINJA_STMT_END Expression cond = toExpression(ctx.condition); List thenBody = toElementList(ctx.thenBody);

```

```

        List<JinjaElifClause> elifs = ctx.elifClause().stream()
            .map(e -> (JinjaElifClause) visit(e))
            .collect(Collectors.toList());

        JinjaElseClause elseClause = ctx.elseClause() == null
            ? null
            : (JinjaElseClause) visit(ctx.elseClause());

        return new JinjaIfStmt(lineOf(ctx), cond, thenBody, elifs, elseClause);

```

```

}

```

```

@Override public Object visitElifClause(JinjaParser.ElifClauseContext ctx) { // JINJA_STMT_START ELIF condition=expr JINJA_STMT_END body=templateBody Expression
cond = toExpression(ctx.condition); List body = toElementList(ctx.body); return new JinjaElifClause(lineOf(ctx), cond, body); }

```

```

@Override public Object visitElseClause(JinjaParser.ElseClauseContext ctx) { // JINJA_STMT_START ELSE JINJA_STMT_END body=templateBody List body =
toElementList(ctx.body); return new JinjaElseClause(lineOf(ctx), body); }

```

```

@Override public Object visitForStatement(JinjaParser.ForStatementContext ctx) { // JINJA_STMT_START FOR variable=IDENTIFIER IN iterable=expr JINJA_STMT_END //
body=templateBody // JINJA_STMT_START ENDFOR JINJA_STMT_END Identifier var = new Identifier(lineOf(ctx.variable), ctx.variable.getText()); Expression iterable =
toExpression(ctx.iterable); List body = toElementList(ctx.body); return new JinjaForStmt(lineOf(ctx), var, iterable, body); }

```

```

/* ----- Expression visitors (labels) - operator precedence chain ----- */

```

```

@Override public Object visitExpr(JinjaParser.ExprContext ctx) { return visit(ctx.logicalOr()); }

```

```

@Override public Object visitOrExpr(JinjaParser.OrExprContext ctx) { Expression left = (Expression) visit(ctx.left); Expression right = (Expression) visit(ctx.right); return new
BinaryExpr(lineOf(ctx), left, BinaryOperator.OR, right); }

```

```

@Override public Object visitLogicalAndExpr(JinjaParser.LogicalAndExprContext ctx) { // Pass-through: logicalOr -> logicalAnd return visit(ctx.logicalAnd()); }

```

```

@Override public Object visitAndExpr(JinjaParser.AndExprContext ctx) { Expression left = (Expression) visit(ctx.left); Expression right = (Expression) visit(ctx.right); return new
BinaryExpr(lineOf(ctx), left, BinaryOperator.AND, right); }

```

```

@Override public Object visitComparisonExpr(JinjaParser.ComparisonExprContext ctx) { // Pass-through: logicalAnd -> comparison return visit(ctx.comparison()); }

```

```

@Override public Object visitComparisonOpExpr(JinjaParser.ComparisonOpExprContext ctx) { Expression left = (Expression) visit(ctx.left); Expression right = (Expression)
visit(ctx.right); BinaryOperator op = mapComparisonOperator(ctx.op.getText()); return new BinaryExpr(lineOf(ctx), left, op, right); }

```

```

@Override public Object visitAdditiveExpr(JinjaParser.AdditiveExprContext ctx) { // Pass-through: comparison -> additive return visit(ctx.additive()); }

```

```

@Override public Object visitAddExpr(JinjaParser.AddExprContext ctx) { Expression left = (Expression) visit(ctx.left); Expression right = (Expression) visit(ctx.right);
BinaryOperator op = ctx.op.getText().equals("+") ? BinaryOperator.ADD : BinaryOperator.SUBTRACT; return new BinaryExpr(lineOf(ctx), left, op, right); }

```

```

@Override public Object visitMultiplicativeExpr(JinjaParser.MultiplicativeExprContext ctx) { // Pass-through: additive -> multiplicative return visit(ctx.multiplicative()); }

```

```

@Override public Object visitMulExpr(JinjaParser.MulExprContext ctx) { Expression left = (Expression) visit(ctx.left); Expression right = (Expression) visit(ctx.right);
BinaryOperator op; String opText = ctx.op.getText(); if (opText.equals("****")) op = BinaryOperator.MULTIPLY; else if (opText.equals("/")) op = BinaryOperator.DIVIDE; else op =
BinaryOperator.MODULO; return new BinaryExpr(lineOf(ctx), left, op, right); }

```

```

@Override public Object visitUnaryExpr(JinjaParser.UnaryExprContext ctx) { // Pass-through: multiplicative -> unary return visit(ctx.unary()); }

```

```

@Override public Object visitNotExpr(JinjaParser.NotExprContext ctx) { Expression operand = (Expression) visit(ctx.operand); return new UnaryExpr(lineOf(ctx),
UnaryOperator.NOT, operand); }

```

```

@Override public Object visitUnaryMinus(JinjaParser.UnaryMinusContext ctx) { Expression operand = (Expression) visit(ctx.operand); return new UnaryExpr(lineOf(ctx),
UnaryOperator.MINUS, operand); }

```

```

@Override public Object visitPrimaryExpr(JinjaParser.PrimaryExprContext ctx) { // Pass-through: unary -> primary return visit(ctx.primary()); }

```

```

/* ----- Primary / literal visitors ----- */

```

```

@Override public Object visitParenExpr(JinjaParser.ParenExprContext ctx) { return toExpression(ctx.expression); }

@Override public Object visitLiteralExpr(JinjaParser.LiteralExprContext ctx) { // Pass-through: primary -> literal return visit(ctx.literal()); }

@Override public Object visitIdentifierExpr(JinjaParser.IdentifierExprContext ctx) { // identifier=IDENTIFIER trailers+=trailer* Expression base = new Identifier(lineOf(ctx),
ctx.identifier.getText()); for (JinjaParser.TrailerContext t : ctx.trailer()) { base = applyTrailer(base, t); } return base; }

@Override public Object visitStringLiteral(JinjaParser.StringLiteralContext ctx) { return new StringLiteral(lineOf(ctx), unquote(ctx.STRING().getText())); }

@Override public Object visitIntLiteral(JinjaParser.IntLiteralContext ctx) { long v = Long.parseLong(ctx.INT().getText()); return new IntegerLiteral(lineOf(ctx), v); }

@Override public Object visitFloatLiteral(JinjaParser.FloatLiteralContext ctx) { double v = Double.parseDouble(ctx.FLOAT().getText()); return new FloatLiteral(lineOf(ctx), v); }

@Override public Object visitTrueLiteral(JinjaParser.TrueLiteralContext ctx) { return new BooleanLiteral(lineOf(ctx), true); }

@Override public Object visitFalseLiteral(JinjaParser.FalseLiteralContext ctx) { return new BooleanLiteral(lineOf(ctx), false); }

@Override public Object visitNoneLiteral(JinjaParser.NoneLiteralContext ctx) { return new NoneLiteral(lineOf(ctx)); }

/* ----- Trailer helpers (mirrors Python's applyTrailer) ----- */

private Expression applyTrailer(Expression base, JinjaParser.TrailerContext t) {

    // trailer alternatives (labeled):
    //   DOT attribute=IDENTIFIER           #attributeAccess
    //   | LBRACK index=expr RBRACK         #indexAccess
    //   | LPAREN arguments=argList? RPAREN #callTrailer

    if (t instanceof JinjaParser.AttributeAccessContext) {
        JinjaParser.AttributeAccessContext ac = (JinjaParser.AttributeAccessContext) t;
        String attr = ac.attribute.getText();
        return new AttributeAccess(lineOf(ac), base, attr);
    }

    if (t instanceof JinjaParser.IndexAccessContext) {
        JinjaParser.IndexAccessContext ic = (JinjaParser.IndexAccessContext) t;
        Expression idx = toExpression(ic.index);
        return new IndexAccess(lineOf(ic), base, idx);
    }

    if (t instanceof JinjaParser.CallTrailerContext) {
        JinjaParser.CallTrailerContext cc = (JinjaParser.CallTrailerContext) t;
        List<Argument> args = toArgumentList(cc.arguments);
        return new CallExpr(lineOf(cc), base, args);
    }

    throw new IllegalStateException("Unknown trailer alternative at line " + lineOf(t));

}

/* ----- Fallbacks ----- */

@Override public Object visitChildren(org.antlr.v4.runtime.tree.RuleNode node) { return super.visitChildren(node); }

@Override protected Object defaultResult() { return null; } }

```

?? File: AstBuilder.java

```
package compiler.frontend.python;
```

```
import compiler.ast.python.; import compiler.ast.common.; import compiler.generated.python.PythonParser; import compiler.generated.python.PythonParserBaseVisitor; import
org.antlr.v4.runtime.tree.TerminalNode;
```

```
import java.util.*; import java.util.stream.Collectors;
```

```
/**
```

- AstBuilder: converts ANTLR parse tree (compiler.generated.python.PythonParser)

- into the project's AST nodes (compiler.ast.python.*).

-

- This implementation matches the labeled alternatives in your PythonParser.g4.

- It returns AST nodes as Object (the base visitor is parameterized with Object). */ public final class AstBuilder extends PythonParserBaseVisitor {

```
/* ----- Public entry point ----- */

public Program build(PythonParser.ProgramContext ctx) { List statements = ctx.statement().stream() .map(this::toStatement) .collect(Collectors.toList()); return new
Program(lineOf(ctx), statements); }

/* ----- Helpers ----- */

private int lineOf(org.antlr.v4.runtime.ParserRuleContext ctx) { return ctx == null ? -1 : ctx.getStart().getLine(); }

private int lineOf(TerminalNode node) { return node == null ? -1 : node.getSymbol().getLine(); }

private Statement toStatement(PythonParser.StatementContext ctx) { Object o = visit(ctx); if (o == null) return null; if (!(o instanceof Statement)) { throw new
IllegalStateException("Expected Statement but got " + o.getClass().getSimpleName() + " at line " + lineOf(ctx)); } return (Statement) o; }

private Expression toExpression(PythonParser.ExprContext ctx) { Object o = visit(ctx); if (o == null) return null; if (!(o instanceof Expression)) { throw new
IllegalStateException("Expected Expression but got " + o.getClass().getSimpleName() + " at line " + lineOf(ctx)); } return (Expression) o; }

private List toArgumentList(PythonParser.ArgListContext ctx) { if (ctx == null) return Collections.emptyList(); return ctx.argument().stream() .map(a -> (Argument)
visit(a)) .collect(Collectors.toList()); }

private String unquote(String s) { if (s == null || s.length() < 2) return ""; String inner = s.substring(1, s.length() - 1); return inner.replace("\"", "").replace("\",",
""),.replace("\n", "\n").replace("\t", "\t"); }

private BinaryOperator mapComparisonOperator(String op) { switch (op) { case "==": return BinaryOperator.EQ; case "!=": return BinaryOperator.NE; case "<": return
BinaryOperator.LT; case "<=": return BinaryOperator.LE; case ">": return BinaryOperator.GT; case ">=": return BinaryOperator.GE; default: throw new
IllegalArgumentException("Unknown comparison operator: " + op); } }

/* ----- Statement visitors (labels from grammar) ----- */

@Override public Object visitProgram(PythonParser.ProgramContext ctx) { return build(ctx); }

@Override public Object visitReturnStatement(PythonParser.ReturnStatementContext ctx) { // RETURN expr (COMMA expr)* NEWLINE
List<PythonParser.ExprContext> exprs = ctx.expr(); if (exprs == null || exprs.isEmpty()) { return new ReturnStmt(lineOf(ctx), null); } // For now, handle single expr
returns. Tuple returns would need a TupleExpr AST node. // Taking the first expression as the return value. Expression expr = toExpression(exprs.get(0)); return new
ReturnStmt(lineOf(ctx), expr); }

@Override public Object visitAssignOrExprStatement(PythonParser.AssignOrExprStatementContext ctx) { // expr (ASSIGN expr)* NEWLINE
List<PythonParser.ExprContext> exprs = ctx.expr(); if (ctx.ASSIGN().size() > 0) { List targets = exprs.subList(0, exprs.size() - 1).stream() .map(this::toExpression)
.collect(Collectors.toList()); Expression value = toExpression(exprs.get(exprs.size() - 1)); return new Assign(lineOf(ctx), targets, value); } else { Expression e =
toExpression(exprs.get(0)); return new ExprStmt(lineOf(ctx), e); } }

@Override public Object visitFunctionDef(PythonParser.FunctionDefContext ctx) { // decorator* DEF IDENTIFIER LPAREN paramList? RPAREN COLON suite List
decorators = ctx.decorator().stream() .map(d -> (Decorator) visit(d)) .collect(Collectors.toList());
```

```
String name = ctx.IDENTIFIER().getText();

List<FunctionParameter> params = Collections.emptyList();
if (ctx.paramList() != null) {
    params = ctx.paramList().IDENTIFIER().stream()
        .map(t -> new FunctionParameter(lineOf(t), t.getText()))
        .collect(Collectors.toList());
}

List<Statement> body = visitSuiteAsStatements(ctx.suite());

// Match your FunctionDef constructor:
// public FunctionDef(int line, String name, List<FunctionParameter> parameters,
//                    List<Decorator> decorators, List<Statement> body)
return new FunctionDef(lineOf(ctx), name, params, decorators, body);
```

```
}
```

```
@Override public Object visitIfStatement(PythonParser.IfStatementContext ctx) { // IF expr COLON suite (ELIF expr COLON suite)* (ELSE COLON suite)? Expression
cond = toExpression(ctx.expr(0)); List thenBody = visitSuiteAsStatements(ctx.suite(0));
```

```

    List<ElifClause> elifs = new ArrayList<>();
    int elifCount = ctx.ELIF().size();
    for (int i = 0; i < elifCount; i++) {
        Expression elifCond = toExpression(ctx.expr(i + 1));
        List<Statement> elifBody = visitSuiteAsStatements(ctx.suite(i + 1));
        elifs.add(new ElifClause(lineOf(ctx), elifCond, elifBody));
    }

    ElseClause elseClause = null;
    if (ctx.ELSE() != null) {
        PythonParser.SuiteContext elseSuite = ctx.suite(ctx.suite().size() - 1);
        List<Statement> elseBody = visitSuiteAsStatements(elseSuite);
        elseClause = new ElseClause(lineOf(ctx), elseBody);
    }

    return new IfStmt(lineOf(ctx), cond, thenBody, elifs, elseClause);
}

```

```
}
```

```
@Override public Object visitForStatement(PythonParser.ForStatementContext ctx) { // FOR IDENTIFIER IN expr COLON suite Identifier var = new
Identifier(lineOf(ctx.IDENTIFIER()), ctx.IDENTIFIER().getText()); Expression iterable = toExpression(ctx.expr()); List body = visitSuiteAsStatements(ctx.suite()); return
new ForStmt(lineOf(ctx), var, iterable, body); }
```

```
@Override public Object visitWhileStatement(PythonParser.WhileStatementContext ctx) { // WHILE expr COLON suite Expression cond = toExpression(ctx.expr());
List body = visitSuiteAsStatements(ctx.suite()); return new WhileStmt(lineOf(ctx), cond, body); }
```

```
/* ----- Decorator ----- */
```

```
@Override public Object visitDecorator(PythonParser.DecoratorContext ctx) { // AT IDENTIFIER (DOT IDENTIFIER)* LPAREN argList? RPAREN NEWLINE List ids =
ctx.IDENTIFIER(); Expression target = new Identifier(lineOf(ctx), ids.get(0).getText()); for (int i = 1; i < ids.size(); i++) { target = new AttributeAccess(lineOf(ctx), target,
ids.get(i).getText()); } List args = ctx.argList() == null ? Collections.emptyList() : toArgumentList(ctx.argList()); return new Decorator(lineOf(ctx), target, args); }
```

```
/* ----- Expression visitors (labels) ----- */
```

```
@Override public Object visitOrExpr(PythonParser.OrExprContext ctx) { Expression left = toExpression(ctx.expr(0)); Expression right = toExpression(ctx.expr(1));
return new BinaryExpr(lineOf(ctx), left, BinaryOperator.OR, right); }
```

```
@Override public Object visitAndExpr(PythonParser.AndExprContext ctx) { Expression left = toExpression(ctx.expr(0)); Expression right = toExpression(ctx.expr(1));
return new BinaryExpr(lineOf(ctx), left, BinaryOperator.AND, right); }
```

```
@Override public Object visitNotExpr(PythonParser.NotExprContext ctx) { // Represent 'not x' as (x == False) using BinaryExpr and BooleanLiteral(false). Expression
inner = toExpression(ctx.expr()); return new BinaryExpr(lineOf(ctx), inner, BinaryOperator.EQ, new BooleanLiteral(lineOf(ctx), false)); }
```

```
@Override public Object visitComparisonExpr(PythonParser.ComparisonExprContext ctx) { Expression left = toExpression(ctx.expr(0)); Expression right =
toExpression(ctx.expr(1)); String op = ctx.getChild(1).getText(); BinaryOperator binOp = mapComparisonOperator(op); return new BinaryExpr(lineOf(ctx), left, binOp,
right); }
```

```
@Override public Object visitAddExpr(PythonParser.AddExprContext ctx) { Expression left = toExpression(ctx.expr(0)); Expression right = toExpression(ctx.expr(1));
BinaryOperator op = ctx.PLUS() != null ? BinaryOperator.ADD : BinaryOperator.SUBTRACT; return new BinaryExpr(lineOf(ctx), left, op, right); }
```

```
@Override public Object visitMulExpr(PythonParser.MulExprContext ctx) { Expression left = toExpression(ctx.expr(0)); Expression right = toExpression(ctx.expr(1));
BinaryOperator op; if (ctx.STAR() != null) op = BinaryOperator.MULTIPLY; else if (ctx.SLASH() != null) op = BinaryOperator.DIVIDE; else op =
BinaryOperator.MODULO; return new BinaryExpr(lineOf(ctx), left, op, right); }
```

```
@Override public Object visitUnaryMinus(PythonParser.UnaryMinusContext ctx) { Expression inner = toExpression(ctx.expr()); // Represent unary minus as 0 - inner
using IntegerLiteral(0). Expression zero = new IntegerLiteral(lineOf(ctx), 0); return new BinaryExpr(lineOf(ctx), zero, BinaryOperator.SUBTRACT, inner); }
```

```
@Override public Object visitAtomExpr(PythonParser.AtomExprContext ctx) { return visit(ctx.atom()); }
```

```
/* ----- Atom visitors ----- */
```

```
@Override public Object visitParenExpr(PythonParser.ParenExprContext ctx) { return toExpression(ctx.expr()); }
```

```
@Override public Object visitListExpr(PythonParser.ListExprContext ctx) { List items = ctx.listItems() == null ? Collections.emptyList() :
ctx.listItems().expr().stream().map(this::toExpression).collect(Collectors.toList()); return new ListExpr(lineOf(ctx), items); }
```



```
@Override public Object visitDictExpr(PythonParser.DictExprContext ctx) { List entries = Collections.emptyList(); if (ctx.dictItems() != null) { entries = ctx.dictItems().dictItem().stream().map(di -> { Expression key; if (di.STRING_LITERAL() != null) { key = new StringLiteral(lineOf(di), unquote(di.STRING_LITERAL().getText())); } else { key = new Identifier(lineOf(di), di.IDENTIFIER().getText()); } Expression value = toExpression(di.expr()); return new DictEntry(lineOf(di), key, value); }).collect(Collectors.toList()); } return new DictExpr(lineOf(ctx), entries); }
```

```
@Override public Object visitStringLiteral(PythonParser.StringLiteralContext ctx) { return new StringLiteral(lineOf(ctx), unquote(ctx.STRING_LITERAL().getText())); }
```

```
@Override public Object visitIntLiteral(PythonParser.IntLiteralContext ctx) { int v = Integer.parseInt(ctx.INTEGER_LITERAL().getText()); return new IntegerLiteral(lineOf(ctx), v); }
```

```
@Override public Object visitFloatLiteral(PythonParser.FloatLiteralContext ctx) { double v = Double.parseDouble(ctx.FLOAT_LITERAL().getText()); return new FloatLiteral(lineOf(ctx), v); }
```

```
@Override public Object visitTrueLiteral(PythonParser.TrueLiteralContext ctx) { return new BooleanLiteral(lineOf(ctx), true); }
```

```
@Override public Object visitFalseLiteral(PythonParser.FalseLiteralContext ctx) { return new BooleanLiteral(lineOf(ctx), false); }
```

```
@Override public Object visitNoneLiteral(PythonParser.NoneLiteralContext ctx) { return new NoneLiteral(lineOf(ctx)); }
```

```
@Override public Object visitIdentifierAtom(PythonParser.IdentifierAtomContext ctx) { // IDENTIFIER atomTrailer* Expression base = new Identifier(lineOf(ctx), ctx.IDENTIFIER().getText()); for (PythonParser.AtomTrailerContext t : ctx.atomTrailer()) { base = applyTrailer(base, t); } return base; }
```

```
/* ----- AtomTrailer helpers ----- */
```

```
// Replace your existing applyTrailer(...) with this implementation private Expression applyTrailer(Expression base, compiler.generated.python.PythonParser.AtomTrailerContext t) {
```

```
    // atomTrailer alternatives (labeled):
    //   DOT IDENTIFIER                #attributeAccess
    //   | LBRACK expr RBRACK          #indexAccess
    //   | LPAREN argList? RPAREN      #callTrailer

    // Attribute access: subclass is PythonParser.AttributeAccessContext
    if (t instanceof compiler.generated.python.PythonParser.AttributeAccessContext) {
        compiler.generated.python.PythonParser.AttributeAccessContext ac =
            (compiler.generated.python.PythonParser.AttributeAccessContext) t;
        // DOT IDENTIFIER -> IDENTIFIER() exists on the AttributeAccessContext
        String attr = ac.IDENTIFIER().getText();
        return new AttributeAccess(lineOf(ac), base, attr);
    }

    // Index access: subclass is PythonParser.IndexAccessContext
    if (t instanceof compiler.generated.python.PythonParser.IndexAccessContext) {
        compiler.generated.python.PythonParser.IndexAccessContext ic =
            (compiler.generated.python.PythonParser.IndexAccessContext) t;
        Expression idx = toExpression(ic.expr());
        return new IndexAccess(lineOf(ic), base, idx);
    }

    // Call trailer: subclass is PythonParser.CallTrailerContext
    if (t instanceof compiler.generated.python.PythonParser.CallTrailerContext) {
        compiler.generated.python.PythonParser.CallTrailerContext cc =
            (compiler.generated.python.PythonParser.CallTrailerContext) t;
        List<Argument> args = cc.argList() == null ? Collections.emptyList() : toArgumentList(cc.argList());
        return new CallExpr(lineOf(cc), base, args);
    }

    throw new IllegalStateException("Unknown atomTrailer alternative at line " + lineOf(t));
}
```

```
}
```

```
/* ----- Arguments ----- */
```

```
@Override public Object visitPositionalArg(PythonParser.PositionalArgContext ctx) { Expression e = toExpression(ctx.expr()); return new PositionalArgument(lineOf(ctx), e); }
```

```
@Override public Object visitKeywordArg(PythonParser.KeywordArgContext ctx) { String name = ctx.IDENTIFIER().getText(); Expression value = toExpression(ctx.expr()); return new KeywordArgument(lineOf(ctx), name, value); }
```

```
@Override public Object visitArgList(PythonParser.ArgListContext ctx) { // Return a List as Object to match base visitor signature return toArgumentList(ctx); }
```

```
/* ----- Suite handling (single-line and indented) ----- */
```

```
// Replace your existing visitSuiteAsStatements(...) with this implementation private List visitSuiteAsStatements(compiler.generated.python.PythonParser.SuiteContext suite) { if (suite == null) return Collections.emptyList();
```

```
List<Statement> stmts = new ArrayList<>();

// Indented suite: NEWLINE INDENT statement+ DEDENT
if (suite instanceof compiler.generated.python.PythonParser.IndentedSuiteContext) {
    compiler.generated.python.PythonParser.IndentedSuiteContext ind =
        (compiler.generated.python.PythonParser.IndentedSuiteContext) suite;

    // ind.statement() is available on IndentedSuiteContext
    for (compiler.generated.python.PythonParser.StatementContext s : ind.statement()) {
        stmts.add(toStatement(s));
    }
    return stmts;
}

// Single-line suite: simpleLine
if (suite instanceof compiler.generated.python.PythonParser.SingleSuiteContext) {
    compiler.generated.python.PythonParser.SingleSuiteContext single =
        (compiler.generated.python.PythonParser.SingleSuiteContext) suite;

    // single.simpleLine() returns the SimpleLineContext
    compiler.generated.python.PythonParser.SimpleLineContext sl = single.simpleLine();
    if (sl != null) {
        List<compiler.generated.python.PythonParser.ExprContext> exprs = sl.expr();
        if (sl.ASSIGN().size() > 0) {
            List<Expression> targets = exprs.subList(0, exprs.size() - 1).stream()
                .map(this::toExpression)
                .collect(Collectors.toList());
            Expression value = toExpression(exprs.get(exprs.size() - 1));
            stmts.add(new Assign(lineOf(sl), targets, value));
        } else {
            stmts.add(new ExprStmt(lineOf(sl), toExpression(exprs.get(0))));
        }
    }
    return stmts;
}

// Fallback: try to find statement children (defensive)
for (int i = 0; i < suite.getChildCount(); i++) {
    org.antlr.v4.runtime.tree.ParseTree child = suite.getChild(i);
    if (child instanceof compiler.generated.python.PythonParser.StatementContext) {
        stmts.add(toStatement((compiler.generated.python.PythonParser.StatementContext) child));
    }
}
return stmts;
```

```
}
```

```
/* ----- Fallbacks ----- */
```

```
@Override public Object visitChildren(org.antlr.v4.runtime.tree.RuleNode node) { return super.visitChildren(node); }
```

```
@Override protected Object defaultResult() { return null; } }
```

```
.
```

?? File: JinjaLexer.java

```
// Generated from D:/Compiler/PyJinjaCompiler/src/grammar/jinja/JinjaLexer.g4 by ANTLR 4.13.2 package compiler.generated.jinja; import org.antlr.v4.runtime.Lexer;
import org.antlr.v4.runtime.CharStream; import org.antlr.v4.runtime.Token; import org.antlr.v4.runtime.TokenStream; import org.antlr.v4.runtime.; import
org.antlr.v4.runtime.atn.; import org.antlr.v4.runtime.dfa.DFA; import org.antlr.v4.runtime.misc.*;
```

```
@SuppressWarnings({"all", "warnings", "unchecked", "unused", "cast", "CheckReturnValue", "this-escape"}) public class JinjaLexer extends Lexer { static {  
    RuntimeMetadata.checkVersion("4.13.2", RuntimeMetadata.VERSION); }
```

```

protected static final DFA[] _decisionToDFA;
protected static final PredictionContextCache _sharedContextCache =
    new PredictionContextCache();
public static final int
    JINJA_EXPR_START=1, JINJA_STMT_START=2, HTML_TEXT=3, JINJA_EXPR_END=4,
    JINJA_STMT_END=5, IF=6, ELIF=7, ELSE=8, ENDIF=9, FOR=10, IN=11, ENDFOR=12,
    AND=13, OR=14, NOT=15, TRUE=16, FALSE=17, NONE=18, EQ=19, NE=20, LT=21,
    GT=22, LE=23, GE=24, PLUS=25, MINUS=26, STAR=27, SLASH=28, MOD=29, DOT=30,
    LPAREN=31, RPAREN=32, LBRACK=33, RBRACK=34, COMMA=35, STRING=36, INT=37,
    FLOAT=38, IDENTIFIER=39, WS=40;
public static final int
    JINJA_MODE=1;
public static String[] channelNames = {
    "DEFAULT_TOKEN_CHANNEL", "HIDDEN"
};

public static String[] modeNames = {
    "DEFAULT_MODE", "JINJA_MODE"
};

private static String[] makeRuleNames() {
    return new String[] {
        "JINJA_EXPR_START", "JINJA_STMT_START", "HTML_TEXT", "JINJA_EXPR_END",
        "JINJA_STMT_END", "IF", "ELIF", "ELSE", "ENDIF", "FOR", "IN", "ENDFOR",
        "AND", "OR", "NOT", "TRUE", "FALSE", "NONE", "EQ", "NE", "LT", "GT",
        "LE", "GE", "PLUS", "MINUS", "STAR", "SLASH", "MOD", "DOT", "LPAREN",
        "RPAREN", "LBRACK", "RBRACK", "COMMA", "STRING", "INT", "FLOAT", "IDENTIFIER",
        "WS"
    };
}

public static final String[] ruleNames = makeRuleNames();

private static String[] makeLiteralNames() {
    return new String[] {
        null, "'{'", "'{'", null, "'}'", "'%'", "'if'", "'elif'", "'else'",
        "'endif'", "'for'", "'in'", "'endfor'", "'and'", "'or'", "'not'", "'true'",
        "'false'", "'none'", "'=='", "'!='", "'<'", "'>'", "'<='", "'>='", "'+'",
        "'-'", "'*' ", "'/'", "'%'", "'.'", "'('", "')'", "'['", "']'", "'.'",
    };
}

private static final String[] _LITERAL_NAMES = makeLiteralNames();
private static String[] makeSymbolicNames() {
    return new String[] {
        null, "JINJA_EXPR_START", "JINJA_STMT_START", "HTML_TEXT", "JINJA_EXPR_END",
        "JINJA_STMT_END", "IF", "ELIF", "ELSE", "ENDIF", "FOR", "IN", "ENDFOR",
        "AND", "OR", "NOT", "TRUE", "FALSE", "NONE", "EQ", "NE", "LT", "GT",
        "LE", "GE", "PLUS", "MINUS", "STAR", "SLASH", "MOD", "DOT", "LPAREN",
        "RPAREN", "LBRACK", "RBRACK", "COMMA", "STRING", "INT", "FLOAT", "IDENTIFIER",
        "WS"
    };
}

private static final String[] _SYMBOLIC_NAMES = makeSymbolicNames();
public static final Vocabulary VOCABULARY = new VocabularyImpl(_LITERAL_NAMES, _SYMBOLIC_NAMES);

/**
 * @deprecated Use {@link #VOCABULARY} instead.
 */
@Deprecated
public static final String[] tokenNames;
static {
    tokenNames = new String[_SYMBOLIC_NAMES.length];
    for (int i = 0; i < tokenNames.length; i++) {
        tokenNames[i] = VOCABULARY.getLiteralName(i);
        if (tokenNames[i] == null) {
            tokenNames[i] = VOCABULARY.getSymbolicName(i);

```

```
}

        if (tokenNames[i] == null) {
            tokenNames[i] = "<INVALID>";
        }
    }
}

@Override
@Deprecated
public String[] getTokenNames() {
    return tokenNames;
}

@Override
public Vocabulary getVocabulary() {
    return VOCABULARY;
}

public JinjaLexer(CharStream input) {
    super(input);
    _interp = new LexerATNSimulator(this, _ATN, _decisionToDFA, _sharedContextCache);
}

@Override
public String getGrammarFileName() { return "JinjaLexer.g4"; }

@Override
public String[] getRuleNames() { return ruleNames; }

@Override
public String getSerializedATN() { return _serializedATN; }

@Override
public String[] getChannelNames() { return channelNames; }

@Override
public String[] getModeNames() { return modeNames; }

@Override
public ATN getATN() { return _ATN; }

public static final String _serializedATN =
    "\u0004\u0000(\u00f8\u0006\u00ff\u00ff\u0006\u00ff\u00ff\u0002\u0000\u0007"+
    "\u0000\u0002\u0001\u0007\u0001\u0002\u0002\u0002\u0007\u0002\u0002\u0003\u0007"+
    "\u0003\u0002\u0004\u0007\u0004\u0002\u0005\u0007\u0005\u0002\u0006\u0007"+
    "\u0006\u0002\u0007\u0007\u0007\u0002\b\u0007\b\u0002\t\u0007\t\u0002\n"+
    "\u0007\n\u0002\u000b\u0007\u000b\u0002\f\u0007\f\u0002\r\u0007\r\u0002"+
    "\u000e\u0007\u000e\u0002\u000f\u0007\u000f\u0002\u0010\u0007\u0010\u0002"+
    "\u0011\u0007\u0011\u0002\u0012\u0007\u0012\u0002\u0013\u0007\u0013\u0002"+
    "\u0014\u0007\u0014\u0002\u0015\u0007\u0015\u0002\u0016\u0007\u0016\u0002"+
    "\u0017\u0007\u0017\u0002\u0018\u0007\u0018\u0002\u0019\u0007\u0019\u0002"+
    "\u001a\u0007\u001a\u0002\u001b\u0007\u001b\u0002\u001c\u0007\u001c\u0002"+
    "\u001d\u0007\u001d\u0002\u001e\u0007\u001e\u0002\u001f\u0007\u001f\u0002"+
    " \u0007 \u0002!\u0007!\u0002\"\u0007\" \u0002#\u0007#\u0002$\u0007$\u0002"+
    "%\u0007%\u0002&\u0007&\u0002'\u0007' \u0001\u0000\u0001\u0000\u0001\u0000"+
    "\u0001\u0000\u0001\u0001\u0000\u0001\u0001\u0001\u0001\u0001\u0001\u0001"+
    "\u0001\u0001\u0001\u0002\u0001\u0002\u0001\u0002\u0001\u0002\u0001\u0002"+
    "\u000b\u0002\f\u0002a\u0001\u0003\u0001\u0003\u0001\u0003\u0001\u0003"+
    "\u0001\u0003\u0001\u0004\u0001\u0004\u0001\u0004\u0001\u0004\u0001\u0004"+
    "\u0001\u0005\u0001\u0005\u0001\u0005\u0001\u0005\u0001\u0006\u0001\u0006"+
    "\u0001\u0006\u0001\u0006\u0001\u0007\u0001\u0007\u0001\u0007\u0001\u0007"+
    "\u0001\u0007\u0001\b\u0001\b\u0001\b\u0001\b\u0001\b\u0001\b\u0001\b\u0001\b"+
    "\t\u0001\t\u0001\t\u0001\t\u0001\t\u0001\t\u0001\t\u0001\t\u0001\t\u0001\t"+
```

[illegible]

"\u0000jk\u0001\u0000\u0000\u0000k1\u0006\u0004\u0001\u0000\u0001\u000b\u0001"+
"\u0000\u0000\u0000mn\u0005i\u0000\u0000no\u0005f\u0000\u0000o\r\u0001"+
"\u0000\u0000\u0000pq\u0005e\u0000\u0000qr\u0005l\u0000\u0000rs\u0005i"+
"\u0000\u0000\u0000st\u0005f\u0000\u0000t\u0000f\u0001\u0000\u0000\u0000uv\u0005"+
"e\u0000\u0000\u0000vw\u0005l\u0000\u0000wx\u0005s\u0000\u0000xy\u0005e\u0000"+
"\u0000y\u0001l\u0001\u0000\u0000\u0000\u0000z{\u0005e\u0000\u0000{\|\u0005n\u0000"+
"\u0000|}\u0005d\u0000\u0000}\~\u0005i\u0000\u0000\u0000~\u0007f\u0005f\u0000\u0000"+
"\u0007f\u00013\u0001\u0000\u0000\u0000\u0000\u0080\u0081\u0005f\u0000\u0000\u0000\u0081"+
"\u0082\u0005o\u0000\u0000\u0082\u0083\u0005x\u0000\u0000\u0083\u0015\u0001"+
"\u0000\u0000\u0000\u0084\u0085\u0005i\u0000\u0000\u0085\u0086\u0005n\u0000"+
"\u0000\u0086\u0017\u0001\u0000\u0000\u0000\u0000\u0087\u0088\u0005e\u0000\u0000"+
"\u0088\u0089\u0005n\u0000\u0000\u0089\u008a\u0005d\u0000\u0000\u008a\u008b"+
"\u0005f\u0000\u0000\u008b\u008c\u0005o\u0000\u0000\u008c\u008d\u0005x"+
"\u0000\u0000\u008d\u0019\u0001\u0000\u0000\u0000\u0000\u008e\u008f\u0005a\u0000"+
"\u0000\u008f\u0090\u0005n\u0000\u0000\u0090\u0091\u0005d\u0000\u0000\u0091"+
"\u0001b\u0001\u0000\u0000\u0000\u0092\u0093\u0005o\u0000\u0000\u0093\u0094"+
"\u0005x\u0000\u0000\u0094\u001d\u0001\u0000\u0000\u0000\u0000\u0095\u0096\u0005"+
"n\u0000\u0000\u0096\u0097\u0005o\u0000\u0000\u0097\u0098\u0005t\u0000"+
"\u0000\u0098\u001f\u0001\u0000\u0000\u0000\u0099\u009a\u0005t\u0000\u0000"+
"\u009a\u009b\u0005x\u0000\u0000\u009b\u009c\u0005u\u0000\u0000\u009c\u009d"+
"\u0005e\u0000\u0000\u009d!\u0001\u0000\u0000\u0000\u009e\u009f\u0005f"+
"\u0000\u0000\u009f\u00a0\u0005a\u0000\u0000\u00a0\u00a1\u0005l\u0000\u0000"+
"\u00a1\u00a2\u0005s\u0000\u0000\u00a2\u00a3\u0005e\u0000\u0000\u00a3#"+
"\u0001\u0000\u0000\u0000\u0000\u00a4\u00a5\u0005n\u0000\u0000\u00a5\u00a6\u0005"+
"o\u0000\u0000\u0000\u00a6\u00a7\u0005n\u0000\u0000\u00a7\u00a8\u0005e\u0000"+
"\u0000\u00a8%\u0001\u0000\u0000\u0000\u0000\u00a9\u00aa\u0005=\u0000\u0000\u00aa"+
"\u00ab\u0005=\u0000\u0000\u0000\u00ab\"'\u0001\u0000\u0000\u0000\u00ac\u00ad"+
"\u0005!\u0000\u0000\u0000\u00ad\u00ae\u0005=\u0000\u0000\u00ae)\u0001\u0000"+
"\u0000\u0000\u0000\u00af\u00b0\u0005<\u0000\u0000\u00b0+\u0001\u0000\u0000\u0000"+
"\u00b1\u00b2\u0005>\u0000\u0000\u00b2-\u0001\u0000\u0000\u0000\u00b3\u00b4"+
"\u0005<\u0000\u0000\u00b4\u00b5\u0005=\u0000\u0000\u00b5/\u0001\u0000"+
"\u0000\u0000\u00b6\u00b7\u0005>\u0000\u0000\u00b7\u00b8\u0005=\u0000\u0000"+
"\u00b8\u0001\u0001\u0000\u0000\u00b9\u00ba\u0005+\u0000\u0000\u00ba3"+
"\u0001\u0000\u0000\u0000\u0000\u00bb\u00bc\u0005-\u0000\u0000\u00bc5\u0001\u0000"+
"\u0000\u0000\u00bd\u00be\u0005*\u0000\u0000\u00be7\u0001\u0000\u0000\u0000\u0000"+
"\u00bf\u00c0\u0005/\u0000\u0000\u00c09\u0001\u0000\u0000\u0000\u00c1\u00c2"+
"\u0005%\u0000\u0000\u00c2;\u0001\u0000\u0000\u0000\u00c3\u00c4\u0005.\""+
"\u0000\u0000\u00c4=\u0001\u0000\u0000\u0000\u00c5\u00c6\u0005(\u0000\u0000\u0000"+
"\u00c6?\u0001\u0000\u0000\u0000\u00c7\u00c8\u0005)\u0000\u0000\u00c8A"+
"\u0001\u0000\u0000\u0000\u0000\u00c9\u00ca\u0005[\u0000\u0000\u0000\u00ca\u0001\u0000"+
"\u0000\u0000\u0000\u00cb\u00cc\u0005]\u0000\u0000\u0000\u00ccE\u0001\u0000\u0000\u0000\u0000"+
"\u00cd\u00ce\u0005,\u0000\u0000\u0000\u00ceG\u0001\u0000\u0000\u0000\u00cf\u00d5"+
"\u0005\"'\u0000\u0000\u00d0\u00d4\b\u0002\u0000\u0000\u00d1\u00d2\u0005"+
"\\\u0000\u0000\u00d2\u00d4\t\u0000\u0000\u0000\u00d3\u00d0\u0001\u0000\u0000"+
"\u0000\u0000\u00d3\u00d1\u0001\u0000\u0000\u0000\u00d4\u00d7\u0001\u0000\u0000"+
"\u0000\u0000\u00d5\u00d3\u0001\u0000\u0000\u0000\u00d5\u00d6\u0001\u0000\u0000"+
"\u0000\u0000\u00d6\u00d8\u0001\u0000\u0000\u0000\u00d7\u00d5\u0001\u0000\u0000"+
"\u0000\u0000\u00d8\u00d9\u0005\"'\u0000\u0000\u00d9I\u0001\u0000\u0000\u0000"+
"\u0000\u00da\u00dc\u0007\u0003\u0000\u0000\u00db\u00da\u0001\u0000\u0000\u0000"+
"\u0000\u00dc\u00dd\u0001\u0000\u0000\u0000\u00dd\u00db\u0001\u0000\u0000\u0000"+
"\u0000\u00dd\u00de\u0001\u0000\u0000\u0000\u00deK\u0001\u0000\u0000\u0000\u0000"+
"\u00df\u00e1\u0007\u0003\u0000\u0000\u00e0\u00df\u0001\u0000\u0000\u0000\u0000"+
"\u00e1\u00e2\u0001\u0000\u0000\u0000\u00e2\u00e0\u0001\u0000\u0000\u0000\u0000"+
"\u00e2\u00e3\u0001\u0000\u0000\u0000\u00e3\u00e4\u0001\u0000\u0000\u0000\u0000"+
"\u00e4\u00e6\u0005.\u0000\u0000\u00e5\u00e7\u0007\u0003\u0000\u0000\u0000\u00e6"+
"\u00e5\u0001\u0000\u0000\u0000\u00e7\u00e8\u0001\u0000\u0000\u0000\u0000\u00e8"+
"\u00e6\u0001\u0000\u0000\u0000\u00e8\u00e9\u0001\u0000\u0000\u0000\u0000\u00e9"+
"M\u0001\u0000\u0000\u0000\u0000\u00ea\u00ee\u0007\u0004\u0000\u0000\u0000\u00eb\u00ed"+
"\u0007\u0005\u0000\u0000\u00ec\u00eb\u0001\u0000\u0000\u0000\u0000\u00ed\u00f0"+
"\u0001\u0000\u0000\u0000\u0000\u00ee\u00ec\u0001\u0000\u0000\u0000\u0000\u00ee\u00ef"+
"\u0001\u0000\u0000\u0000\u0000\u00efO\u0001\u0000\u0000\u0000\u00f0\u00ee\u0001"+
"\u0000\u0000\u0000\u0000\u00f1\u00f3\u0007\u0006\u0000\u0000\u00f2\u00f1\u0001"+
"\u0000\u0000\u0000\u0000\u00f3\u00f4\u0001\u0000\u0000\u0000\u0000\u00f4\u00f2\u0001"+
"\u0000\u0000\u0000\u0000\u00f4\u00f5\u0001\u0000\u0000\u0000\u0000\u00f5\u00f6\u0001"+
"\u0000\u0000\u0000\u0000\u00f6\u00f7\u0006\"'\u0002\u0000\u0000\u00f7Q\u0001\u0000\u0000"+
"\u0000\u0000\u0000\u0000\u00b\u0000\u0001_a\u00d3\u00d5\u00dd\u00e2\u00e8\u00ee\u00f4"+

```

        "\u0003\u0005\u0001\u0000\u0004\u0000\u0000\u0006\u0000\u0000";
    public static final ATN _ATN =
        new ATNDeserializer().deserialize(_serializedATN.toCharArray());
    static {
        _decisionToDFA = new DFA[_ATN.getNumberOfDecisions()];
        for (int i = 0; i < _ATN.getNumberOfDecisions(); i++) {
            _decisionToDFA[i] = new DFA(_ATN.getDecisionState(i), i);
        }
    }
}

```

}.

?? File: JinjaParser.java

```

// Generated from D:/Compiler/PyJinjaCompiler/src/grammar/jinja/JinjaParser.g4 by ANTLR 4.13.2 package compiler.generated.jinja; import org.antlr.v4.runtime.atn.;
import org.antlr.v4.runtime.dfa.DFA; import org.antlr.v4.runtime.; import org.antlr.v4.runtime.misc.; import org.antlr.v4.runtime.tree.; import java.util.List; import
java.util.Iterator; import java.util.ArrayList;

```

```

@SuppressWarnings({"all", "warnings", "unchecked", "unused", "cast", "CheckReturnValue", "this-escape"}) public class JinjaParser extends Parser { static {
    RuntimeMetaData.checkVersion("4.13.2", RuntimeMetaData.VERSION); }

```



```

protected static final DFA[] _decisionToDFA;
protected static final PredictionContextCache _sharedContextCache =
    new PredictionContextCache();
public static final int
    JINJA_EXPR_START=1, JINJA_STMT_START=2, HTML_TEXT=3, JINJA_EXPR_END=4,
    JINJA_STMT_END=5, IF=6, ELIF=7, ELSE=8, ENDIF=9, FOR=10, IN=11, ENDFOR=12,
    AND=13, OR=14, NOT=15, TRUE=16, FALSE=17, NONE=18, EQ=19, NE=20, LT=21,
    GT=22, LE=23, GE=24, PLUS=25, MINUS=26, STAR=27, SLASH=28, MOD=29, DOT=30,
    LPAREN=31, RPAREN=32, LBRACK=33, RBRACK=34, COMMA=35, STRING=36, INT=37,
    FLOAT=38, IDENTIFIER=39, WS=40;
public static final int
    RULE_template = 0, RULE_templateBody = 1, RULE_element = 2, RULE_expressionOutput = 3,
    RULE_ifStatement = 4, RULE_elifClause = 5, RULE_elseClause = 6, RULE_forStatement = 7,
    RULE_expr = 8, RULE_logicalOr = 9, RULE_logicalAnd = 10, RULE_comparison = 11,
    RULE_additive = 12, RULE_multiplicative = 13, RULE_unary = 14, RULE_primary = 15,
    RULE_literal = 16, RULE_trailer = 17, RULE_argList = 18;
private static String[] makeRuleNames() {
    return new String[] {
        "template", "templateBody", "element", "expressionOutput", "ifStatement",
        "elifClause", "elseClause", "forStatement", "expr", "logicalOr", "logicalAnd",
        "comparison", "additive", "multiplicative", "unary", "primary", "literal",
        "trailer", "argList"
    };
}
}
public static final String[] ruleNames = makeRuleNames();

private static String[] makeLiteralNames() {
    return new String[] {
        null, "'{'", "'{'", null, "'}'", "'%'", "'if'", "'elif'", "'else'",
        "'endif'", "'for'", "'in'", "'endfor'", "'and'", "'or'", "'not'", "'true'",
        "'false'", "'none'", "'=='", "'!='", "'<' ", "'>' ", "'<='", "'>='", "'+'",
        "'-'", "'*'", "'/'", "'%'", "'.'", "'('", "')'", "'['", "']'", "','",
    };
}
}
private static final String[] _LITERAL_NAMES = makeLiteralNames();
private static String[] makeSymbolicNames() {
    return new String[] {
        null, "JINJA_EXPR_START", "JINJA_STMT_START", "HTML_TEXT", "JINJA_EXPR_END",
        "JINJA_STMT_END", "IF", "ELIF", "ELSE", "ENDIF", "FOR", "IN", "ENDFOR",
        "AND", "OR", "NOT", "TRUE", "FALSE", "NONE", "EQ", "NE", "LT", "GT",
        "LE", "GE", "PLUS", "MINUS", "STAR", "SLASH", "MOD", "DOT", "LPAREN",
        "RPAREN", "LBRACK", "RBRACK", "COMMA", "STRING", "INT", "FLOAT", "IDENTIFIER",
        "WS"
    };
}
}
private static final String[] _SYMBOLIC_NAMES = makeSymbolicNames();
public static final Vocabulary VOCABULARY = new VocabularyImpl(_LITERAL_NAMES, _SYMBOLIC_NAMES);

/**
 * @deprecated Use {@link #VOCABULARY} instead.
 */
@Deprecated
public static final String[] tokenNames;
static {
    tokenNames = new String[_SYMBOLIC_NAMES.length];
    for (int i = 0; i < tokenNames.length; i++) {
        tokenNames[i] = VOCABULARY.getLiteralName(i);
        if (tokenNames[i] == null) {
            tokenNames[i] = VOCABULARY.getSymbolicName(i);
        }

        if (tokenNames[i] == null) {
            tokenNames[i] = "<INVALID>";
        }
    }
}
}

```

```

    }

    @Override
    @Deprecated
    public String[] getTokenNames() {
        return tokenNames;
    }

    @Override

    public Vocabulary getVocabulary() {
        return VOCABULARY;
    }

    @Override
    public String getGrammarFileName() { return "JinjaParser.g4"; }

    @Override
    public String[] getRuleNames() { return ruleNames; }

    @Override
    public String getSerializedATN() { return _serializedATN; }

    @Override
    public ATN getATN() { return _ATN; }

    public JinjaParser(TokenStream input) {
        super(input);
        _interp = new ParserATNSimulator(this, _ATN, _decisionToDFA, _sharedContextCache);
    }

    @SuppressWarnings("CheckReturnValue")
    public static class TemplateContext extends ParserRuleContext {
        public TemplateBodyContext body;
        public TerminalNode EOF() { return getToken(JinjaParser.EOF, 0); }
        public TemplateBodyContext templateBody() {
            return getRuleContext(TemplateBodyContext.class, 0);
        }
        public TemplateContext(ParserRuleContext parent, int invokingState) {
            super(parent, invokingState);
        }
        @Override public int getRuleIndex() { return RULE_template; }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterTemplate(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitTemplate(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitTempl
            else return visitor.visitChildren(this);
        }
    }

    public final TemplateContext template() throws RecognitionException {
        TemplateContext _localctx = new TemplateContext(_ctx, getState());
        enterRule(_localctx, 0, RULE_template);
        try {
            enterOuterAlt(_localctx, 1);
            {
                setState(38);
                ((TemplateContext)_localctx).body = templateBody();
                setState(39);
            }
        }
    }

```

```

        match(EOF);
    }

}

catch (RecognitionException re) {
    _localctx.exception = re;
    _errHandler.reportError(this, re);
    _errHandler.recover(this, re);
}

finally {
    exitRule();
}

return _localctx;
}

}

@SuppressWarnings("CheckReturnValue")
public static class TemplateBodyContext extends ParserRuleContext {
    public List<ElementContext> element() {
        return getRuleContexts(ElementContext.class);
    }

    public ElementContext element(int i) {
        return getRuleContext(ElementContext.class,i);
    }

    public TemplateBodyContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_templateBody; }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterTemplateBody(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitTemplateBody(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitTemplateBody(this);
        else return visitor.visitChildren(this);
    }
}

}

public final TemplateBodyContext templateBody() throws RecognitionException {
    TemplateBodyContext _localctx = new TemplateBodyContext(_ctx, getState());
    enterRule(_localctx, 2, RULE_templateBody);
    try {
        int _alt;
        enterOuterAlt(_localctx, 1);
        {
            setState(44);
            _errHandler.sync(this);
            _alt = getInterpreter().adaptivePredict(_input,0,_ctx);
            while ( _alt!=2 && _alt!=org.antlr.v4.runtime.atn.ATN.INVALID_ALT_NUMBER ) {
                if ( _alt==1 ) {
                    {
                        {
                            setState(41);
                            element();
                        }
                    }
                }
                setState(46);
                _errHandler.sync(this);
                _alt = getInterpreter().adaptivePredict(_input,0,_ctx);
            }
        }
    }
}
}

```

```

        catch (RecognitionException re) {
            _localctx.exception = re;
            _errHandler.reportError(this, re);
            _errHandler.recover(this, re);
        }

        finally {
            exitRule();
        }

        return _localctx;
    }

    @SuppressWarnings("CheckReturnValue")
    public static class ElementContext extends ParserRuleContext {
        public ElementContext(ParserRuleContext parent, int invokingState) {
            super(parent, invokingState);
        }

        @Override public int getRuleIndex() { return RULE_element; }

        public ElementContext() { }
        public void copyFrom(ElementContext ctx) {
            super.copyFrom(ctx);
        }
    }

    @SuppressWarnings("CheckReturnValue")
    public static class ForElementContext extends ElementContext {
        public ForStatementContext forStatement() {
            return getRuleContext(ForStatementContext.class,0);
        }

        public ForElementContext(ElementContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterForElement(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitForElement(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitForElement(this);
            else return visitor.visitChildren(this);
        }
    }

    @SuppressWarnings("CheckReturnValue")
    public static class IfElementContext extends ElementContext {
        public IfStatementContext ifStatement() {
            return getRuleContext(IfStatementContext.class,0);
        }

        public IfElementContext(ElementContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterIfElement(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitIfElement(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitIfElement(this);
            else return visitor.visitChildren(this);
        }
    }

    @SuppressWarnings("CheckReturnValue")
    public static class HtmlElementContext extends ElementContext {
        public TerminalNode HTML_TEXT() { return getToken(JinjaParser.HTML_TEXT, 0); }
    }

```

```

        public HtmlElementContext(ElementContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterHtmlElement(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitHtmlElement(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitHtmlElement(this);
            else return visitor.visitChildren(this);
        }
    }
    @SuppressWarnings("CheckReturnValue")
    public static class ExpressionElementContext extends ElementContext {
        public ExpressionOutputContext expressionOutput() {
            return getRuleContext(ExpressionOutputContext.class,0);
        }
        public ExpressionElementContext(ElementContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterExpressionElement(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitExpressionElement(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitExpressionElement(this);
            else return visitor.visitChildren(this);
        }
    }
}

public final ElementContext element() throws RecognitionException {
    ElementContext _localctx = new ElementContext(_ctx, getState());
    enterRule(_localctx, 4, RULE_element);
    try {
        setState(51);
        _errHandler.sync(this);
        switch ( getInterpreter().adaptivePredict(_input,1,_ctx) ) {
            case 1:
                _localctx = new HtmlElementContext(_localctx);
                enterOuterAlt(_localctx, 1);
                {
                    setState(47);
                    match(HTML_TEXT);
                }
                break;
            case 2:
                _localctx = new ExpressionElementContext(_localctx);
                enterOuterAlt(_localctx, 2);
                {
                    setState(48);
                    expressionOutput();
                }
                break;
            case 3:
                _localctx = new IfElementContext(_localctx);
                enterOuterAlt(_localctx, 3);
                {
                    setState(49);
                    ifStatement();
                }
        }
    }
}

```

```

        break;
    case 4:
        _localctx = new ForElementContext(_localctx);
        enterOuterAlt(_localctx, 4);
        {
            setState(50);
            forStatement();
        }
        break;
    }
}

catch (RecognitionException re) {
    _localctx.exception = re;
    _errHandler.reportError(this, re);
    _errHandler.recover(this, re);
}

finally {
    exitRule();
}

return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class ExpressionOutputContext extends ParserRuleContext {
    public ExprContext expression;
    public TerminalNode JINJA_EXPR_START() { return getToken(JinjaParser.JINJA_EXPR_START, 0); }
    public TerminalNode JINJA_EXPR_END() { return getToken(JinjaParser.JINJA_EXPR_END, 0); }
    public ExprContext expr() {
        return getRuleContext(ExprContext.class,0);
    }
    public ExpressionOutputContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }
    @Override public int getRuleIndex() { return RULE_expressionOutput; }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterExpressionOutput(this)
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitExpressionOutput(this)
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitExpressionOutput(this)
        else return visitor.visitChildren(this);
    }
}

public final ExpressionOutputContext expressionOutput() throws RecognitionException {
    ExpressionOutputContext _localctx = new ExpressionOutputContext(_ctx, getState());
    enterRule(_localctx, 6, RULE_expressionOutput);
    try {
        enterOuterAlt(_localctx, 1);
        {
            setState(53);
            match(JINJA_EXPR_START);
            setState(54);
            ((ExpressionOutputContext)_localctx).expression = expr();
            setState(55);
            match(JINJA_EXPR_END);
        }
    }
    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
    }
}

```

```

        _errHandler.recover(this, re);
    }
    finally {
        exitRule();
    }
    return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class IfStatementContext extends ParserRuleContext {
    public ExprContext condition;
    public TemplateBodyContext thenBody;
    public List<TerminalNode> JINJA_STMT_START() { return getTokens(JinjaParser.JINJA_STMT_START); }
    public TerminalNode JINJA_STMT_START(int i) {
        return getToken(JinjaParser.JINJA_STMT_START, i);
    }
    public TerminalNode IF() { return getToken(JinjaParser.IF, 0); }
    public List<TerminalNode> JINJA_STMT_END() { return getTokens(JinjaParser.JINJA_STMT_END); }
    public TerminalNode JINJA_STMT_END(int i) {
        return getToken(JinjaParser.JINJA_STMT_END, i);
    }
    public TerminalNode ENDIF() { return getToken(JinjaParser.ENDIF, 0); }
    public ExprContext expr() {
        return getRuleContext(ExprContext.class, 0);
    }
    public TemplateBodyContext templateBody() {
        return getRuleContext(TemplateBodyContext.class, 0);
    }
    public List<ElifClauseContext> elifClause() {
        return getRuleContexts(ElifClauseContext.class);
    }
    public ElifClauseContext elifClause(int i) {
        return getRuleContext(ElifClauseContext.class, i);
    }
    public ElseClauseContext elseClause() {
        return getRuleContext(ElseClauseContext.class, 0);
    }
    public IfStatementContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }
    @Override public int getRuleIndex() { return RULE_ifStatement; }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterIfStatement(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitIfStatement(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitIfSta
        else return visitor.visitChildren(this);
    }
}

public final IfStatementContext ifStatement() throws RecognitionException {
    IfStatementContext _localctx = new IfStatementContext(_ctx, getState());
    enterRule(_localctx, 8, RULE_ifStatement);
    try {
        int _alt;
        enterOuterAlt(_localctx, 1);
        {
            setState(57);
            match(JINJA_STMT_START);
            setState(58);

```

```

        match(IF);
        setState(59);
        ((IfStatementContext)_localctx).condition = expr();
        setState(60);
        match(JINJA_STMT_END);
        setState(61);
        ((IfStatementContext)_localctx).thenBody = templateBody();
        setState(65);
        _errHandler.sync(this);
        _alt = getInterpreter().adaptivePredict(_input,2,_ctx);
        while ( _alt!=2 && _alt!=org.antlr.v4.runtime.atn.ATN.INVALID_ALT_NUMBER ) {
            if ( _alt==1 ) {
                {
                    {
                        setState(62);
                        elifClause();
                    }
                }
                setState(67);
                _errHandler.sync(this);
                _alt = getInterpreter().adaptivePredict(_input,2,_ctx);
            }
            setState(69);
            _errHandler.sync(this);
            switch ( getInterpreter().adaptivePredict(_input,3,_ctx) ) {
                case 1:
                    {
                        setState(68);
                        elseClause();
                    }
                    break;
            }
            setState(71);
            match(JINJA_STMT_START);
            setState(72);
            match(ENDIF);
            setState(73);
            match(JINJA_STMT_END);
        }
    }
    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }
    finally {
        exitRule();
    }
    return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class ElifClauseContext extends ParserRuleContext {
    public ExprContext condition;
    public TemplateBodyContext body;
    public TerminalNode JINJA_STMT_START() { return getToken(JinjaParser.JINJA_STMT_START, 0); }
    public TerminalNode ELIF() { return getToken(JinjaParser.ELIF, 0); }
    public TerminalNode JINJA_STMT_END() { return getToken(JinjaParser.JINJA_STMT_END, 0); }
    public ExprContext expr() {
        return getRuleContext(ExprContext.class,0);
    }
    public TemplateBodyContext templateBody() {
        return getRuleContext(TemplateBodyContext.class,0);
    }
    public ElifClauseContext(ParserRuleContext parent, int invokingState) {

```



```

        super(parent, invokingState);
    }
    @Override public int getRuleIndex() { return RULE_elifClause; }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterElifClause(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitElifClause(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitElifC
        else return visitor.visitChildren(this);
    }
}

public final ElifClauseContext elifClause() throws RecognitionException {
    ElifClauseContext _localctx = new ElifClauseContext(_ctx, getState());
    enterRule(_localctx, 10, RULE_elifClause);
    try {
        enterOuterAlt(_localctx, 1);
        {
            setState(75);
            match(JINJA_STMT_START);
            setState(76);
            match(ELIF);
            setState(77);
            ((ElifClauseContext)_localctx).condition = expr();
            setState(78);
            match(JINJA_STMT_END);
            setState(79);
            ((ElifClauseContext)_localctx).body = templateBody();
        }
    }
    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }
    finally {
        exitRule();
    }
    return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class ElseClauseContext extends ParserRuleContext {
    public TemplateBodyContext body;
    public TerminalNode JINJA_STMT_START() { return getToken(JinjaParser.JINJA_STMT_START, 0); }
    public TerminalNode ELSE() { return getToken(JinjaParser.ELSE, 0); }
    public TerminalNode JINJA_STMT_END() { return getToken(JinjaParser.JINJA_STMT_END, 0); }
    public TemplateBodyContext templateBody() {
        return getRuleContext(TemplateBodyContext.class,0);
    }
    public ElseClauseContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }
    @Override public int getRuleIndex() { return RULE_elseClause; }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterElseClause(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {

```

```

        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitElseClause(this);
    }

    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitElseC
        else return visitor.visitChildren(this);
    }
}

public final ElseClauseContext elseClause() throws RecognitionException {
    ElseClauseContext _localctx = new ElseClauseContext(_ctx, getState());
    enterRule(_localctx, 12, RULE_elseClause);
    try {
        enterOuterAlt(_localctx, 1);
        {
            setState(81);
            match(JINJA_STMT_START);
            setState(82);
            match(ELSE);
            setState(83);
            match(JINJA_STMT_END);
            setState(84);
            ((ElseClauseContext)_localctx).body = templateBody();
        }
    }
    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }
    finally {
        exitRule();
    }
    return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class ForStatementContext extends ParserRuleContext {
    public Token variable;
    public ExprContext iterable;
    public TemplateBodyContext body;
    public List<TerminalNode> JINJA_STMT_START() { return getTokens(JinjaParser.JINJA_STMT_START); }
    public TerminalNode JINJA_STMT_START(int i) {
        return getToken(JinjaParser.JINJA_STMT_START, i);
    }
    public TerminalNode FOR() { return getToken(JinjaParser.FOR, 0); }
    public TerminalNode IN() { return getToken(JinjaParser.IN, 0); }
    public List<TerminalNode> JINJA_STMT_END() { return getTokens(JinjaParser.JINJA_STMT_END); }
    public TerminalNode JINJA_STMT_END(int i) {
        return getToken(JinjaParser.JINJA_STMT_END, i);
    }
    public TerminalNode ENDFOR() { return getToken(JinjaParser.ENDFOR, 0); }
    public TerminalNode IDENTIFIER() { return getToken(JinjaParser.IDENTIFIER, 0); }
    public ExprContext expr() {
        return getRuleContext(ExprContext.class,0);
    }
    public TemplateBodyContext templateBody() {
        return getRuleContext(TemplateBodyContext.class,0);
    }
    public ForStatementContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }
    @Override public int getRuleIndex() { return RULE_forStatement; }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterForStatement(this);
    }

```

```

    }

    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitForStatement(this);
    }

    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitForSt
        else return visitor.visitChildren(this);
    }
}

public final ForStatementContext forStatement() throws RecognitionException {
    ForStatementContext _localctx = new ForStatementContext(_ctx, getState());
    enterRule(_localctx, 14, RULE_forStatement);
    try {
        enterOuterAlt(_localctx, 1);
        {
            setState(86);
            match(JINJA_STMT_START);
            setState(87);
            match(FOR);
            setState(88);
            ((ForStatementContext)_localctx).variable = match(IDENTIFIER);
            setState(89);
            match(IN);
            setState(90);
            ((ForStatementContext)_localctx).iterable = expr();
            setState(91);
            match(JINJA_STMT_END);
            setState(92);
            ((ForStatementContext)_localctx).body = templateBody();
            setState(93);
            match(JINJA_STMT_START);
            setState(94);
            match(ENDFOR);
            setState(95);
            match(JINJA_STMT_END);
        }
    }
    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }
    finally {
        exitRule();
    }
    return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class ExprContext extends ParserRuleContext {
    public LogicalOrContext logicalOr() {
        return getRuleContext(LogicalOrContext.class,0);
    }

    public ExprContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_expr; }

    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterExpr(this);
    }

    @Override
    public void exitRule(ParseTreeListener listener) {

```

```

        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitExpr(this);
    }

    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitExpr(this);
        else return visitor.visitChildren(this);
    }
}

public final ExprContext expr() throws RecognitionException {
    ExprContext _localctx = new ExprContext(_ctx, getState());
    enterRule(_localctx, 16, RULE_expr);
    try {
        enterOuterAlt(_localctx, 1);
        {
            setState(97);
            logicalOr(0);
        }
    }
    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }
    finally {
        exitRule();
    }
    return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class LogicalOrContext extends ParserRuleContext {
    public LogicalOrContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_logicalOr; }

    public LogicalOrContext() { }
    public void copyFrom(LogicalOrContext ctx) {
        super.copyFrom(ctx);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class LogicalAndExprContext extends LogicalOrContext {
    public LogicalAndContext logicalAnd() {
        return getRuleContext(LogicalAndContext.class,0);
    }

    public LogicalAndExprContext(LogicalOrContext ctx) { copyFrom(ctx); }

    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterLogicalAndExpr(this);
    }

    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitLogicalAndExpr(this);
    }

    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitLogicalAndExpr(this);
        else return visitor.visitChildren(this);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class OrExprContext extends LogicalOrContext {
    public LogicalOrContext left;
    public Token op;
}

```

```

    public LogicalAndContext right;

    public LogicalOrContext logicalOr() {
        return getRuleContext(LogicalOrContext.class,0);
    }

    public TerminalNode OR() { return getToken(JinjaParser.OR, 0); }

    public LogicalAndContext logicalAnd() {
        return getRuleContext(LogicalAndContext.class,0);
    }

    public OrExprContext(LogicalOrContext ctx) { copyFrom(ctx); }

    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterOrExpr(this);
    }

    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitOrExpr(this);
    }

    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitOrExpr(this);
        else return visitor.visitChildren(this);
    }
}

public final LogicalOrContext logicalOr() throws RecognitionException {
    return logicalOr(0);
}

private LogicalOrContext logicalOr(int _p) throws RecognitionException {
    ParserRuleContext _parentctx = _ctx;
    int _parentState = getState();
    LogicalOrContext _localctx = new LogicalOrContext(_ctx, _parentState);
    LogicalOrContext _prevctx = _localctx;
    int _startState = 18;
    enterRecursionRule(_localctx, 18, RULE_logicalOr, _p);
    try {
        int _alt;
        enterOuterAlt(_localctx, 1);
        {
            {
                _localctx = new LogicalAndExprContext(_localctx);
                _ctx = _localctx;
                _prevctx = _localctx;

                setState(100);
                logicalAnd(0);
            }
            _ctx.stop = _input.LT(-1);
            setState(107);
            _errHandler.sync(this);
            _alt = getInterpreter().adaptivePredict(_input,4,_ctx);
            while ( _alt!=2 && _alt!=org.antlr.v4.runtime.atn.ATN.INVALID_ALT_NUMBER ) {
                if ( _alt==1 ) {
                    if ( _parseListeners!=null ) triggerExitRuleEvent();
                    _prevctx = _localctx;
                    {
                        {
                            _localctx = new OrExprContext(new LogicalOrContext(_parentctx, _parentState));
                            ((OrExprContext)_localctx).left = _prevctx;
                            pushNewRecursionContext(_localctx, _startState, RULE_logicalOr);
                            setState(102);
                            if (!(precpred(_ctx, 2))) throw new FailedPredicateException(this, "precpred(_ctx, 2)");
                            setState(103);
                            ((OrExprContext)_localctx).op = match(OR);
                            setState(104);
                            ((OrExprContext)_localctx).right = logicalAnd(0);

```

```

        }
        }

        }

        setState(109);
        _errHandler.sync(this);
        _alt = getInterpreter().adaptivePredict(_input,4,_ctx);
    }
}

}

catch (RecognitionException re) {
    _localctx.exception = re;
    _errHandler.reportError(this, re);
    _errHandler.recover(this, re);
}

finally {
    unrollRecursionContexts(_parentctx);
}

return _localctx;
}

}

@SuppressWarnings("CheckReturnValue")
public static class LogicalAndContext extends ParserRuleContext {
    public LogicalAndContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_logicalAnd; }

    public LogicalAndContext() { }
    public void copyFrom(LogicalAndContext ctx) {
        super.copyFrom(ctx);
    }
}

}

@SuppressWarnings("CheckReturnValue")
public static class ComparisonExprContext extends LogicalAndContext {
    public ComparisonContext comparison() {
        return getRuleContext(ComparisonContext.class,0);
    }

    public ComparisonExprContext(LogicalAndContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterComparisonExpr(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitComparisonExpr(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitComparisonExpr(this);
        else return visitor.visitChildren(this);
    }
}

}

@SuppressWarnings("CheckReturnValue")
public static class AndExprContext extends LogicalAndContext {
    public LogicalAndContext left;
    public Token op;
    public ComparisonContext right;
    public LogicalAndContext logicalAnd() {
        return getRuleContext(LogicalAndContext.class,0);
    }

    public TerminalNode AND() { return getToken(JinjaParser.AND, 0); }
    public ComparisonContext comparison() {
        return getRuleContext(ComparisonContext.class,0);
    }
}

public AndExprContext(LogicalAndContext ctx) { copyFrom(ctx); }
@Override

```

```

        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterAndExpr(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitAndExpr(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitAndExpr(this);
            else return visitor.visitChildren(this);
        }
    }

    public final LogicalAndContext logicalAnd() throws RecognitionException {
        return logicalAnd(0);
    }

    private LogicalAndContext logicalAnd(int _p) throws RecognitionException {
        ParserRuleContext _parentctx = _ctx;
        int _parentState = getState();
        LogicalAndContext _localctx = new LogicalAndContext(_ctx, _parentState);
        LogicalAndContext _prevctx = _localctx;
        int _startState = 20;
        enterRecursionRule(_localctx, 20, RULE_logicalAnd, _p);
        try {
            int _alt;
            enterOuterAlt(_localctx, 1);
            {
                {
                    _localctx = new ComparisonExprContext(_localctx);
                    _ctx = _localctx;
                    _prevctx = _localctx;

                    setState(111);
                    comparison(0);
                }
                _ctx.stop = _input.LT(-1);
                setState(118);
                _errHandler.sync(this);
                _alt = getInterpreter().adaptivePredict(_input,5,_ctx);
                while ( _alt!=2 && _alt!=org.antlr.v4.runtime.atn.ATN.INVALID_ALT_NUMBER ) {
                    if ( _alt==1 ) {
                        if ( _parseListeners!=null ) triggerExitRuleEvent();
                        _prevctx = _localctx;
                        {
                            {
                                _localctx = new AndExprContext(new LogicalAndContext(_parentctx, _parentState));
                                ((AndExprContext)_localctx).left = _prevctx;
                                pushNewRecursionContext(_localctx, _startState, RULE_logicalAnd);
                                setState(113);
                                if (!(precpred(_ctx, 2))) throw new FailedPredicateException(this, "precpred(_ctx, 2)");
                                setState(114);
                                ((AndExprContext)_localctx).op = match(AND);
                                setState(115);
                                ((AndExprContext)_localctx).right = comparison(0);
                            }
                        }
                    }
                    setState(120);
                    _errHandler.sync(this);
                    _alt = getInterpreter().adaptivePredict(_input,5,_ctx);
                }
            }
        }
        catch (RecognitionException re) {

```

```

        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }

    finally {
        unrollRecursionContexts(_parentctx);
    }

    return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class ComparisonContext extends ParserRuleContext {
    public ComparisonContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_comparison; }

    public ComparisonContext() { }
    public void copyFrom(ComparisonContext ctx) {
        super.copyFrom(ctx);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class ComparisonOpExprContext extends ComparisonContext {
    public ComparisonContext left;
    public Token op;
    public AdditiveContext right;
    public ComparisonContext comparison() {
        return getRuleContext(ComparisonContext.class,0);
    }

    public AdditiveContext additive() {
        return getRuleContext(AdditiveContext.class,0);
    }

    public TerminalNode EQ() { return getToken(JinjaParser.EQ, 0); }
    public TerminalNode NE() { return getToken(JinjaParser.NE, 0); }
    public TerminalNode LT() { return getToken(JinjaParser.LT, 0); }
    public TerminalNode GT() { return getToken(JinjaParser.GT, 0); }
    public TerminalNode LE() { return getToken(JinjaParser.LE, 0); }
    public TerminalNode GE() { return getToken(JinjaParser.GE, 0); }
    public ComparisonOpExprContext(ComparisonContext ctx) { copyFrom(ctx); }

    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterComparisonOpExpr(this)
    }

    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitComparisonOpExpr(this)
    }

    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitComparisonOpExpr(this)
        else return visitor.visitChildren(this);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class AdditiveExprContext extends ComparisonContext {
    public AdditiveContext additive() {
        return getRuleContext(AdditiveContext.class,0);
    }

    public AdditiveExprContext(ComparisonContext ctx) { copyFrom(ctx); }

    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterAdditiveExpr(this);
    }

    @Override
    public void exitRule(ParseTreeListener listener) {

```



```

        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitAdditiveExpr(this);
    }

    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitAdditiveExpr(this);
        else return visitor.visitChildren(this);
    }
}

public final ComparisonContext comparison() throws RecognitionException {
    return comparison(0);
}

private ComparisonContext comparison(int _p) throws RecognitionException {
    ParserRuleContext _parentctx = _ctx;
    int _parentState = getState();
    ComparisonContext _localctx = new ComparisonContext(_ctx, _parentState);
    ComparisonContext _prevctx = _localctx;
    int _startState = 22;
    enterRecursionRule(_localctx, 22, RULE_comparison, _p);
    int _la;
    try {
        int _alt;
        enterOuterAlt(_localctx, 1);
        {
            {
                _localctx = new AdditiveExprContext(_localctx);
                _ctx = _localctx;
                _prevctx = _localctx;

                setState(122);
                additive(0);
            }
            _ctx.stop = _input.LT(-1);
            setState(129);
            _errHandler.sync(this);
            _alt = getInterpreter().adaptivePredict(_input,6,_ctx);
            while ( _alt!=2 && _alt!=org.antlr.v4.runtime.atn.ATN.INVALID_ALT_NUMBER ) {
                if ( _alt==1 ) {
                    if ( _parseListeners!=null ) triggerExitRuleEvent();
                    _prevctx = _localctx;
                    {
                        {
                            _localctx = new ComparisonOpExprContext(new ComparisonContext(_parentctx, _parentState));
                            ((ComparisonOpExprContext)_localctx).left = _prevctx;
                            pushNewRecursionContext(_localctx, _startState, RULE_comparison);
                            setState(124);
                            if (!(precpred(_ctx, 2))) throw new FailedPredicateException(this, "precpred(_ctx, 2)");
                            setState(125);
                            ((ComparisonOpExprContext)_localctx).op = _input.LT(1);
                            _la = _input.LA(1);
                            if ( !((((_la) & ~0x3f) == 0 && ((1L << _la) & 33030144L) != 0)) ) {
                                ((ComparisonOpExprContext)_localctx).op = (Token)_errHandler.recoverInline(this);
                            }
                        }
                        else {
                            if ( _input.LA(1)==Token.EOF ) matchedEOF = true;
                            _errHandler.reportMatch(this);
                            consume();
                        }
                        setState(126);
                        ((ComparisonOpExprContext)_localctx).right = additive(0);
                    }
                }
                _alt = getInterpreter().adaptivePredict(_input,7,_ctx);
            }
            setState(131);
            _errHandler.sync(this);

```

```

        _alt = getInterpreter().adaptivePredict(_input,6,_ctx);
    }
}

}

catch (RecognitionException re) {
    _localctx.exception = re;
    _errHandler.reportError(this, re);
    _errHandler.recover(this, re);
}

finally {
    unrollRecursionContexts(_parentctx);
}

return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class AdditiveContext extends ParserRuleContext {
    public AdditiveContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_additive; }

    public AdditiveContext() { }
    public void copyFrom(AdditiveContext ctx) {
        super.copyFrom(ctx);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class AddExprContext extends AdditiveContext {
    public AdditiveContext left;
    public Token op;
    public MultiplicativeContext right;
    public AdditiveContext additive() {
        return getRuleContext(AdditiveContext.class,0);
    }
    public MultiplicativeContext multiplicative() {
        return getRuleContext(MultiplicativeContext.class,0);
    }
    public TerminalNode PLUS() { return getToken(JinjaParser.PLUS, 0); }
    public TerminalNode MINUS() { return getToken(JinjaParser.MINUS, 0); }
    public AddExprContext(AdditiveContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterAddExpr(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitAddExpr(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitAddExpr(this);
        else return visitor.visitChildren(this);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class MultiplicativeExprContext extends AdditiveContext {
    public MultiplicativeContext multiplicative() {
        return getRuleContext(MultiplicativeContext.class,0);
    }
    public MultiplicativeExprContext(AdditiveContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterMultiplicativeExpr(th
    }
    @Override

```

```

    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitMultiplicativeExpr(this);
    }

    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitMultiplicativeExpr(this);
        else return visitor.visitChildren(this);
    }
}

public final AdditiveContext additive() throws RecognitionException {
    return additive(0);
}

private AdditiveContext additive(int _p) throws RecognitionException {
    ParserRuleContext _parentctx = _ctx;
    int _parentState = getState();
    AdditiveContext _localctx = new AdditiveContext(_ctx, _parentState);
    AdditiveContext _prevctx = _localctx;
    int _startState = 24;
    enterRecursionRule(_localctx, 24, RULE_additive, _p);
    int _la;
    try {
        int _alt;
        enterOuterAlt(_localctx, 1);
        {
            {
                _localctx = new MultiplicativeExprContext(_localctx);
                _ctx = _localctx;
                _prevctx = _localctx;

                setState(133);
                multiplicative(0);
            }
            _ctx.stop = _input.LT(-1);
            setState(140);
            _errHandler.sync(this);
            _alt = getInterpreter().adaptivePredict(_input,7,_ctx);
            while ( _alt!=2 && _alt!=org.antlr.v4.runtime.atn.ATN.INVALID_ALT_NUMBER ) {
                if ( _alt==1 ) {
                    if ( _parseListeners!=null ) triggerExitRuleEvent();
                    _prevctx = _localctx;
                    {
                        {
                            _localctx = new AddExprContext(new AdditiveContext(_parentctx, _parentState));
                            ((AddExprContext)_localctx).left = _prevctx;
                            pushNewRecursionContext(_localctx, _startState, RULE_additive);
                            setState(135);
                            if (!(precpred(_ctx, 2))) throw new FailedPredicateException(this, "precpred(_ctx, 2)");
                            setState(136);
                            ((AddExprContext)_localctx).op = _input.LT(1);
                            _la = _input.LA(1);
                            if ( !(_la==PLUS || _la==MINUS) ) {
                                ((AddExprContext)_localctx).op = (Token)_errHandler.recoverInline(this);
                            }
                        }
                        else {
                            if ( _input.LA(1)==Token.EOF ) matchedEOF = true;
                            _errHandler.reportMatch(this);
                            consume();
                        }
                    }
                    setState(137);
                    ((AddExprContext)_localctx).right = multiplicative(0);
                }
            }
        }
        setState(142);
    }
}

```

```

        _errHandler.sync(this);
        _alt = getInterpreter().adaptivePredict(_input,7,_ctx);
    }
}

}

catch (RecognitionException re) {
    _localctx.exception = re;
    _errHandler.reportError(this, re);
    _errHandler.recover(this, re);
}

finally {
    unrollRecursionContexts(_parentctx);
}

return _localctx;
}

}

@SuppressWarnings("CheckReturnValue")
public static class MultiplicativeContext extends ParserRuleContext {
    public MultiplicativeContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_multiplicative; }

    public MultiplicativeContext() { }
    public void copyFrom(MultiplicativeContext ctx) {
        super.copyFrom(ctx);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class UnaryExprContext extends MultiplicativeContext {
    public UnaryContext unary() {
        return getRuleContext(UnaryContext.class,0);
    }

    public UnaryExprContext(MultiplicativeContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterUnaryExpr(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitUnaryExpr(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitUnaryExpr(this);
        else return visitor.visitChildren(this);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class MulExprContext extends MultiplicativeContext {
    public MultiplicativeContext left;
    public Token op;
    public UnaryContext right;
    public MultiplicativeContext multiplicative() {
        return getRuleContext(MultiplicativeContext.class,0);
    }
    public UnaryContext unary() {
        return getRuleContext(UnaryContext.class,0);
    }
    public TerminalNode STAR() { return getToken(JinjaParser.STAR, 0); }
    public TerminalNode SLASH() { return getToken(JinjaParser.SLASH, 0); }
    public TerminalNode MOD() { return getToken(JinjaParser.MOD, 0); }
    public MulExprContext(MultiplicativeContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterMulExpr(this);
    }
}

```

```

    }

    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitMulExpr(this);
    }

    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitMulExpr(this);
        else return visitor.visitChildren(this);
    }
}

public final MultiplicativeContext multiplicative() throws RecognitionException {
    return multiplicative(0);
}

private MultiplicativeContext multiplicative(int _p) throws RecognitionException {
    ParserRuleContext _parentctx = _ctx;
    int _parentState = getState();
    MultiplicativeContext _localctx = new MultiplicativeContext(_ctx, _parentState);
    MultiplicativeContext _prevctx = _localctx;
    int _startState = 26;
    enterRecursionRule(_localctx, 26, RULE_multiplicative, _p);
    int _la;
    try {
        int _alt;
        enterOuterAlt(_localctx, 1);
        {
            {
                _localctx = new UnaryExprContext(_localctx);
                _ctx = _localctx;
                _prevctx = _localctx;

                setState(144);
                unary();
            }
            _ctx.stop = _input.LT(-1);
            setState(151);
            _errHandler.sync(this);
            _alt = getInterpreter().adaptivePredict(_input,8,_ctx);
            while ( _alt!=2 && _alt!=org.antlr.v4.runtime.atn.ATN.INVALID_ALT_NUMBER ) {
                if ( _alt==1 ) {
                    if ( _parseListeners!=null ) triggerExitRuleEvent();
                    _prevctx = _localctx;
                    {
                        {
                            _localctx = new MulExprContext(new MultiplicativeContext(_parentctx, _parentState));
                            ((MulExprContext)_localctx).left = _prevctx;
                            pushNewRecursionContext(_localctx, _startState, RULE_multiplicative);
                            setState(146);
                            if (!(precpred(_ctx, 2))) throw new FailedPredicateException(this, "precpred(_ctx, 2)");
                            setState(147);
                            ((MulExprContext)_localctx).op = _input.LT(1);
                            _la = _input.LA(1);
                            if ( !((((_la) & ~0x3f) == 0 && ((1L << _la) & 939524096L) != 0)) ) {
                                ((MulExprContext)_localctx).op = (Token)_errHandler.recoverInline(this);
                            }
                        }
                        else {
                            if ( _input.LA(1)==Token.EOF ) matchedEOF = true;
                            _errHandler.reportMatch(this);
                            consume();
                        }
                    }
                    setState(148);
                    ((MulExprContext)_localctx).right = unary();
                }
            }
        }
    }
}

```

```

        }
        setState(153);
        _errHandler.sync(this);
        _alt = getInterpreter().adaptivePredict(_input,8,_ctx);
    }
}

}
catch (RecognitionException re) {
    _localctx.exception = re;
    _errHandler.reportError(this, re);
    _errHandler.recover(this, re);
}
finally {
    unrollRecursionContexts(_parentctx);
}
return _localctx;
}

}

@SuppressWarnings("CheckReturnValue")
public static class UnaryContext extends ParserRuleContext {
    public UnaryContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_unary; }

    public UnaryContext() { }
    public void copyFrom(UnaryContext ctx) {
        super.copyFrom(ctx);
    }
}

}

@SuppressWarnings("CheckReturnValue")
public static class NotExprContext extends UnaryContext {
    public Token op;
    public UnaryContext operand;
    public TerminalNode NOT() { return getToken(JinjaParser.NOT, 0); }
    public UnaryContext unary() {
        return getRuleContext(UnaryContext.class,0);
    }
    public NotExprContext(UnaryContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterNotExpr(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitNotExpr(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitNotExpr(this);
        else return visitor.visitChildren(this);
    }
}

}

@SuppressWarnings("CheckReturnValue")
public static class PrimaryExprContext extends UnaryContext {
    public PrimaryContext primary() {
        return getRuleContext(PrimaryContext.class,0);
    }
    public PrimaryExprContext(UnaryContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterPrimaryExpr(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitPrimaryExpr(this);
    }
}

```

```

    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitPrimaryExpression(this);
        else return visitor.visitChildren(this);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class UnaryMinusContext extends UnaryContext {
    public Token op;
    public UnaryContext operand;
    public TerminalNode MINUS() { return getToken(JinjaParser.MINUS, 0); }
    public UnaryContext unary() {
        return getRuleContext(UnaryContext.class,0);
    }
    public UnaryMinusContext(UnaryContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterUnaryMinus(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitUnaryMinus(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitUnaryMinus(this);
        else return visitor.visitChildren(this);
    }
}

public final UnaryContext unary() throws RecognitionException {
    UnaryContext _localctx = new UnaryContext(_ctx, getState());
    enterRule(_localctx, 28, RULE_unary);
    try {
        setState(159);
        _errHandler.sync(this);
        switch (_input.LA(1)) {
            case NOT:
                _localctx = new NotExprContext(_localctx);
                enterOuterAlt(_localctx, 1);
                {
                    setState(154);
                    ((NotExprContext)_localctx).op = match(NOT);
                    setState(155);
                    ((NotExprContext)_localctx).operand = unary();
                }
                break;
            case MINUS:
                _localctx = new UnaryMinusContext(_localctx);
                enterOuterAlt(_localctx, 2);
                {
                    setState(156);
                    ((UnaryMinusContext)_localctx).op = match(MINUS);
                    setState(157);
                    ((UnaryMinusContext)_localctx).operand = unary();
                }
                break;
            case TRUE:
            case FALSE:
            case NONE:
            case LPAREN:
            case STRING:
            case INT:
            case FLOAT:
            case IDENTIFIER:

```

```

        _localctx = new PrimaryExprContext(_localctx);
        enterOuterAlt(_localctx, 3);
        {
            setState(158);
            primary();
        }
        break;
    default:
        throw new NoViableAltException(this);
    }
}

catch (RecognitionException re) {
    _localctx.exception = re;
    _errHandler.reportError(this, re);
    _errHandler.recover(this, re);
}

finally {
    exitRule();
}

return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class PrimaryContext extends ParserRuleContext {
    public PrimaryContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_primary; }

    public PrimaryContext() { }
    public void copyFrom(PrimaryContext ctx) {
        super.copyFrom(ctx);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class LiteralExprContext extends PrimaryContext {
    public LiteralContext literal() {
        return getRuleContext(LiteralContext.class,0);
    }

    public LiteralExprContext(PrimaryContext ctx) { copyFrom(ctx); }

    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterLiteralExpr(this);
    }

    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitLiteralExpr(this);
    }

    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitLiteralExpr(this);
        else return visitor.visitChildren(this);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class ParenExprContext extends PrimaryContext {
    public ExprContext expression;

    public TerminalNode LPAREN() { return getToken(JinjaParser.LPAREN, 0); }
    public TerminalNode RPAREN() { return getToken(JinjaParser.RPAREN, 0); }
    public ExprContext expr() {
        return getRuleContext(ExprContext.class,0);
    }

    public ParenExprContext(PrimaryContext ctx) { copyFrom(ctx); }

    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterParenExpr(this);

```



```

    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitParenExpr(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitParenExpr(this);
        else return visitor.visitChildren(this);
    }
}
@SuppressWarnings("CheckReturnValue")
public static class IdentifierExprContext extends PrimaryContext {
    public Token identifier;
    public TrailerContext trailer;
    public List<TrailerContext> trailers = new ArrayList<TrailerContext>();
    public TerminalNode IDENTIFIER() { return getToken(JinjaParser.IDENTIFIER, 0); }
    public List<TrailerContext> trailer() {
        return getRuleContexts(TrailerContext.class);
    }
    public TrailerContext trailer(int i) {
        return getRuleContext(TrailerContext.class,i);
    }
    public IdentifierExprContext(PrimaryContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterIdentifierExpr(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitIdentifierExpr(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitIdentifierExpr(this);
        else return visitor.visitChildren(this);
    }
}

public final PrimaryContext primary() throws RecognitionException {
    PrimaryContext _localctx = new PrimaryContext(_ctx, getState());
    enterRule(_localctx, 30, RULE_primary);
    try {
        int _alt;
        setState(173);
        _errHandler.sync(this);
        switch (_input.LA(1)) {
            case LPAREN:
                _localctx = new ParenExprContext(_localctx);
                enterOuterAlt(_localctx, 1);
                {
                    setState(161);
                    match(LPAREN);
                    setState(162);
                    ((ParenExprContext)_localctx).expression = expr();
                    setState(163);
                    match(RPAREN);
                }
                break;
            case TRUE:
            case FALSE:
            case NONE:
            case STRING:
            case INT:
            case FLOAT:
                _localctx = new LiteralExprContext(_localctx);

```

```

        enterOuterAlt(_localctx, 2);
        {
            setState(165);
            literal();
        }
        break;
    case IDENTIFIER:
        _localctx = new IdentifierExprContext(_localctx);
        enterOuterAlt(_localctx, 3);
        {
            setState(166);
            ((IdentifierExprContext)_localctx).identifier = match(IDENTIFIER);
            setState(170);
            _errHandler.sync(this);
            _alt = getInterpreter().adaptivePredict(_input,10,_ctx);
            while ( _alt!=2 && _alt!=org.antlr.v4.runtime.atn.ATN.INVALID_ALT_NUMBER ) {
                if ( _alt==1 ) {
                    {
                        {
                            setState(167);
                            ((IdentifierExprContext)_localctx).trailer = trailer();
                            ((IdentifierExprContext)_localctx).trailers.add(((IdentifierExprContext)_localctx)
                                .trailer());
                        }
                    }
                }
                setState(172);
                _errHandler.sync(this);
                _alt = getInterpreter().adaptivePredict(_input,10,_ctx);
            }
        }
        break;
    default:
        throw new NoViableAltException(this);
    }
}

}

catch (RecognitionException re) {
    _localctx.exception = re;
    _errHandler.reportError(this, re);
    _errHandler.recover(this, re);
}

finally {
    exitRule();
}

return _localctx;
}

}

@SuppressWarnings("CheckReturnValue")
public static class LiteralContext extends ParserRuleContext {
    public LiteralContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_literal; }

    public LiteralContext() { }
    public void copyFrom(LiteralContext ctx) {
        super.copyFrom(ctx);
    }
}

}

@SuppressWarnings("CheckReturnValue")
public static class FalseLiteralContext extends LiteralContext {
    public TerminalNode FALSE() { return getToken(JinjaParser.FALSE, 0); }
    public FalseLiteralContext(LiteralContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterFalseLiteral(this);
    }
}

```

```

        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitFalseLiteral(this);
        }

        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitFalseLiteral(this);
            else return visitor.visitChildren(this);
        }
    }

    @SuppressWarnings("CheckReturnValue")
    public static class NoneLiteralContext extends LiteralContext {
        public TerminalNode NONE() { return getToken(JinjaParser.NONE, 0); }
        public NoneLiteralContext(LiteralContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterNoneLiteral(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitNoneLiteral(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitNoneLiteral(this);
            else return visitor.visitChildren(this);
        }
    }

    @SuppressWarnings("CheckReturnValue")
    public static class StringLiteralContext extends LiteralContext {
        public TerminalNode STRING() { return getToken(JinjaParser.STRING, 0); }
        public StringLiteralContext(LiteralContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterStringLiteral(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitStringLiteral(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitStringLiteral(this);
            else return visitor.visitChildren(this);
        }
    }

    @SuppressWarnings("CheckReturnValue")
    public static class IntLiteralContext extends LiteralContext {
        public TerminalNode INT() { return getToken(JinjaParser.INT, 0); }
        public IntLiteralContext(LiteralContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterIntLiteral(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitIntLiteral(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitIntLiteral(this);
            else return visitor.visitChildren(this);
        }
    }

    @SuppressWarnings("CheckReturnValue")
    public static class FloatLiteralContext extends LiteralContext {

```

```

public TerminalNode FLOAT() { return getToken(JinjaParser.FLOAT, 0); }

public FloatLiteralContext(LiteralContext ctx) { copyFrom(ctx); }

@Override
public void enterRule(ParseTreeListener listener) {
    if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterFloatLiteral(this);
}

@Override
public void exitRule(ParseTreeListener listener) {
    if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitFloatLiteral(this);
}

@Override
public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
    if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitFloatLiteral(this);
    else return visitor.visitChildren(this);
}
}

@SuppressWarnings("CheckReturnValue")
public static class TrueLiteralContext extends LiteralContext {
    public TerminalNode TRUE() { return getToken(JinjaParser.TRUE, 0); }
    public TrueLiteralContext(LiteralContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterTrueLiteral(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitTrueLiteral(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitTrueLiteral(this);
        else return visitor.visitChildren(this);
    }
}

}

public final LiteralContext literal() throws RecognitionException {
    LiteralContext _localctx = new LiteralContext(_ctx, getState());
    enterRule(_localctx, 32, RULE_literal);
    try {
        setState(181);
        _errHandler.sync(this);
        switch (_input.LA(1)) {
            case STRING:
                _localctx = new StringLiteralContext(_localctx);
                enterOuterAlt(_localctx, 1);
                {
                    setState(175);
                    match(STRING);
                }
                break;
            case INT:
                _localctx = new IntLiteralContext(_localctx);
                enterOuterAlt(_localctx, 2);
                {
                    setState(176);
                    match(INT);
                }
                break;
            case FLOAT:
                _localctx = new FloatLiteralContext(_localctx);
                enterOuterAlt(_localctx, 3);
                {
                    setState(177);
                    match(FLOAT);
                }
                break;
        }
    }
}

```

```

        case TRUE:
            _localctx = new TrueLiteralContext(_localctx);
            enterOuterAlt(_localctx, 4);
            {
                setState(178);
                match(TRUE);
            }
            break;
        case FALSE:
            _localctx = new FalseLiteralContext(_localctx);
            enterOuterAlt(_localctx, 5);
            {
                setState(179);
                match(FALSE);
            }
            break;
        case NONE:
            _localctx = new NoneLiteralContext(_localctx);
            enterOuterAlt(_localctx, 6);
            {
                setState(180);
                match(NONE);
            }
            break;
        default:
            throw new NoViableAltException(this);
    }

    }

    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }

    finally {
        exitRule();
    }

    return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class TrailerContext extends ParserRuleContext {
    public TrailerContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_trailer; }

    public TrailerContext() { }
    public void copyFrom(TrailerContext ctx) {
        super.copyFrom(ctx);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class IndexAccessContext extends TrailerContext {
    public ExprContext index;
    public TerminalNode LBRACK() { return getToken(JinjaParser.LBRACK, 0); }
    public TerminalNode RBRACK() { return getToken(JinjaParser.RBRACK, 0); }
    public ExprContext expr() {
        return getRuleContext(ExprContext.class,0);
    }

    public IndexAccessContext(TrailerContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterIndexAccess(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {

```

```

        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitIndexAccess(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitIndexAccess(this);
        else return visitor.visitChildren(this);
    }
}
@SuppressWarnings("CheckReturnValue")
public static class AttributeAccessContext extends TrailerContext {
    public Token attribute;
    public TerminalNode DOT() { return getToken(JinjaParser.DOT, 0); }
    public TerminalNode IDENTIFIER() { return getToken(JinjaParser.IDENTIFIER, 0); }
    public AttributeAccessContext(TrailerContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterAttributeAccess(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitAttributeAccess(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitAttributeAccess(this);
        else return visitor.visitChildren(this);
    }
}
@SuppressWarnings("CheckReturnValue")
public static class CallTrailerContext extends TrailerContext {
    public ArgListContext arguments;
    public TerminalNode LPAREN() { return getToken(JinjaParser.LPAREN, 0); }
    public TerminalNode RPAREN() { return getToken(JinjaParser.RPAREN, 0); }
    public ArgListContext argList() {
        return getRuleContext(ArgListContext.class,0);
    }
    public CallTrailerContext(TrailerContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterCallTrailer(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitCallTrailer(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitCallTrailer(this);
        else return visitor.visitChildren(this);
    }
}

public final TrailerContext trailer() throws RecognitionException {
    TrailerContext _localctx = new TrailerContext(_ctx, getState());
    enterRule(_localctx, 34, RULE_trailer);
    int _la;
    try {
        setState(194);
        _errHandler.sync(this);
        switch (_input.LA(1)) {
            case DOT:
                _localctx = new AttributeAccessContext(_localctx);
                enterOuterAlt(_localctx, 1);
                {
                    setState(183);
                    match(DOT);
                }
            default:
                throw new RecognitionException(_localctx, _errHandler, _input);
        }
    } catch (RecognitionException re) {
        reportError(re);
        recover(re);
    }
}

```

```

        setState(184);
        ((AttributeAccessContext)_localctx).attribute = match(IDENTIFIER);
    }
    break;
case LBRACK:
    _localctx = new IndexAccessContext(_localctx);
    enterOuterAlt(_localctx, 2);
    {
        setState(185);
        match(LBRACK);
        setState(186);
        ((IndexAccessContext)_localctx).index = expr();
        setState(187);
        match(RBRACK);
    }
    break;
case LPAREN:
    _localctx = new CallTrailerContext(_localctx);
    enterOuterAlt(_localctx, 3);
    {
        setState(189);
        match(LPAREN);
        setState(191);
        _errHandler.sync(this);
        _la = _input.LA(1);
        if ((((_la) & ~0x3f) == 0 && ((1L << _la) & 1033007235072L) != 0)) {
            {
                setState(190);
                ((CallTrailerContext)_localctx).arguments = argList();
            }
        }

        setState(193);
        match(RPAREN);
    }
    break;
default:
    throw new NoViableAltException(this);
}

}

catch (RecognitionException re) {
    _localctx.exception = re;
    _errHandler.reportError(this, re);
    _errHandler.recover(this, re);
}

finally {
    exitRule();
}

return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class ArgListContext extends ParserRuleContext {
    public List<ExprContext> expr() {
        return getRuleContexts(ExprContext.class);
    }

    public ExprContext expr(int i) {
        return getRuleContext(ExprContext.class,i);
    }

    public List<TerminalNode> COMMA() { return getTokens(JinjaParser.COMMA); }
    public TerminalNode COMMA(int i) {
        return getToken(JinjaParser.COMMA, i);
    }

    public ArgListContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }
}

```

```

@Override public int getRuleIndex() { return RULE_argList; }

@Override
public void enterRule(ParseTreeListener listener) {
    if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).enterArgList(this);
}

@Override
public void exitRule(ParseTreeListener listener) {
    if ( listener instanceof JinjaParserListener ) ((JinjaParserListener)listener).exitArgList(this);
}

@Override
public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
    if ( visitor instanceof JinjaParserVisitor ) return ((JinjaParserVisitor<? extends T>)visitor).visitArgList(this);
    else return visitor.visitChildren(this);
}
}

public final ArgListContext argList() throws RecognitionException {
    ArgListContext _localctx = new ArgListContext(_ctx, getState());
    enterRule(_localctx, 36, RULE_argList);
    int _la;
    try {
        int _alt;
        enterOuterAlt(_localctx, 1);
        {
            setState(196);
            expr();
            setState(201);
            _errHandler.sync(this);
            _alt = getInterpreter().adaptivePredict(_input,15,_ctx);
            while ( _alt!=2 && _alt!=org.antlr.v4.runtime.atn.ATN.INVALID_ALT_NUMBER ) {
                if ( _alt==1 ) {
                    {
                        {
                            setState(197);
                            match(COMMA);
                            setState(198);
                            expr();
                        }
                    }
                }
                setState(203);
                _errHandler.sync(this);
                _alt = getInterpreter().adaptivePredict(_input,15,_ctx);
            }
            setState(205);
            _errHandler.sync(this);
            _la = _input.LA(1);
            if (_la==COMMA) {
                {
                    {
                        setState(204);
                        match(COMMA);
                    }
                }
            }
        }
    }
    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }
    finally {
        exitRule();
    }
    return _localctx;
}

```



```

public boolean sempred(RuleContext _localctx, int ruleIndex, int predIndex) {
    switch (ruleIndex) {
        case 9:
            return logicalOr_sempred((LogicalOrContext)_localctx, predIndex);
        case 10:
            return logicalAnd_sempred((LogicalAndContext)_localctx, predIndex);
        case 11:
            return comparison_sempred((ComparisonContext)_localctx, predIndex);
        case 12:
            return additive_sempred((AdditiveContext)_localctx, predIndex);
        case 13:
            return multiplicative_sempred((MultiplicativeContext)_localctx, predIndex);
    }
    return true;
}

private boolean logicalOr_sempred(LogicalOrContext _localctx, int predIndex) {
    switch (predIndex) {
        case 0:
            return precpred(_ctx, 2);
    }
    return true;
}

private boolean logicalAnd_sempred(LogicalAndContext _localctx, int predIndex) {
    switch (predIndex) {
        case 1:
            return precpred(_ctx, 2);
    }
    return true;
}

private boolean comparison_sempred(ComparisonContext _localctx, int predIndex) {
    switch (predIndex) {
        case 2:
            return precpred(_ctx, 2);
    }
    return true;
}

private boolean additive_sempred(AdditiveContext _localctx, int predIndex) {
    switch (predIndex) {
        case 3:
            return precpred(_ctx, 2);
    }
    return true;
}

private boolean multiplicative_sempred(MultiplicativeContext _localctx, int predIndex) {
    switch (predIndex) {
        case 4:
            return precpred(_ctx, 2);
    }
    return true;
}

}

public static final String _serializedATN =
    "\u0004\u0001(\u00d0\u0002\u0000\u0007\u0000\u0002\u0001\u0007\u0001\u0002"+
    "\u0002\u0007\u0002\u0002\u0003\u0007\u0003\u0002\u0004\u0007\u0004\u0002"+
    "\u0005\u0007\u0005\u0002\u0006\u0007\u0006\u0002\u0007\u0007\u0007\u0002"+
    "\b\u0007\b\u0002\t\u0007\t\u0002\n\u0007\n\u0002\u000b\u0007\u000b\u0002"+
    "\f\u0007\f\u0002\u0007\u0007\u0002\u000e\u0007\u000e\u0002\u000f\u0007\u000f"+
    "\u0002\u0010\u0007\u0010\u0002\u0011\u0007\u0011\u0002\u0012\u0007\u0012"+
    "\u0001\u0000\u0001\u0000\u0001\u0000\u0001\u0001\u0000\u0001\u0005\u0001+\b\u0001"+
    "\n\u0001\f\u0001.\t\u0001\u0001\u0002\u0001\u0002\u0001\u0002\u0001\u0002"+
    "\u0003\u00024\b\u0002\u0001\u0003\u0001\u0003\u0001\u0003\u0001\u0003\u0003"+
    "\u0001\u0004\u0001\u0004\u0001\u0004\u0001\u0004\u0001\u0004\u0001\u0004"+
    "\u0005\u0004@\b\u0004\n\u0004\u0004\u0004\u0004\u0001\u0004\u0003\u0004"+
    "F\b\u0004\u0001\u0004\u0001\u0004\u0001\u0004\u0001\u0004\u0001\u0005"+
    "\u0001\u0005\u0001\u0005\u0001\u0005\u0001\u0005\u0001\u0005\u0001\u0006"+

```

[illegible]

```

"\u0082\u0001\u0000\u0000\u0000\u0000\u0082\u0017\u0001\u0000\u0000\u0000\u0000\u0083"+
"\u0081\u0001\u0000\u0000\u0000\u0000\u0084\u0085\u0006\u0000\u0000\u0000\u0000\u0085"+
"\u0086\u0003\u001a\u0000\u0000\u0086\u008c\u0001\u0000\u0000\u0000\u0000\u0087\u0088"+
"\u0002\u0000\u0000\u0000\u0088\u0089\u0007\u0001\u0000\u0000\u0000\u0089\u008b\u0003"+
"\u001a\u0000\u0000\u008a\u0087\u0001\u0000\u0000\u0000\u0000\u008b\u008e\u0001\u0000"+
"\u0000\u0000\u008c\u008a\u0001\u0000\u0000\u0000\u0000\u008c\u008d\u0001\u0000"+
"\u0000\u0000\u008d\u0019\u0001\u0000\u0000\u0000\u0000\u008e\u008c\u0001\u0000"+
"\u0000\u0000\u008f\u0090\u0006\u0000\u0000\u0000\u0000\u0090\u0091\u0003\u001c"+
"\u000e\u0000\u0091\u0097\u0001\u0000\u0000\u0000\u0000\u0092\u0093\u0002\u0000"+
"\u0000\u0093\u0094\u0007\u0002\u0000\u0000\u0000\u0094\u0096\u0003\u001c\u000e"+
"\u0000\u0095\u0092\u0001\u0000\u0000\u0000\u0096\u0099\u0001\u0000\u0000"+
"\u0000\u0097\u0095\u0001\u0000\u0000\u0000\u0097\u0098\u0001\u0000\u0000"+
"\u0000\u0098\u001b\u0001\u0000\u0000\u0000\u0099\u0097\u0001\u0000\u0000"+
"\u0000\u009a\u009b\u0005\u000f\u0000\u0000\u009b\u00a0\u0003\u001c\u000e"+
"\u0000\u009c\u009d\u0005\u001a\u0000\u0000\u009d\u00a0\u0003\u001c\u000e"+
"\u0000\u009e\u00a0\u0003\u001e\u000f\u0000\u009f\u009a\u0001\u0000\u0000"+
"\u0000\u009f\u009c\u0001\u0000\u0000\u0000\u009f\u009e\u0001\u0000\u0000"+
"\u0000\u00a0\u001d\u0001\u0000\u0000\u0000\u00a1\u00a2\u0005\u001f\u0000"+
"\u0000\u00a2\u00a3\u0003\u0010\u00b\u0000\u00a3\u00a4\u0005 \u0000\u0000"+
"\u00a4\u00ae\u0001\u0000\u0000\u0000\u00a5\u00ae\u0003 \u0010\u0000\u00a6"+
"\u00aa\u0005'\u0000\u0000\u00a7\u00a9\u0003'\u0011\u0000\u00a8\u00a7"+
"\u0001\u0000\u0000\u0000\u00a9\u00ac\u0001\u0000\u0000\u0000\u0000\u00aa\u00a8"+
"\u0001\u0000\u0000\u0000\u0000\u00ab\u0001\u0000\u0000\u0000\u0000\u00ab\u00ae"+
"\u0001\u0000\u0000\u0000\u0000\u00ac\u00aa\u0001\u0000\u0000\u0000\u0000\u00ad\u00a1"+
"\u0001\u0000\u0000\u0000\u0000\u00ad\u00a5\u0001\u0000\u0000\u0000\u0000\u00ad\u00a6"+
"\u0001\u0000\u0000\u0000\u0000\u00ae\u001f\u0001\u0000\u0000\u0000\u0000\u00af\u00b6"+
"\u0005$\u0000\u0000\u00b0\u00b6\u0005%\u0000\u0000\u00b1\u00b6\u0005\u0005"+
"\u0000\u0000\u00b0\u00b6\u0005\u0010\u0000\u0000\u00b3\u00b6\u0005\u0011"+
"\u0000\u0000\u00b4\u00b6\u0005\u0012\u0000\u0000\u00b5\u00af\u0001\u0000"+
"\u0000\u0000\u00b5\u00b0\u0001\u0000\u0000\u0000\u00b5\u00b1\u0001\u0000"+
"\u0000\u0000\u00b5\u00b2\u0001\u0000\u0000\u0000\u00b5\u00b3\u0001\u0000"+
"\u0000\u0000\u00b5\u00b4\u0001\u0000\u0000\u0000\u00b6!\u0001\u0000\u0000"+
"\u0000\u00b7\u00b8\u0005\u001e\u0000\u0000\u00b8\u00c3\u0005'\u0000\u0000"+
"\u00b9\u00ba\u0005!\u0000\u0000\u00ba\u00bb\u0003\u0010\u0000\u00bb"+
"\u00bc\u0005'\u0000\u0000\u00bc\u00c3\u0001\u0000\u0000\u0000\u00bd\u00bf"+
"\u0005\u001f\u0000\u0000\u00be\u00c0\u0003$\u0012\u0000\u00bf\u00be\u0001"+
"\u0000\u0000\u0000\u0000\u00bf\u00c0\u0001\u0000\u0000\u0000\u0000\u00c0\u00c1\u0001"+
"\u0000\u0000\u0000\u0000\u00c1\u00c3\u0005 \u0000\u0000\u00c2\u00b7\u0001\u0000"+
"\u0000\u0000\u0000\u00c2\u00b9\u0001\u0000\u0000\u0000\u00c2\u00bd\u0001\u0000"+
"\u0000\u0000\u00c3#\u0001\u0000\u0000\u0000\u00c4\u00c9\u0003\u0010\u00b"+
"\u0000\u00c5\u00c6\u0005#\u0000\u0000\u00c6\u00c8\u0003\u0010\u00b\u0000"+
"\u00c7\u00c5\u0001\u0000\u0000\u0000\u00c8\u00cb\u0001\u0000\u0000\u0000"+
"\u00c9\u00c7\u0001\u0000\u0000\u0000\u00c9\u00ca\u0001\u0000\u0000\u0000"+
"\u00ca\u00cd\u0001\u0000\u0000\u0000\u00cb\u00c9\u0001\u0000\u0000\u0000"+
"\u00cc\u00ce\u0005#\u0000\u0000\u00cd\u00cc\u0001\u0000\u0000\u0000\u00cd"+
"\u00ce\u0001\u0000\u0000\u0000\u00ce%\u0001\u0000\u0000\u0000\u0011,3"+
"Aekv\u0081\u008c\u0097\u009f\u000a\u00ad\u00b5\u00bf\u00c2\u00c9\u00cd";

public static final ATN _ATN =
    new ATNDeserializer().deserialize(_serializedATN.toCharArray());

static {
    _decisionToDFA = new DFA[_ATN.getNumberOfDecisions()];
    for (int i = 0; i < _ATN.getNumberOfDecisions(); i++) {
        _decisionToDFA[i] = new DFA(_ATN.getDecisionState(i), i);
    }
}

```

}.

?? File: JinjaParserBaseListener.java

// Generated from D:/Compiler/PyJinjaCompiler/src/grammar/jinja/JinjaParser.g4 by ANTLR 4.13.2 package compiler.generated.jinja;

import org.antlr.v4.runtime.ParserRuleContext; import org.antlr.v4.runtime.tree.ErrorNode; import org.antlr.v4.runtime.tree.TerminalNode;

/**

- This class provides an empty implementation of [@link JinjaParserListener](#),

- which can be extended to create a listener which only needs to handle a subset
 - of the available methods. */ @SuppressWarnings("CheckReturnValue") public class JinjaParserBaseListener implements JinjaParserListener { /**
 - *{@inheritDoc}*
 -
 - The default implementation does nothing.
- / @Override public void enterTemplate(JinjaParser.TemplateContext ctx) /**
- *{@inheritDoc}*
 -
 - The default implementation does nothing.
- / @Override public void exitTemplate(JinjaParser.TemplateContext ctx) /**
- *{@inheritDoc}*
 -
 - The default implementation does nothing.
- / @Override public void enterTemplateBody(JinjaParser.TemplateBodyContext ctx) /**
- *{@inheritDoc}*
 -
 - The default implementation does nothing.
- / @Override public void exitTemplateBody(JinjaParser.TemplateBodyContext ctx) /**
- *{@inheritDoc}*
 -
 - The default implementation does nothing.
- / @Override public void enterHtmlElement(JinjaParser.HtmlElementContext ctx) /**
- *{@inheritDoc}*
 -
 - The default implementation does nothing.
- / @Override public void exitHtmlElement(JinjaParser.HtmlElementContext ctx) /**
- *{@inheritDoc}*
 -
 - The default implementation does nothing.
- / @Override public void enterExpressionElement(JinjaParser.ExpressionElementContext ctx) /**
- *{@inheritDoc}*
 -
 - The default implementation does nothing.
- / @Override public void exitExpressionElement(JinjaParser.ExpressionElementContext ctx) /**
- *{@inheritDoc}*
 -
 - The default implementation does nothing.
- / @Override public void enterIfElement(JinjaParser.IfElementContext ctx) /**
- *{@inheritDoc}*
 -
 - The default implementation does nothing.
- / @Override public void exitIfElement(JinjaParser.IfElementContext ctx) /**
- *{@inheritDoc}*
 -

- The default implementation does nothing.

*/ @Override public void enterForElement(JinjaParser.ForElementContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitForElement(JinjaParser.ForElementContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterExpressionOutput(JinjaParser.ExpressionOutputContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitExpressionOutput(JinjaParser.ExpressionOutputContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterIfStatement(JinjaParser.IfStatementContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitIfStatement(JinjaParser.IfStatementContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterElifClause(JinjaParser.ElifClauseContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitElifClause(JinjaParser.ElifClauseContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterElseClause(JinjaParser.ElseClauseContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitElseClause(JinjaParser.ElseClauseContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterForStatement(JinjaParser.ForStatementContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitForStatement(JinjaParser.ForStatementContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void enterExpr(JinjaParser.ExprContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void exitExpr(JinjaParser.ExprContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void enterLogicalAndExpr(JinjaParser.LogicalAndExprContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void exitLogicalAndExpr(JinjaParser.LogicalAndExprContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void enterOrExpr(JinjaParser.OrExprContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void exitOrExpr(JinjaParser.OrExprContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void enterComparisonExpr(JinjaParser.ComparisonExprContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void exitComparisonExpr(JinjaParser.ComparisonExprContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void enterAndExpr(JinjaParser.AndExprContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void exitAndExpr(JinjaParser.AndExprContext ctx) /**

- *{@inheritDoc}*
-

- The default implementation does nothing.

*/ @Override public void enterComparisonOpExpr(JinjaParser.ComparisonOpExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitComparisonOpExpr(JinjaParser.ComparisonOpExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterAdditiveExpr(JinjaParser.AdditiveExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitAdditiveExpr(JinjaParser.AdditiveExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterAddExpr(JinjaParser.AddExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitAddExpr(JinjaParser.AddExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterMultiplicativeExpr(JinjaParser.MultiplicativeExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitMultiplicativeExpr(JinjaParser.MultiplicativeExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterUnaryExpr(JinjaParser.UnaryExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitUnaryExpr(JinjaParser.UnaryExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterMulExpr(JinjaParser.MulExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitMulExpr(JinjaParser.MulExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterNotExpr(JinjaParser.NotExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitNotExpr(JinjaParser.NotExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterUnaryMinus(JinjaParser.UnaryMinusContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitUnaryMinus(JinjaParser.UnaryMinusContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterPrimaryExpr(JinjaParser.PrimaryExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitPrimaryExpr(JinjaParser.PrimaryExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterParenExpr(JinjaParser.ParenExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitParenExpr(JinjaParser.ParenExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterLiteralExpr(JinjaParser.LiteralExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitLiteralExpr(JinjaParser.LiteralExprContext ctx) /**

- {@inheritDoc}
-

- The default implementation does nothing.

*/ @Override public void enterIdentifierExpr(JinjaParser.IdentifierExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitIdentifierExpr(JinjaParser.IdentifierExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterStringLiteral(JinjaParser.StringLiteralContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitStringLiteral(JinjaParser.StringLiteralContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterIntLiteral(JinjaParser.IntLiteralContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitIntLiteral(JinjaParser.IntLiteralContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterFloatLiteral(JinjaParser.FloatLiteralContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitFloatLiteral(JinjaParser.FloatLiteralContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterTrueLiteral(JinjaParser.TrueLiteralContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitTrueLiteral(JinjaParser.TrueLiteralContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterFalseLiteral(JinjaParser.FalseLiteralContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitFalseLiteral(JinjaParser.FalseLiteralContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterNoneLiteral(JinjaParser.NoneLiteralContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitNoneLiteral(JinjaParser.NoneLiteralContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterAttributeAccess(JinjaParser.AttributeAccessContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitAttributeAccess(JinjaParser.AttributeAccessContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterIndexAccess(JinjaParser.IndexAccessContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitIndexAccess(JinjaParser.IndexAccessContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterCallTrailer(JinjaParser.CallTrailerContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitCallTrailer(JinjaParser.CallTrailerContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterArgList(JinjaParser.ArgListContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

**/ @Override public void exitArgList(JinjaParser.ArgListContext ctx)*

*/***

- {@inheritDoc}

-
- The default implementation does nothing.

```
/ @Override public void enterEveryRule(ParserRuleContext ctx) /*
```

- {@inheritDoc}
-
- The default implementation does nothing.

```
/ @Override public void exitEveryRule(ParserRuleContext ctx) /*
```

- {@inheritDoc}
-
- The default implementation does nothing.

```
/ @Override public void visitTerminal(TerminalNode node) /*
```

- {@inheritDoc}
-
- The default implementation does nothing.

```
*/ @Override public void visitErrorNode(ErrorNode node) } .
```

?? File: JinjaParserBaseVisitor.java

// Generated from D:/Compiler/PyJinjaCompiler/src/grammar/jinja/JinjaParser.g4 by ANTLR 4.13.2 package compiler.generated.jinja; import org.antlr.v4.runtime.tree.AbstractParseTreeVisitor;

*/***

- This class provides an empty implementation of {@link JinjaParserVisitor},
- which can be extended to create a visitor which only needs to handle a subset
- of the available methods.
-
- @param The return type of the visit operation. Use {@link Void} for
- operations with no return type. */ @SuppressWarnings("CheckReturnValue") public class JinjaParserBaseVisitor extends AbstractParseTreeVisitor implements JinjaParserVisitor { /**
- {@inheritDoc}
-
- The default implementation returns the result of calling
- {@link #visitChildren} on {@code ctx}.
- / @Override public T visitTemplate(JinjaParser.TemplateContext ctx) { return visitChildren(ctx); } /**
- {@inheritDoc}
-
- The default implementation returns the result of calling
- {@link #visitChildren} on {@code ctx}.
- / @Override public T visitTemplateBody(JinjaParser.TemplateBodyContext ctx) { return visitChildren(ctx); } /**
- {@inheritDoc}
-
- The default implementation returns the result of calling
- {@link #visitChildren} on {@code ctx}.
- / @Override public T visitHtmlElement(JinjaParser.HtmlElementContext ctx) { return visitChildren(ctx); } /**
- {@inheritDoc}
-
- The default implementation returns the result of calling
- {@link #visitChildren} on {@code ctx}.
- / @Override public T visitExpressionElement(JinjaParser.ExpressionElementContext ctx) { return visitChildren(ctx); } /**
- {@inheritDoc}
-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
/ @Override public T visitIfElement(JinjaParser.IfElementContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
/ @Override public T visitForElement(JinjaParser.ForElementContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
/ @Override public T visitExpressionOutput(JinjaParser.ExpressionOutputContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
/ @Override public T visitIfStatement(JinjaParser.IfStatementContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
/ @Override public T visitElifClause(JinjaParser.ElifClauseContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
/ @Override public T visitElseClause(JinjaParser.ElseClauseContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
/ @Override public T visitForStatement(JinjaParser.ForStatementContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
/ @Override public T visitExpr(JinjaParser.ExprContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
/ @Override public T visitLogicalAndExpr(JinjaParser.LogicalAndExprContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```

    / @Override public T visitOrExpr(JinjaParser.OrExprContext ctx) { return visitChildren(ctx); } /*
    ■ { @inheritDoc }
    ■
    ■ The default implementation returns the result of calling
    ■ { @link #visitChildren } on { @code ctx }.

    / @Override public T visitComparisonExpr(JinjaParser.ComparisonExprContext ctx) { return visitChildren(ctx); } /*
    ■ { @inheritDoc }
    ■
    ■ The default implementation returns the result of calling
    ■ { @link #visitChildren } on { @code ctx }.

    / @Override public T visitAndExpr(JinjaParser.AndExprContext ctx) { return visitChildren(ctx); } /*
    ■ { @inheritDoc }
    ■
    ■ The default implementation returns the result of calling
    ■ { @link #visitChildren } on { @code ctx }.

    / @Override public T visitComparisonOpExpr(JinjaParser.ComparisonOpExprContext ctx) { return visitChildren(ctx); } /*
    ■ { @inheritDoc }
    ■
    ■ The default implementation returns the result of calling
    ■ { @link #visitChildren } on { @code ctx }.

    / @Override public T visitAdditiveExpr(JinjaParser.AdditiveExprContext ctx) { return visitChildren(ctx); } /*
    ■ { @inheritDoc }
    ■
    ■ The default implementation returns the result of calling
    ■ { @link #visitChildren } on { @code ctx }.

    / @Override public T visitAddExpr(JinjaParser.AddExprContext ctx) { return visitChildren(ctx); } /*
    ■ { @inheritDoc }
    ■
    ■ The default implementation returns the result of calling
    ■ { @link #visitChildren } on { @code ctx }.

    / @Override public T visitMultiplicativeExpr(JinjaParser.MultiplicativeExprContext ctx) { return visitChildren(ctx); } /*
    ■ { @inheritDoc }
    ■
    ■ The default implementation returns the result of calling
    ■ { @link #visitChildren } on { @code ctx }.

    / @Override public T visitUnaryExpr(JinjaParser.UnaryExprContext ctx) { return visitChildren(ctx); } /*
    ■ { @inheritDoc }
    ■
    ■ The default implementation returns the result of calling
    ■ { @link #visitChildren } on { @code ctx }.

    / @Override public T visitMulExpr(JinjaParser.MulExprContext ctx) { return visitChildren(ctx); } /*
    ■ { @inheritDoc }
    ■
    ■ The default implementation returns the result of calling
    ■ { @link #visitChildren } on { @code ctx }.

    / @Override public T visitNotExpr(JinjaParser.NotExprContext ctx) { return visitChildren(ctx); } /*
    ■ { @inheritDoc }
    ■

```

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
// @Override public T visitUnaryMinus(JinjaParser.UnaryMinusContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

▪

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
// @Override public T visitPrimaryExpr(JinjaParser.PrimaryExprContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

▪

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
// @Override public T visitParenExpr(JinjaParser.ParenExprContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

▪

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
// @Override public T visitLiteralExpr(JinjaParser.LiteralExprContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

▪

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
// @Override public T visitIdentifierExpr(JinjaParser.IdentifierExprContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

▪

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
// @Override public T visitStringLiteral(JinjaParser.StringLiteralContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

▪

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
// @Override public T visitIntLiteral(JinjaParser.IntLiteralContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

▪

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
// @Override public T visitFloatLiteral(JinjaParser.FloatLiteralContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

▪

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
// @Override public T visitTrueLiteral(JinjaParser.TrueLiteralContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

▪

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```

    / @Override public T visitFalseLiteral(JinjaParser.FalseLiteralContext ctx) { return visitChildren(ctx); } /*
    ■ { @inheritDoc }
    ■

    ■ The default implementation returns the result of calling

    ■ { @link #visitChildren } on { @code ctx }.

    / @Override public T visitNoneLiteral(JinjaParser.NoneLiteralContext ctx) { return visitChildren(ctx); } /*
    ■ { @inheritDoc }
    ■

    ■ The default implementation returns the result of calling

    ■ { @link #visitChildren } on { @code ctx }.

    / @Override public T visitAttributeAccess(JinjaParser.AttributeAccessContext ctx) { return visitChildren(ctx); } /*
    ■ { @inheritDoc }
    ■

    ■ The default implementation returns the result of calling

    ■ { @link #visitChildren } on { @code ctx }.

    / @Override public T visitIndexAccess(JinjaParser.IndexAccessContext ctx) { return visitChildren(ctx); } /*
    ■ { @inheritDoc }
    ■

    ■ The default implementation returns the result of calling

    ■ { @link #visitChildren } on { @code ctx }.

    / @Override public T visitCallTrailer(JinjaParser.CallTrailerContext ctx) { return visitChildren(ctx); } /*
    ■ { @inheritDoc }
    ■

    ■ The default implementation returns the result of calling

    ■ { @link #visitChildren } on { @code ctx }.

    */ @Override public T visitArgList(JinjaParser.ArgListContext ctx) { return visitChildren(ctx); } } .

```

?? File: JinjaParserListener.java

// Generated from D:/Compiler/PyJinjaCompiler/src/grammar/jinja/JinjaParser.g4 by ANTLR 4.13.2 package compiler.generated.jinja; import org.antlr.v4.runtime.tree.ParseTreeListener;

/**

- This interface defines a complete listener for a parse tree produced by
- { @link JinjaParser }. / public interface JinjaParserListener extends ParseTreeListener { /*
 - Enter a parse tree produced by { @link JinjaParser#template }.
 - @param ctx the parse tree / void enterTemplate(JinjaParser.TemplateContext ctx); /*
 - Exit a parse tree produced by { @link JinjaParser#template }.
 - @param ctx the parse tree / void exitTemplate(JinjaParser.TemplateContext ctx); /*
 - Enter a parse tree produced by { @link JinjaParser#templateBody }.
 - @param ctx the parse tree / void enterTemplateBody(JinjaParser.TemplateBodyContext ctx); /*
 - Exit a parse tree produced by { @link JinjaParser#templateBody }.
 - @param ctx the parse tree / void exitTemplateBody(JinjaParser.TemplateBodyContext ctx); /*
 - Enter a parse tree produced by the { @code htmlElement }
 - labeled alternative in { @link JinjaParser#element }.
 - @param ctx the parse tree / void enterHtmlElement(JinjaParser.HtmlElementContext ctx); /*
 - Exit a parse tree produced by the { @code htmlElement }
 - labeled alternative in { @link JinjaParser#element }.
 - @param ctx the parse tree / void exitHtmlElement(JinjaParser.HtmlElementContext ctx); /*
 - Enter a parse tree produced by the { @code expressionElement }
 - labeled alternative in { @link JinjaParser#element }.
 - @param ctx the parse tree / void enterExpressionElement(JinjaParser.ExpressionElementContext ctx); /*
 - Exit a parse tree produced by the { @code expressionElement }
 - labeled alternative in { @link JinjaParser#element }.
 - @param ctx the parse tree / void exitExpressionElement(JinjaParser.ExpressionElementContext ctx); /*
 - Enter a parse tree produced by the { @code ifElement }

- labeled alternative in {@link JinjaParser#element}.
- @param ctx the parse tree / void enterIfElement(JinjaParser.IfElementContext ctx); /*
- Exit a parse tree produced by the {@code ifElement}
- labeled alternative in {@link JinjaParser#element}.
- @param ctx the parse tree / void exitIfElement(JinjaParser.IfElementContext ctx); /*
- Enter a parse tree produced by the {@code forElement}
- labeled alternative in {@link JinjaParser#element}.
- @param ctx the parse tree / void enterForElement(JinjaParser.ForElementContext ctx); /*
- Exit a parse tree produced by the {@code forElement}
- labeled alternative in {@link JinjaParser#element}.
- @param ctx the parse tree / void exitForElement(JinjaParser.ForElementContext ctx); /*
- Enter a parse tree produced by {@link JinjaParser#expressionOutput}.
- @param ctx the parse tree / void enterExpressionOutput(JinjaParser.ExpressionOutputContext ctx); /*
- Exit a parse tree produced by {@link JinjaParser#expressionOutput}.
- @param ctx the parse tree / void exitExpressionOutput(JinjaParser.ExpressionOutputContext ctx); /*
- Enter a parse tree produced by {@link JinjaParser#ifStatement}.
- @param ctx the parse tree / void enterIfStatement(JinjaParser.IfStatementContext ctx); /*
- Exit a parse tree produced by {@link JinjaParser#ifStatement}.
- @param ctx the parse tree / void exitIfStatement(JinjaParser.IfStatementContext ctx); /*
- Enter a parse tree produced by {@link JinjaParser#elifClause}.
- @param ctx the parse tree / void enterElifClause(JinjaParser.ElifClauseContext ctx); /*
- Exit a parse tree produced by {@link JinjaParser#elifClause}.
- @param ctx the parse tree / void exitElifClause(JinjaParser.ElifClauseContext ctx); /*
- Enter a parse tree produced by {@link JinjaParser#elseClause}.
- @param ctx the parse tree / void enterElseClause(JinjaParser.ElseClauseContext ctx); /*
- Exit a parse tree produced by {@link JinjaParser#elseClause}.
- @param ctx the parse tree / void exitElseClause(JinjaParser.ElseClauseContext ctx); /*
- Enter a parse tree produced by {@link JinjaParser#forStatement}.
- @param ctx the parse tree / void enterForStatement(JinjaParser.ForStatementContext ctx); /*
- Exit a parse tree produced by {@link JinjaParser#forStatement}.
- @param ctx the parse tree / void exitForStatement(JinjaParser.ForStatementContext ctx); /*
- Enter a parse tree produced by {@link JinjaParser#expr}.
- @param ctx the parse tree / void enterExpr(JinjaParser.ExprContext ctx); /*
- Exit a parse tree produced by {@link JinjaParser#expr}.
- @param ctx the parse tree / void exitExpr(JinjaParser.ExprContext ctx); /*
- Enter a parse tree produced by the {@code logicalAndExpr}
- labeled alternative in {@link JinjaParser#logicalOr}.
- @param ctx the parse tree / void enterLogicalAndExpr(JinjaParser.LogicalAndExprContext ctx); /*
- Exit a parse tree produced by the {@code logicalAndExpr}
- labeled alternative in {@link JinjaParser#logicalOr}.
- @param ctx the parse tree / void exitLogicalAndExpr(JinjaParser.LogicalAndExprContext ctx); /*
- Enter a parse tree produced by the {@code orExpr}
- labeled alternative in {@link JinjaParser#logicalOr}.
- @param ctx the parse tree / void enterOrExpr(JinjaParser.OrExprContext ctx); /*
- Exit a parse tree produced by the {@code orExpr}
- labeled alternative in {@link JinjaParser#logicalOr}.
- @param ctx the parse tree / void exitOrExpr(JinjaParser.OrExprContext ctx); /*
- Enter a parse tree produced by the {@code comparisonExpr}
- labeled alternative in {@link JinjaParser#logicalAnd}.
- @param ctx the parse tree / void enterComparisonExpr(JinjaParser.ComparisonExprContext ctx); /*
- Exit a parse tree produced by the {@code comparisonExpr}
- labeled alternative in {@link JinjaParser#logicalAnd}.
- @param ctx the parse tree / void exitComparisonExpr(JinjaParser.ComparisonExprContext ctx); /*
- Enter a parse tree produced by the {@code andExpr}
- labeled alternative in {@link JinjaParser#logicalAnd}.
- @param ctx the parse tree / void enterAndExpr(JinjaParser.AndExprContext ctx); /*
- Exit a parse tree produced by the {@code andExpr}
- labeled alternative in {@link JinjaParser#logicalAnd}.
- @param ctx the parse tree / void exitAndExpr(JinjaParser.AndExprContext ctx); /*
- Enter a parse tree produced by the {@code comparisonOpExpr}
- labeled alternative in {@link JinjaParser#comparison}.
- @param ctx the parse tree / void enterComparisonOpExpr(JinjaParser.ComparisonOpExprContext ctx); /*
- Exit a parse tree produced by the {@code comparisonOpExpr}
- labeled alternative in {@link JinjaParser#comparison}.
- @param ctx the parse tree / void exitComparisonOpExpr(JinjaParser.ComparisonOpExprContext ctx); /*
- Enter a parse tree produced by the {@code additiveExpr}

- labeled alternative in {@link JinjaParser#comparison}.
- @param ctx the parse tree / void enterAdditiveExpr(JinjaParser.AdditiveExprContext ctx); /*
- Exit a parse tree produced by the {@code additiveExpr}
- labeled alternative in {@link JinjaParser#comparison}.
- @param ctx the parse tree / void exitAdditiveExpr(JinjaParser.AdditiveExprContext ctx); /*
- Enter a parse tree produced by the {@code addExpr}
- labeled alternative in {@link JinjaParser#additive}.
- @param ctx the parse tree / void enterAddExpr(JinjaParser.AddExprContext ctx); /*
- Exit a parse tree produced by the {@code addExpr}
- labeled alternative in {@link JinjaParser#additive}.
- @param ctx the parse tree / void exitAddExpr(JinjaParser.AddExprContext ctx); /*
- Enter a parse tree produced by the {@code multiplicativeExpr}
- labeled alternative in {@link JinjaParser#additive}.
- @param ctx the parse tree / void enterMultiplicativeExpr(JinjaParser.MultiplicativeExprContext ctx); /*
- Exit a parse tree produced by the {@code multiplicativeExpr}
- labeled alternative in {@link JinjaParser#additive}.
- @param ctx the parse tree / void exitMultiplicativeExpr(JinjaParser.MultiplicativeExprContext ctx); /*
- Enter a parse tree produced by the {@code unaryExpr}
- labeled alternative in {@link JinjaParser#multiplicative}.
- @param ctx the parse tree / void enterUnaryExpr(JinjaParser.UnaryExprContext ctx); /*
- Exit a parse tree produced by the {@code unaryExpr}
- labeled alternative in {@link JinjaParser#multiplicative}.
- @param ctx the parse tree / void exitUnaryExpr(JinjaParser.UnaryExprContext ctx); /*
- Enter a parse tree produced by the {@code mulExpr}
- labeled alternative in {@link JinjaParser#multiplicative}.
- @param ctx the parse tree / void enterMulExpr(JinjaParser.MulExprContext ctx); /*
- Exit a parse tree produced by the {@code mulExpr}
- labeled alternative in {@link JinjaParser#multiplicative}.
- @param ctx the parse tree / void exitMulExpr(JinjaParser.MulExprContext ctx); /*
- Enter a parse tree produced by the {@code notExpr}
- labeled alternative in {@link JinjaParser#unary}.
- @param ctx the parse tree / void enterNotExpr(JinjaParser.NotExprContext ctx); /*
- Exit a parse tree produced by the {@code notExpr}
- labeled alternative in {@link JinjaParser#unary}.
- @param ctx the parse tree / void exitNotExpr(JinjaParser.NotExprContext ctx); /*
- Enter a parse tree produced by the {@code unaryMinus}
- labeled alternative in {@link JinjaParser#unary}.
- @param ctx the parse tree / void enterUnaryMinus(JinjaParser.UnaryMinusContext ctx); /*
- Exit a parse tree produced by the {@code unaryMinus}
- labeled alternative in {@link JinjaParser#unary}.
- @param ctx the parse tree / void exitUnaryMinus(JinjaParser.UnaryMinusContext ctx); /*
- Enter a parse tree produced by the {@code primaryExpr}
- labeled alternative in {@link JinjaParser#unary}.
- @param ctx the parse tree / void enterPrimaryExpr(JinjaParser.PrimaryExprContext ctx); /*
- Exit a parse tree produced by the {@code primaryExpr}
- labeled alternative in {@link JinjaParser#unary}.
- @param ctx the parse tree / void exitPrimaryExpr(JinjaParser.PrimaryExprContext ctx); /*
- Enter a parse tree produced by the {@code parenExpr}
- labeled alternative in {@link JinjaParser#primary}.
- @param ctx the parse tree / void enterParenExpr(JinjaParser.ParenExprContext ctx); /*
- Exit a parse tree produced by the {@code parenExpr}
- labeled alternative in {@link JinjaParser#primary}.
- @param ctx the parse tree / void exitParenExpr(JinjaParser.ParenExprContext ctx); /*
- Enter a parse tree produced by the {@code literalExpr}
- labeled alternative in {@link JinjaParser#primary}.
- @param ctx the parse tree / void enterLiteralExpr(JinjaParser.LiteralExprContext ctx); /*
- Exit a parse tree produced by the {@code literalExpr}
- labeled alternative in {@link JinjaParser#primary}.
- @param ctx the parse tree / void exitLiteralExpr(JinjaParser.LiteralExprContext ctx); /*
- Enter a parse tree produced by the {@code identifierExpr}
- labeled alternative in {@link JinjaParser#primary}.
- @param ctx the parse tree / void enterIdentifierExpr(JinjaParser.IdentifierExprContext ctx); /*
- Exit a parse tree produced by the {@code identifierExpr}
- labeled alternative in {@link JinjaParser#primary}.
- @param ctx the parse tree / void exitIdentifierExpr(JinjaParser.IdentifierExprContext ctx); /*
- Enter a parse tree produced by the {@code stringLiteral}

- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree / void enterStringLiteral(JinjaParser.StringLiteralContext ctx); /*
- Exit a parse tree produced by the {@code stringLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree / void exitStringLiteral(JinjaParser.StringLiteralContext ctx); /*
- Enter a parse tree produced by the {@code intLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree / void enterIntLiteral(JinjaParser.IntLiteralContext ctx); /*
- Exit a parse tree produced by the {@code intLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree / void exitIntLiteral(JinjaParser.IntLiteralContext ctx); /*
- Enter a parse tree produced by the {@code floatLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree / void enterFloatLiteral(JinjaParser.FloatLiteralContext ctx); /*
- Exit a parse tree produced by the {@code floatLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree / void exitFloatLiteral(JinjaParser.FloatLiteralContext ctx); /*
- Enter a parse tree produced by the {@code trueLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree / void enterTrueLiteral(JinjaParser.TrueLiteralContext ctx); /*
- Exit a parse tree produced by the {@code trueLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree / void exitTrueLiteral(JinjaParser.TrueLiteralContext ctx); /*
- Enter a parse tree produced by the {@code falseLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree / void enterFalseLiteral(JinjaParser.FalseLiteralContext ctx); /*
- Exit a parse tree produced by the {@code falseLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree / void exitFalseLiteral(JinjaParser.FalseLiteralContext ctx); /*
- Enter a parse tree produced by the {@code noneLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree / void enterNoneLiteral(JinjaParser.NoneLiteralContext ctx); /*
- Exit a parse tree produced by the {@code noneLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree / void exitNoneLiteral(JinjaParser.NoneLiteralContext ctx); /*
- Enter a parse tree produced by the {@code attributeAccess}
- labeled alternative in {@link JinjaParser#trailer}.
- @param ctx the parse tree / void enterAttributeAccess(JinjaParser.AttributeAccessContext ctx); /*
- Exit a parse tree produced by the {@code attributeAccess}
- labeled alternative in {@link JinjaParser#trailer}.
- @param ctx the parse tree / void exitAttributeAccess(JinjaParser.AttributeAccessContext ctx); /*
- Enter a parse tree produced by the {@code indexAccess}
- labeled alternative in {@link JinjaParser#trailer}.
- @param ctx the parse tree / void enterIndexAccess(JinjaParser.IndexAccessContext ctx); /*
- Exit a parse tree produced by the {@code indexAccess}
- labeled alternative in {@link JinjaParser#trailer}.
- @param ctx the parse tree / void exitIndexAccess(JinjaParser.IndexAccessContext ctx); /*
- Enter a parse tree produced by the {@code callTrailer}
- labeled alternative in {@link JinjaParser#trailer}.
- @param ctx the parse tree / void enterCallTrailer(JinjaParser.CallTrailerContext ctx); /*
- Exit a parse tree produced by the {@code callTrailer}
- labeled alternative in {@link JinjaParser#trailer}.
- @param ctx the parse tree / void exitCallTrailer(JinjaParser.CallTrailerContext ctx); /*
- Enter a parse tree produced by {@link JinjaParser#argList}.
- @param ctx the parse tree / void enterArgList(JinjaParser.ArgListContext ctx); /*
- Exit a parse tree produced by {@link JinjaParser#argList}.
- @param ctx the parse tree */ void exitArgList(JinjaParser.ArgListContext ctx); } .

?? File: JinjaParserVisitor.java

```
// Generated from D:/Compiler/PyJinjaCompiler/src/grammar/jinja/JinjaParser.g4 by ANTLR 4.13.2 package compiler.generated.jinja; import
org.antlr.v4.runtime.tree.ParseTreeVisitor;
```

```
/**
```

- This interface defines a complete generic visitor for a parse tree produced
- by {@link JinjaParser}.

-
- @param The return type of the visit operation. Use {@link Void} for
- operations with no return type. / *public interface JinjaParserVisitor extends ParseTreeVisitor { /**
 - Visit a parse tree produced by {@link JinjaParser#template}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitTemplate(JinjaParser.TemplateContext ctx); /**
 - Visit a parse tree produced by {@link JinjaParser#templateBody}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitTemplateBody(JinjaParser.TemplateBodyContext ctx); /**
 - Visit a parse tree produced by the {@code htmlElement}
 - labeled alternative in {@link JinjaParser#element}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitHtmlElement(JinjaParser.HtmlElementContext ctx); /**
 - Visit a parse tree produced by the {@code expressionElement}
 - labeled alternative in {@link JinjaParser#element}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitExpressionElement(JinjaParser.ExpressionElementContext ctx); /**
 - Visit a parse tree produced by the {@code ifElement}
 - labeled alternative in {@link JinjaParser#element}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitIfElement(JinjaParser.IfElementContext ctx); /**
 - Visit a parse tree produced by the {@code forElement}
 - labeled alternative in {@link JinjaParser#element}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitForElement(JinjaParser.ForElementContext ctx); /**
 - Visit a parse tree produced by {@link JinjaParser#expressionOutput}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitExpressionOutput(JinjaParser.ExpressionOutputContext ctx); /**
 - Visit a parse tree produced by {@link JinjaParser#ifStatement}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitIfStatement(JinjaParser.IfStatementContext ctx); /**
 - Visit a parse tree produced by {@link JinjaParser#elifClause}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitElifClause(JinjaParser.ElifClauseContext ctx); /**
 - Visit a parse tree produced by {@link JinjaParser#elseClause}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitElseClause(JinjaParser.ElseClauseContext ctx); /**
 - Visit a parse tree produced by {@link JinjaParser#forStatement}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitForStatement(JinjaParser.ForStatementContext ctx); /**
 - Visit a parse tree produced by {@link JinjaParser#expr}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitExpr(JinjaParser.ExprContext ctx); /**
 - Visit a parse tree produced by the {@code logicalAndExpr}
 - labeled alternative in {@link JinjaParser#logicalOr}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitLogicalAndExpr(JinjaParser.LogicalAndExprContext ctx); /**
 - Visit a parse tree produced by the {@code orExpr}
 - labeled alternative in {@link JinjaParser#logicalOr}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitOrExpr(JinjaParser.OrExprContext ctx); /**
 - Visit a parse tree produced by the {@code comparisonExpr}
 - labeled alternative in {@link JinjaParser#logicalAnd}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitComparisonExpr(JinjaParser.ComparisonExprContext ctx); /**
 - Visit a parse tree produced by the {@code andExpr}
 - labeled alternative in {@link JinjaParser#logicalAnd}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitAndExpr(JinjaParser.AndExprContext ctx); /**
 - Visit a parse tree produced by the {@code comparisonOpExpr}
 - labeled alternative in {@link JinjaParser#comparison}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitComparisonOpExpr(JinjaParser.ComparisonOpExprContext ctx); /**
 - Visit a parse tree produced by the {@code additiveExpr}
 - labeled alternative in {@link JinjaParser#comparison}.
 - @param ctx the parse tree

- @return the visitor result / *T visitAdditiveExpr(JinjaParser.AdditiveExprContext ctx); /**
- Visit a parse tree produced by the {@code addExpr}
- labeled alternative in {@link JinjaParser#additive}.
- @param ctx the parse tree
- @return the visitor result / *T visitAddExpr(JinjaParser.AddExprContext ctx); /**
- Visit a parse tree produced by the {@code multiplicativeExpr}
- labeled alternative in {@link JinjaParser#additive}.
- @param ctx the parse tree
- @return the visitor result / *T visitMultiplicativeExpr(JinjaParser.MultiplicativeExprContext ctx); /**
- Visit a parse tree produced by the {@code unaryExpr}
- labeled alternative in {@link JinjaParser#multiplicative}.
- @param ctx the parse tree
- @return the visitor result / *T visitUnaryExpr(JinjaParser.UnaryExprContext ctx); /**
- Visit a parse tree produced by the {@code mulExpr}
- labeled alternative in {@link JinjaParser#multiplicative}.
- @param ctx the parse tree
- @return the visitor result / *T visitMulExpr(JinjaParser.MulExprContext ctx); /**
- Visit a parse tree produced by the {@code notExpr}
- labeled alternative in {@link JinjaParser#unary}.
- @param ctx the parse tree
- @return the visitor result / *T visitNotExpr(JinjaParser.NotExprContext ctx); /**
- Visit a parse tree produced by the {@code unaryMinus}
- labeled alternative in {@link JinjaParser#unary}.
- @param ctx the parse tree
- @return the visitor result / *T visitUnaryMinus(JinjaParser.UnaryMinusContext ctx); /**
- Visit a parse tree produced by the {@code primaryExpr}
- labeled alternative in {@link JinjaParser#unary}.
- @param ctx the parse tree
- @return the visitor result / *T visitPrimaryExpr(JinjaParser.PrimaryExprContext ctx); /**
- Visit a parse tree produced by the {@code parenExpr}
- labeled alternative in {@link JinjaParser#primary}.
- @param ctx the parse tree
- @return the visitor result / *T visitParenExpr(JinjaParser.ParenExprContext ctx); /**
- Visit a parse tree produced by the {@code literalExpr}
- labeled alternative in {@link JinjaParser#primary}.
- @param ctx the parse tree
- @return the visitor result / *T visitLiteralExpr(JinjaParser.LiteralExprContext ctx); /**
- Visit a parse tree produced by the {@code identifierExpr}
- labeled alternative in {@link JinjaParser#primary}.
- @param ctx the parse tree
- @return the visitor result / *T visitIdentifierExpr(JinjaParser.IdentifierExprContext ctx); /**
- Visit a parse tree produced by the {@code stringLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree
- @return the visitor result / *T visitStringLiteral(JinjaParser.StringLiteralContext ctx); /**
- Visit a parse tree produced by the {@code intLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree
- @return the visitor result / *T visitIntLiteral(JinjaParser.IntLiteralContext ctx); /**
- Visit a parse tree produced by the {@code floatLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree
- @return the visitor result / *T visitFloatLiteral(JinjaParser.FloatLiteralContext ctx); /**
- Visit a parse tree produced by the {@code trueLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree
- @return the visitor result / *T visitTrueLiteral(JinjaParser.TrueLiteralContext ctx); /**
- Visit a parse tree produced by the {@code falseLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree
- @return the visitor result / *T visitFalseLiteral(JinjaParser.FalseLiteralContext ctx); /**
- Visit a parse tree produced by the {@code noneLiteral}
- labeled alternative in {@link JinjaParser#literal}.
- @param ctx the parse tree
- @return the visitor result / *T visitNoneLiteral(JinjaParser.NoneLiteralContext ctx); /**
- Visit a parse tree produced by the {@code attributeAccess}

- labeled alternative in {@link JinjaParser#trailer}.
- @param ctx the parse tree
- @return the visitor result / *T visitAttributeAccess(JinjaParser.AttributeAccessContext ctx); /**
- Visit a parse tree produced by the {@code indexAccess}
- labeled alternative in {@link JinjaParser#trailer}.
- @param ctx the parse tree
- @return the visitor result / *T visitIndexAccess(JinjaParser.IndexAccessContext ctx); /**
- Visit a parse tree produced by the {@code callTrailer}
- labeled alternative in {@link JinjaParser#trailer}.
- @param ctx the parse tree
- @return the visitor result / *T visitCallTrailer(JinjaParser.CallTrailerContext ctx); /**
- Visit a parse tree produced by {@link JinjaParser#argList}.
- @param ctx the parse tree
- @return the visitor result */ *T visitArgList(JinjaParser.ArgListContext ctx); } .*

?? File: BasePythonLexer.java

package compiler.generated.python;

import org.antlr.v4.runtime.*; import org.antlr.v4.runtime.misc.Pair;

import java.util.ArrayDeque; import java.util.Deque; import java.util.Queue;

public class BasePythonLexer extends PythonLexer {

```

private final Deque<Integer> indents = new ArrayDeque<>();
private final Queue<Token> pending = new ArrayDeque<>();
private int opened = 0;

private final int NEWLINE_T;
private final int INDENT_T;
private final int DEDENT_T;
private final int LPAREN_T;
private final int RPAREN_T;
private final int LBRACK_T;
private final int RBRACK_T;
private final int LBRACE_T;
private final int RBRACE_T;

public BasePythonLexer(CharStream input) {
    super(input);

    NEWLINE_T = resolveTokenType("NEWLINE");
    INDENT_T = resolveTokenType("INDENT");
    DEDENT_T = resolveTokenType("DEDENT");
    LPAREN_T = resolveTokenType("LPAREN");
    RPAREN_T = resolveTokenType("RPAREN");
    LBRACK_T = resolveTokenType("LBRACK");
    RBRACK_T = resolveTokenType("RBRACK");
    LBRACE_T = resolveTokenType("LBRACE");
    RBRACE_T = resolveTokenType("RBRACE");

    indents.addFirst(0);
}

@Override
public Token nextToken() {

    if (!pending.isEmpty()) {
        return pending.poll();
    }

    Token raw = super.nextToken();
    if (raw == null) return null;

    // Skip NEWLINE inside grouping
    while (raw.getType() == NEWLINE_T && opened > 0) {
        raw = super.nextToken();
        if (raw == null) return null;
    }

    // EOF: emit final NEWLINE + all DEDENTS
    if (raw.getType() == Token.EOF) {

        if (indents.getFirst() != 0) {
            pending.add(makeSynthetic(NEWLINE_T, "\n"));
        }

        while (indents.getFirst() > 0) {
            indents.removeFirst();
            pending.add(makeSynthetic(DEDENT_T, ""));
        }

        if (!pending.isEmpty()) {
            return pending.poll();
        }

        return raw;
    }
}

```

```

// Track grouping
if (raw.getType() == LPAREN_T || raw.getType() == LBRACK_T || raw.getType() == LBRACE_T) {
    opened++;
    return raw;
}

if (raw.getType() == RPAREN_T || raw.getType() == RBRACK_T || raw.getType() == RBRACE_T) {
    if (opened > 0) opened--;
    return raw;
}

// Handle NEWLINE (only when opened == 0)
if (raw.getType() == NEWLINE_T) {

    IntStream input = _input;
    int startIndex = input.index();
    int indent = 0;
    int la = input.LA(1);

    while (la == ' ' || la == '\t') {
        indent += (la == ' ') ? 1 : 8;
        input.consume();
        la = input.LA(1);
    }

    // Blank or comment-only line â†’ skip entirely
    if (la == '\r' || la == '\n' || la == '#' || la == IntStream.EOF) {
        input.seek(startIndex);
        return nextToken();
    }

    input.seek(startIndex);

    pending.add(makeSynthetic(raw)); // NEWLINE

    int prev = indents.getFirst();

    if (indent > prev) {
        indents.addFirst(indent);
        pending.add(makeSynthetic(INDENT_T, ""));
    } else if (indent < prev) {
        while (indents.getFirst() > indent) {
            indents.removeFirst();
            pending.add(makeSynthetic(DEDENT_T, ""));
        }
        if (indents.getFirst() != indent) {
            throw new RuntimeException("Unindent does not match any outer indentation level at line " + raw.getLine());
        }
    }

    return pending.poll();
}

return raw;
}

/**
 * Create synthetic tokens with correct TokenSource + InputStream
 */
private Token makeSynthetic(int type, String text) {
    int start = getCharIndex();
    int stop = start + text.length() - 1;

    CommonToken t = new CommonToken(
        new Pair<>(this, _input),
        type,
        Token.DEFAULT_CHANNEL,

```

```

        start,
        stop
    );
    t.setText(text);
    return t;
}

private Token makeSynthetic(Token original) {
    CommonToken t = new CommonToken(
        new Pair<>(this, _input),
        original.getType(),
        original.getChannel(),
        original.getStartIndex(),
        original.getStopIndex()
    );
    t.setText(original.getText());
    return t;
}

private int resolveTokenType(String name) {
    try {
        java.lang.reflect.Field f = this.getClass().getSuperclass().getField(name);
        if (f.getType() == int.class) return f.getInt(null);
    } catch (Exception ignored) {}

    try {
        Class<?> parserClass = Class.forName("compiler.generated.python.PythonParser");
        java.lang.reflect.Field pf = parserClass.getField(name);
        if (pf.getType() == int.class) return pf.getInt(null);
    } catch (Exception ignored) {}

    Vocabulary vocab = this.getVocabulary();
    String[] tokenNames = this.getTokenNames();
    if (tokenNames != null) {
        for (int i = 0; i < tokenNames.length; i++) {
            String sym = vocab.getSymbolicName(i);
            if (name.equals(sym)) return i;
        }
    }

    throw new RuntimeException("Token type constant not found: " + name);
}
}

```

```

}

```

?? File: PythonLexer.java

```

// Generated from D:/Compiler/PyJinjaCompiler/src/grammar/python/PythonLexer.g4 by ANTLR 4.13.2 package compiler.generated.python; import
org.antlr.v4.runtime.Lexer; import org.antlr.v4.runtime.CharStream; import org.antlr.v4.runtime.Token; import org.antlr.v4.runtime.TokenStream; import
org.antlr.v4.runtime.; import org.antlr.v4.runtime.atn.; import org.antlr.v4.runtime.dfa.DFA; import org.antlr.v4.runtime.misc.*;

```

```

@SuppressWarnings({"all", "warnings", "unchecked", "unused", "cast", "CheckReturnValue", "this-escape"}) public class PythonLexer extends Lexer { static {
RuntimeMetaData.checkVersion("4.13.2", RuntimeMetaData.VERSION); }

```



```

protected static final DFA[] _decisionToDFA;
protected static final PredictionContextCache _sharedContextCache =
    new PredictionContextCache();
public static final int
    DEF=1, RETURN=2, IF=3, ELIF=4, ELSE=5, FOR=6, IN=7, WHILE=8, TRUE=9, FALSE=10,
    NONE=11, AND=12, OR=13, NOT=14, EQ=15, NE=16, LE=17, GE=18, ASSIGN=19,
    LT=20, GT=21, PLUS=22, MINUS=23, STAR=24, SLASH=25, MOD=26, LPAREN=27,
    RPAREN=28, LBRACK=29, RBRACK=30, LBRACE=31, RBRACE=32, COLON=33, COMMA=34,
    DOT=35, AT=36, FLOAT_LITERAL=37, INTEGER_LITERAL=38, STRING_LITERAL=39,
    IDENTIFIER=40, NEWLINE=41, COMMENT=42, WS=43;
public static String[] channelNames = {
    "DEFAULT_TOKEN_CHANNEL", "HIDDEN"
};

public static String[] modeNames = {
    "DEFAULT_MODE"
};

private static String[] makeRuleNames() {
    return new String[] {
        "DEF", "RETURN", "IF", "ELIF", "ELSE", "FOR", "IN", "WHILE", "TRUE",
        "FALSE", "NONE", "AND", "OR", "NOT", "EQ", "NE", "LE", "GE", "ASSIGN",
        "LT", "GT", "PLUS", "MINUS", "STAR", "SLASH", "MOD", "LPAREN", "RPAREN",
        "LBRACK", "RBRACK", "LBRACE", "RBRACE", "COLON", "COMMA", "DOT", "AT",
        "FLOAT_LITERAL", "INTEGER_LITERAL", "STRING_LITERAL", "IDENTIFIER", "NEWLINE",
        "COMMENT", "WS", "DIGIT", "ESC_SEQ"
    };
}

public static final String[] ruleNames = makeRuleNames();

private static String[] makeLiteralNames() {
    return new String[] {
        null, "'def'", "'return'", "'if'", "'elif'", "'else'", "'for'", "'in'",
        "'while'", "'true'", "'false'", "'none'", "'and'", "'or'", "'not'", "'=='",
        "'!='", "'<='", "'>='", "'='", "'<'", "'>'", "'+'", "'-'", "'*'", "'/'",
        "'%'", "'('", "')'", "'['", "']'", "'{'", "'}'", "':'", "';'", "'.'",
        "'@'"
    };
}

private static final String[] _LITERAL_NAMES = makeLiteralNames();
private static String[] makeSymbolicNames() {
    return new String[] {
        null, "DEF", "RETURN", "IF", "ELIF", "ELSE", "FOR", "IN", "WHILE", "TRUE",
        "FALSE", "NONE", "AND", "OR", "NOT", "EQ", "NE", "LE", "GE", "ASSIGN",
        "LT", "GT", "PLUS", "MINUS", "STAR", "SLASH", "MOD", "LPAREN", "RPAREN",
        "LBRACK", "RBRACK", "LBRACE", "RBRACE", "COLON", "COMMA", "DOT", "AT",
        "FLOAT_LITERAL", "INTEGER_LITERAL", "STRING_LITERAL", "IDENTIFIER", "NEWLINE",
        "COMMENT", "WS"
    };
}

private static final String[] _SYMBOLIC_NAMES = makeSymbolicNames();
public static final Vocabulary VOCABULARY = new VocabularyImpl(_LITERAL_NAMES, _SYMBOLIC_NAMES);

/**
 * @deprecated Use {@link #VOCABULARY} instead.
 */
@Deprecated
public static final String[] tokenNames;
static {
    tokenNames = new String[_SYMBOLIC_NAMES.length];
    for (int i = 0; i < tokenNames.length; i++) {
        tokenNames[i] = VOCABULARY.getLiteralName(i);
        if (tokenNames[i] == null) {
            tokenNames[i] = VOCABULARY.getSymbolicName(i);
        }
    }
}

```

[illegible]

[illegible]

"\u0000ij\u0005e\u0000\u0000jk\u0005l\u0000\u0000kl\u0005i\u0000\u0000"+
"lm\u0005f\u0000\u0000m\b\u0001\u0000\u0000\u0000no\u0005e\u0000\u0000"+
"op\u0005l\u0000\u0000pq\u0005s\u0000\u0000qr\u0005e\u0000\u0000r\n\u0001"+
"\u0000\u0000\u0000st\u0005f\u0000\u0000tu\u0005o\u0000\u0000uv\u0005x"+
"\u0000\u0000v\f\u0001\u0000\u0000\u0000wx\u0005i\u0000\u0000xy\u0005n"+
"\u0000\u0000y\u000e\u0001\u0000\u0000\u0000z{\u0005w\u0000\u0000{|\u0005"+
"h\u0000\u0000}|}\u0005i\u0000\u0000}\u0005l\u0000\u0000\u0000\u0007f\u0005e"+
"\u0000\u0000\u0007f\u0010\u0001\u0000\u0000\u0000\u0000\u0080\u0081\u0005T\u0000"+
"\u0000\u0081\u0082\u0005x\u0000\u0000\u0082\u0083\u0005u\u0000\u0000\u0083"+
"\u0084\u0005e\u0000\u0000\u0084\u0012\u0001\u0000\u0000\u0000\u0085\u0086"+
"\u0005F\u0000\u0000\u0086\u0087\u0005a\u0000\u0000\u0087\u0088\u00051"+
"\u0000\u0000\u0088\u0089\u0005s\u0000\u0000\u0089\u008a\u0005e\u0000\u0000"+
"\u008a\u0014\u0001\u0000\u0000\u0000\u0000\u008b\u008c\u0005N\u0000\u0000\u008c"+
"\u008d\u0005o\u0000\u0000\u008d\u008e\u0005n\u0000\u0000\u008e\u008f\u0005"+
"e\u0000\u0000\u008f\u0016\u0001\u0000\u0000\u0000\u0090\u0091\u0005a\u0000"+
"\u0000\u0091\u0092\u0005n\u0000\u0000\u0092\u0093\u0005d\u0000\u0000\u0093"+
"\u0018\u0001\u0000\u0000\u0000\u0094\u0095\u0005o\u0000\u0000\u0095\u0096"+
"\u0005x\u0000\u0000\u0096\u001a\u0001\u0000\u0000\u0000\u0097\u0098\u0005"+
"n\u0000\u0000\u0098\u0099\u0005o\u0000\u0000\u0099\u009a\u0005t\u0000"+
"\u0000\u009a\u001c\u0001\u0000\u0000\u0000\u009b\u009c\u0005=\u0000\u0000"+
"\u009c\u009d\u0005=\u0000\u0000\u009d\u001e\u0001\u0000\u0000\u0000\u009e"+
"\u009f\u00005!\u0000\u0000\u009f\u00a0\u0005=\u0000\u0000\u00a0 \u0001"+
"\u0000\u0000\u0000\u00a1\u00a2\u00005<\u0000\u0000\u00a2\u00a3\u0005=\u0000"+
"\u0000\u00a3\"'\u0001\u0000\u0000\u0000\u0000\u00a4\u00a5\u00005>\u0000\u0000"+
"\u00a5\u00a6\u00005=\u0000\u0000\u00a6\$\u0001\u0000\u0000\u0000\u00a7\u00a8"+
"\u0005=\u0000\u0000\u0000\u00a8&\u0001\u0000\u0000\u0000\u0000\u00a9\u00aa\u00005<"+
"\u0000\u0000\u0000\u00aa{\u0001\u0000\u0000\u0000\u0000\u00ab\u00ac\u00005>\u0000\u0000"+
"\u00ac*\u0001\u0000\u0000\u0000\u00ad\u00ae\u00005+\u0000\u0000\u0000\u00ae,\""+
"\u0001\u0000\u0000\u0000\u0000\u00af\u00b0\u0005-\u0000\u0000\u00b0.\u0001\u0000"+
"\u0000\u0000\u00b1\u00b2\u00005*\u0000\u0000\u00b20\u0001\u0000\u0000\u0000\u0000"+
"\u00b3\u00b4\u00005/\u0000\u0000\u00b42\u0001\u0000\u0000\u0000\u0000\u00b5\u00b6"+
"\u0005%\u0000\u0000\u00b64\u0001\u0000\u0000\u0000\u0000\u00b7\u00b8\u00005(\"+
"\u0000\u0000\u0000\u00b86\u0001\u0000\u0000\u0000\u0000\u00b9\u00ba\u00005)\u0000\u0000"+
"\u00ba8\u0001\u0000\u0000\u0000\u0000\u00bb\u00bc\u00005[\u0000\u0000\u00bc.\"+
"\u0001\u0000\u0000\u0000\u0000\u00bd\u00be\u00005]\u0000\u0000\u00be<\u0001\u0000"+
"\u0000\u0000\u0000\u00bf\u00c0\u00005{\u0000\u0000\u0000\u00c0>\u0001\u0000\u0000\u0000\u0000"+
"\u00c1\u00c2\u00005}\u0000\u0000\u00c2@ \u0001\u0000\u0000\u0000\u0000\u00c3\u00c4"+
"\u00005:\u0000\u0000\u0000\u00c4B\u0001\u0000\u0000\u0000\u0000\u00c5\u00c6\u00005,\"+
"\u0000\u0000\u0000\u00c6D\u0001\u0000\u0000\u0000\u0000\u00c7\u00c8\u00005.\u0000\u0000\u0000"+
"\u00c8F\u0001\u0000\u0000\u0000\u0000\u00c9\u00ca\u00005@ \u0000\u0000\u00caH"+
"\u0001\u0000\u0000\u0000\u0000\u00cb\u00cd\u00003W+\u0000\u0000\u00cc\u00cb\u0001\u0000"+
"\u0000\u0000\u0000\u00cd\u00ce\u0001\u0000\u0000\u0000\u0000\u00ce\u00cc\u0001\u0000"+
"\u0000\u0000\u0000\u00ce\u00cf\u0001\u0000\u0000\u0000\u0000\u00cf\u00d0\u0001\u0000"+
"\u0000\u0000\u0000\u00d0\u00d2\u00005.\u0000\u0000\u0000\u00d1\u00d3\u00003W+\u0000"+
"\u00d2\u00d1\u0001\u0000\u0000\u0000\u0000\u00d3\u00d4\u0001\u0000\u0000\u0000\u0000"+
"\u00d4\u00d2\u0001\u0000\u0000\u0000\u0000\u00d4\u00d5\u0001\u0000\u0000\u0000\u0000"+
"\u00d5J\u0001\u0000\u0000\u0000\u0000\u00d6\u00d8\u00003W+\u0000\u0000\u00d7\u00d6"+
"\u0001\u0000\u0000\u0000\u0000\u00d8\u00d9\u0001\u0000\u0000\u0000\u0000\u0000\u00d9\u00d7"+
"\u0001\u0000\u0000\u0000\u0000\u0000\u00d9\u00da\u0001\u0000\u0000\u0000\u0000\u0000\u00daI\u0001"+
"\u0000\u0000\u0000\u0000\u00db\u00e0\u00005\"'\u0000\u0000\u00dc\u00df\u00003Y\"+
"/\u0000\u0000\u00dd\u00df\b\u0000\u0000\u0000\u00de\u00dc\u0001\u0000\u0000\u0000"+
"\u0000\u00de\u00dd\u0001\u0000\u0000\u0000\u0000\u00df\u00e2\u0001\u0000\u0000\u0000"+
"\u0000\u00e0\u00de\u0001\u0000\u0000\u0000\u0000\u00e0\u00e1\u0001\u0000\u0000\u0000"+
"\u0000\u00e1\u00e3\u0001\u0000\u0000\u0000\u0000\u00e2\u00e0\u0001\u0000\u0000\u0000"+
"\u0000\u00e3\u00e0e\u00005\"'\u0000\u0000\u00e4\u00e9\u00005\"'\u0000\u0000\u0000"+
"\u00e5\u00e8\u00003Y,\u0000\u00e6\u00e8\b\u0001\u0000\u0000\u0000\u00e7\u00e5"+
"\u0001\u0000\u0000\u0000\u0000\u00e7\u00e6\u0001\u0000\u0000\u0000\u0000\u00e8\u00eb"+
"\u0001\u0000\u0000\u0000\u0000\u00e9\u00e7\u0001\u0000\u0000\u0000\u0000\u00e9\u00ea"+
"\u0001\u0000\u0000\u0000\u0000\u00ea\u00ec\u0001\u0000\u0000\u0000\u0000\u00eb\u00e9"+
"\u0001\u0000\u0000\u0000\u0000\u0000\u00ec\u00ee\u00005\"'\u0000\u0000\u00ed\u00db\u0001"+
"\u0000\u0000\u0000\u0000\u00ed\u00e4\u0001\u0000\u0000\u0000\u0000\u00eeN\u0001\u0000"+
"\u0000\u0000\u0000\u00ef\u00f3\u0007\u0002\u0000\u0000\u0000\u00f0\u00f2\u0007\u0003"+
"\u0000\u0000\u0000\u00f1\u00f0\u0001\u0000\u0000\u0000\u00f2\u00f5\u0001\u0000\u0000"+
"\u0000\u0000\u0000\u00f3\u00f1\u0001\u0000\u0000\u0000\u0000\u00f3\u00f4\u0001\u0000\u0000"+
"\u0000\u0000\u0000\u00f4P\u0001\u0000\u0000\u0000\u0000\u00f5\u00f3\u0001\u0000\u0000\u0000"+
"\u0000\u0000\u00f6\u00f8\u00005x\u0000\u0000\u0000\u00f7\u00f6\u0001\u0000\u0000\u0000\u0000"+

```

"\u00f7\u00f8\u0001\u0000\u0000\u0000\u0000\u00f8\u00f9\u0001\u0000\u0000\u0000\u0000"+
"\u00f9\u00fc\u0005\n\u0000\u0000\u00fa\u00fc\u0005\r\u0000\u0000\u00fb"+
"\u00f7\u0001\u0000\u0000\u0000\u0000\u00fb\u00fa\u0001\u0000\u0000\u0000\u00fc"+
"R\u0001\u0000\u0000\u0000\u00fd\u0101\u0005#\u0000\u0000\u00fe\u0100\b"+
"\u0004\u0000\u0000\u00ff\u00fe\u0001\u0000\u0000\u0000\u0100\u0103\u0001"+
"\u0000\u0000\u0000\u0101\u00ff\u0001\u0000\u0000\u0000\u0101\u0102\u0001"+
"\u0000\u0000\u0000\u0102\u0104\u0001\u0000\u0000\u0000\u0103\u0101\u0001"+
"\u0000\u0000\u0000\u0104\u0105\u0006)\u0000\u0000\u0105T\u0001\u0000\u0000"+
"\u0000\u0106\u0108\u0007\u0005\u0000\u0000\u0107\u0106\u0001\u0000\u0000"+
"\u0000\u0108\u0109\u0001\u0000\u0000\u0000\u0109\u0107\u0001\u0000\u0000"+
"\u0000\u0109\u010a\u0001\u0000\u0000\u0000\u010a\u010b\u0001\u0000\u0000"+
"\u0000\u010b\u010c\u0006*\u0000\u0000\u010cV\u0001\u0000\u0000\u0000\u010d"+
"\u010e\u0007\u0006\u0000\u0000\u010eX\u0001\u0000\u0000\u0000\u010f\u0110"+
"\u0005\\u0000\u0000\u0110\u0111\t\u0000\u0000\u0000\u0111Z\u0001\u0000"+
"\u0000\u0000\u000e\u0000\u0000\u00ce\u00d4\u00d9\u00de\u00e0\u00e7\u00e9\u00ed"+
"\u00f3\u00f7\u00fb\u0101\u0109\u0001\u0006\u0000\u0000";

public static final ATN _ATN =
    new ATNDeserializer().deserialize(_serializedATN.toCharArray());
static {
    _decisionToDFA = new DFA[_ATN.getNumberOfDecisions()];
    for (int i = 0; i < _ATN.getNumberOfDecisions(); i++) {
        _decisionToDFA[i] = new DFA(_ATN.getDecisionState(i), i);
    }
}
}

```

}).

?? File: PythonParser.java

```

// Generated from D:/Compiler/PyJinjaCompiler/src/grammar/python/PythonParser.g4 by ANTLR 4.13.2 package compiler.generated.python; import
org.antlr.v4.runtime.atn.; import org.antlr.v4.runtime.dfa.DFA; import org.antlr.v4.runtime.; import org.antlr.v4.runtime.misc.; import org.antlr.v4.runtime.tree.; import
java.util.List; import java.util.Iterator; import java.util.ArrayList;

```

```

@SuppressWarnings({"all", "warnings", "unchecked", "unused", "cast", "CheckReturnValue", "this-escape"}) public class PythonParser extends Parser { static {
RuntimeMetaData.checkVersion("4.13.2", RuntimeMetaData.VERSION); }

```

```

protected static final DFA[] _decisionToDFA;
protected static final PredictionContextCache _sharedContextCache =
    new PredictionContextCache();
public static final int
    DEF=1, RETURN=2, IF=3, ELIF=4, ELSE=5, FOR=6, IN=7, WHILE=8, TRUE=9, FALSE=10,
    NONE=11, AND=12, OR=13, NOT=14, EQ=15, NE=16, LE=17, GE=18, ASSIGN=19,
    LT=20, GT=21, PLUS=22, MINUS=23, STAR=24, SLASH=25, MOD=26, LPAREN=27,
    RPAREN=28, LBRACK=29, RBRACK=30, LBRACE=31, RBRACE=32, COLON=33, COMMA=34,
    DOT=35, AT=36, FLOAT_LITERAL=37, INTEGER_LITERAL=38, STRING_LITERAL=39,
    IDENTIFIER=40, NEWLINE=41, COMMENT=42, WS=43, INDENT=44, DEDENT=45;
public static final int
    RULE_program = 0, RULE_statement = 1, RULE_decorator = 2, RULE_paramList = 3,
    RULE_suite = 4, RULE_simpleLine = 5, RULE_expr = 6, RULE_listItems = 7,
    RULE_atom = 8, RULE_atomTrailer = 9, RULE_argList = 10, RULE_argument = 11,
    RULE_dictItems = 12, RULE_dictItem = 13;
private static String[] makeRuleNames() {
    return new String[] {
        "program", "statement", "decorator", "paramList", "suite", "simpleLine",
        "expr", "listItems", "atom", "atomTrailer", "argList", "argument", "dictItems",
        "dictItem"
    };
}
public static final String[] ruleNames = makeRuleNames();

private static String[] makeLiteralNames() {
    return new String[] {
        null, "def", "return", "if", "elif", "else", "for", "in",
        "while", "True", "False", "None", "and", "or", "not", "==",
        "!=", "<=", ">=", "=", "<", ">", "+", "-", "*", "/",
        "%", "(", ")", "[", "]", "{", "}", ":", ";", ",", ".",
        "@",
    };
}
private static final String[] _LITERAL_NAMES = makeLiteralNames();
private static String[] makeSymbolicNames() {
    return new String[] {
        null, "DEF", "RETURN", "IF", "ELIF", "ELSE", "FOR", "IN", "WHILE", "TRUE",
        "FALSE", "NONE", "AND", "OR", "NOT", "EQ", "NE", "LE", "GE", "ASSIGN",
        "LT", "GT", "PLUS", "MINUS", "STAR", "SLASH", "MOD", "LPAREN", "RPAREN",
        "LBRACK", "RBRACK", "LBRACE", "RBRACE", "COLON", "COMMA", "DOT", "AT",
        "FLOAT_LITERAL", "INTEGER_LITERAL", "STRING_LITERAL", "IDENTIFIER", "NEWLINE",
        "COMMENT", "WS", "INDENT", "DEDENT"
    };
}
private static final String[] _SYMBOLIC_NAMES = makeSymbolicNames();
public static final Vocabulary VOCABULARY = new VocabularyImpl(_LITERAL_NAMES, _SYMBOLIC_NAMES);

/**
 * @deprecated Use {@link #VOCABULARY} instead.
 */
@Deprecated
public static final String[] tokenNames;
static {
    tokenNames = new String[_SYMBOLIC_NAMES.length];
    for (int i = 0; i < tokenNames.length; i++) {
        tokenNames[i] = VOCABULARY.getLiteralName(i);
        if (tokenNames[i] == null) {
            tokenNames[i] = VOCABULARY.getSymbolicName(i);
        }

        if (tokenNames[i] == null) {
            tokenNames[i] = "<INVALID>";
        }
    }
}

```

```

@Override
@Deprecated
public String[] getTokenNames() {
    return tokenNames;
}

@Override

public Vocabulary getVocabulary() {
    return VOCABULARY;
}

@Override

public String getGrammarFileName() { return "PythonParser.g4"; }

@Override

public String[] getRuleNames() { return ruleNames; }

@Override

public String getSerializedATN() { return _serializedATN; }

@Override

public ATN getATN() { return _ATN; }

public PythonParser(TokenStream input) {
    super(input);
    _interp = new ParserATNSimulator(this, _ATN, _decisionToDFA, _sharedContextCache);
}

@SuppressWarnings("CheckReturnValue")
public static class ProgramContext extends ParserRuleContext {
    public TerminalNode EOF() { return getToken(PythonParser.EOF, 0); }
    public List<StatementContext> statement() {
        return getRuleContexts(StatementContext.class);
    }
    public StatementContext statement(int i) {
        return getRuleContext(StatementContext.class,i);
    }
    public List<TerminalNode> NEWLINE() { return getTokens(PythonParser.NEWLINE); }
    public TerminalNode NEWLINE(int i) {
        return getToken(PythonParser.NEWLINE, i);
    }
    public ProgramContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }
    @Override public int getRuleIndex() { return RULE_program; }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterProgram(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitProgram(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitPro
        else return visitor.visitChildren(this);
    }
}

public final ProgramContext program() throws RecognitionException {
    ProgramContext _localctx = new ProgramContext(_ctx, getState());
    enterRule(_localctx, 0, RULE_program);
    int _la;

```

```

try {
    enterOuterAlt(_localctx, 1);
    {
        setState(32);
        _errHandler.sync(this);
        _la = _input.LA(1);
        while (((_la) & ~0x3f) == 0 && ((1L << _la) & 4332154015566L) != 0)) {
            {
                setState(30);
                _errHandler.sync(this);
                switch ( getInterpreter().adaptivePredict(_input,0,_ctx) ) {
                    case 1:
                        {
                            setState(28);
                            statement();
                        }
                        break;
                    case 2:
                        {
                            setState(29);
                            match(NEWLINE);
                        }
                        break;
                }
            }
            setState(34);
            _errHandler.sync(this);
            _la = _input.LA(1);
        }
        setState(35);
        match(EOF);
    }
}

catch (RecognitionException re) {
    _localctx.exception = re;
    _errHandler.reportError(this, re);
    _errHandler.recover(this, re);
}

finally {
    exitRule();
}

return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class StatementContext extends ParserRuleContext {
    public StatementContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_statement; }

    public StatementContext() { }
    public void copyFrom(StatementContext ctx) {
        super.copyFrom(ctx);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class WhileStatementContext extends StatementContext {
    public TerminalNode WHILE() { return getToken(PythonParser.WHILE, 0); }
    public ExprContext expr() {
        return getRuleContext(ExprContext.class,0);
    }

    public TerminalNode COLON() { return getToken(PythonParser.COLON, 0); }
    public SuiteContext suite() {
        return getRuleContext(SuiteContext.class,0);
    }
}

```



```

        public WhileStatementContext(StatementContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterWhileStatement(this)
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitWhileStatement(this)
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitWhileStatement(this)
            else return visitor.visitChildren(this);
        }
    }
    @SuppressWarnings("CheckReturnValue")
    public static class AssignOrExprStatementContext extends StatementContext {
        public List<ExprContext> expr() {
            return getRuleContexts(ExprContext.class);
        }
        public ExprContext expr(int i) {
            return getRuleContext(ExprContext.class,i);
        }
        public List<TerminalNode> ASSIGN() { return getTokens(PythonParser.ASSIGN); }
        public TerminalNode ASSIGN(int i) {
            return getToken(PythonParser.ASSIGN, i);
        }
        public TerminalNode NEWLINE() { return getToken(PythonParser.NEWLINE, 0); }
        public AssignOrExprStatementContext(StatementContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterAssignOrExprStatement(this)
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitAssignOrExprStatement(this)
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitAssignOrExprStatement(this)
            else return visitor.visitChildren(this);
        }
    }
    @SuppressWarnings("CheckReturnValue")
    public static class FunctionDefContext extends StatementContext {
        public TerminalNode DEF() { return getToken(PythonParser.DEF, 0); }
        public TerminalNode IDENTIFIER() { return getToken(PythonParser.IDENTIFIER, 0); }
        public TerminalNode LPAREN() { return getToken(PythonParser.LPAREN, 0); }
        public TerminalNode RPAREN() { return getToken(PythonParser.RPAREN, 0); }
        public TerminalNode COLON() { return getToken(PythonParser.COLON, 0); }
        public SuiteContext suite() {
            return getRuleContext(SuiteContext.class,0);
        }
        public List<DecoratorContext> decorator() {
            return getRuleContexts(DecoratorContext.class);
        }
        public DecoratorContext decorator(int i) {
            return getRuleContext(DecoratorContext.class,i);
        }
        public TerminalNode NEWLINE() { return getToken(PythonParser.NEWLINE, 0); }
        public ParamListContext paramList() {
            return getRuleContext(ParamListContext.class,0);
        }
        public FunctionDefContext(StatementContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {

```

```

        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterFunctionDef(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitFunctionDef(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitFun
        else return visitor.visitChildren(this);
    }
}
@SuppressWarnings("CheckReturnValue")
public static class ForStatementContext extends StatementContext {
    public TerminalNode FOR() { return getToken(PythonParser.FOR, 0); }
    public TerminalNode IDENTIFIER() { return getToken(PythonParser.IDENTIFIER, 0); }
    public TerminalNode IN() { return getToken(PythonParser.IN, 0); }
    public ExprContext expr() {
        return getRuleContext(ExprContext.class,0);
    }
    public TerminalNode COLON() { return getToken(PythonParser.COLON, 0); }
    public SuiteContext suite() {
        return getRuleContext(SuiteContext.class,0);
    }
    public ForStatementContext(StatementContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterForStatement(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitForStatement(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitFor
        else return visitor.visitChildren(this);
    }
}
@SuppressWarnings("CheckReturnValue")
public static class ReturnStatementContext extends StatementContext {
    public TerminalNode RETURN() { return getToken(PythonParser.RETURN, 0); }
    public List<ExprContext> expr() {
        return getRuleContexts(ExprContext.class);
    }
    public ExprContext expr(int i) {
        return getRuleContext(ExprContext.class,i);
    }
    public List<TerminalNode> COMMA() { return getTokens(PythonParser.COMMA); }
    public TerminalNode COMMA(int i) {
        return getToken(PythonParser.COMMA, i);
    }
    public TerminalNode NEWLINE() { return getToken(PythonParser.NEWLINE, 0); }
    public ReturnStatementContext(StatementContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterReturnStatement(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitReturnStatement(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitRet
        else return visitor.visitChildren(this);
    }
}

```

```

    }
}

@SuppressWarnings("CheckReturnValue")
public static class IfStatementContext extends StatementContext {
    public TerminalNode IF() { return getToken(PythonParser.IF, 0); }
    public List<ExprContext> expr() {
        return getRuleContexts(ExprContext.class);
    }
    public ExprContext expr(int i) {
        return getRuleContext(ExprContext.class,i);
    }
    public List<TerminalNode> COLON() { return getTokens(PythonParser.COLON); }
    public TerminalNode COLON(int i) {
        return getToken(PythonParser.COLON, i);
    }
    public List<SuiteContext> suite() {
        return getRuleContexts(SuiteContext.class);
    }
    public SuiteContext suite(int i) {
        return getRuleContext(SuiteContext.class,i);
    }
    public List<TerminalNode> ELIF() { return getTokens(PythonParser.ELIF); }
    public TerminalNode ELIF(int i) {
        return getToken(PythonParser.ELIF, i);
    }
    public TerminalNode ELSE() { return getToken(PythonParser.ELSE, 0); }
    public IfStatementContext(StatementContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterIfStatement(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitIfStatement(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitIfS
        else return visitor.visitChildren(this);
    }
}

public final StatementContext statement() throws RecognitionException {
    StatementContext _localctx = new StatementContext(_ctx, getState());
    enterRule(_localctx, 2, RULE_statement);
    int _la;
    try {
        setState(109);
        _errHandler.sync(this);
        switch (_input.LA(1)) {
            case RETURN:
                _localctx = new ReturnStatementContext(_localctx);
                enterOuterAlt(_localctx, 1);
                {
                    setState(37);
                    match(RETURN);
                    setState(38);
                    expr(0);
                    setState(43);
                    _errHandler.sync(this);
                    _la = _input.LA(1);
                    while (_la==COMMA) {
                        {
                            {
                                setState(39);
                                match(COMMA);

```

```

        setState(40);
        expr(0);
    }
}

setState(45);
_errHandler.sync(this);
_la = _input.LA(1);
}

setState(47);
_errHandler.sync(this);
switch ( getInterpreter().adaptivePredict(_input,3,_ctx) ) {
case 1:
    {
        setState(46);
        match(NEWLINE);
    }
    break;
}
}
break;

case TRUE:
case FALSE:
case NONE:
case NOT:
case MINUS:
case LPAREN:
case LBRACK:
case LBRACE:
case FLOAT_LITERAL:
case INTEGER_LITERAL:
case STRING_LITERAL:
case IDENTIFIER:
    _localctx = new AssignOrExprStatementContext(_localctx);
    enterOuterAlt(_localctx, 2);
    {
        setState(49);
        expr(0);
        setState(54);
        _errHandler.sync(this);
        _la = _input.LA(1);
        while (_la==ASSIGN) {
            {
                {
                    setState(50);
                    match(ASSIGN);
                    setState(51);
                    expr(0);
                }
            }
            setState(56);
            _errHandler.sync(this);
            _la = _input.LA(1);
        }
        setState(58);
        _errHandler.sync(this);
        switch ( getInterpreter().adaptivePredict(_input,5,_ctx) ) {
case 1:
            {
                setState(57);
                match(NEWLINE);
            }
            break;
        }
    }
    break;

case DEF:

```

```

case AT:
case NEWLINE:
    _localctx = new FunctionDefContext(_localctx);
    enterOuterAlt(_localctx, 3);
    {
        setState(63);
        _errHandler.sync(this);
        _la = _input.LA(1);
        while (_la==AT) {
            {
                {
                    setState(60);
                    decorator();
                }
            }
            setState(65);
            _errHandler.sync(this);
            _la = _input.LA(1);
        }
        setState(67);
        _errHandler.sync(this);
        _la = _input.LA(1);
        if (_la==NEWLINE) {
            {
                setState(66);
                match(NEWLINE);
            }
        }

        setState(69);
        match(DEF);
        setState(70);
        match(IDENTIFIER);
        setState(71);
        match(LPAREN);
        setState(73);
        _errHandler.sync(this);
        _la = _input.LA(1);
        if (_la==IDENTIFIER) {
            {
                setState(72);
                paramList();
            }
        }

        setState(75);
        match(RPAREN);
        setState(76);
        match(COLON);
        setState(77);
        suite();
    }
    break;
case IF:
    _localctx = new IfStatementContext(_localctx);
    enterOuterAlt(_localctx, 4);
    {
        setState(78);
        match(IF);
        setState(79);
        expr(0);
        setState(80);
        match(COLON);
        setState(81);
        suite();
        setState(89);
    }

```

```

_errHandler.sync(this);
_la = _input.LA(1);
while (_la==ELIF) {
    {
        {
            setState(82);
            match(ELIF);
            setState(83);
            expr(0);
            setState(84);
            match(COLON);
            setState(85);
            suite();
        }
    }
    setState(91);
    _errHandler.sync(this);
    _la = _input.LA(1);
}
setState(95);
_errHandler.sync(this);
_la = _input.LA(1);
if (_la==ELSE) {
    {
        setState(92);
        match(ELSE);
        setState(93);
        match(COLON);
        setState(94);
        suite();
    }
}

}

break;
case WHILE:
    _localctx = new WhileStatementContext(_localctx);
    enterOuterAlt(_localctx, 5);
    {
        setState(97);
        match(WHILE);
        setState(98);
        expr(0);
        setState(99);
        match(COLON);
        setState(100);
        suite();
    }
    break;
case FOR:
    _localctx = new ForStatementContext(_localctx);
    enterOuterAlt(_localctx, 6);
    {
        setState(102);
        match(FOR);
        setState(103);
        match(IDENTIFIER);
        setState(104);
        match(IN);
        setState(105);
        expr(0);
        setState(106);
        match(COLON);
        setState(107);
        suite();
    }
}

```

```

        break;
    default:
        throw new NoViableAltException(this);
    }
}

catch (RecognitionException re) {
    _localctx.exception = re;
    _errHandler.reportError(this, re);
    _errHandler.recover(this, re);
}

finally {
    exitRule();
}

return _localctx;
}

}

@SuppressWarnings("CheckReturnValue")
public static class DecoratorContext extends ParserRuleContext {
    public TerminalNode AT() { return getToken(PythonParser.AT, 0); }
    public List<TerminalNode> IDENTIFIER() { return getTokens(PythonParser.IDENTIFIER); }
    public TerminalNode IDENTIFIER(int i) {
        return getToken(PythonParser.IDENTIFIER, i);
    }
    public TerminalNode LPAREN() { return getToken(PythonParser.LPAREN, 0); }
    public TerminalNode RPAREN() { return getToken(PythonParser.RPAREN, 0); }
    public List<TerminalNode> DOT() { return getTokens(PythonParser.DOT); }
    public TerminalNode DOT(int i) {
        return getToken(PythonParser.DOT, i);
    }
    public ArgListContext argList() {
        return getRuleContext(ArgListContext.class,0);
    }
    public DecoratorContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }
    @Override public int getRuleIndex() { return RULE_decorator; }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterDecorator(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitDecorator(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitDeco
        else return visitor.visitChildren(this);
    }
}

}

public final DecoratorContext decorator() throws RecognitionException {
    DecoratorContext _localctx = new DecoratorContext(_ctx, getState());
    enterRule(_localctx, 4, RULE_decorator);
    int _la;
    try {
        enterOuterAlt(_localctx, 1);
        {
            setState(111);
            match(AT);
            setState(112);
            match(IDENTIFIER);
            setState(117);
            _errHandler.sync(this);
            _la = _input.LA(1);
            while (_la==DOT) {

```

```

        {
        {
            setState(113);
            match(DOT);
            setState(114);
            match(IDENTIFIER);
        }
        }

        setState(119);
        _errHandler.sync(this);
        _la = _input.LA(1);
    }

    setState(120);
    match(LPAREN);
    setState(122);
    _errHandler.sync(this);
    _la = _input.LA(1);
    if ((((_la) & ~0x3f) == 0 && ((1L << _la) & 2064411282944L) != 0)) {
        {
            setState(121);
            argList();
        }
    }

    setState(124);
    match(RPAREN);
}

catch (RecognitionException re) {
    _localctx.exception = re;
    _errHandler.reportError(this, re);
    _errHandler.recover(this, re);
}

finally {
    exitRule();
}

return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class ParamListContext extends ParserRuleContext {
    public List<TerminalNode> IDENTIFIER() { return getTokens(PythonParser.IDENTIFIER); }
    public TerminalNode IDENTIFIER(int i) {
        return getToken(PythonParser.IDENTIFIER, i);
    }

    public List<TerminalNode> COMMA() { return getTokens(PythonParser.COMMA); }
    public TerminalNode COMMA(int i) {
        return getToken(PythonParser.COMMA, i);
    }

    public ParamListContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_paramList; }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterParamList(this);
    }

    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitParamList(this);
    }

    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitParamList(this);
        else return visitor.visitChildren(this);
    }
}

```



```

    }

    public final ParamListContext paramList() throws RecognitionException {
        ParamListContext _localctx = new ParamListContext(_ctx, getState());
        enterRule(_localctx, 6, RULE_paramList);
        int _la;
        try {
            enterOuterAlt(_localctx, 1);
            {
                setState(126);
                match(IDENTIFIER);
                setState(131);
                _errHandler.sync(this);
                _la = _input.LA(1);
                while (_la==COMMA) {
                    {
                        {
                            setState(127);
                            match(COMMA);
                            setState(128);
                            match(IDENTIFIER);
                        }
                    }
                    setState(133);
                    _errHandler.sync(this);
                    _la = _input.LA(1);
                }
            }
        }
        catch (RecognitionException re) {
            _localctx.exception = re;
            _errHandler.reportError(this, re);
            _errHandler.recover(this, re);
        }
        finally {
            exitRule();
        }
        return _localctx;
    }

    @SuppressWarnings("CheckReturnValue")
    public static class SuiteContext extends ParserRuleContext {
        public SuiteContext(ParserRuleContext parent, int invokingState) {
            super(parent, invokingState);
        }

        @Override public int getRuleIndex() { return RULE_suite; }

        public SuiteContext() { }
        public void copyFrom(SuiteContext ctx) {
            super.copyFrom(ctx);
        }
    }

    @SuppressWarnings("CheckReturnValue")
    public static class IndentedSuiteContext extends SuiteContext {
        public TerminalNode NEWLINE() { return getToken(PythonParser.NEWLINE, 0); }
        public TerminalNode INDENT() { return getToken(PythonParser.INDENT, 0); }
        public TerminalNode DEDENT() { return getToken(PythonParser.DEDENT, 0); }
        public List<StatementContext> statement() {
            return getRuleContexts(StatementContext.class);
        }

        public StatementContext statement(int i) {
            return getRuleContext(StatementContext.class,i);
        }

        public IndentedSuiteContext(SuiteContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {

```

```

        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterIndentedSuite(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitIndentedSuite(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitIndentedSuite(this);
        else return visitor.visitChildren(this);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class SingleSuiteContext extends SuiteContext {
    public SimpleLineContext simpleLine() {
        return getRuleContext(SimpleLineContext.class,0);
    }
    public SingleSuiteContext(SuiteContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterSingleSuite(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitSingleSuite(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitSingleSuite(this);
        else return visitor.visitChildren(this);
    }
}

public final SuiteContext suite() throws RecognitionException {
    SuiteContext _localctx = new SuiteContext(_ctx, getState());
    enterRule(_localctx, 8, RULE_suite);
    int _la;
    try {
        setState(144);
        _errHandler.sync(this);
        switch (_input.LA(1)) {
            case TRUE:
            case FALSE:
            case NONE:
            case NOT:
            case MINUS:
            case LPAREN:
            case LBRACK:
            case LBRACE:
            case FLOAT_LITERAL:
            case INTEGER_LITERAL:
            case STRING_LITERAL:
            case IDENTIFIER:
                _localctx = new SingleSuiteContext(_localctx);
                enterOuterAlt(_localctx, 1);
                {
                    setState(134);
                    simpleLine();
                }
                break;
            case NEWLINE:
                _localctx = new IndentedSuiteContext(_localctx);
                enterOuterAlt(_localctx, 2);
                {
                    setState(135);
                    match(NEWLINE);
                }
            default:
                throw new RecognitionException(_localctx, _errHandler, _input);
        }
    } catch (RecognitionException re) {
        reportError(re);
        recover(re);
    }
}

```

```

        setState(136);
        match(INDENT);
        setState(138);
        _errHandler.sync(this);
        _la = _input.LA(1);
        do {
            {
                {
                    setState(137);
                    statement();
                }
            }
            setState(140);
            _errHandler.sync(this);
            _la = _input.LA(1);
        } while ( (((_la) & ~0x3f) == 0 && ((1L << _la) & 4332154015566L) != 0) );
        setState(142);
        match(DEDENT);
    }
    break;
default:
    throw new NoViableAltException(this);
}
}

catch (RecognitionException re) {
    _localctx.exception = re;
    _errHandler.reportError(this, re);
    _errHandler.recover(this, re);
}

finally {
    exitRule();
}

return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class SimpleLineContext extends ParserRuleContext {
    public List<ExprContext> expr() {
        return getRuleContexts(ExprContext.class);
    }

    public ExprContext expr(int i) {
        return getRuleContext(ExprContext.class,i);
    }

    public List<TerminalNode> ASSIGN() { return getTokens(PythonParser.ASSIGN); }
    public TerminalNode ASSIGN(int i) {
        return getToken(PythonParser.ASSIGN, i);
    }

    public TerminalNode NEWLINE() { return getToken(PythonParser.NEWLINE, 0); }
    public SimpleLineContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_simpleLine; }

    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterSimpleLine(this);
    }

    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitSimpleLine(this);
    }

    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitSimpleLine(this);
        else return visitor.visitChildren(this);
    }
}
}

```

```

public final SimpleLineContext simpleLine() throws RecognitionException {
    SimpleLineContext _localctx = new SimpleLineContext(_ctx, getState());
    enterRule(_localctx, 10, RULE_simpleLine);
    int _la;
    try {
        enterOuterAlt(_localctx, 1);
        {
            setState(146);
            expr(0);
            setState(151);
            _errHandler.sync(this);
            _la = _input.LA(1);
            while (_la==ASSIGN) {
                {
                    {
                        setState(147);
                        match(ASSIGN);
                        setState(148);
                        expr(0);
                    }
                }
                setState(153);
                _errHandler.sync(this);
                _la = _input.LA(1);
            }
            setState(155);
            _errHandler.sync(this);
            switch ( getInterpreter().adaptivePredict(_input,18,_ctx) ) {
                case 1:
                    {
                        setState(154);
                        match(NEWLINE);
                    }
                    break;
            }
        }
    }
    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }
    finally {
        exitRule();
    }
    return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class ExprContext extends ParserRuleContext {
    public ExprContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }
    @Override public int getRuleIndex() { return RULE_expr; }

    public ExprContext() { }
    public void copyFrom(ExprContext ctx) {
        super.copyFrom(ctx);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class NotExprContext extends ExprContext {
    public TerminalNode NOT() { return getToken(PythonParser.NOT, 0); }
    public ExprContext expr() {
        return getRuleContext(ExprContext.class,0);
    }
}

```

```

    }
    public NotExprContext(ExprContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterNotExpr(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitNotExpr(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitNotExpr(this);
        else return visitor.visitChildren(this);
    }
}
@SuppressWarnings("CheckReturnValue")
public static class AddExprContext extends ExprContext {
    public List<ExprContext> expr() {
        return getRuleContexts(ExprContext.class);
    }
    public ExprContext expr(int i) {
        return getRuleContext(ExprContext.class,i);
    }
    public TerminalNode PLUS() { return getToken(PythonParser.PLUS, 0); }
    public TerminalNode MINUS() { return getToken(PythonParser.MINUS, 0); }
    public AddExprContext(ExprContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterAddExpr(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitAddExpr(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitAddExpr(this);
        else return visitor.visitChildren(this);
    }
}
@SuppressWarnings("CheckReturnValue")
public static class UnaryMinusContext extends ExprContext {
    public TerminalNode MINUS() { return getToken(PythonParser.MINUS, 0); }
    public ExprContext expr() {
        return getRuleContext(ExprContext.class,0);
    }
    public UnaryMinusContext(ExprContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterUnaryMinus(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitUnaryMinus(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitUnaryMinus(this);
        else return visitor.visitChildren(this);
    }
}
@SuppressWarnings("CheckReturnValue")
public static class MulExprContext extends ExprContext {
    public List<ExprContext> expr() {
        return getRuleContexts(ExprContext.class);
    }

```

```

    }

    public ExprContext expr(int i) {
        return getRuleContext(ExprContext.class,i);
    }

    public TerminalNode STAR() { return getToken(PythonParser.STAR, 0); }
    public TerminalNode SLASH() { return getToken(PythonParser.SLASH, 0); }
    public TerminalNode MOD() { return getToken(PythonParser.MOD, 0); }
    public MulExprContext(ExprContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterMulExpr(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitMulExpr(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitMulExpr(this);
        else return visitor.visitChildren(this);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class AtomExprContext extends ExprContext {
    public AtomContext atom() {
        return getRuleContext(AtomContext.class,0);
    }
    public AtomExprContext(ExprContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterAtomExpr(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitAtomExpr(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitAtomExpr(this);
        else return visitor.visitChildren(this);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class OrExprContext extends ExprContext {
    public List<ExprContext> expr() {
        return getRuleContexts(ExprContext.class);
    }
    public ExprContext expr(int i) {
        return getRuleContext(ExprContext.class,i);
    }
    public TerminalNode OR() { return getToken(PythonParser.OR, 0); }
    public OrExprContext(ExprContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterOrExpr(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitOrExpr(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitOrExpr(this);
        else return visitor.visitChildren(this);
    }
}
}

```

```

@SuppressWarnings("CheckReturnValue")
public static class ComparisonExprContext extends ExprContext {
    public List<ExprContext> expr() {
        return getRuleContexts(ExprContext.class);
    }

    public ExprContext expr(int i) {
        return getRuleContext(ExprContext.class,i);
    }

    public TerminalNode EQ() { return getToken(PythonParser.EQ, 0); }
    public TerminalNode NE() { return getToken(PythonParser.NE, 0); }
    public TerminalNode LT() { return getToken(PythonParser.LT, 0); }
    public TerminalNode GT() { return getToken(PythonParser.GT, 0); }
    public TerminalNode LE() { return getToken(PythonParser.LE, 0); }
    public TerminalNode GE() { return getToken(PythonParser.GE, 0); }
    public ComparisonExprContext(ExprContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterComparisonExpr(this)
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitComparisonExpr(this)
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitComparisonExpr(this)
        else return visitor.visitChildren(this);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class AndExprContext extends ExprContext {
    public List<ExprContext> expr() {
        return getRuleContexts(ExprContext.class);
    }

    public ExprContext expr(int i) {
        return getRuleContext(ExprContext.class,i);
    }

    public TerminalNode AND() { return getToken(PythonParser.AND, 0); }
    public AndExprContext(ExprContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterAndExpr(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitAndExpr(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitAndExpr(this)
        else return visitor.visitChildren(this);
    }
}

}

public final ExprContext expr() throws RecognitionException {
    return expr(0);
}

private ExprContext expr(int _p) throws RecognitionException {
    ParserRuleContext _parentctx = _ctx;
    int _parentState = getState();
    ExprContext _localctx = new ExprContext(_ctx, _parentState);
    ExprContext _prevctx = _localctx;
    int _startState = 12;
    enterRecursionRule(_localctx, 12, RULE_expr, _p);
    int _la;

```

```

try {
    int _alt;
    enterOuterAlt(_localctx, 1);
    {
        setState(163);
        _errHandler.sync(this);
        switch ( _input.LA(1)) {
        case NOT:
            {
                _localctx = new NotExprContext(_localctx);
                _ctx = _localctx;
                _prevctx = _localctx;

                setState(158);
                match(NOT);
                setState(159);
                expr(6);
            }
            break;
        case MINUS:
            {
                _localctx = new UnaryMinusContext(_localctx);
                _ctx = _localctx;
                _prevctx = _localctx;
                setState(160);
                match(MINUS);
                setState(161);
                expr(2);
            }
            break;
        case TRUE:
        case FALSE:
        case NONE:
        case LPAREN:
        case LBRACK:
        case LBRACE:
        case FLOAT_LITERAL:
        case INTEGER_LITERAL:
        case STRING_LITERAL:
        case IDENTIFIER:
            {
                _localctx = new AtomExprContext(_localctx);
                _ctx = _localctx;
                _prevctx = _localctx;
                setState(162);
                atom();
            }
            break;
        default:
            throw new NoViableAltException(this);
        }

        _ctx.stop = _input.LT(-1);
        setState(182);
        _errHandler.sync(this);
        _alt = getInterpreter().adaptivePredict(_input,21,_ctx);
        while ( _alt!=2 && _alt!=org.antlr.v4.runtime.atn.ATN.INVALID_ALT_NUMBER ) {
            if ( _alt==1 ) {
                if ( _parseListeners!=null ) triggerExitRuleEvent();
                _prevctx = _localctx;
                {
                    setState(180);
                    _errHandler.sync(this);
                    switch ( getInterpreter().adaptivePredict(_input,20,_ctx) ) {
                    case 1:
                        {
                            _localctx = new OrExprContext(new ExprContext(_parentctx, _parentState));

```



```

        pushNewRecursionContext(_localctx, _startState, RULE_expr);
        setState(165);
        if (!(precpred(_ctx, 8))) throw new FailedPredicateException(this, "precpred(_ctx
        setState(166);
        match(OR);
        setState(167);
        expr(9);
    }
    break;
case 2:
    {
        _localctx = new AndExprContext(new ExprContext(_parentctx, _parentState));
        pushNewRecursionContext(_localctx, _startState, RULE_expr);
        setState(168);
        if (!(precpred(_ctx, 7))) throw new FailedPredicateException(this, "precpred(_ctx
        setState(169);
        match(AND);
        setState(170);
        expr(8);
    }
    break;
case 3:
    {
        _localctx = new ComparisonExprContext(new ExprContext(_parentctx, _parentState));
        pushNewRecursionContext(_localctx, _startState, RULE_expr);
        setState(171);
        if (!(precpred(_ctx, 5))) throw new FailedPredicateException(this, "precpred(_ctx
        setState(172);
        _la = _input.LA(1);
        if ( !((( (_la) & ~0x3f) == 0 && ((1L << _la) & 3637248L) != 0)) ) {
            _errHandler.recoverInline(this);
        }
        else {
            if ( _input.LA(1)==Token.EOF ) matchedEOF = true;
            _errHandler.reportMatch(this);
            consume();
        }
        setState(173);
        expr(6);
    }
    break;
case 4:
    {
        _localctx = new AddExprContext(new ExprContext(_parentctx, _parentState));
        pushNewRecursionContext(_localctx, _startState, RULE_expr);
        setState(174);
        if (!(precpred(_ctx, 4))) throw new FailedPredicateException(this, "precpred(_ctx
        setState(175);
        _la = _input.LA(1);
        if ( !(_la==PLUS || _la==MINUS) ) {
            _errHandler.recoverInline(this);
        }
        else {
            if ( _input.LA(1)==Token.EOF ) matchedEOF = true;
            _errHandler.reportMatch(this);
            consume();
        }
        setState(176);
        expr(5);
    }
    break;
case 5:
    {
        _localctx = new MulExprContext(new ExprContext(_parentctx, _parentState));
        pushNewRecursionContext(_localctx, _startState, RULE_expr);
        setState(177);

```

```

        if (!(precpred(_ctx, 3))) throw new FailedPredicateException(this, "precpred(_ctx
        setState(178);
        _la = _input.LA(1);
        if ( !((( _la) & ~0x3f) == 0 && ((1L << _la) & 117440512L) != 0) ) {
            _errHandler.recoverInline(this);
        }
        else {
            if ( _input.LA(1)==Token.EOF ) matchedEOF = true;
            _errHandler.reportMatch(this);
            consume();
        }
        setState(179);
        expr(4);
    }
    break;
}
}
}
    setState(184);
    _errHandler.sync(this);
    _alt = getInterpreter().adaptivePredict(_input,21,_ctx);
}
}

}
catch (RecognitionException re) {
    _localctx.exception = re;
    _errHandler.reportError(this, re);
    _errHandler.recover(this, re);
}
finally {
    unrollRecursionContexts(_parentctx);
}
return _localctx;
}

}

@SuppressWarnings("CheckReturnValue")
public static class ListItemsContext extends ParserRuleContext {
    public List<ExprContext> expr() {
        return getRuleContexts(ExprContext.class);
    }
    public ExprContext expr(int i) {
        return getRuleContext(ExprContext.class,i);
    }
    public List<TerminalNode> COMMA() { return getTokens(PythonParser.COMMA); }
    public TerminalNode COMMA(int i) {
        return getToken(PythonParser.COMMA, i);
    }
    public ListItemsContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }
    @Override public int getRuleIndex() { return RULE_listItems; }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterListItems(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitListItems(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitListItems(this)
        else return visitor.visitChildren(this);
    }
}
}

```

```

public final ListItemsContext listItems() throws RecognitionException {
    ListItemsContext _localctx = new ListItemsContext(_ctx, getState());
    enterRule(_localctx, 14, RULE_listItems);
    int _la;
    try {
        int _alt;
        enterOuterAlt(_localctx, 1);
        {
            setState(185);
            expr(0);
            setState(190);
            _errHandler.sync(this);
            _alt = getInterpreter().adaptivePredict(_input,22,_ctx);
            while ( _alt!=2 && _alt!=org.antlr.v4.runtime.atn.ATN.INVALID_ALT_NUMBER ) {
                if ( _alt==1 ) {
                    {
                        {
                            setState(186);
                            match(COMMA);
                            setState(187);
                            expr(0);
                        }
                    }
                }
                setState(192);
                _errHandler.sync(this);
                _alt = getInterpreter().adaptivePredict(_input,22,_ctx);
            }
            setState(194);
            _errHandler.sync(this);
            _la = _input.LA(1);
            if (_la==COMMA) {
                {
                    {
                        setState(193);
                        match(COMMA);
                    }
                }
            }
        }
    }
    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }
    finally {
        exitRule();
    }
    return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class AtomContext extends ParserRuleContext {
    public AtomContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }
    @Override public int getRuleIndex() { return RULE_atom; }

    public AtomContext() { }
    public void copyFrom(AtomContext ctx) {
        super.copyFrom(ctx);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class FalseLiteralContext extends AtomContext {
    public TerminalNode FALSE() { return getToken(PythonParser.FALSE, 0); }
}

```

```

        public FalseLiteralContext(AtomContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterFalseLiteral(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitFalseLiteral(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitFalseLiteral(this);
            else return visitor.visitChildren(this);
        }
    }
    @SuppressWarnings("CheckReturnValue")
    public static class NoneLiteralContext extends AtomContext {
        public TerminalNode NONE() { return getToken(PythonParser.NONE, 0); }
        public NoneLiteralContext(AtomContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterNoneLiteral(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitNoneLiteral(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitNoneLiteral(this);
            else return visitor.visitChildren(this);
        }
    }
    @SuppressWarnings("CheckReturnValue")
    public static class StringLiteralContext extends AtomContext {
        public TerminalNode STRING_LITERAL() { return getToken(PythonParser.STRING_LITERAL, 0); }
        public StringLiteralContext(AtomContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterStringLiteral(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitStringLiteral(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitStringLiteral(this);
            else return visitor.visitChildren(this);
        }
    }
    @SuppressWarnings("CheckReturnValue")
    public static class IdentifierAtomContext extends AtomContext {
        public TerminalNode IDENTIFIER() { return getToken(PythonParser.IDENTIFIER, 0); }
        public List<AtomTrailerContext> atomTrailer() {
            return getRuleContexts(AtomTrailerContext.class);
        }
        public AtomTrailerContext atomTrailer(int i) {
            return getRuleContext(AtomTrailerContext.class,i);
        }
        public IdentifierAtomContext(AtomContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterIdentifierAtom(this);
        }
        @Override

```

```

        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitIdentifierAtom(this)
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitIdentifierAtom(this);
            else return visitor.visitChildren(this);
        }
    }
    @SuppressWarnings("CheckReturnValue")
    public static class IntLiteralContext extends AtomContext {
        public TerminalNode INTEGER_LITERAL() { return getToken(PythonParser.INTEGER_LITERAL, 0); }
        public IntLiteralContext(AtomContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterIntLiteral(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitIntLiteral(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitIntLiteral(this);
            else return visitor.visitChildren(this);
        }
    }
    @SuppressWarnings("CheckReturnValue")
    public static class FloatLiteralContext extends AtomContext {
        public TerminalNode FLOAT_LITERAL() { return getToken(PythonParser.FLOAT_LITERAL, 0); }
        public FloatLiteralContext(AtomContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterFloatLiteral(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitFloatLiteral(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitFloatLiteral(this);
            else return visitor.visitChildren(this);
        }
    }
    @SuppressWarnings("CheckReturnValue")
    public static class TrueLiteralContext extends AtomContext {
        public TerminalNode TRUE() { return getToken(PythonParser.TRUE, 0); }
        public TrueLiteralContext(AtomContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterTrueLiteral(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitTrueLiteral(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitTrueLiteral(this);
            else return visitor.visitChildren(this);
        }
    }
    @SuppressWarnings("CheckReturnValue")
    public static class DictExprContext extends AtomContext {
        public TerminalNode LBRACE() { return getToken(PythonParser.LBRACE, 0); }

```

```

        public TerminalNode RBACE() { return getToken(PythonParser.RBACE, 0); }

        public DictItemsContext dictItems() {
            return getRuleContext(DictItemsContext.class,0);
        }

        public DictExprContext (AtomContext ctx) { copyFrom(ctx); }

        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterDictExpr(this);
        }

        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitDictExpr(this);
        }

        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitDictExpr(this);
            else return visitor.visitChildren(this);
        }
    }

    @SuppressWarnings("CheckReturnValue")
    public static class ListExprContext extends AtomContext {
        public TerminalNode LBRACK() { return getToken(PythonParser.LBRACK, 0); }
        public TerminalNode RBRACK() { return getToken(PythonParser.RBRACK, 0); }
        public ListItemsContext listItems() {
            return getRuleContext(ListItemsContext.class,0);
        }

        public ListExprContext (AtomContext ctx) { copyFrom(ctx); }

        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterListExpr(this);
        }

        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitListExpr(this);
        }

        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitListExpr(this);
            else return visitor.visitChildren(this);
        }
    }

    @SuppressWarnings("CheckReturnValue")
    public static class ParenExprContext extends AtomContext {
        public TerminalNode LPAREN() { return getToken(PythonParser.LPAREN, 0); }
        public ExprContext expr() {
            return getRuleContext(ExprContext.class,0);
        }

        public TerminalNode RPAREN() { return getToken(PythonParser.RPAREN, 0); }
        public ParenExprContext (AtomContext ctx) { copyFrom(ctx); }

        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterParenExpr(this);
        }

        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitParenExpr(this);
        }

        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitParenExpr(this);
            else return visitor.visitChildren(this);
        }
    }

    public final AtomContext atom() throws RecognitionException {
        AtomContext _localctx = new AtomContext(_ctx, getState());

```

```

enterRule(_localctx, 16, RULE_atom);
int _la;
try {
    int _alt;
    setState(223);
    _errHandler.sync(this);
    switch (_input.LA(1)) {
    case LPAREN:
        _localctx = new ParenExprContext(_localctx);
        enterOuterAlt(_localctx, 1);
        {
            setState(196);
            match(LPAREN);
            setState(197);
            expr(0);
            setState(198);
            match(RPAREN);
        }
        break;
    case LBRACK:
        _localctx = new ListExprContext(_localctx);
        enterOuterAlt(_localctx, 2);
        {
            setState(200);
            match(LBRACK);
            setState(202);
            _errHandler.sync(this);
            _la = _input.LA(1);
            if (((_la) & ~0x3f) == 0 && ((1L << _la) & 2064411282944L) != 0) {
                {
                    setState(201);
                    listItems();
                }
            }

            setState(204);
            match(RBRACK);
        }
        break;
    case LBRACE:
        _localctx = new DictExprContext(_localctx);
        enterOuterAlt(_localctx, 3);
        {
            setState(205);
            match(LBRACE);
            setState(207);
            _errHandler.sync(this);
            _la = _input.LA(1);
            if (_la==STRING_LITERAL || _la==IDENTIFIER) {
                {
                    setState(206);
                    dictItems();
                }
            }

            setState(209);
            match(RBRACE);
        }
        break;
    case STRING_LITERAL:
        _localctx = new StringLiteralContext(_localctx);
        enterOuterAlt(_localctx, 4);
        {
            setState(210);
            match(STRING_LITERAL);
        }
    }
}

```

```

        break;
    case INTEGER_LITERAL:
        _localctx = new IntLiteralContext(_localctx);
        enterOuterAlt(_localctx, 5);
        {
            setState(211);
            match(INTEGER_LITERAL);
        }
        break;
    case FLOAT_LITERAL:
        _localctx = new FloatLiteralContext(_localctx);
        enterOuterAlt(_localctx, 6);
        {
            setState(212);
            match(FLOAT_LITERAL);
        }
        break;
    case TRUE:
        _localctx = new TrueLiteralContext(_localctx);
        enterOuterAlt(_localctx, 7);
        {
            setState(213);
            match(TRUE);
        }
        break;
    case FALSE:
        _localctx = new FalseLiteralContext(_localctx);
        enterOuterAlt(_localctx, 8);
        {
            setState(214);
            match(FALSE);
        }
        break;
    case NONE:
        _localctx = new NoneLiteralContext(_localctx);
        enterOuterAlt(_localctx, 9);
        {
            setState(215);
            match(NONE);
        }
        break;
    case IDENTIFIER:
        _localctx = new IdentifierAtomContext(_localctx);
        enterOuterAlt(_localctx, 10);
        {
            setState(216);
            match(IDENTIFIER);
            setState(220);
            _errHandler.sync(this);
            _alt = getInterpreter().adaptivePredict(_input,26,_ctx);
            while ( _alt!=2 && _alt!=org.antlr.v4.runtime.atn.ATN.INVALID_ALT_NUMBER ) {
                if ( _alt==1 ) {
                    {
                        {
                            setState(217);
                            atomTrailer();
                        }
                    }
                }
                setState(222);
                _errHandler.sync(this);
                _alt = getInterpreter().adaptivePredict(_input,26,_ctx);
            }
        }
        break;
    default:

```



```

        throw new NoViableAltException(this);
    }

    }

    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }

    finally {
        exitRule();
    }

    return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class AtomTrailerContext extends ParserRuleContext {
    public AtomTrailerContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_atomTrailer; }

    public AtomTrailerContext() { }

    public void copyFrom(AtomTrailerContext ctx) {
        super.copyFrom(ctx);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class IndexAccessContext extends AtomTrailerContext {
    public TerminalNode LBRACK() { return getToken(PythonParser.LBRACK, 0); }
    public ExprContext expr() {
        return getRuleContext(ExprContext.class,0);
    }

    public TerminalNode RBRACK() { return getToken(PythonParser.RBRACK, 0); }
    public IndexAccessContext(AtomTrailerContext ctx) { copyFrom(ctx); }

    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterIndexAccess(this);
    }

    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitIndexAccess(this);
    }

    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitIndexAccess(this);
        else return visitor.visitChildren(this);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class AttributeAccessContext extends AtomTrailerContext {
    public TerminalNode DOT() { return getToken(PythonParser.DOT, 0); }
    public TerminalNode IDENTIFIER() { return getToken(PythonParser.IDENTIFIER, 0); }
    public AttributeAccessContext(AtomTrailerContext ctx) { copyFrom(ctx); }

    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterAttributeAccess(this);
    }

    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitAttributeAccess(this);
    }

    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitAttributeAccess(this);
        else return visitor.visitChildren(this);
    }
}

```

```

    }

    @SuppressWarnings("CheckReturnValue")
    public static class CallTrailerContext extends AtomTrailerContext {
        public TerminalNode LPAREN() { return getToken(PythonParser.LPAREN, 0); }
        public TerminalNode RPAREN() { return getToken(PythonParser.RPAREN, 0); }
        public ArgListContext argList() {
            return getRuleContext(ArgListContext.class,0);
        }
        public CallTrailerContext(AtomTrailerContext ctx) { copyFrom(ctx); }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterCallTrailer(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitCallTrailer(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitCallTrailer(this)
            else return visitor.visitChildren(this);
        }
    }

    public final AtomTrailerContext atomTrailer() throws RecognitionException {
        AtomTrailerContext _localctx = new AtomTrailerContext(_ctx, getState());
        enterRule(_localctx, 18, RULE_atomTrailer);
        int _la;
        try {
            setState(236);
            _errHandler.sync(this);
            switch (_input.LA(1)) {
                case DOT:
                    _localctx = new AttributeAccessContext(_localctx);
                    enterOuterAlt(_localctx, 1);
                    {
                        setState(225);
                        match(DOT);
                        setState(226);
                        match(IDENTIFIER);
                    }
                    break;
                case LBRACK:
                    _localctx = new IndexAccessContext(_localctx);
                    enterOuterAlt(_localctx, 2);
                    {
                        setState(227);
                        match(LBRACK);
                        setState(228);
                        expr(0);
                        setState(229);
                        match(RBRACK);
                    }
                    break;
                case LPAREN:
                    _localctx = new CallTrailerContext(_localctx);
                    enterOuterAlt(_localctx, 3);
                    {
                        setState(231);
                        match(LPAREN);
                        setState(233);
                        _errHandler.sync(this);
                        _la = _input.LA(1);
                        if ((((_la) & ~0x3f) == 0 && ((1L << _la) & 2064411282944L) != 0)) {
                            {
                                setState(232);

```

```

        argList();
    }

    }

    setState(235);
    match(RPAREN);
    }
    break;
default:
    throw new NoViableAltException(this);
}

}

catch (RecognitionException re) {
    _localctx.exception = re;
    _errHandler.reportError(this, re);
    _errHandler.recover(this, re);
}

finally {
    exitRule();
}

return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class ArgListContext extends ParserRuleContext {
    public List<ArgumentContext> argument() {
        return getRuleContexts(ArgumentContext.class);
    }

    public ArgumentContext argument(int i) {
        return getRuleContext(ArgumentContext.class,i);
    }

    public List<TerminalNode> COMMA() { return getTokens(PythonParser.COMMA); }
    public TerminalNode COMMA(int i) {
        return getToken(PythonParser.COMMA, i);
    }

    public ArgListContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_argList; }

    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterArgList(this);
    }

    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitArgList(this);
    }

    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitArgList(this);
        else return visitor.visitChildren(this);
    }

}

public final ArgListContext argList() throws RecognitionException {
    ArgListContext _localctx = new ArgListContext(_ctx, getState());
    enterRule(_localctx, 20, RULE_argList);
    int _la;
    try {
        int _alt;
        enterOuterAlt(_localctx, 1);
        {
            setState(238);
            argument();
            setState(243);
            _errHandler.sync(this);

```

```

        _alt = getInterpreter().adaptivePredict(_input,30,_ctx);
        while ( _alt!=2 && _alt!=org.antlr.v4.runtime.atn.ATN.INVALID_ALT_NUMBER ) {
            if ( _alt==1 ) {
                {
                    {
                        setState(239);
                        match(COMMA);
                        setState(240);
                        argument();
                    }
                }
            }
            setState(245);
            _errHandler.sync(this);
            _alt = getInterpreter().adaptivePredict(_input,30,_ctx);
        }
        setState(247);
        _errHandler.sync(this);
        _la = _input.LA(1);
        if ( _la==COMMA ) {
            {
                {
                    setState(246);
                    match(COMMA);
                }
            }
        }
    }

    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }

    finally {
        exitRule();
    }

    return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class ArgumentContext extends ParserRuleContext {
    public ArgumentContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_argument; }

    public ArgumentContext() { }
    public void copyFrom(ArgumentContext ctx) {
        super.copyFrom(ctx);
    }
}

@SuppressWarnings("CheckReturnValue")
public static class PositionalArgContext extends ArgumentContext {
    public ExprContext expr() {
        return getRuleContext(ExprContext.class,0);
    }

    public PositionalArgContext(ArgumentContext ctx) { copyFrom(ctx); }

    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterPositionalArg(this);
    }

    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitPositionalArg(this);
    }

    @Override

```

```

        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitPos
            else return visitor.visitChildren(this);
        }
    }
}

@SuppressWarnings("CheckReturnValue")
public static class KeywordArgContext extends ArgumentContext {
    public TerminalNode IDENTIFIER() { return getToken(PythonParser.IDENTIFIER, 0); }
    public TerminalNode ASSIGN() { return getToken(PythonParser.ASSIGN, 0); }
    public ExprContext expr() {
        return getRuleContext(ExprContext.class,0);
    }
    public KeywordArgContext(ArgumentContext ctx) { copyFrom(ctx); }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterKeywordArg(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitKeywordArg(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitKey
        else return visitor.visitChildren(this);
    }
}

public final ArgumentContext argument() throws RecognitionException {
    ArgumentContext _localctx = new ArgumentContext(_ctx, getState());
    enterRule(_localctx, 22, RULE_argument);
    try {
        setState(253);
        _errHandler.sync(this);
        switch ( getInterpreter().adaptivePredict(_input,32,_ctx) ) {
            case 1:
                _localctx = new PositionalArgContext(_localctx);
                enterOuterAlt(_localctx, 1);
                {
                    setState(249);
                    expr(0);
                }
                break;
            case 2:
                _localctx = new KeywordArgContext(_localctx);
                enterOuterAlt(_localctx, 2);
                {
                    setState(250);
                    match(IDENTIFIER);
                    setState(251);
                    match(ASSIGN);
                    setState(252);
                    expr(0);
                }
                break;
        }
    }
    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }
    finally {
        exitRule();
    }
    return _localctx;
}

```

```

    }

    @SuppressWarnings("CheckReturnValue")
    public static class DictItemsContext extends ParserRuleContext {
        public List<DictItemContext> dictItem() {
            return getRuleContexts(DictItemContext.class);
        }
        public DictItemContext dictItem(int i) {
            return getRuleContext(DictItemContext.class,i);
        }
        public List<TerminalNode> COMMA() { return getTokens(PythonParser.COMMA); }
        public TerminalNode COMMA(int i) {
            return getToken(PythonParser.COMMA, i);
        }
        public DictItemsContext(ParserRuleContext parent, int invokingState) {
            super(parent, invokingState);
        }
        @Override public int getRuleIndex() { return RULE_dictItems; }
        @Override
        public void enterRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterDictItems(this);
        }
        @Override
        public void exitRule(ParseTreeListener listener) {
            if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitDictItems(this);
        }
        @Override
        public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
            if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitDictItems(this);
            else return visitor.visitChildren(this);
        }
    }

    }

    public final DictItemsContext dictItems() throws RecognitionException {
        DictItemsContext _localctx = new DictItemsContext(_ctx, getState());
        enterRule(_localctx, 24, RULE_dictItems);
        int _la;
        try {
            int _alt;
            enterOuterAlt(_localctx, 1);
            {
                setState(255);
                dictItem();
                setState(260);
                _errHandler.sync(this);
                _alt = getInterpreter().adaptivePredict(_input,33,_ctx);
                while ( _alt!=2 && _alt!=org.antlr.v4.runtime.atn.ATN.INVALID_ALT_NUMBER ) {
                    if ( _alt==1 ) {
                        {
                            {
                                setState(256);
                                match(COMMA);
                                setState(257);
                                dictItem();
                            }
                        }
                    }
                    setState(262);
                    _errHandler.sync(this);
                    _alt = getInterpreter().adaptivePredict(_input,33,_ctx);
                }
                setState(264);
                _errHandler.sync(this);
                _la = _input.LA(1);
                if (_la==COMMA) {
                    {

```

```

        setState(263);
        match(COMMA);
    }

    }

    }

    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }

    finally {
        exitRule();
    }

    return _localctx;
}

@SuppressWarnings("CheckReturnValue")
public static class DictItemContext extends ParserRuleContext {
    public TerminalNode COLON() { return getToken(PythonParser.COLON, 0); }
    public ExprContext expr() {
        return getRuleContext(ExprContext.class,0);
    }

    public TerminalNode STRING_LITERAL() { return getToken(PythonParser.STRING_LITERAL, 0); }
    public TerminalNode IDENTIFIER() { return getToken(PythonParser.IDENTIFIER, 0); }
    public DictItemContext(ParserRuleContext parent, int invokingState) {
        super(parent, invokingState);
    }

    @Override public int getRuleIndex() { return RULE_dictItem; }
    @Override
    public void enterRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).enterDictItem(this);
    }
    @Override
    public void exitRule(ParseTreeListener listener) {
        if ( listener instanceof PythonParserListener ) ((PythonParserListener)listener).exitDictItem(this);
    }
    @Override
    public <T> T accept(ParseTreeVisitor<? extends T> visitor) {
        if ( visitor instanceof PythonParserVisitor ) return ((PythonParserVisitor<? extends T>)visitor).visitDictItem(this);
        else return visitor.visitChildren(this);
    }
}

public final DictItemContext dictItem() throws RecognitionException {
    DictItemContext _localctx = new DictItemContext(_ctx, getState());
    enterRule(_localctx, 26, RULE_dictItem);
    int _la;
    try {
        enterOuterAlt(_localctx, 1);
        {
            setState(266);
            _la = _input.LA(1);
            if ( !(_la==STRING_LITERAL || _la==IDENTIFIER) ) {
                _errHandler.recoverInline(this);
            }
            else {
                if ( _input.LA(1)==Token.EOF ) matchedEOF = true;
                _errHandler.reportMatch(this);
                consume();
            }
        }
        setState(267);
        match(COLON);
        setState(268);
        expr(0);
    }
}

```

```

    }

    catch (RecognitionException re) {
        _localctx.exception = re;
        _errHandler.reportError(this, re);
        _errHandler.recover(this, re);
    }

    finally {
        exitRule();
    }

    return _localctx;
}

public boolean sempred(RuleContext _localctx, int ruleIndex, int predIndex) {
    switch (ruleIndex) {
        case 6:
            return expr_sempred((ExprContext)_localctx, predIndex);
    }
    return true;
}

private boolean expr_sempred(ExprContext _localctx, int predIndex) {
    switch (predIndex) {
        case 0:
            return precpred(_ctx, 8);
        case 1:
            return precpred(_ctx, 7);
        case 2:
            return precpred(_ctx, 5);
        case 3:
            return precpred(_ctx, 4);
        case 4:
            return precpred(_ctx, 3);
    }
    return true;
}

public static final String _serializedATN =
    "\u0004\u0001-\u010f\u0002\u0000\u0007\u0000\u0002\u0001\u0007\u0001\u0002"+
    "\u0002\u0007\u0002\u0002\u0003\u0007\u0003\u0002\u0004\u0007\u0004\u0002"+
    "\u0005\u0007\u0005\u0002\u0006\u0007\u0006\u0002\u0007\u0007\u0007\u0002"+
    "\b\u0007\b\u0002\t\u0007\t\u0002\n\u0007\n\u0002\u000b\u0007\u000b\u0002"+
    "\f\u0007\f\u0002\u0007\u0001\u0000\u0001\u0000\u0005\u0000\u0001f\b"+
    "\u0000\n\u0000\u0000\u0000\u0000\u0001\u0000\u0001\u0000\u0001\u0001"+
    "\u0001\u0001\u0001\u0001\u0001\u0005\u0001*\b\u0001\n\u0001\u0001-\t"+
    "\u0001\u0001\u0001\u0003\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0001"+
    "\u0001\u0005\u00015\b\u0001\n\u0001\u0001\u0001\u0001\u0001\u0001\u0003"+
    "\u0001;\b\u0001\u0001\u0001\u0005\u0001>\b\u0001\n\u0001\u0001\u0001A\t\u0001"+
    "\u0001\u0001\u0003\u0001D\b\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0001"+
    "\u0001\u0001\u0003\u0001J\b\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0001"+
    "\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0005\u0001X\b\u0001\n\u0001\u0001\u0001"+
    "\t\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0003\u0001`\b\u0001\u0001\u0001"+
    "\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0001"+
    "\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0001\u0003"+
    "\u0001n\b\u0001\u0001\u0002\u0001\u0002\u0001\u0002\u0001\u0002\u0001\u0002\u0005"+
    "\u0002t\b\u0002\n\u0002\u0002\u0002w\t\u0002\u0001\u0002\u0001\u0002\u0003"+
    "\u0002{\b\u0002\u0001\u0002\u0001\u0002\u0002\u0001\u0003\u0001\u0003\u0001\u0001"+
    "\u0003\u0005\u0003\u0002\u0003\b\u0003\n\u0003\u0003\u0003\u0005\u0003\u0003\u0001\u0004"+
    "\u0001\u0004\u0001\u0004\u0001\u0004\u0004\u0004\u0004\u0008\b\u0004\u000b\u0004\u0004"+
    "\u0004\u0004\u000c\u0001\u0004\u0001\u0004\u0003\u0004\u0004\u0009\b\u0004\u0001\u0001"+
    "\u0005\u0001\u0005\u0001\u0005\u0005\u0005\u0005\u0006\b\u0005\u0005\u0005\u0005"+
    "\u0099\t\u0005\u0001\u0005\u0003\u0005\u0005\u0005\u0009c\b\u0005\u0005\u0006\u0001"+
    "\u0006\u0001\u0006\u0001\u0006\u0001\u0006\u0001\u0006\u0003\u0006\u0006\u0006\u0006"+
    "\b\u0006\u0001\u0006\u0001\u0006\u0006\u0001\u0006\u0006\u0001\u0006\u0006\u0001"+
    "\u0006\u0001\u0006\u0001\u0006\u0006\u0001\u0006\u0006\u0001\u0006\u0006\u0001"+
    "\u0006\u0001\u0006\u0001\u0006\u0006\u0005\u0006\u0006\u000b5\b\u0006\n"+

```


[illegible]

"\u0000\u0086\u0091\u0003\u0005\u0000\u0087\u0088\u0005)\u0000\u0000"+
"\u0088\u008a\u0005,\u0000\u0000\u0089\u008b\u0003\u0002\u0001\u0000\u008a"+
"\u0089\u0001\u0000\u0000\u0000\u008b\u008c\u0001\u0000\u0000\u0000\u008c"+
"\u008a\u0001\u0000\u0000\u0000\u008c\u008d\u0001\u0000\u0000\u0000\u008d"+
"\u008e\u0001\u0000\u0000\u0000\u008e\u008f\u0005-\u0000\u0000\u008f\u0091"+
"\u0001\u0000\u0000\u0000\u0090\u0086\u0001\u0000\u0000\u0000\u0090\u0087"+
"\u0001\u0000\u0000\u0000\u0091\t\u0001\u0000\u0000\u0000\u0092\u0097\u0003"+
"\f\u0006\u0000\u0093\u0094\u0005\u0013\u0000\u0000\u0094\u0096\u0003\f"+
"\u0006\u0000\u0095\u0093\u0001\u0000\u0000\u0000\u0096\u0099\u0001\u0000"+
"\u0000\u0000\u0097\u0095\u0001\u0000\u0000\u0000\u0097\u0098\u0001\u0000"+
"\u0000\u0000\u0098\u009b\u0001\u0000\u0000\u0000\u0099\u0097\u0001\u0000"+
"\u0000\u0000\u009a\u009c\u0005)\u0000\u0000\u009b\u009a\u0001\u0000\u0000"+
"\u0000\u009b\u009c\u0001\u0000\u0000\u0000\u009c\u000b\u0001\u0000\u0000"+
"\u0000\u009d\u009e\u0006\u0006\u0000\u0000\u0000\u009e\u009f\u0005\u000e"+
"\u0000\u0000\u009f\u00a4\u0003\u0006\u0006\u00a0\u00a1\u0005\u0017\u0000"+
"\u0000\u00a1\u00a4\u0003\u0006\u0002\u00a2\u00a4\u0003\u0010\b\u0000"+
"\u00a3\u009d\u0001\u0000\u0000\u0000\u00a3\u00a0\u0001\u0000\u0000\u0000"+
"\u00a3\u00a2\u0001\u0000\u0000\u0000\u00a4\u00b6\u0001\u0000\u0000\u0000"+
"\u00a5\u00a6\n\b\u0000\u0000\u00a6\u00a7\u0005\r\u0000\u0000\u00a7\u00b5"+
"\u0003\u0006\t\u00a8\u00a9\n\u0007\u0000\u0000\u00a9\u00aa\u0005\u0000\u0000"+
"\u0000\u00aa\u00b5\u0003\u0006\b\u00ab\u00ac\n\u0005\u0000\u0000\u00ac"+
"\u00ad\u0007\u0000\u0000\u0000\u00ad\u00b5\u0003\u0006\u0006\u00ae\u00af"+
"\n\u0004\u0000\u0000\u0000\u00af\u00b0\u0007\u0001\u0000\u0000\u0000\u00b0\u00b5\u0003"+
"\f\u0006\u0005\u00b1\u00b2\n\u0003\u0000\u0000\u00b2\u00b3\u0007\u0002"+
"\u0000\u0000\u00b3\u00b5\u0003\u0006\u0004\u00b4\u00a5\u0001\u0000\u0000"+
"\u0000\u00b4\u00a8\u0001\u0000\u0000\u0000\u00b4\u00ab\u0001\u0000\u0000"+
"\u0000\u00b5\u000e\u0001\u0000\u0000\u0000\u00b5\u00b1\u0001\u0000\u0000"+
"\u0000\u00b5\u00b8\u0001\u0000\u0000\u0000\u00b6\u00b4\u0001\u0000\u0000"+
"\u0000\u00b6\u00b7\u0001\u0000\u0000\u0000\u00b7\r\u0001\u0000\u0000\u0000"+
"\u00b8\u00b6\u0001\u0000\u0000\u0000\u00b9\u00be\u0003\u0006\u0000\u00ba"+
"\u00bb\u0005"\u0000\u0000\u0000\u00bb\u00bd\u0003\u0006\u0000\u00bc\u00ba"+
"\u0001\u0000\u0000\u0000\u00bd\u00c0\u0001\u0000\u0000\u0000\u00be\u00bc"+
"\u0001\u0000\u0000\u0000\u00be\u00bf\u0001\u0000\u0000\u0000\u0000\u00bf\u00c2"+
"\u0001\u0000\u0000\u0000\u00c0\u00be\u0001\u0000\u0000\u0000\u00c1\u00c3"+
"\u0005\u0000\u0000\u0000\u00c2\u00c1\u0001\u0000\u0000\u0000\u00c2\u00c3\u0001"+
"\u0000\u0000\u0000\u0000\u00c3\u000f\u0001\u0000\u0000\u0000\u00c4\u00c5\u0005"+
"\u001b\u0000\u0000\u00c5\u00c6\u0003\u0006\u0000\u00c6\u00c7\u0005\u001c"+
"\u0000\u0000\u0000\u00c7\u00e0\u0001\u0000\u0000\u0000\u00c8\u00ca\u0005\u001d"+
"\u0000\u0000\u0000\u00c9\u00cb\u0003\u000e\u0007\u0000\u0000\u00ca\u00c9\u0001\u0000"+
"\u0000\u0000\u00ca\u00cb\u0001\u0000\u0000\u0000\u00cb\u00cc\u0001\u0000"+
"\u0000\u0000\u0000\u00cc\u00e0\u0005\u001e\u0000\u0000\u00cd\u00cf\u0005\u001f"+
"\u0000\u0000\u0000\u00ce\u00d0\u0003\u0018\u0000\u00cf\u000e\u0001\u0000\u0000"+
"\u0000\u00cf\u00d0\u0001\u0000\u0000\u0000\u00d0\u00d1\u0001\u0000\u0000"+
"\u0000\u00d1\u00e0\u0005 \u0000\u0000\u00d2\u00e0\u0005\''\u0000\u0000"+
"\u00d3\u00e0\u0005&\u0000\u0000\u00d4\u00e0\u0005%\u0000\u0000\u00d5\u00e0"+
"\u0005\t\u0000\u0000\u00d6\u00e0\u0005\n\u0000\u0000\u00d7\u00e0\u0005"+
"\u000b\u0000\u0000\u00d8\u00dc\u0005(\u0000\u0000\u00d9\u00db\u0003\u0012"+
"\t\u0000\u00da\u00d9\u0001\u0000\u0000\u0000\u00db\u00de\u0001\u0000\u0000"+
"\u0000\u00dc\u00da\u0001\u0000\u0000\u0000\u00dc\u00dd\u0001\u0000\u0000"+
"\u0000\u00dd\u00e0\u0001\u0000\u0000\u0000\u00de\u00dc\u0001\u0000\u0000"+
"\u0000\u00df\u00c4\u0001\u0000\u0000\u0000\u00df\u00c8\u0001\u0000\u0000"+
"\u0000\u00df\u00cd\u0001\u0000\u0000\u0000\u00df\u00d2\u0001\u0000\u0000"+
"\u0000\u00df\u00d3\u0001\u0000\u0000\u0000\u00df\u00d4\u0001\u0000\u0000"+
"\u0000\u00df\u00d5\u0001\u0000\u0000\u0000\u00df\u00d6\u0001\u0000\u0000"+
"\u0000\u00df\u00d7\u0001\u0000\u0000\u0000\u00df\u00d8\u0001\u0000\u0000"+
"\u0000\u00e0\u0011\u0001\u0000\u0000\u0000\u00e1\u00e2\u0005#\u0000\u0000"+
"\u00e2\u00ed\u0005(\u0000\u0000\u00e3\u00e4\u0005\u001d\u0000\u0000\u00e4"+
"\u00e5\u0003\u0006\u0000\u0005\u00e6\u0005\u001e\u0000\u0000\u00e6\u00ed"+
"\u0001\u0000\u0000\u0000\u0000\u00e7\u00e9\u0005\u001b\u0000\u0000\u00e8\u00ea"+
"\u0003\u0014\n\u0000\u00e9\u00e8\u0001\u0000\u0000\u0000\u00e9\u00ea\u0001"+
"\u0000\u0000\u0000\u00ea\u00eb\u0001\u0000\u0000\u0000\u00eb\u00ed\u0005"+
"\u001c\u0000\u0000\u00ec\u00e1\u0001\u0000\u0000\u0000\u00ec\u00e3\u0001"+
"\u0000\u0000\u0000\u00ec\u00e7\u0001\u0000\u0000\u0000\u00ed\u0013\u0001"+
"\u0000\u0000\u0000\u00ee\u00f3\u0003\u0016\u000b\u0000\u00ef\u00f0\u0005"+
"\u0000\u0000\u0000\u00f0\u00f2\u0003\u0016\u000b\u0000\u00f1\u00ef\u0001\u0000"+
"\u0000\u0000\u00f2\u00f5\u0001\u0000\u0000\u0000\u00f3\u00f1\u0001\u0000"+

```

"\u0000\u0000\u00f3\u00f4\u0001\u0000\u0000\u0000\u00f4\u00f7\u0001\u0000"+
"\u0000\u0000\u00f5\u00f3\u0001\u0000\u0000\u0000\u00f6\u00f8\u0005\u0000"+
"\u0000\u00f7\u00f6\u0001\u0000\u0000\u0000\u00f7\u00f8\u0001\u0000\u0000"+
"\u0000\u00f8\u0015\u0001\u0000\u0000\u0000\u00f9\u00fe\u0003\u0006\u0000"+
"\u00fa\u00fb\u0005(\u0000\u0000\u00fb\u00fc\u0005\u0013\u0000\u0000\u00fc"+
"\u00fe\u0003\u0006\u0000\u00fd\u00f9\u0001\u0000\u0000\u0000\u00fd\u00fa"+
"\u0001\u0000\u0000\u0000\u00fe\u0017\u0001\u0000\u0000\u0000\u00ff\u0104"+
"\u0003\u001a\u0000\u0100\u0101\u0005\u0000\u0000\u0101\u0103\u0003"+
"\u001a\u0000\u0102\u0100\u0001\u0000\u0000\u0000\u0103\u0106\u0001\u0000"+
"\u0000\u0000\u0104\u0102\u0001\u0000\u0000\u0000\u0104\u0105\u0001\u0000"+
"\u0000\u0000\u0105\u0108\u0001\u0000\u0000\u0000\u0106\u0104\u0001\u0000"+
"\u0000\u0000\u0107\u0109\u0005\u0000\u0000\u0108\u0107\u0001\u0000\u0000"+
"\u0000\u0108\u0109\u0001\u0000\u0000\u0000\u0109\u0019\u0001\u0000\u0000"+
"\u0000\u010a\u010b\u0007\u0003\u0000\u0000\u010b\u010c\u0005!\u0000\u0000"+
"\u010c\u010d\u0003\u0006\u0000\u010d\u001b\u0001\u0000\u0000\u0000\u0000#"+
"\u001e +/6: ?CIY_muz\u0083\u008c\u0090\u0097\u009b\u00a3\u00b4\u00b6\u00be"+
"\u00c2\u00ca\u00cf\u00dc\u00e9\u00ec\u00f3\u00f7\u00fd\u0104\u0108";

public static final ATN _ATN =
    new ATNDeserializer().deserialize(_serializedATN.toCharArray());
static {
    _decisionToDFA = new DFA[_ATN.getNumberOfDecisions()];
    for (int i = 0; i < _ATN.getNumberOfDecisions(); i++) {
        _decisionToDFA[i] = new DFA(_ATN.getDecisionState(i), i);
    }
}

```

}).

?? File: PythonParserBaseListener.java

// Generated from D:/Compiler/PyJinjaCompiler/src/grammar/python/PythonParser.g4 by ANTLR 4.13.2 package compiler.generated.python;

import org.antlr.v4.runtime.ParserRuleContext; import org.antlr.v4.runtime.tree.ErrorNode; import org.antlr.v4.runtime.tree.TerminalNode;

/**

- This class provides an empty implementation of {@link PythonParserListener}.
- which can be extended to create a listener which only needs to handle a subset
- of the available methods. / *@SuppressWarnings("CheckReturnValue") public class PythonParserBaseListener implements PythonParserListener { /**

- *{@inheritDoc}*

-

- The default implementation does nothing.

*/ @Override public void enterProgram(PythonParser.ProgramContext ctx) /**

- *{@inheritDoc}*

-

- The default implementation does nothing.

*/ @Override public void exitProgram(PythonParser.ProgramContext ctx) /**

- *{@inheritDoc}*

-

- The default implementation does nothing.

*/ @Override public void enterReturnStatement(PythonParser.ReturnStatementContext ctx) /**

- *{@inheritDoc}*

-

- The default implementation does nothing.

*/ @Override public void exitReturnStatement(PythonParser.ReturnStatementContext ctx) /**

- *{@inheritDoc}*

-

- The default implementation does nothing.

*/ @Override public void enterAssignOrExprStatement(PythonParser.AssignOrExprStatementContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitAssignOrExprStatement(PythonParser.AssignOrExprStatementContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterFunctionDef(PythonParser.FunctionDefContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitFunctionDef(PythonParser.FunctionDefContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterIfStatement(PythonParser.IfStatementContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitIfStatement(PythonParser.IfStatementContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterWhileStatement(PythonParser.WhileStatementContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitWhileStatement(PythonParser.WhileStatementContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterForStatement(PythonParser.ForStatementContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitForStatement(PythonParser.ForStatementContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterDecorator(PythonParser.DecoratorContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitDecorator(PythonParser.DecoratorContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterParamList(PythonParser.ParamListContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitParamList(PythonParser.ParamListContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterSingleSuite(PythonParser.SingleSuiteContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitSingleSuite(PythonParser.SingleSuiteContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterIndentedSuite(PythonParser.IndentedSuiteContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitIndentedSuite(PythonParser.IndentedSuiteContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterSimpleLine(PythonParser.SimpleLineContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitSimpleLine(PythonParser.SimpleLineContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterNotExpr(PythonParser.NotExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitNotExpr(PythonParser.NotExprContext ctx) /**

- {@inheritDoc}
-

- The default implementation does nothing.

*/ @Override public void enterAddExpr(PythonParser.AddExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitAddExpr(PythonParser.AddExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterUnaryMinus(PythonParser.UnaryMinusContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitUnaryMinus(PythonParser.UnaryMinusContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterMulExpr(PythonParser.MulExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitMulExpr(PythonParser.MulExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterAtomExpr(PythonParser.AtomExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitAtomExpr(PythonParser.AtomExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterOrExpr(PythonParser.OrExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitOrExpr(PythonParser.OrExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterComparisonExpr(PythonParser.ComparisonExprContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitComparisonExpr(PythonParser.ComparisonExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterAndExpr(PythonParser.AndExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitAndExpr(PythonParser.AndExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterListItems(PythonParser.ListItemsContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitListItems(PythonParser.ListItemsContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterParenExpr(PythonParser.ParenExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitParenExpr(PythonParser.ParenExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterListExpr(PythonParser.ListExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitListExpr(PythonParser.ListExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterDictExpr(PythonParser.DictExprContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitDictExpr(PythonParser.DictExprContext ctx) /**

- {@inheritDoc}
-

- The default implementation does nothing.

*/ @Override public void enterStringLiteral(PythonParser.StringLiteralContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void exitStringLiteral(PythonParser.StringLiteralContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void enterIntLiteral(PythonParser.IntLiteralContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void exitIntLiteral(PythonParser.IntLiteralContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void enterFloatLiteral(PythonParser.FloatLiteralContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void exitFloatLiteral(PythonParser.FloatLiteralContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void enterTrueLiteral(PythonParser.TrueLiteralContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void exitTrueLiteral(PythonParser.TrueLiteralContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void enterFalseLiteral(PythonParser.FalseLiteralContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void exitFalseLiteral(PythonParser.FalseLiteralContext ctx) /**

- *{@inheritDoc}*
-
- The default implementation does nothing.

*/ @Override public void enterNoneLiteral(PythonParser.NoneLiteralContext ctx) /**

- *{@inheritDoc}*
-

- The default implementation does nothing.

*/ @Override public void exitNoneLiteral(PythonParser.NoneLiteralContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterIdentifierAtom(PythonParser.IdentifierAtomContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitIdentifierAtom(PythonParser.IdentifierAtomContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterAttributeAccess(PythonParser.AttributeAccessContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitAttributeAccess(PythonParser.AttributeAccessContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterIndexAccess(PythonParser.IndexAccessContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitIndexAccess(PythonParser.IndexAccessContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterCallTrailer(PythonParser.CallTrailerContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitCallTrailer(PythonParser.CallTrailerContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void enterArgList(PythonParser.ArgListContext ctx) /**

- {@inheritDoc}
-
- The default implementation does nothing.

*/ @Override public void exitArgList(PythonParser.ArgListContext ctx) /**

- {@inheritDoc}
-

- The default implementation does nothing.

*/ @Override public void enterPositionalArg(PythonParser.PositionalArgContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitPositionalArg(PythonParser.PositionalArgContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterKeywordArg(PythonParser.KeywordArgContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitKeywordArg(PythonParser.KeywordArgContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterDictItems(PythonParser.DictItemsContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitDictItems(PythonParser.DictItemsContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterDictItem(PythonParser.DictItemContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

**/ @Override public void exitDictItem(PythonParser.DictItemContext ctx)*

*/***

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void enterEveryRule(ParserRuleContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void exitEveryRule(ParserRuleContext ctx) /**

- {@inheritDoc}

▪

- The default implementation does nothing.

*/ @Override public void visitTerminal(TerminalNode node) /**

- {@inheritDoc}

-
- The default implementation does nothing.

```
*/ @Override public void visitErrorNode(ErrorNode node) } .
```

?? File: PythonParserBaseVisitor.java

```
// Generated from D:/Compiler/PyJinjaCompiler/src/grammar/python/PythonParser.g4 by ANTLR 4.13.2 package compiler.generated.python; import
org.antlr.v4.runtime.tree.AbstractParseTreeVisitor;
```

```
/*
```

- This class provides an empty implementation of {@link PythonParserVisitor},
- which can be extended to create a visitor which only needs to handle a subset
- of the available methods.
-
- @param The return type of the visit operation. Use {@link Void} for
- operations with no return type. / @SuppressWarnings("CheckReturnValue") public class PythonParserBaseVisitor extends AbstractParseTreeVisitor
- implements PythonParserVisitor { /*

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
 / @Override public T visitProgram(PythonParser.ProgramContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
 / @Override public T visitReturnStatement(PythonParser.ReturnStatementContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
 / @Override public T visitAssignOrExprStatement(PythonParser.AssignOrExprStatementContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
 / @Override public T visitFunctionDef(PythonParser.FunctionDefContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
 / @Override public T visitIfStatement(PythonParser.IfStatementContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```
 / @Override public T visitWhileStatement(PythonParser.WhileStatementContext ctx) { return visitChildren(ctx); } /*
```

- {@inheritDoc}

-

- The default implementation returns the result of calling

- {@link #visitChildren} on {@code ctx}.

```

    / @Override public T visitForStatement(PythonParser.ForStatementContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitDecorator(PythonParser.DecoratorContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitParamList(PythonParser.ParamListContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitSingleSuite(PythonParser.SingleSuiteContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitIndentedSuite(PythonParser.IndentedSuiteContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitSimpleLine(PythonParser.SimpleLineContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitNotExpr(PythonParser.NotExprContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitAddExpr(PythonParser.AddExprContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitUnaryMinus(PythonParser.UnaryMinusContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitMulExpr(PythonParser.MulExprContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

```

- The default implementation returns the result of calling

- `{@link #visitChildren}` on `{@code ctx}`.

```
// @Override public T visitAtomExpr(PythonParser.AtomExprContext ctx) { return visitChildren(ctx); } /*
```

- `{@inheritDoc}`

-

- The default implementation returns the result of calling

- `{@link #visitChildren}` on `{@code ctx}`.

```
// @Override public T visitOrExpr(PythonParser.OrExprContext ctx) { return visitChildren(ctx); } /*
```

- `{@inheritDoc}`

-

- The default implementation returns the result of calling

- `{@link #visitChildren}` on `{@code ctx}`.

```
// @Override public T visitComparisonExpr(PythonParser.ComparisonExprContext ctx) { return visitChildren(ctx); } /*
```

- `{@inheritDoc}`

-

- The default implementation returns the result of calling

- `{@link #visitChildren}` on `{@code ctx}`.

```
// @Override public T visitAndExpr(PythonParser.AndExprContext ctx) { return visitChildren(ctx); } /*
```

- `{@inheritDoc}`

-

- The default implementation returns the result of calling

- `{@link #visitChildren}` on `{@code ctx}`.

```
// @Override public T visitListItems(PythonParser.ListItemsContext ctx) { return visitChildren(ctx); } /*
```

- `{@inheritDoc}`

-

- The default implementation returns the result of calling

- `{@link #visitChildren}` on `{@code ctx}`.

```
// @Override public T visitParenExpr(PythonParser.ParenExprContext ctx) { return visitChildren(ctx); } /*
```

- `{@inheritDoc}`

-

- The default implementation returns the result of calling

- `{@link #visitChildren}` on `{@code ctx}`.

```
// @Override public T visitListExpr(PythonParser.ListExprContext ctx) { return visitChildren(ctx); } /*
```

- `{@inheritDoc}`

-

- The default implementation returns the result of calling

- `{@link #visitChildren}` on `{@code ctx}`.

```
// @Override public T visitDictExpr(PythonParser.DictExprContext ctx) { return visitChildren(ctx); } /*
```

- `{@inheritDoc}`

-

- The default implementation returns the result of calling

- `{@link #visitChildren}` on `{@code ctx}`.

```
// @Override public T visitStringLiteral(PythonParser.StringLiteralContext ctx) { return visitChildren(ctx); } /*
```

- `{@inheritDoc}`

-

- The default implementation returns the result of calling

- `{@link #visitChildren}` on `{@code ctx}`.

```

    / @Override public T visitIntLiteral(PythonParser.IntLiteralContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitFloatLiteral(PythonParser.FloatLiteralContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitTrueLiteral(PythonParser.TrueLiteralContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitFalseLiteral(PythonParser.FalseLiteralContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitNoneLiteral(PythonParser.NoneLiteralContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitIdentifierAtom(PythonParser.IdentifierAtomContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitAttributeAccess(PythonParser.AttributeAccessContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitIndexAccess(PythonParser.IndexAccessContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitCallTrailer(PythonParser.CallTrailerContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

    ■ The default implementation returns the result of calling

    ■ {@link #visitChildren} on {@code ctx}.

    / @Override public T visitArgList(PythonParser.ArgListContext ctx) { return visitChildren(ctx); } /*
    ■ @inheritDoc
    ■

```

- The default implementation returns the result of calling
- {@link #visitChildren} on {@code ctx}.


```

      / @Override public T visitPositionalArg(PythonParser.PositionalArgContext ctx) { return visitChildren(ctx); } /*
      {@inheritDoc}
      
```
- The default implementation returns the result of calling
- {@link #visitChildren} on {@code ctx}.


```

      / @Override public T visitKeywordArg(PythonParser.KeywordArgContext ctx) { return visitChildren(ctx); } /*
      {@inheritDoc}
      
```
- The default implementation returns the result of calling
- {@link #visitChildren} on {@code ctx}.


```

      / @Override public T visitDictItems(PythonParser.DictItemsContext ctx) { return visitChildren(ctx); } /*
      {@inheritDoc}
      
```
- The default implementation returns the result of calling
- {@link #visitChildren} on {@code ctx}.


```

      /* @Override public T visitDictItem(PythonParser.DictItemContext ctx) { return visitChildren(ctx); } .
      
```

?? File: PythonParserListener.java

// Generated from D:/Compiler/PyJinjaCompiler/src/grammar/python/PythonParser.g4 by ANTLR 4.13.2 package compiler.generated.python; import org.antlr.v4.runtime.tree.ParseTreeListener;

/**

- This interface defines a complete listener for a parse tree produced by
- {@link PythonParser}. / *public interface PythonParserListener extends ParseTreeListener { /*

 - Enter a parse tree produced by {@link PythonParser#program}.
 - @param ctx the parse tree / *void enterProgram(PythonParser.ProgramContext ctx); /**
 - Exit a parse tree produced by {@link PythonParser#program}.
 - @param ctx the parse tree / *void exitProgram(PythonParser.ProgramContext ctx); /**
 - Enter a parse tree produced by the {@code returnStatement}
 - labeled alternative in {@link PythonParser#statement}.
 - @param ctx the parse tree / *void enterReturnStatement(PythonParser.ReturnStatementContext ctx); /**
 - Exit a parse tree produced by the {@code returnStatement}
 - labeled alternative in {@link PythonParser#statement}.
 - @param ctx the parse tree / *void exitReturnStatement(PythonParser.ReturnStatementContext ctx); /**
 - Enter a parse tree produced by the {@code assignOrExprStatement}
 - labeled alternative in {@link PythonParser#statement}.
 - @param ctx the parse tree / *void enterAssignOrExprStatement(PythonParser.AssignOrExprStatementContext ctx); /**
 - Exit a parse tree produced by the {@code assignOrExprStatement}
 - labeled alternative in {@link PythonParser#statement}.
 - @param ctx the parse tree / *void exitAssignOrExprStatement(PythonParser.AssignOrExprStatementContext ctx); /**
 - Enter a parse tree produced by the {@code functionDef}
 - labeled alternative in {@link PythonParser#statement}.
 - @param ctx the parse tree / *void enterFunctionDef(PythonParser.FunctionDefContext ctx); /**
 - Exit a parse tree produced by the {@code functionDef}
 - labeled alternative in {@link PythonParser#statement}.
 - @param ctx the parse tree / *void exitFunctionDef(PythonParser.FunctionDefContext ctx); /**
 - Enter a parse tree produced by the {@code ifStatement}
 - labeled alternative in {@link PythonParser#statement}.
 - @param ctx the parse tree / *void enterIfStatement(PythonParser.IfStatementContext ctx); /**
 - Exit a parse tree produced by the {@code ifStatement}
 - labeled alternative in {@link PythonParser#statement}.
 - @param ctx the parse tree / *void exitIfStatement(PythonParser.IfStatementContext ctx); /**
 - Enter a parse tree produced by the {@code whileStatement}
 - labeled alternative in {@link PythonParser#statement}.
 - @param ctx the parse tree / *void enterWhileStatement(PythonParser.WhileStatementContext ctx); /***

- Exit a parse tree produced by the {@code whileStatement}
- labeled alternative in {@link PythonParser#statement}.
- @param ctx the parse tree / *void exitWhileStatement(PythonParser.WhileStatementContext ctx); /**
- Enter a parse tree produced by the {@code forStatement}
- labeled alternative in {@link PythonParser#statement}.
- @param ctx the parse tree / *void enterForStatement(PythonParser.ForStatementContext ctx); /**
- Exit a parse tree produced by the {@code forStatement}
- labeled alternative in {@link PythonParser#statement}.
- @param ctx the parse tree / *void exitForStatement(PythonParser.ForStatementContext ctx); /**
- Enter a parse tree produced by {@link PythonParser#decorator}.
- @param ctx the parse tree / *void enterDecorator(PythonParser.DecoratorContext ctx); /**
- Exit a parse tree produced by {@link PythonParser#decorator}.
- @param ctx the parse tree / *void exitDecorator(PythonParser.DecoratorContext ctx); /**
- Enter a parse tree produced by {@link PythonParser#paramList}.
- @param ctx the parse tree / *void enterParamList(PythonParser.ParamListContext ctx); /**
- Exit a parse tree produced by {@link PythonParser#paramList}.
- @param ctx the parse tree / *void exitParamList(PythonParser.ParamListContext ctx); /**
- Enter a parse tree produced by the {@code singleSuite}
- labeled alternative in {@link PythonParser#suite}.
- @param ctx the parse tree / *void enterSingleSuite(PythonParser.SingleSuiteContext ctx); /**
- Exit a parse tree produced by the {@code singleSuite}
- labeled alternative in {@link PythonParser#suite}.
- @param ctx the parse tree / *void exitSingleSuite(PythonParser.SingleSuiteContext ctx); /**
- Enter a parse tree produced by the {@code indentedSuite}
- labeled alternative in {@link PythonParser#suite}.
- @param ctx the parse tree / *void enterIndentedSuite(PythonParser.IndentedSuiteContext ctx); /**
- Exit a parse tree produced by the {@code indentedSuite}
- labeled alternative in {@link PythonParser#suite}.
- @param ctx the parse tree / *void exitIndentedSuite(PythonParser.IndentedSuiteContext ctx); /**
- Enter a parse tree produced by {@link PythonParser#simpleLine}.
- @param ctx the parse tree / *void enterSimpleLine(PythonParser.SimpleLineContext ctx); /**
- Exit a parse tree produced by {@link PythonParser#simpleLine}.
- @param ctx the parse tree / *void exitSimpleLine(PythonParser.SimpleLineContext ctx); /**
- Enter a parse tree produced by the {@code notExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / *void enterNotExpr(PythonParser.NotExprContext ctx); /**
- Exit a parse tree produced by the {@code notExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / *void exitNotExpr(PythonParser.NotExprContext ctx); /**
- Enter a parse tree produced by the {@code addExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / *void enterAddExpr(PythonParser.AddExprContext ctx); /**
- Exit a parse tree produced by the {@code addExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / *void exitAddExpr(PythonParser.AddExprContext ctx); /**
- Enter a parse tree produced by the {@code unaryMinus}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / *void enterUnaryMinus(PythonParser.UnaryMinusContext ctx); /**
- Exit a parse tree produced by the {@code unaryMinus}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / *void exitUnaryMinus(PythonParser.UnaryMinusContext ctx); /**
- Enter a parse tree produced by the {@code mulExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / *void enterMulExpr(PythonParser.MulExprContext ctx); /**
- Exit a parse tree produced by the {@code mulExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / *void exitMulExpr(PythonParser.MulExprContext ctx); /**
- Enter a parse tree produced by the {@code atomExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / *void enterAtomExpr(PythonParser.AtomExprContext ctx); /**
- Exit a parse tree produced by the {@code atomExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / *void exitAtomExpr(PythonParser.AtomExprContext ctx); /**
- Enter a parse tree produced by the {@code orExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / *void enterOrExpr(PythonParser.OrExprContext ctx); /**

- Exit a parse tree produced by the {@code orExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / void exitOrExpr(PythonParser.OrExprContext ctx); /*
- Enter a parse tree produced by the {@code comparisonExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / void enterComparisonExpr(PythonParser.ComparisonExprContext ctx); /*
- Exit a parse tree produced by the {@code comparisonExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / void exitComparisonExpr(PythonParser.ComparisonExprContext ctx); /*
- Enter a parse tree produced by the {@code andExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / void enterAndExpr(PythonParser.AndExprContext ctx); /*
- Exit a parse tree produced by the {@code andExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree / void exitAndExpr(PythonParser.AndExprContext ctx); /*
- Enter a parse tree produced by {@link PythonParser#listItems}.
- @param ctx the parse tree / void enterListItems(PythonParser.ListItemsContext ctx); /*
- Exit a parse tree produced by {@link PythonParser#listItems}.
- @param ctx the parse tree / void exitListItems(PythonParser.ListItemsContext ctx); /*
- Enter a parse tree produced by the {@code parenExpr}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void enterParenExpr(PythonParser.ParenExprContext ctx); /*
- Exit a parse tree produced by the {@code parenExpr}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void exitParenExpr(PythonParser.ParenExprContext ctx); /*
- Enter a parse tree produced by the {@code listExpr}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void enterListExpr(PythonParser.ListExprContext ctx); /*
- Exit a parse tree produced by the {@code listExpr}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void exitListExpr(PythonParser.ListExprContext ctx); /*
- Enter a parse tree produced by the {@code dictExpr}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void enterDictExpr(PythonParser.DictExprContext ctx); /*
- Exit a parse tree produced by the {@code dictExpr}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void exitDictExpr(PythonParser.DictExprContext ctx); /*
- Enter a parse tree produced by the {@code stringLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void enterStringLiteral(PythonParser.StringLiteralContext ctx); /*
- Exit a parse tree produced by the {@code stringLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void exitStringLiteral(PythonParser.StringLiteralContext ctx); /*
- Enter a parse tree produced by the {@code intLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void enterIntLiteral(PythonParser.IntLiteralContext ctx); /*
- Exit a parse tree produced by the {@code intLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void exitIntLiteral(PythonParser.IntLiteralContext ctx); /*
- Enter a parse tree produced by the {@code floatLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void enterFloatLiteral(PythonParser.FloatLiteralContext ctx); /*
- Exit a parse tree produced by the {@code floatLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void exitFloatLiteral(PythonParser.FloatLiteralContext ctx); /*
- Enter a parse tree produced by the {@code trueLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void enterTrueLiteral(PythonParser.TrueLiteralContext ctx); /*
- Exit a parse tree produced by the {@code trueLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void exitTrueLiteral(PythonParser.TrueLiteralContext ctx); /*
- Enter a parse tree produced by the {@code falseLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void enterFalseLiteral(PythonParser.FalseLiteralContext ctx); /*
- Exit a parse tree produced by the {@code falseLiteral}
- labeled alternative in {@link PythonParser#atom}.

- @param ctx the parse tree / void exitFalseLiteral(PythonParser.FalseLiteralContext ctx); /*
- Enter a parse tree produced by the {@code noneLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void enterNoneLiteral(PythonParser.NoneLiteralContext ctx); /*
- Exit a parse tree produced by the {@code noneLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void exitNoneLiteral(PythonParser.NoneLiteralContext ctx); /*
- Enter a parse tree produced by the {@code identifierAtom}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void enterIdentifierAtom(PythonParser.IdentifierAtomContext ctx); /*
- Exit a parse tree produced by the {@code identifierAtom}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree / void exitIdentifierAtom(PythonParser.IdentifierAtomContext ctx); /*
- Enter a parse tree produced by the {@code attributeAccess}
- labeled alternative in {@link PythonParser#atomTrailer}.
- @param ctx the parse tree / void enterAttributeAccess(PythonParser.AttributeAccessContext ctx); /*
- Exit a parse tree produced by the {@code attributeAccess}
- labeled alternative in {@link PythonParser#atomTrailer}.
- @param ctx the parse tree / void exitAttributeAccess(PythonParser.AttributeAccessContext ctx); /*
- Enter a parse tree produced by the {@code indexAccess}
- labeled alternative in {@link PythonParser#atomTrailer}.
- @param ctx the parse tree / void enterIndexAccess(PythonParser.IndexAccessContext ctx); /*
- Exit a parse tree produced by the {@code indexAccess}
- labeled alternative in {@link PythonParser#atomTrailer}.
- @param ctx the parse tree / void exitIndexAccess(PythonParser.IndexAccessContext ctx); /*
- Enter a parse tree produced by the {@code callTrailer}
- labeled alternative in {@link PythonParser#atomTrailer}.
- @param ctx the parse tree / void enterCallTrailer(PythonParser.CallTrailerContext ctx); /*
- Exit a parse tree produced by the {@code callTrailer}
- labeled alternative in {@link PythonParser#atomTrailer}.
- @param ctx the parse tree / void exitCallTrailer(PythonParser.CallTrailerContext ctx); /*
- Enter a parse tree produced by {@link PythonParser#argList}.
- @param ctx the parse tree / void enterArgList(PythonParser.ArgListContext ctx); /*
- Exit a parse tree produced by {@link PythonParser#argList}.
- @param ctx the parse tree / void exitArgList(PythonParser.ArgListContext ctx); /*
- Enter a parse tree produced by the {@code positionalArg}
- labeled alternative in {@link PythonParser#argument}.
- @param ctx the parse tree / void enterPositionalArg(PythonParser.PositionalArgContext ctx); /*
- Exit a parse tree produced by the {@code positionalArg}
- labeled alternative in {@link PythonParser#argument}.
- @param ctx the parse tree / void exitPositionalArg(PythonParser.PositionalArgContext ctx); /*
- Enter a parse tree produced by the {@code keywordArg}
- labeled alternative in {@link PythonParser#argument}.
- @param ctx the parse tree / void enterKeywordArg(PythonParser.KeywordArgContext ctx); /*
- Exit a parse tree produced by the {@code keywordArg}
- labeled alternative in {@link PythonParser#argument}.
- @param ctx the parse tree / void exitKeywordArg(PythonParser.KeywordArgContext ctx); /*
- Enter a parse tree produced by {@link PythonParser#dictItems}.
- @param ctx the parse tree / void enterDictItems(PythonParser.DictItemsContext ctx); /*
- Exit a parse tree produced by {@link PythonParser#dictItems}.
- @param ctx the parse tree / void exitDictItems(PythonParser.DictItemsContext ctx); /*
- Enter a parse tree produced by {@link PythonParser#dictItem}.
- @param ctx the parse tree / void enterDictItem(PythonParser.DictItemContext ctx); /*
- Exit a parse tree produced by {@link PythonParser#dictItem}.
- @param ctx the parse tree */ void exitDictItem(PythonParser.DictItemContext ctx); } .

?? File: PythonParserVisitor.java

// Generated from D:/Compiler/PyJinjaCompiler/src/grammar/python/PythonParser.g4 by ANTLR 4.13.2 package compiler.generated.python; import org.antlr.v4.runtime.tree.ParseTreeVisitor;

/**

- This interface defines a complete generic visitor for a parse tree produced
- by {@link PythonParser}.
-
- @param The return type of the visit operation. Use {@link Void} for

- operations with no return type. / *public interface PythonParserVisitor extends ParseTreeVisitor { /**
 - Visit a parse tree produced by {@link PythonParser#program}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitProgram(PythonParser.ProgramContext ctx); /**
 - Visit a parse tree produced by the {@code returnStatement}
 - labeled alternative in {@link PythonParser#statement}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitReturnStatement(PythonParser.ReturnStatementContext ctx); /**
 - Visit a parse tree produced by the {@code assignOrExprStatement}
 - labeled alternative in {@link PythonParser#statement}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitAssignOrExprStatement(PythonParser.AssignOrExprStatementContext ctx); /**
 - Visit a parse tree produced by the {@code functionDef}
 - labeled alternative in {@link PythonParser#statement}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitFunctionDef(PythonParser.FunctionDefContext ctx); /**
 - Visit a parse tree produced by the {@code ifStatement}
 - labeled alternative in {@link PythonParser#statement}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitIfStatement(PythonParser.IfStatementContext ctx); /**
 - Visit a parse tree produced by the {@code whileStatement}
 - labeled alternative in {@link PythonParser#statement}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitWhileStatement(PythonParser.WhileStatementContext ctx); /**
 - Visit a parse tree produced by the {@code forStatement}
 - labeled alternative in {@link PythonParser#statement}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitForStatement(PythonParser.ForStatementContext ctx); /**
 - Visit a parse tree produced by {@link PythonParser#decorator}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitDecorator(PythonParser.DecoratorContext ctx); /**
 - Visit a parse tree produced by {@link PythonParser#paramList}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitParamList(PythonParser.ParamListContext ctx); /**
 - Visit a parse tree produced by the {@code singleSuite}
 - labeled alternative in {@link PythonParser#suite}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitSingleSuite(PythonParser.SingleSuiteContext ctx); /**
 - Visit a parse tree produced by the {@code indentedSuite}
 - labeled alternative in {@link PythonParser#suite}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitIndentedSuite(PythonParser.IndentedSuiteContext ctx); /**
 - Visit a parse tree produced by {@link PythonParser#simpleLine}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitSimpleLine(PythonParser.SimpleLineContext ctx); /**
 - Visit a parse tree produced by the {@code notExpr}
 - labeled alternative in {@link PythonParser#expr}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitNotExpr(PythonParser.NotExprContext ctx); /**
 - Visit a parse tree produced by the {@code addExpr}
 - labeled alternative in {@link PythonParser#expr}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitAddExpr(PythonParser.AddExprContext ctx); /**
 - Visit a parse tree produced by the {@code unaryMinus}
 - labeled alternative in {@link PythonParser#expr}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitUnaryMinus(PythonParser.UnaryMinusContext ctx); /**
 - Visit a parse tree produced by the {@code mulExpr}
 - labeled alternative in {@link PythonParser#expr}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitMulExpr(PythonParser.MulExprContext ctx); /**
 - Visit a parse tree produced by the {@code atomExpr}
 - labeled alternative in {@link PythonParser#expr}.
 - @param ctx the parse tree
 - @return the visitor result / *T visitAtomExpr(PythonParser.AtomExprContext ctx); /**
 - Visit a parse tree produced by the {@code orExpr}

- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree
- @return the visitor result / *T visitOrExpr(PythonParser.OrExprContext ctx); /**
- Visit a parse tree produced by the {@code comparisonExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree
- @return the visitor result / *T visitComparisonExpr(PythonParser.ComparisonExprContext ctx); /**
- Visit a parse tree produced by the {@code andExpr}
- labeled alternative in {@link PythonParser#expr}.
- @param ctx the parse tree
- @return the visitor result / *T visitAndExpr(PythonParser.AndExprContext ctx); /**
- Visit a parse tree produced by {@link PythonParser#listItems}.
- @param ctx the parse tree
- @return the visitor result / *T visitListItems(PythonParser.ListItemsContext ctx); /**
- Visit a parse tree produced by the {@code parenExpr}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree
- @return the visitor result / *T visitParenExpr(PythonParser.ParenExprContext ctx); /**
- Visit a parse tree produced by the {@code listExpr}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree
- @return the visitor result / *T visitListExpr(PythonParser.ListExprContext ctx); /**
- Visit a parse tree produced by the {@code dictExpr}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree
- @return the visitor result / *T visitDictExpr(PythonParser.DictExprContext ctx); /**
- Visit a parse tree produced by the {@code stringLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree
- @return the visitor result / *T visitStringLiteral(PythonParser.StringLiteralContext ctx); /**
- Visit a parse tree produced by the {@code intLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree
- @return the visitor result / *T visitIntLiteral(PythonParser.IntLiteralContext ctx); /**
- Visit a parse tree produced by the {@code floatLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree
- @return the visitor result / *T visitFloatLiteral(PythonParser.FloatLiteralContext ctx); /**
- Visit a parse tree produced by the {@code trueLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree
- @return the visitor result / *T visitTrueLiteral(PythonParser.TrueLiteralContext ctx); /**
- Visit a parse tree produced by the {@code falseLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree
- @return the visitor result / *T visitFalseLiteral(PythonParser.FalseLiteralContext ctx); /**
- Visit a parse tree produced by the {@code noneLiteral}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree
- @return the visitor result / *T visitNoneLiteral(PythonParser.NoneLiteralContext ctx); /**
- Visit a parse tree produced by the {@code identifierAtom}
- labeled alternative in {@link PythonParser#atom}.
- @param ctx the parse tree
- @return the visitor result / *T visitIdentifierAtom(PythonParser.IdentifierAtomContext ctx); /**
- Visit a parse tree produced by the {@code attributeAccess}
- labeled alternative in {@link PythonParser#atomTrailer}.
- @param ctx the parse tree
- @return the visitor result / *T visitAttributeAccess(PythonParser.AttributeAccessContext ctx); /**
- Visit a parse tree produced by the {@code indexAccess}
- labeled alternative in {@link PythonParser#atomTrailer}.
- @param ctx the parse tree
- @return the visitor result / *T visitIndexAccess(PythonParser.IndexAccessContext ctx); /**
- Visit a parse tree produced by the {@code callTrailer}
- labeled alternative in {@link PythonParser#atomTrailer}.
- @param ctx the parse tree
- @return the visitor result / *T visitCallTrailer(PythonParser.CallTrailerContext ctx); /**

- Visit a parse tree produced by {@link PythonParser#argList}.
- @param ctx the parse tree
- @return the visitor result / *T visitArgList(PythonParser.ArgListContext ctx); /**
- Visit a parse tree produced by the {@code positionalArg}
- labeled alternative in {@link PythonParser#argument}.
- @param ctx the parse tree
- @return the visitor result / *T visitPositionalArg(PythonParser.PositionalArgContext ctx); /**
- Visit a parse tree produced by the {@code keywordArg}
- labeled alternative in {@link PythonParser#argument}.
- @param ctx the parse tree
- @return the visitor result / *T visitKeywordArg(PythonParser.KeywordArgContext ctx); /**
- Visit a parse tree produced by {@link PythonParser#dictItems}.
- @param ctx the parse tree
- @return the visitor result / *T visitDictItems(PythonParser.DictItemsContext ctx); /**
- Visit a parse tree produced by {@link PythonParser#dictItem}.
- @param ctx the parse tree
- @return the visitor result */ *T visitDictItem(PythonParser.DictItemContext ctx); }* .

?? File: ExpressionEvaluator.java

package compiler.generator;

import compiler.ast.common.*; import compiler.ast.python.DictEntry; import compiler.ast.python.DictExpr; import compiler.ast.python.ListExpr;

import java.util.ArrayList; import java.util.LinkedHashMap; import java.util.List; import java.util.Map;

/**

- Evaluates Expression nodes against a runtime environment
- (Map<String, Object>), producing plain Java values: Long, Double,
- String, Boolean, null, List, or Map<String, Object>.
-
- Since compiler.ast.common's expression nodes (Identifier, literals,
- BinaryExpr, UnaryExpr, AttributeAccess, IndexAccess, CallExpr) are
- shared by both the Python tree and the Jinja tree, this one evaluator
- covers both:
- - Generator uses it to turn `products = [{"name": ..., ...}, ...]`

- into a real Java List/Map.

- - Renderer uses it to evaluate `{{ p.price }} / {% if p.price > 350 %}`

- against that data while walking the Jinja Template.

-
- Deliberately restricted: no user-defined function calls, no
- assignment, no side effects - just the data-literal + arithmetic +
- comparison subset both grammars actually support. */ public final class ExpressionEvaluator {
- public Object evaluate(Expression expr, Map<String, Object> env) {

```

    if (expr instanceof IntegerLiteral) {
        return ((IntegerLiteral) expr).getValue();

    } else if (expr instanceof FloatLiteral) {
        return ((FloatLiteral) expr).getValue();

    } else if (expr instanceof StringLiteral) {
        return ((StringLiteral) expr).getValue();

    } else if (expr instanceof BooleanLiteral) {
        return ((BooleanLiteral) expr).getValue();

    } else if (expr instanceof NoneLiteral) {
        return null;

    } else if (expr instanceof Identifier) {
        return env.get(((Identifier) expr).getName());

    } else if (expr instanceof ListExpr) {
        List<Object> list = new ArrayList<>();
        for (Expression element : ((ListExpr) expr).getElements()) {
            list.add(evaluate(element, env));
        }
        return list;

    } else if (expr instanceof DictExpr) {
        Map<String, Object> map = new LinkedHashMap<>();
        for (DictEntry entry : ((DictExpr) expr).getEntries()) {
            map.put(keyOf(entry.getKey()), evaluate(entry.getValue(), env));
        }
        return map;

    } else if (expr instanceof UnaryExpr) {
        return evaluateUnary((UnaryExpr) expr, env);

    } else if (expr instanceof BinaryExpr) {
        return evaluateBinary((BinaryExpr) expr, env);

    } else if (expr instanceof AttributeAccess) {
        AttributeAccess access = (AttributeAccess) expr;
        Object target = evaluate(access.getTarget(), env);
        if (target instanceof Map) {
            return ((Map<?, ?>) target).get(access.getAttribute());
        }
        return null;

    } else if (expr instanceof IndexAccess) {
        IndexAccess access = (IndexAccess) expr;
        Object target = evaluate(access.getTarget(), env);
        Object index = evaluate(access.getIndex(), env);
        return evaluateIndex(target, index);

    } else if (expr instanceof CallExpr) {
        return evaluateCall((CallExpr) expr, env);
    }

    return null;

```

```

}

```

```

private String keyOf(Expression key) { if (key instanceof StringLiteral) { return ((StringLiteral) key).getValue(); } if (key instanceof Identifier) { return ((Identifier) key).getName(); } return String.valueOf(key); }

```

```

private Object evaluateIndex(Object target, Object index) {

```

```

    if (target instanceof List && index instanceof Number) {
        List<?> list = (List<?>) target;
        int i = ((Number) index).intValue();
        return (i >= 0 && i < list.size()) ? list.get(i) : null;
    }

    if (target instanceof Map) {
        return ((Map<?, ?>) target).get(String.valueOf(index));
    }

    if (target instanceof String && index instanceof Number) {
        String s = (String) target;
        int i = ((Number) index).intValue();
        return (i >= 0 && i < s.length()) ? String.valueOf(s.charAt(i)) : null;
    }

    return null;
}

```

```

}

```

```

private Object evaluateUnary(UnaryExpr expr, Map<String, Object> env) {

```

```

    Object value = evaluate(expr.getExpr(), env);

    switch (expr.getOperator()) {
        case NOT:
            return !truthy(value);
        case MINUS:
            if (value instanceof Long) return -((Long) value);
            if (value instanceof Double) return -((Double) value);
            return null;
        case PLUS:
            return value;
        default:
            return null;
    }
}

```

```

}

```

```

private Object evaluateBinary(BinaryExpr expr, Map<String, Object> env) {

```

```

BinaryOperator op = expr.getOperator();

// AND / OR short-circuit like Python: return the operand value,
// not a coerced boolean.
if (op == BinaryOperator.AND) {
    Object left = evaluate(expr.getLeft(), env);
    return truthy(left) ? evaluate(expr.getRight(), env) : left;
}
if (op == BinaryOperator.OR) {
    Object left = evaluate(expr.getLeft(), env);
    return truthy(left) ? left : evaluate(expr.getRight(), env);
}

Object left = evaluate(expr.getLeft(), env);
Object right = evaluate(expr.getRight(), env);

switch (op) {
    case EQ: return equalsValue(left, right);
    case NE: return !equalsValue(left, right);
    case LT: return compare(left, right) < 0;
    case LE: return compare(left, right) <= 0;
    case GT: return compare(left, right) > 0;
    case GE: return compare(left, right) >= 0;
    default: break;
}

if (op == BinaryOperator.ADD && left instanceof String && right instanceof String) {
    return (String) left + (String) right;
}

if (isNumeric(left) && isNumeric(right)) {
    boolean useDouble = (left instanceof Double) || (right instanceof Double);
    double l = ((Number) left).doubleValue();
    double r = ((Number) right).doubleValue();

    Double result;
    switch (op) {
        case ADD: result = l + r; break;
        case SUBTRACT: result = l - r; break;
        case MULTIPLY: result = l * r; break;
        case DIVIDE: result = r == 0 ? null : l / r; break;
        case MODULO: result = r == 0 ? null : l % r; break;
        default: return null;
    }

    if (result == null) return null;
    return useDouble ? result : (Object) result.longValue();
}

return null;

```

```

}

```

```

private Object evaluateCall(CallExpr call, Map<String, Object> env) {

```



```

    if (!(call.getCallee() instanceof Identifier)) {
        return null;
    }

    String name = ((Identifier) call.getCallee()).getName();

    List<Object> args = new ArrayList<>();
    for (Argument arg : call.getArguments()) {
        if (arg instanceof PositionalArgument) {
            args.add(evaluate(((PositionalArgument) arg).getValue(), env));
        }
    }

    switch (name) {
        case "len":
            if (args.isEmpty()) return null;
            Object a0 = args.get(0);
            if (a0 instanceof List) return (long) ((List<?>) a0).size();
            if (a0 instanceof Map) return (long) ((Map<?, ?>) a0).size();
            if (a0 instanceof String) return (long) ((String) a0).length();
            return null;

        case "str":
            return args.isEmpty() ? "" : toDisplayString(args.get(0));

        case "int":
            if (args.isEmpty()) return 0L;
            if (args.get(0) instanceof Number) return ((Number) args.get(0)).longValue();
            try { return Long.parseLong(String.valueOf(args.get(0))); } catch (NumberFormatException e) { return null; }

        case "float":
            if (args.isEmpty()) return 0.0;
            if (args.get(0) instanceof Number) return ((Number) args.get(0)).doubleValue();
            try { return Double.parseDouble(String.valueOf(args.get(0))); } catch (NumberFormatException e) { return null; }

        case "bool":
            return args.isEmpty() ? Boolean.FALSE : truthy(args.get(0));

        case "range":
            return evaluateRange(args);

        default:
            // No user-defined functions - not something this evaluator executes.
            return null;
    }
}

```

```

}

```

```

private List evaluateRange(List args) { long start = 0, stop, step = 1; if (args.size() == 1) { stop = ((Number) args.get(0)).longValue(); } else if (args.size() >= 2) {
start = ((Number) args.get(0)).longValue(); stop = ((Number) args.get(1)).longValue(); if (args.size() >= 3) step = ((Number) args.get(2)).longValue(); } else {
return new ArrayList<>(); } List result = new ArrayList<>(); if (step > 0) { for (long i = start; i < stop; i += step) result.add(i); } else if (step < 0) { for (long i = start; i
> stop; i += step) result.add(i); } return result; }

```

```

private boolean isNumeric(Object o) { return o instanceof Long || o instanceof Double; }

```

```

private boolean equalsValue(Object left, Object right) { if (isNumeric(left) && isNumeric(right)) { return ((Number) left).doubleValue() == ((Number)
right).doubleValue(); } if (left == null) return right == null; return left.equals(right); }

```

```

@SuppressWarnings("unchecked") private int compare(Object left, Object right) { if (isNumeric(left) && isNumeric(right)) { return Double.compare(((Number)
left).doubleValue(), ((Number) right).doubleValue()); } if (left instanceof String && right instanceof String) { return ((String) left).compareTo((String) right); } if (left
instanceof Comparable && right != null && left.getClass().isInstance(right)) { return ((Comparable) left).compareTo(right); } return 0; // Not meaningfully
comparable - treat as equal rather than throw. }

```

```

public boolean truthy(Object o) { if (o == null) return false; if (o instanceof Boolean) return (Boolean) o; if (o instanceof Number) return ((Number)
o).doubleValue() != 0; if (o instanceof String) return !((String) o).isEmpty(); if (o instanceof List) return !((List) o).isEmpty(); if (o instanceof Map)
o).isEmpty(); return true; }

```

```

/** Python-flavored display string, since the data originated from Python literals. */ public String toDisplayString(Object value) { if (value == null) return "None"; if
(value instanceof Boolean) return ((Boolean) value) ? "True" : "False"; if (value instanceof Double) { double d = (Double) value; if (d == Math.floor(d) &&
!Double.isInfinite(d)) { return String.valueOf((long) d) + ".0"; } return String.valueOf(d); } return String.valueOf(value); } }

```

?? File: Generator.java

```
package compiler.generator;
```

```
import compiler.ast.python.; import compiler.ast.common.;
```

```
import compiler.semantic.common.Scope; import compiler.semantic.common.Symbol; import compiler.semantic.common.SymbolTable; import
compiler.semantic.common.Type;
```

```
import java.util.ArrayList; import java.util.LinkedHashMap; import java.util.List; import java.util.Map;
```

```
/**
```

- Generator: bridges the Python tree and the Jinja tree (requirement 2).
-
- PythonSemanticAnalyzer.getSymbolTable(), once analyze() has returned,
- sits back at the GLOBAL scope - every FunctionDef enters a FUNCTION
- scope and symmetrically exits it, so a variable declared inside a
- route function (e.g. products inside def index(): products = [...])
- is no longer reachable through SymbolTable.lookup() from that global
- scope. Handing that table straight to JinjaSemanticAnalyzer therefore
- only works for module-level Python variables, not the Flask-style
- render_template("index.jinja", title=title, products=products)
- pattern used in test01.py / test02.py.
-
- This class finds that render_template(...) call and, while walking
- toward it, maintains a real evaluated environment (via
- ExpressionEvaluator) of every simple assignment seen along the way.
- That gives two things per call site:
- - contextValues: actual Java values (List/Map/String/...) - what

```
Rendered needs to produce real HTML.
```

- - contextSymbolTable: a fresh SymbolTable of the same names, typed

```
from those values - what JinjaSemanticAnalyzer needs to type-check
```

```
the template before it's ever rendered.
```

- Where a value can't be statically evaluated (e.g. it came from a
- user-defined function call), the PythonSemanticAnalyzer-produced
- SymbolTable is used as a type-only fallback. */ public final class Generator {

```
private static final String RENDER_TEMPLATE = "render_template"; private static final String RENDER_TEMPLATE_STRING =
"render_template_string";
```

```
private final ExpressionEvaluator evaluator = new ExpressionEvaluator();
```

```
/**
```

- One resolved render_template(...) call site. */ public static final class RenderCall {

```
private final String templateName; private final int line; private final Map<String, Object> contextValues; private final SymbolTable contextSymbolTable;
```

```
public RenderCall( String templateName, int line, Map<String, Object> contextValues, SymbolTable contextSymbolTable) {
```

```
    this.templateName = templateName;
    this.line = line;
    this.contextValues = contextValues;
    this.contextSymbolTable = contextSymbolTable;
```

```
}
```

```
public String getTemplateName() { return templateName; }
```

```
public int getLine() { return line; }
```

```
/** Real data, ready for Renderer.render(template, contextValues). */ public Map<String, Object> getContextValues() { return contextValues; }
```

```
/** Ready to pass straight into new JinjaSemanticAnalyzer(...). */ public SymbolTable getContextSymbolTable() { return contextSymbolTable; }}
```

```
/**
```

- @param program the parsed Python AST.
- @param analyzedSymbols the SymbolTable produced by

- PythonSemanticAnalyzer.analyze(program) -

- used only as a type-only fallback for

- values ExpressionEvaluator can't statically

- resolve (searches every scope, not just

- the currently-active one).

- @return one RenderCall per render_template(...) / render_template_string(...)

- call found anywhere in the program (typically one per route

- function).

```
*/ public List generate(Program program, SymbolTable analyzedSymbols) {
```

```
    Map<String, Symbol> allSymbolsByName = indexAllScopes(analyzedSymbols);

    List<RenderCall> results = new ArrayList<>();
    for (Statement stmt : program.getStatements()) {
        scanStatement(stmt, new LinkedHashMap<>(), allSymbolsByName, results);
    }

    return results;
```

```
}
```

```
/* ----- Indexing every scope by symbol name (type-only fallback) ----- */
```

```
private Map<String, Symbol> indexAllScopes(SymbolTable symbolTable) { Map<String, Symbol> byName = new LinkedHashMap<>(); for (Scope
scope : symbolTable.getAllScopes()) { byName.putAll(scope.getSymbols()); } return byName; }
```

```
/* ----- Statement scan, tracking a real evaluated environment ----- */
```

```
private void scanStatement( Statement stmt, Map<String, Object> env, Map<String, Symbol> symbols, List results) {
```

```
    if (stmt instanceof FunctionDef) {
        // Fresh environment per function - Python variables don't
        // leak across functions.
        Map<String, Object> functionEnv = new LinkedHashMap<>();
        for (Statement inner : ((FunctionDef) stmt).getBody()) {
            scanStatement(inner, functionEnv, symbols, results);
        }

    } else if (stmt instanceof Assign) {
        handleAssign((Assign) stmt, env);
        checkForRenderCall(((Assign) stmt).getValue(), env, symbols, results);

    } else if (stmt instanceof ExprStmt) {
        checkForRenderCall(((ExprStmt) stmt).getExpression(), env, symbols, results);

    } else if (stmt instanceof ReturnStmt) {
        ((ReturnStmt) stmt).getValue()
            .ifPresent(expr -> checkForRenderCall(expr, env, symbols, results));

    } else if (stmt instanceof IfStmt) {
        IfStmt ifStmt = (IfStmt) stmt;
        for (Statement inner : ifStmt.getThenBody()) scanStatement(inner, env, symbols, results);
        for (ElifClause elif : ifStmt.getElifClauses()) {
            for (Statement inner : elif.getBody()) scanStatement(inner, env, symbols, results);
        }
        if (ifStmt.getElseClause() != null) {
            for (Statement inner : ifStmt.getElseClause().getBody()) scanStatement(inner, env, symbols, results);
        }

    } else if (stmt instanceof ForStmt) {
        for (Statement inner : ((ForStmt) stmt).getBody()) scanStatement(inner, env, symbols, results);

    } else if (stmt instanceof WhileStmt) {
        for (Statement inner : ((WhileStmt) stmt).getBody()) scanStatement(inner, env, symbols, results);
    }
}
```

```
}
```

```
private void handleAssign(Assign assign, Map<String, Object> env) {
```

```
    if (assign.getTargets().size() != 1) {
        return; // a = b = c = 0 style chains aren't data sources here
    }

    Expression target = assign.getTargets().get(0);
    if (!(target instanceof Identifier)) {
        return;
    }

    String name = ((Identifier) target).getName();
    env.put(name, evaluator.evaluate(assign.getValue(), env));
}
```

```
}
```

```
private void checkForRenderCall( Expression expr, Map<String, Object> env, Map<String, Symbol> symbols, List results) {
```

```

        if (!(expr instanceof CallExpr)) {
            return;
        }

        CallExpr call = (CallExpr) expr;
        if (!(call.getCallee() instanceof Identifier)) {
            return;
        }

        String calleeName = ((Identifier) call.getCallee()).getName();
        if (!calleeName.equals(RENDER_TEMPLATE) && !calleeName.equals(RENDER_TEMPLATE_STRING)) {
            return;
        }

        String templateName = null;
        Map<String, Object> contextValues = new LinkedHashMap<>();
        SymbolTable contextTable = new SymbolTable();

        for (Argument arg : call.getArguments()) {

            if (arg instanceof PositionalArgument) {
                Object value = evaluator.evaluate(((PositionalArgument) arg).getValue(), env);
                if (templateName == null && value instanceof String) {
                    templateName = (String) value;
                }
            }

            else if (arg instanceof KeywordArgument) {
                KeywordArgument kw = (KeywordArgument) arg;

                Object value = evaluator.evaluate(kw.getValue(), env);
                Type type = typeOf(value);

                if (value == null) {
                    // Couldn't statically evaluate it (or it's a real
                    // None) - fall back to the symbol's declared type,
                    // if one is known.
                    Symbol resolved = resolveSymbol(kw.getName(), kw.getValue(), symbols);
                    if (resolved != null) {
                        type = resolved.getInferredType();
                    }
                }

                contextValues.put(kw.getName(), value);

                Symbol contextSymbol = new Symbol(kw.getName(), Symbol.Kind.VARIABLE, kw.getLine());
                contextSymbol.setInferredType(type);
                contextTable.addSymbol(contextSymbol);
            }
        }

        results.add(new RenderCall(templateName, call.getLine(), contextValues, contextTable));
    }

}

/**

```

- Resolves a `render_template(...)` keyword argument's value back to a
- declared `Symbol`, as a type-only fallback for whatever
- `ExpressionEvaluator` couldn't statically resolve (e.g. the result
- of a user-defined function call). `*/ private Symbol resolveSymbol(String keywordName, Expression value, Map<String, Symbol> symbols) { if (value instanceof Identifier) { return symbols.get(((Identifier) value).getName()); } return symbols.get(keywordName); }`

```

private Type typeOf(Object value) { if (value == null) return Type.NONE; if (value instanceof Long) return Type.INTEGER; if (value instanceof Double)
return Type.FLOAT; if (value instanceof String) return Type.STRING; if (value instanceof Boolean) return Type.BOOLEAN; if (value instanceof List)
return Type.LIST; if (value instanceof Map) return Type.DICTIONARY; return Type.UNKNOWN; } }

```

?? File: Renderer.java

```
package compiler.generator;
```

```
import compiler.ast.jinja.*;
```

```
import java.util.ArrayList; import java.util.HashMap; import java.util.List; import java.util.Map;
```

```
/**
```

- Renderer: the actual "code generation" step (requirement 5). Walks a
- Jinja Template AST together with a data context (typically
- Generator.RenderCall.getContextValues()) and produces the final
- rendered HTML as a String - proving the Python-extracted data and the
- Jinja tree work together end to end.

-

- Expression evaluation ({% ... %}), {% if ... %}, {% for ... %}'s
- iterable) is delegated to ExpressionEvaluator, the same evaluator
- Generator uses to pull data out of the Python tree in the first
- place. */ public final class Renderer {

```
    private final ExpressionEvaluator evaluator = new ExpressionEvaluator();
```

```
    public String render(Template template, Map<String, Object> context) { StringBuilder out = new StringBuilder();
    renderElements(template.getElements(), context, out); return out.toString(); }
```

```
    private void renderElements(List elements, Map<String, Object> env, StringBuilder out) { for (TemplateElement element : elements) {
    renderElement(element, env, out); } }
```

```
    private void renderElement(TemplateElement element, Map<String, Object> env, StringBuilder out) {
```

```
        if (element instanceof HtmlText) {
            out.append(((HtmlText) element).getText());

        } else if (element instanceof ExpressionOutput) {
            Object value = evaluator.evaluate(((ExpressionOutput) element).getExpression(), env);
            out.append(evaluator.toString(value));

        } else if (element instanceof JinjaIfStmt) {
            renderIf((JinjaIfStmt) element, env, out);

        } else if (element instanceof JinjaForStmt) {
            renderFor((JinjaForStmt) element, env, out);
        }
    }
```

```
}
```

```
    private void renderIf(JinjaIfStmt node, Map<String, Object> env, StringBuilder out) {
```

```

        if (evaluator.truthy(evaluator.evaluate(node.getCondition(), env))) {
            renderElements(node.getThenBody(), env, out);
            return;
        }

        for (JinjaElifClause elif : node.getElifClauses()) {
            if (evaluator.truthy(evaluator.evaluate(elif.getCondition(), env))) {
                renderElements(elif.getBody(), env, out);
                return;
            }
        }

        if (node.getElseClause() != null) {
            renderElements(node.getElseClause().getBody(), env, out);
        }
    }
}

```

```

}

```

```

private void renderFor(JinjaForStmt node, Map<String, Object> env, StringBuilder out) {

```

```

    Object iterableValue = evaluator.evaluate(node.getIterable(), env);
    List<Object> items = toIterable(iterableValue);
    String varName = node.getVariable().getName();

    for (Object item : items) {
        Map<String, Object> loopEnv = new HashMap<>(env);
        loopEnv.put(varName, item);
        renderElements(node.getBody(), loopEnv, out);
    }
}

```

```

}

```

```

@SuppressWarnings("unchecked") private List toIterable(Object value) {

```

```

    if (value instanceof List) {
        return (List<Object>) value;
    }

    if (value instanceof Map) {
        // Python dict iteration yields keys.
        return new ArrayList<>(((Map<Object, Object>) value).keySet());
    }

    if (value instanceof String) {
        List<Object> chars = new ArrayList<>();
        for (char c : ((String) value).toCharArray()) {
            chars.add(String.valueOf(c));
        }
        return chars;
    }

    return List.of(); // non-iterable - NON_ITERABLE_IN_FOR should already have flagged this
}

```

```

}}

```

```

.

```

?? File: ASTPrinter.java

```

package compiler.printer;

```

```

import compiler.ast.common.AstNode;

```

```

import javax.swing.; import java.awt.; import java.awt.event.*; import java.lang.reflect.Method; import java.util.ArrayList; import java.util.Collection;
import java.util.List;

```

/**

- Printer for custom AST nodes with console text and interactive GUI visualization modes.

- Uses reflection to traverse AST node structure since nodes don't extend ANTLR classes. */ public class ASTPrinter {

/**

- Prints a highly legible console text outline with indentation mapping custom AST nodes.
- Displays node types, properties, and hierarchical structure. */ public static void printText(AstNode root) { if (root == null) { System.out.println("AST is null"); return; } String text = toTextTree(root, 0, ""); System.out.println(text); }

private static String toTextTree(AstNode node, int level, String prefix) { StringBuilder builder = new StringBuilder();

```
// Build indentation
for (int i = 0; i < level; i++) {
    builder.append("  ", "");
}

// Add branch marker for non-root
if (level > 0) {
    builder.append("â€”â€”â€” ");
}

// Add node name and line
builder.append(node.getNodeName())
    .append(" [line ")
    .append(node.getLine())
    .append("]");

// Add properties using reflection
List<PropertyInfo> properties = extractProperties(node);
if (!properties.isEmpty()) {
    builder.append(" (");
    for (int i = 0; i < properties.size(); i++) {
        if (i > 0) {
            builder.append(", ");
        }
        PropertyInfo prop = properties.get(i);
        builder.append(prop.name).append("=").append(formatValue(prop.value));
    }
    builder.append(")");
}

builder.append("\n");

// Recursively process child AST nodes
List<AstNode> children = extractChildNodes(node);
for (AstNode child : children) {
    builder.append(toTextTree(child, level + 1, prefix));
}

return builder.toString();
```

}

/**

- Extracts properties from an AST node using reflection.

- Returns primitive/string properties (not collections or other AST nodes). */ private static List extractProperties(AstNode node) { List properties = new ArrayList<>(); Class<?> clazz = node.getClass();

for (Method method : clazz.getMethods()) { String methodName = method.getName();


```

        // Skip methods from Object, AstNode, or non-getters
        if (methodName.equals("getClass") ||
            methodName.equals("getNodeName") ||
            methodName.equals("getLine") ||
            methodName.equals("accept") ||
            methodName.equals("prettyPrint") ||
            methodName.equals("toString") ||
            !methodName.startsWith("get") && !methodName.startsWith("is")) {
            continue;
        }

        // Skip if no parameters or not public
        if (method.getParameterCount() != 0 || !java.lang.reflect.Modifier.isPublic(method.getModifiers()))
            continue;
    }

    try {
        Object value = method.invoke(node);

        // Skip null values, AST nodes, and collections
        if (value == null || value instanceof AstNode || value instanceof Collection) {
            continue;
        }

        // Extract property name from method name
        String propName;
        if (methodName.startsWith("get")) {
            propName = methodName.substring(3);
            propName = Character.toLowerCase(propName.charAt(0)) + propName.substring(1);
        } else if (methodName.startsWith("is")) {
            propName = methodName.substring(2);
            propName = Character.toLowerCase(propName.charAt(0)) + propName.substring(1);
        } else {
            continue;
        }

        properties.add(new PropertyInfo(propName, value));
    } catch (Exception e) {
        // Skip methods that can't be invoked
    }
}

return properties; }

/**

```

- Extracts child AST nodes from a node using reflection.
- Looks for methods that return single AST nodes or collections of AST nodes. `*/ private static List extractChildNodes(AstNode node) { List children = new ArrayList<>(); Class<?> clazz = node.getClass();`

`for (Method method : clazz.getMethods()) { String methodName = method.getName();`

```

// Skip methods from Object, AstNode, or non-getters
if (methodName.equals("getClass") ||
    methodName.equals("getNodeName") ||
    methodName.equals("getLine") ||
    methodName.equals("accept") ||
    methodName.equals("prettyPrint") ||
    methodName.equals("toString") ||
    !methodName.startsWith("get") && !methodName.startsWith("is")) {
    continue;
}

if (method.getParameterCount() != 0 || !java.lang.reflect.Modifier.isPublic(method.getModifiers()))
    continue;
}

try {
    Object value = method.invoke(node);

    if (value == null) {
        continue;
    }

    // Handle single AST node
    if (value instanceof AstNode) {
        children.add((AstNode) value);
    }
    // Handle collections of AST nodes
    else if (value instanceof Collection) {
        for (Object item : (Collection<?>) value) {
            if (item instanceof AstNode) {
                children.add((AstNode) item);
            }
        }
    }
} catch (Exception e) {
    // Skip methods that can't be invoked
}

```

```

}

```

```

return children;
}

```

```

private static String formatValue(Object value) { if (value == null) { return "null"; } if (value instanceof String) { return "" + value + ""; } return
value.toString(); }

```

```

/**

```

- Opens a custom visual layout panel with zoom-and-drag support.
- Displays custom AST Node classes and properties with color-coding. */ public static void showGuiTree(AstNode root) { if (root == null) {
JOptionPane.showMessageDialog(null, "AST is null", "Error", JOptionPane.ERROR_MESSAGE); return; }

```

JFrame frame = new JFrame("AST Visualizer (Drag to Pan | Scroll to Zoom)");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); frame.setLayout(new BorderLayout());

```

```

// Create the custom AST canvas ASTCanvas canvas = new ASTCanvas(root);

```

```

// Wrap in scroll pane JScrollPane scrollPane = new JScrollPane(canvas); scrollPane.getVerticalScrollBar().setUnitIncrement(16);
scrollPane.getHorizontalScrollBar().setUnitIncrement(16); scrollPane.setPreferredSize(new Dimension(1200, 800));

```

```

frame.add(scrollPane, BorderLayout.CENTER); frame.pack(); frame.setLocationRelativeTo(null); frame.setVisible(true); }

```

```

/**

```

- Custom canvas component for rendering AST trees with zoom and pan support. */ private static class ASTCanvas extends JPanel {
private final AstNode root; private double scale = 1.0; private Point dragStartPoint; private final List nodeLayouts = new ArrayList<>();

private static final int NODE_WIDTH = 160; private static final int NODE_HEIGHT = 50; private static final int VERTICAL_SPACING =
70; private static final int HORIZONTAL_SPACING = 30;

```
// Colors private static final Color NODE_COLOR = new Color(255, 228, 196); // Light tan/bisque private static final Color
BORDER_COLOR = new Color(139, 69, 19); // Saddle brown private static final Color TEXT_COLOR = Color.BLACK; private static final
Color PROPERTY_COLOR = new Color(0, 100, 0); // Dark green private static final Color LINE_COLOR = new Color(100, 100, 100);
```

```
public ASTCanvas(AstNode root) { this.root = root; setBackground(Color.WHITE); setLayout(null);
```

```
    // Calculate node layouts
    calculateLayouts();

    // Setup mouse listeners for zoom and pan
    setupMouseListeners();
```

```
}
```

```
private void calculateLayouts() { nodeLayouts.clear(); calculateSubtreeLayout(root, 0, 0); }
```

```
private int calculateSubtreeLayout(AstNode node, int depth, int startX) { List children = extractChildNodes(node); int childCount =
children.size(); int totalWidth = 0;
```

```
    if (childCount == 0) {
        // Leaf node
        int x = startX;
        int y = depth * (NODE_HEIGHT + VERTICAL_SPACING) + 20;
        nodeLayouts.add(new ASTNodeLayout(node, x, y, NODE_WIDTH, NODE_HEIGHT));
        return NODE_WIDTH + HORIZONTAL_SPACING;
    }

    // Internal node - calculate children first
    int currentX = startX;
    for (int i = 0; i < childCount; i++) {
        int childWidth = calculateSubtreeLayout(children.get(i), depth + 1, currentX);
        currentX += childWidth;
        totalWidth += childWidth;
    }

    // Center this node above its children
    int firstChildX = nodeLayouts.stream()
        .filter(l -> isChildOf(l.node, node))
        .mapToInt(l -> l.x)
        .min()
        .orElse(startX);

    int lastChildX = nodeLayouts.stream()
        .filter(l -> isChildOf(l.node, node))
        .mapToInt(l -> l.x + l.width)
        .max()
        .orElse(startX + NODE_WIDTH);

    int x = (firstChildX + lastChildX) / 2 - NODE_WIDTH / 2;
    int y = depth * (NODE_HEIGHT + VERTICAL_SPACING) + 20;

    nodeLayouts.add(new ASTNodeLayout(node, x, y, NODE_WIDTH, NODE_HEIGHT));

    return Math.max(totalWidth, NODE_WIDTH + HORIZONTAL_SPACING);
```

```
}
```

```
private boolean isChildOf(AstNode potentialChild, AstNode parent) { List parentChildren = extractChildNodes(parent); return
parentChildren.contains(potentialChild); }
```

```
private void setupMouseListeners() { // Mouse wheel for zooming addMouseWheelListener(new MouseWheelListener() { @Override
public void mouseWheelMoved(MouseWheelEvent e) { if (e.getWheelRotation() < 0) { scale = Math.min(4.0, scale + 0.1); } else { scale =
Math.max(0.4, scale - 0.1); } revalidate(); repaint(); } });
```

```

// Mouse adapter for drag-to-pan
MouseListener mouseAdapter = new MouseAdapter() {
    @Override
    public void mousePressed(MouseEvent e) {
        dragStartPoint = e.getPoint();
        setCursor(Cursor.getPredefinedCursor(Cursor.MOVE_CURSOR));
    }

    @Override
    public void mouseReleased(MouseEvent e) {
        dragStartPoint = null;
        setCursor(Cursor.getDefaultCursor());
    }

    @Override
    public void mouseDragged(MouseEvent e) {
        if (dragStartPoint == null) return;

        int deltaX = dragStartPoint.x - e.getX();
        int deltaY = dragStartPoint.y - e.getY();

        JScrollPane scrollPane = (JScrollPane) SwingUtilities.getAncestorOfClass(JScrollPane.class,
        if (scrollPane != null) {
            JViewport viewport = scrollPane.getViewport();
            Point viewPos = viewport.getViewPosition();
            viewPos.translate(deltaX, deltaY);

            // Clamp to valid range
            int maxX = Math.max(0, getWidth() - viewport.getWidth());
            int maxY = Math.max(0, getHeight() - viewport.getHeight());

            viewPos.x = Math.max(0, Math.min(viewPos.x, maxX));
            viewPos.y = Math.max(0, Math.min(viewPos.y, maxY));

            viewport.setViewPosition(viewPos);
        }

        dragStartPoint = e.getPoint();
    }
};

addMouseListener(mouseAdapter);
addMouseMotionListener(mouseAdapter);

```

```

}

```

```

@Override public Dimension getPreferredSize() { if (nodeLayouts.isEmpty()) { return new Dimension(800, 600); }

```

```

    int maxX = nodeLayouts.stream().mapToInt(l -> l.x + l.width).max().orElse(0);
    int maxY = nodeLayouts.stream().mapToInt(l -> l.y + l.height).max().orElse(0);

    return new Dimension((int) (maxX * scale) + 50, (int) (maxY * scale) + 50);

```

```

}

```

```

@Override protected void paintComponent(Graphics g) { super.paintComponent(g);

```

```

Graphics2D g2d = (Graphics2D) g;
g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
g2d.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING, RenderingHints.VALUE_TEXT_ANTIALIAS_ON);

g2d.scale(scale, scale);

// Draw connections first (behind nodes)
drawConnections(g2d);

// Draw nodes
for (ASTNodeLayout layout : nodeLayouts) {
    drawNode(g2d, layout);
}

```

}

private void drawConnections(Graphics2D g2d) { g2d.setColor(LINE_COLOR); g2d.setStroke(new BasicStroke(1.5f));

```

for (ASTNodeLayout layout : nodeLayouts) {
    AstNode node = layout.node;
    List<AstNode> children = extractChildNodes(node);

    if (!children.isEmpty()) {
        Point parentCenter = new Point(
            layout.x + layout.width / 2,
            layout.y + layout.height
        );

        for (AstNode child : children) {
            ASTNodeLayout childLayout = findLayout(child);
            if (childLayout != null) {
                Point childCenter = new Point(
                    childLayout.x + childLayout.width / 2,
                    childLayout.y
                );
                g2d.drawLine(parentCenter.x, parentCenter.y, childCenter.x, childCenter.y);
            }
        }
    }
}

```

}

private void drawNode(Graphics2D g2d, ASTNodeLayout layout) { AstNode node = layout.node;

```

// Draw rounded rectangle
g2d.setColor(NODE_COLOR);
g2d.fillRoundRect(layout.x, layout.y, layout.width, layout.height, 10, 10);

// Draw border
g2d.setColor(BORDER_COLOR);
g2d.setStroke(new BasicStroke(2.0f));
g2d.drawRoundRect(layout.x, layout.y, layout.width, layout.height, 10, 10);

// Draw node name (bold)
g2d.setColor(TEXT_COLOR);
g2d.setFont(new Font("SansSerif", Font.BOLD, 12));

String nodeName = node.getNodeName();
FontMetrics boldFm = g2d.getFontMetrics();
int nameWidth = boldFm.stringWidth(nodeName);
int nameX = layout.x + (layout.width - nameWidth) / 2;
int nameY = layout.y + 20;

g2d.drawString(nodeName, nameX, nameY);

// Draw line number (smaller)
g2d.setFont(new Font("SansSerif", Font.PLAIN, 10));
String lineText = "line " + node.getLine();
FontMetrics smallFm = g2d.getFontMetrics();
int lineWidth = smallFm.stringWidth(lineText);
int lineX = layout.x + (layout.width - lineWidth) / 2;
int lineY = layout.y + 32;

g2d.drawString(lineText, lineX, lineY);

// Draw properties (if any)
List<PropertyInfo> properties = extractProperties(node);
if (!properties.isEmpty()) {
    g2d.setColor(PROPERTY_COLOR);
    g2d.setFont(new Font("SansSerif", Font.ITALIC, 9));

    String propText = formatProperties(properties);
    String truncatedProp = truncateText(propText, layout.width - 10, g2d.getFont());

    FontMetrics propFm = g2d.getFontMetrics();
    int propWidth = propFm.stringWidth(truncatedProp);
    int propX = layout.x + (layout.width - propWidth) / 2;
    int propY = layout.y + 44;

    g2d.drawString(truncatedProp, propX, propY);
}

```

```

}

```

```

private String formatProperties(List properties) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < properties.size(); i++) {
        if (i > 0) {
            sb.append(", ");
        }
        PropertyInfo prop = properties.get(i);
        sb.append(prop.name).append("=").append(formatValue(prop.value));
    }
    return sb.toString();
}

```

```

private String truncateText(String text, int maxWidth, Font font) {
    FontMetrics fm = getFontMetrics(font);
    if (fm.stringWidth(text) <=
        maxWidth) {
        return text;
    }
}

```

```
String ellipsis = "...";
int ellipsisWidth = fm.stringWidth(ellipsis);
int availableWidth = maxWidth - ellipsisWidth;

for (int i = text.length() - 1; i > 0; i--) {
    String truncated = text.substring(0, i);
    if (fm.stringWidth(truncated) <= availableWidth) {
        return truncated + ellipsis;
    }
}

return ellipsis;
```

```
}
```

```
private ASTNodeLayout findLayout(AstNode node) { return nodeLayouts.stream().filter(l -> l.node == node).findFirst().orElse(null); }
```

```
private static class ASTNodeLayout { final AstNode node; final int x; final int y; final int width; final int height;
```

```
ASTNodeLayout(AstNode node, int x, int y, int width, int height) {
    this.node = node;
    this.x = x;
    this.y = y;
    this.width = width;
    this.height = height;
}
```

```
}}
```

```
private static class PropertyInfo { final String name; final Object value;
```

```
PropertyInfo(String name, Object value) {
    this.name = name;
    this.value = value;
}
```

```
}}
```

?? File: ParseTreePrinter.java

```
package compiler.printer;
```

```
import org.antlr.v4.runtime.Parser; import org.antlr.v4.runtime.ParserRuleContext; import org.antlr.v4.runtime.Token; import
org.antlr.v4.runtime.tree.ParseTree; import org.antlr.v4.runtime.tree.TerminalNode; import org.antlr.v4.runtime.tree.Trees;
```

```
import javax.swing.; import java.awt.; import java.awt.event.*; import java.util.ArrayList; import java.util.List;
```

```
/**
```

```
■ Printer for ANTLR parse trees with console text and interactive GUI visualization modes. */ public class ParseTreePrinter {
```

```
/**
```

- Outputs a clean console ASCII hierarchical tree structure using branch symbols.
- Resolves rule names dynamically via parser rule vocabulary. */ public static void printText(ParseTree tree, Parser parser) { String
asciiTree = toAsciiTree(tree, parser, 0, new StringBuilder()); System.out.println(asciiTree); }

```
private static String toAsciiTree(ParseTree tree, Parser parser, int level, StringBuilder prefix) { StringBuilder builder = new StringBuilder();
```

```

// Build the prefix for current level
for (int i = 0; i < level; i++) {
    builder.append("  ", "");
}

// Add branch marker for non-root nodes
if (level > 0) {
    builder.append("â€”â€”â€” ");
}

// Get node text
String nodeText = getNodeText(tree, parser);
builder.append(nodeText).append("\n");

// Process children
int childCount = tree.getChildCount();
for (int i = 0; i < childCount; i++) {
    ParseTree child = tree.getChild(i);
    builder.append(toAsciiTree(child, parser, level + 1, prefix));
}

return builder.toString();

```

```

}

```

```

private static String getNodeText(ParseTree tree, Parser parser) { if (tree instanceof TerminalNode) { TerminalNode terminal = (TerminalNode)
tree; Token token = terminal.getSymbol(); String tokenName = parser.getVocabulary().getSymbolicName(token.getType()); if (tokenName ==
null) { tokenName = "TOKEN"; } String tokenText = escapeTokenText(token.getText()); return tokenName + " " + tokenText + ""; } else if (tree
instanceof ParserRuleContext) { ParserRuleContext context = (ParserRuleContext) tree; int ruleIndex = context.getRuleIndex(); String ruleName
= parser.getRuleNames()[ruleIndex]; return ruleName; } return tree.toString(); }

```

```

private static String escapeTokenText(String text) { if (text == null) { return ""; } return text .replace("\n", "\n") .replace("\r", "\r") .replace("\t", "\t"); }

```

```

/**

```

```

    ■ Opens an interactive vector-drawn Swing Canvas with zoom and pan support.

    ■ Features anti-aliasing, MouseWheelListener zooming, MouseAdapter drag-to-pan,

    ■ and color-coding for rule contexts (light blue) vs terminal tokens (light green). */ public static void showGuiTree(ParseTree tree, Parser
parser) { JFrame frame = new JFrame("Parse Tree Visualizer (Drag to Pan | Scroll to Zoom)");
frame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE); frame.setLayout(new BorderLayout());

// Create the custom tree canvas ParseTreeCanvas canvas = new ParseTreeCanvas(tree, parser);

// Wrap in scroll pane JScrollPane scrollPane = new JScrollPane(canvas); scrollPane.getVerticalScrollBar().setUnitIncrement(16);
scrollPane.getHorizontalScrollBar().setUnitIncrement(16); scrollPane.setPreferredSize(new Dimension(1200, 800));

frame.add(scrollPane, BorderLayout.CENTER); frame.pack(); frame.setLocationRelativeTo(null); frame.setVisible(true); }

```

```

/**

```

```

    ■ Custom canvas component for rendering parse trees with zoom and pan support. */ private static class ParseTreeCanvas extends
JPanel { private final ParseTree tree; private final Parser parser; private double scale = 1.0; private Point dragStartPoint; private final List
nodeLayouts = new ArrayList<>();

private static final int NODE_WIDTH = 120; private static final int NODE_HEIGHT = 30; private static final int VERTICAL_SPACING =
50; private static final int HORIZONTAL_SPACING = 20;

// Colors private static final Color RULE_CONTEXT_COLOR = new Color(173, 216, 230); // Light blue private static final Color
TERMINAL_COLOR = new Color(144, 238, 144); // Light green private static final Color BORDER_COLOR = Color.BLACK; private
static final Color TEXT_COLOR = Color.BLACK;

public ParseTreeCanvas(ParseTree tree, Parser parser) { this.tree = tree; this.parser = parser; setBackground(Color.WHITE);
setLayout(null);

```



```

// Calculate node layouts
calculateLayouts();

// Setup mouse listeners for zoom and pan
setupMouseListeners();

```

```

}

```

```

private void calculateLayouts() { nodeLayouts.clear(); calculateSubtreeLayout(tree, 0, 0); }

```

```

private int calculateSubtreeLayout(ParseTree node, int depth, int startX) { int childCount = node.getChildCount(); int totalWidth = 0;

```

```

    if (childCount == 0) {
        // Leaf node
        int x = startX;
        int y = depth * (NODE_HEIGHT + VERTICAL_SPACING) + 20;
        nodeLayouts.add(new TreeNodeLayout(node, x, y, NODE_WIDTH, NODE_HEIGHT));
        return NODE_WIDTH + HORIZONTAL_SPACING;
    }

    // Internal node - calculate children first
    int currentX = startX;
    for (int i = 0; i < childCount; i++) {
        int childWidth = calculateSubtreeLayout(node.getChild(i), depth + 1, currentX);
        currentX += childWidth;
        totalWidth += childWidth;
    }

    // Center this node above its children
    int firstChildX = nodeLayouts.stream()
        .filter(l -> l.node.getParent() == node)
        .mapToInt(l -> l.x)
        .min()
        .orElse(startX);

    int lastChildX = nodeLayouts.stream()
        .filter(l -> l.node.getParent() == node)
        .mapToInt(l -> l.x + l.width)
        .max()
        .orElse(startX + NODE_WIDTH);

    int x = (firstChildX + lastChildX) / 2 - NODE_WIDTH / 2;
    int y = depth * (NODE_HEIGHT + VERTICAL_SPACING) + 20;

    nodeLayouts.add(new TreeNodeLayout(node, x, y, NODE_WIDTH, NODE_HEIGHT));

    return Math.max(totalWidth, NODE_WIDTH + HORIZONTAL_SPACING);

```

```

}

```

```

private void setupMouseListeners() { // Mouse wheel for zooming addMouseListener(new MouseWheelListener() { @Override
public void mouseWheelMoved(MouseWheelEvent e) { if (e.getWheelRotation() < 0) { scale = Math.min(4.0, scale + 0.1); } else { scale =
Math.max(0.4, scale - 0.1); } revalidate(); repaint(); } });

```

```

// Mouse adapter for drag-to-pan
MouseListener mouseAdapter = new MouseAdapter() {
    @Override
    public void mousePressed(MouseEvent e) {
        dragStartPoint = e.getPoint();
        setCursor(Cursor.getPredefinedCursor(Cursor.MOVE_CURSOR));
    }

    @Override
    public void mouseReleased(MouseEvent e) {
        dragStartPoint = null;
        setCursor(Cursor.getDefaultCursor());
    }

    @Override
    public void mouseDragged(MouseEvent e) {
        if (dragStartPoint == null) return;

        int deltaX = dragStartPoint.x - e.getX();
        int deltaY = dragStartPoint.y - e.getY();

        JScrollPane scrollPane = (JScrollPane) SwingUtilities.getAncestorOfClass(JScrollPane.class,
        if (scrollPane != null) {
            JViewport viewport = scrollPane.getViewport();
            Point viewPos = viewport.getViewPosition();
            viewPos.translate(deltaX, deltaY);

            // Clamp to valid range
            int maxX = Math.max(0, getWidth() - viewport.getWidth());
            int maxY = Math.max(0, getHeight() - viewport.getHeight());

            viewPos.x = Math.max(0, Math.min(viewPos.x, maxX));
            viewPos.y = Math.max(0, Math.min(viewPos.y, maxY));

            viewport.setViewPosition(viewPos);
        }

        dragStartPoint = e.getPoint();
    }
};

addMouseListener(mouseAdapter);
addMouseMotionListener(mouseAdapter);

```

```

}

```

```

@Override public Dimension getPreferredSize() { if (nodeLayouts.isEmpty()) { return new Dimension(800, 600); }

```

```

    int maxX = nodeLayouts.stream().mapToInt(l -> l.x + l.width).max().orElse(0);
    int maxY = nodeLayouts.stream().mapToInt(l -> l.y + l.height).max().orElse(0);

    return new Dimension((int) (maxX * scale) + 50, (int) (maxY * scale) + 50);

```

```

}

```

```

@Override protected void paintComponent(Graphics g) { super.paintComponent(g);

```

```

Graphics2D g2d = (Graphics2D) g;
g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
g2d.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING, RenderingHints.VALUE_TEXT_ANTIALIAS_ON);

g2d.scale(scale, scale);

// Draw connections first (behind nodes)
drawConnections(g2d);

// Draw nodes
for (TreeNodeLayout layout : nodeLayouts) {
    drawNode(g2d, layout);
}

```

}

private void drawConnections(Graphics2D g2d) { g2d.setColor(Color.GRAY); g2d.setStroke(new BasicStroke(1.5f));

```

for (TreeNodeLayout layout : nodeLayouts) {
    ParseTree node = layout.node;
    if (node.getChildCount() > 0) {
        Point parentCenter = new Point(
            layout.x + layout.width / 2,
            layout.y + layout.height
        );

        for (int i = 0; i < node.getChildCount(); i++) {
            ParseTree child = node.getChild(i);
            TreeNodeLayout childLayout = findLayout(child);
            if (childLayout != null) {
                Point childCenter = new Point(
                    childLayout.x + childLayout.width / 2,
                    childLayout.y
                );
                g2d.drawLine(parentCenter.x, parentCenter.y, childCenter.x, childCenter.y);
            }
        }
    }
}

```

}

private void drawNode(Graphics2D g2d, TreeNodeLayout layout) { ParseTree node = layout.node;

```

// Determine color based on node type
Color fillColor;
if (node instanceof TerminalNode) {
    fillColor = TERMINAL_COLOR;
} else {
    fillColor = RULE_CONTEXT_COLOR;
}

// Draw rounded rectangle
g2d.setColor(fillColor);
g2d.fillRoundRect(layout.x, layout.y, layout.width, layout.height, 8, 8);

// Draw border
g2d.setColor(BORDER_COLOR);
g2d.setStroke(new BasicStroke(1.5f));
g2d.drawRoundRect(layout.x, layout.y, layout.width, layout.height, 8, 8);

// Draw text
g2d.setColor(TEXT_COLOR);
g2d.setFont(new Font("SansSerif", Font.PLAIN, 11));

String nodeText = getNodeText(node, parser);
String truncatedText = truncateText(nodeText, layout.width - 10);

FontMetrics fm = g2d.getFontMetrics();
int textWidth = fm.stringWidth(truncatedText);
int textX = layout.x + (layout.width - textWidth) / 2;
int textY = layout.y + (layout.height + fm.getAscent()) / 2 - 2;

g2d.drawString(truncatedText, textX, textY);

```

```

}

```

```

private String truncateText(String text, int maxWidth) { FontMetrics fm = getFontMetrics(new Font("SansSerif", Font.PLAIN, 11)); if
(fm.stringWidth(text) <= maxWidth) { return text; }

```

```

String ellipsis = "...";
int ellipsisWidth = fm.stringWidth(ellipsis);
int availableWidth = maxWidth - ellipsisWidth;

for (int i = text.length() - 1; i > 0; i--) {
    String truncated = text.substring(0, i);
    if (fm.stringWidth(truncated) <= availableWidth) {
        return truncated + ellipsis;
    }
}

return ellipsis;

```

```

}

```

```

private TreeNodeLayout findLayout(ParseTree node) { return nodeLayouts.stream().filter(l -> l.node == node).findFirst().orElse(null); }

```

```

private static class TreeNodeLayout { final ParseTree node; final int x; final int y; final int width; final int height;

```

```

TreeNodeLayout(ParseTree node, int x, int y, int width, int height) {
    this.node = node;
    this.x = x;
    this.y = y;
    this.width = width;
    this.height = height;
}

```

```

}}}

```

?? File: TokenPrinter.java

```
package compiler.printer;
```

```
import org.antlr.v4.runtime.Lexer; import org.antlr.v4.runtime.Token;
```

```
import javax.swing.; import javax.swing.table.DefaultTableModel; import java.awt.;
```

```
/**
```

- Printer for lexer tokens with console text and GUI visualization modes. */ public class TokenPrinter {

```
    /**
```

- Streams all tokens from the lexer and prints a clean two-column console layout.

- Columns: TOKEN and TEXT (with escaped \n, \r, \t characters). */ public static void printText(Lexer lexer) { System.out.println(""); System.out.printf("%-25s %-25s\n", "TOKEN", "TEXT"); System.out.println("");

```
    Token token; do { token = lexer.nextToken(); if (token.getType() == Token.EOF) { break; }
```

```
        String tokenName = lexer.getVocabulary().getSymbolicName(token.getType());
        if (tokenName == null) {
            tokenName = "UNKNOWN";
        }
```

```
        String tokenText = escapeTokenText(token.getText());
```

```
        System.out.printf("%-25s %-25s\n", tokenName, "\"" + tokenText + "\"");           } while (token.getT
```

```
    }
```

```
    /**
```

- Escapes special characters in token text for clean console output. */ private static String escapeTokenText(String text) { if (text == null) { return ""; } return text .replace("\n", "\\n") .replace("\r", "\\r") .replace("\t", "\\t"); }

```
    /**
```

- Opens a clean Swing JFrame window displaying Token Name and Text value rows visually. */ public static void showGuiList(Lexer lexer) { JFrame frame = new JFrame("Token Viewer"); frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); frame.setLayout(new BorderLayout());

```
    // Create table model with columns DefaultTableModel model = new DefaultTableModel(); model.addColumn("Token");
    model.addColumn("Text");
```

```
    // Populate table with tokens Token token; do { token = lexer.nextToken(); if (token.getType() == Token.EOF) { break; }
```

```
        String tokenName = lexer.getVocabulary().getSymbolicName(token.getType());
        if (tokenName == null) {
            tokenName = "UNKNOWN";
        }
```

```
        String tokenText = escapeTokenText(token.getText());
        model.addRow(new Object[] { tokenName, tokenText });
```

```
    } while (token.getType() != Token.EOF);
```

```
    // Create table with model JTable table = new JTable(model); table.setAutoResizeMode(JTable.AUTO_RESIZE_ALL_COLUMNS);
    table.setRowHeight(25); table.getTableHeader().setFont(table.getFont().deriveFont(Font.BOLD));
```

```
    // Enable sorting table.setAutoCreateRowSorter(true);
```

```
    // Add to scroll pane JScrollPane scrollPane = new JScrollPane(table); scrollPane.setPreferredSize(new Dimension(800, 600));
```

```
    frame.add(scrollPane, BorderLayout.CENTER); frame.pack(); frame.setLocationRelativeTo(null); frame.setVisible(true); }
```

?? File: TreeVisualizer.java

```
package compiler.printer;
```

```
import org.antlr.v4.gui.TreeViewer; import org.antlr.v4.runtime.Parser; import org.antlr.v4.runtime.tree.ParseTree; import org.antlr.v4.runtime.tree.Trees;
```

```
import javax.swing.; import java.awt.; import java.awt.event.*; import java.util.Arrays; import java.util.List;
```

```
public class TreeVisualizer {
```

```

/**
 * Renders the native ANTLR GUI TreeViewer with interactive Zoom and Drag-to-Pan support.
 */
public static void showGuiTree(ParseTree tree, Parser parser) {
    JFrame frame = new JFrame("Official ANTLR Tree Viewer (Drag to Move | Scroll Wheel to Zoom)");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    // 1. Initialize the official ANTLR GUI component
    List<String> ruleNames = Arrays.asList(parser.getRuleNames());
    TreeViewer viewer = new TreeViewer(ruleNames, tree);

    // 2. Turn off curved edges to ensure lines scale perfectly
    viewer.setUseCurvedEdges(false);

    // 3. Set up the scroll pane container first
    JScrollPane scrollPane = new JScrollPane();
    scrollPane.setVerticalScrollBar().setUnitIncrement(16);
    scrollPane.setHorizontalScrollBar().setUnitIncrement(16);

    // 4. Wrap the ANTLR viewer in our interactive controller panel
    InteractiveWrapperPanel wrapperPanel = new InteractiveWrapperPanel(viewer, scrollPane);
    scrollPane.setViewportView(wrapperPanel);

    frame.add(scrollPane);
    frame.setSize(1200, 800);
    frame.setLocationRelativeTo(null);
    frame.setVisible(true);
}

/**
 * Layout container that captures mouse movements to pan the JScrollPane view.
 */
private static class InteractiveWrapperPanel extends JPanel implements MouseWheelListener {
    private final TreeViewer antlrViewer;
    private final JScrollPane parentScrollPane;
    private double scaleFactor = 1.0;

    // Track the starting position when a drag action begins
    private Point originPoint;

    public InteractiveWrapperPanel(TreeViewer antlrViewer, JScrollPane parentScrollPane) {
        this.antlrViewer = antlrViewer;
        this.parentScrollPane = parentScrollPane;

        setLayout(new GridBagLayout());
        setBackground(Color.WHITE);
        add(antlrViewer);

        // Bind listeners for scrolling and dragging
        addMouseWheelListener(this);
        setupDragToPanSupport();

        updateAntlrViewerScale();
    }

    /**
     * Captures drag vectors to shift the viewport scrollbars smoothly
     */
    private void setupDragToPanSupport() {
        MouseAdapter mouseAdapter = new MouseAdapter() {
            @Override
            public void mousePressed(MouseEvent e) {
                // Save the mouse coordinate point when you click down
                originPoint = e.getPoint();
                setCursor(Cursor.getPredefinedCursor(Cursor.MOVE_CURSOR));
            }
        };
    }
}

```

```

    }

    @Override
    public void mouseReleased(MouseEvent e) {
        setCursor(Cursor.getPredefinedCursor(Cursor.DEFAULT_CURSOR));
    }

    @Override
    public void mouseDragged(MouseEvent e) {
        if (originPoint == null) return;

        // Calculate how far the mouse has traveled since clicking down
        int deltaX = originPoint.x - e.getX();
        int deltaY = originPoint.y - e.getY();

        // Access the viewport container configuration bounds
        JViewport viewport = parentScrollPane.getViewport();
        Point viewPosition = viewport.getViewPosition();

        // Shift the current coordinates by the drag distance
        viewPosition.translate(deltaX, deltaY);

        // Clamp layout viewport values so you don't pan into outer space
        int maxX = InteractiveWrapperPanel.this.getWidth() - viewport.getWidth();
        int maxY = InteractiveWrapperPanel.this.getHeight() - viewport.getHeight();

        if (viewPosition.x < 0) viewPosition.x = 0;
        if (viewPosition.y < 0) viewPosition.y = 0;
        if (viewPosition.x > maxX && maxX > 0) viewPosition.x = maxX;
        if (viewPosition.y > maxY && maxY > 0) viewPosition.y = maxY;

        // Update the layout positions instantly
        viewport.setViewPosition(viewPosition);
    }
};

// Register handlers with the panel infrastructure
addMouseListener(mouseAdapter);
addMouseMotionListener(mouseAdapter);
}

private void updateAntlrViewerScale() {
    antlrViewer.setScale(scaleFactor);
    antlrViewer.invalidate();
    revalidate();
    repaint();
}

@Override
public void mouseWheelMoved(MouseWheelEvent e) {
    if (e.getWheelRotation() < 0) {
        scaleFactor = Math.min(4.0, scaleFactor + 0.1);
    } else {
        scaleFactor = Math.max(0.4, scaleFactor - 0.1);
    }
    updateAntlrViewerScale();
}
}

public static void printTextTree(ParseTree tree, Parser parser) {
    // StringUtils.convertToString handles clean escaping of line breaks
    String prettyTree = toPrettyTree(tree, parser, 0);
    System.out.println(prettyTree);
}

```



```

private static String toPrettyTree(ParseTree tree, Parser parser, int level) {
    StringBuilder builder = new StringBuilder();

    // Create matching indentation spaces
    for (int i = 0; i < level; i++) {
        builder.append("  ", "");
    }

    // Add a visual branch marker
    if (level > 0) {
        builder.append("â€”â€”â€” ");
    }

    // Get the human-readable rule name or literal token value
    String nodeText = Trees.getNodeText(tree, parser);
    builder.append(nodeText).append("\n");

    // Recursively walk through child nodes
    for (int i = 0; i < tree.getChildCount(); i++) {
        builder.append(toPrettyTree(tree.getChild(i), parser, level + 1));
    }

    return builder.toString();
}

```

}

.

?? File: Scope.java

package compiler.semantic.common;

import java.util.HashMap; import java.util.Map;

/**

- Represents a lexical scope in the symbol table. */ public class Scope {

public enum ScopeKind { GLOBAL, FUNCTION, BLOCK }

private final ScopeKind kind; private final Scope parent; private final Map<String, Symbol> symbols; private final int depth;

public Scope(ScopeKind kind, Scope parent) { this.kind = kind; this.parent = parent; this.symbols = new HashMap<>(); this.depth = parent == null ? 0 : parent.depth + 1; }

public ScopeKind getKind() { return kind; }

public Scope getParent() { return parent; }

public int getDepth() { return depth; }

/**

- Add a symbol to this scope.
- Returns true if added, false if already exists in this scope (shadowing not allowed in same scope). */ public boolean addSymbol(Symbol symbol) { if (symbols.containsKey(symbol.getName())) { return false; } symbols.put(symbol.getName(), symbol); return true; }

/**

- Look up a symbol in this scope only. */ public Symbol lookupLocal(String name) { return symbols.get(name); }

/**

- Look up a symbol in this scope and all parent scopes. */ public Symbol lookup(String name) { Symbol symbol = symbols.get(name); if (symbol != null) { return symbol; } if (parent != null) { return parent.lookup(name); } return null; }

/**

- Check if a symbol exists in any parent scope (for shadowing detection). */ public boolean existsInParent(String name) { if (parent == null) { return false; } if (parent.lookupLocal(name) != null) { return true; } return parent.existsInParent(name); }

```
public Map<String, Symbol> getSymbols() { return new HashMap<>(symbols); }
```

?? File: SemanticError.java

```
package compiler.semantic.common;
```

```
/**
```

```
    ■ Represents a semantic error found during analysis. */ public class SemanticError {

    public enum ErrorType { DUPLICATE_FUNCTION_DEFINITION, UNDEFINED_VARIABLE, UNDEFINED_FUNCTION, NOT_CALLABLE,
        DUPLICATE_PARAMETER, WRONG_ARGUMENT_COUNT, NON_ITERABLE_IN_FOR, TYPE_MISMATCH }

    private final int line; private final ErrorType errorType; private final String message;

    public SemanticError(int line, ErrorType errorType, String message)

    public int getLine() { return line; }

    public ErrorType getErrorType() { return errorType; }

    public String getMessage() { return message; }

    @Override public String toString() { return String.format("Line %d: %s", line, message); }}
```

?? File: Symbol.java

```
package compiler.semantic.common;
```

```
/**
```

```
    ■ Represents a symbol in the symbol table. */ public class Symbol {

    public enum Kind { VARIABLE, FUNCTION, PARAMETER }

    private final String name; private final Kind kind; private final int declaredLine; private Type inferredType;

    public Symbol(String name, Kind kind, int declaredLine)

    public String getName() { return name; }

    public Kind getKind() { return kind; }

    public int getDeclaredLine() { return declaredLine; }

    public Type getInferredType() { return inferredType; }

    public void setInferredType(Type inferredType)

    @Override public String toString() { return String.format("%s (%s) [line %d] : %s", name, kind, declaredLine, inferredType); }}
```

?? File: SymbolTable.java

```
package compiler.semantic.common;
```

```
import java.util.ArrayList; import java.util.List;
```

```
/**
```

```
    ■ Manages the symbol table with scope hierarchy. */ public class SymbolTable {

    private Scope currentScope; private final List allScopes;

    public SymbolTable() { this.currentScope = new Scope(Scope.ScopeKind.GLOBAL, null); this.allScopes = new ArrayList<>();
        this.allScopes.add(currentScope); }

    public Scope getCurrentScope() { return currentScope; }
```

```

public void enterScope(Scope.ScopeKind kind) { Scope newScope = new Scope(kind, currentScope); currentScope = newScope;
allScopes.add(newScope); }

public void exitScope() { if (currentScope.getParent() != null) }

public boolean addSymbol(Symbol symbol) { return currentScope.addSymbol(symbol); }

public Symbol lookup(String name) { return currentScope.lookup(name); }

public List getAllScopes() { return new ArrayList<>(allScopes); }

public void print() { prettyPrint(""); }

public void prettyPrint() { prettyPrint(""); } public void prettyPrint(String indent) { for (Scope scope : allScopes) { System.out.println(indent +
"Scope (" + scope.getKind() + ") [depth " + scope.getDepth() + "]"); for (Symbol symbol : scope.getSymbols().values()) {
System.out.println(indent + " " + symbol); } System.out.println(); } } }

```

?? File: Type.java

```

package compiler.semantic.common;

/**
 *
 * ■ Simple type system for semantic analysis. */ public enum Type { UNKNOWN, INTEGER, FLOAT, BOOLEAN, STRING, LIST, DICTIONARY,
NONE, FUNCTION }

```

?? File: JinjaSemanticAnalyzer.java

```

package compiler.semantic.jinja;

import compiler.ast.jinja.; import compiler.ast.common.; import compiler.ast.python.*;

import compiler.semantic.common.Scope; import compiler.semantic.common.SemanticError; import compiler.semantic.common.Symbol; import
compiler.semantic.common.SymbolTable; import compiler.semantic.common.Type;

import java.util.ArrayList;

import java.util.List;

public class JinjaSemanticAnalyzer implements AstVisitor {

```

```

private final SymbolTable symbolTable;
private final List<SemanticError> errors;

public JinjaSemanticAnalyzer(SymbolTable symbolTable) {
    this.symbolTable = symbolTable;
    this.errors = new ArrayList<>();
}

public List<SemanticError> analyze(Template template) {
    template.accept(this);
    return new ArrayList<>(errors);
}

public SymbolTable getSymbolTable() {
    return symbolTable;
}

/* -----
   Jinja-tree nodes
   ----- */

@Override
public Void visitTemplate(Template node) {
    for (TemplateElement element : node.getElements()) {
        element.accept(this);
    }
    return null;
}

@Override
public Void visitHtmlText(HtmlText node) {
    // Raw text - nothing to check.
    return null;
}

@Override
public Void visitExpressionOutput(ExpressionOutput node) {
    node.getExpression().accept(this);
    return null;
}

@Override
public Void visitJinjaIfStmt(JinjaIfStmt node) {
    node.getCondition().accept(this);

    symbolTable.enterScope(Scope.ScopeKind.BLOCK);
    for (TemplateElement element : node.getThenBody()) {
        element.accept(this);
    }
    symbolTable.exitScope();

    for (JinjaElifClause elif : node.getElifClauses()) {
        elif.accept(this);
    }

    if (node.getElseClause() != null) {
        node.getElseClause().accept(this);
    }

    return null;
}

```

```

    }

    @Override
    public Void visitJinjaElifClause(JinjaElifClause node) {
        node.getCondition().accept(this);

        symbolTable.enterScope(Scope.ScopeKind.BLOCK);
        for (TemplateElement element : node.getBody()) {
            element.accept(this);
        }
        symbolTable.exitScope();

        return null;
    }

    @Override
    public Void visitJinjaElseClause(JinjaElseClause node) {
        symbolTable.enterScope(Scope.ScopeKind.BLOCK);
        for (TemplateElement element : node.getBody()) {
            element.accept(this);
        }
        symbolTable.exitScope();

        return null;
    }

    @Override
    public Void visitJinjaForStmt(JinjaForStmt node) {
        // Check if the iterable is actually iterable (visit it first so
        // undefined-variable errors inside the iterable expression are
        // still reported even when the type can't be inferred).
        node.getIterable().accept(this);

        Type iterableType = inferType(node.getIterable());
        if (iterableType != Type.UNKNOWN &&
            iterableType != Type.LIST &&
            iterableType != Type.STRING &&
            iterableType != Type.DICTIONARY) {
            errors.add(new SemanticError(
                node.getLine(),
                SemanticError.ErrorType.NON_ITERABLE_IN_FOR,
                "Non-iterable type '" + iterableType + "' used in for loop"
            ));
        }

        symbolTable.enterScope(Scope.ScopeKind.BLOCK);

        Type elementType = Type.UNKNOWN;

        switch (iterableType) {

            case LIST:
                elementType = Type.UNKNOWN;
                break;

            case STRING:
                elementType = Type.STRING;
                break;

            case DICTIONARY:
                elementType = Type.STRING;
                break;

            default:
                elementType = Type.UNKNOWN;
        }
    }

```

```

        Symbol loopVar = new Symbol(
            node.getVariable().getName(),
            Symbol.Kind.VARIABLE,
            node.getLine()
        );

        loopVar.setInferredType(elementType);

        symbolTable.addSymbol(loopVar);

        for (TemplateElement element : node.getBody()) {
            element.accept(this);
        }

        symbolTable.exitScope();

        return null;
    }

    /* -----
    Common expression nodes
    ----- */

    @Override
    public Void visitIdentifier(Identifier node) {
        Symbol symbol = symbolTable.lookup(node.getName());
        if (symbol == null) {
            errors.add(new SemanticError(
                node.getLine(),
                SemanticError.ErrorType.UNDEFINED_VARIABLE,
                "Undefined variable '" + node.getName() + "'"
            ));
        }
        return null;
    }

    private boolean isBinaryOperationValid(
        BinaryExpr node,
        Type left,
        Type right) {

        switch (node.getOperator()) {

            case ADD:

                if (left == Type.STRING && right == Type.STRING)
                    return true;

                if (left == Type.INTEGER && right == Type.INTEGER)
                    return true;

                if (left == Type.FLOAT && right == Type.FLOAT)
                    return true;

                if ((left == Type.INTEGER && right == Type.FLOAT) ||
                    (left == Type.FLOAT && right == Type.INTEGER))
                    return true;

                return false;

            case SUBTRACT:
            case MULTIPLY:
            case DIVIDE:
            case MODULO:

                return (left == Type.INTEGER || left == Type.FLOAT)

```

```

        &&
        (right == Type.INTEGER || right == Type.FLOAT);

    case EQ:
    case NE:

        return true;

    case LT:
    case GT:
    case LE:
    case GE:

        return left == right;

    case AND:
    case OR:

        return left == Type.BOOLEAN
            &&
            right == Type.BOOLEAN;

    default:
        return true;
    }
}

@Override
public Void visitBinaryExpr(BinaryExpr node) {

    node.getLeft().accept(this);
    node.getRight().accept(this);

    Type leftType = inferType(node.getLeft());
    Type rightType = inferType(node.getRight());

    if (leftType == Type.UNKNOWN || rightType == Type.UNKNOWN) {
        return null;
    }

    if (!isBinaryOperationValid(node, leftType, rightType)) {

        errors.add(new SemanticError(
            node.getLine(),
            SemanticError.ErrorType.TYPE_MISMATCH,
            "Invalid operation: " +
                leftType +
                " " +
                node.getOperator() +
                " " +
                rightType
        ));
    }

    return null;
}

@Override
public Void visitUnaryExpr(UnaryExpr node) {
    node.getExpr().accept(this);
    return null;
}

@Override
public Void visitCallExpr(CallExpr node) {
    String functionName = null;

```

```

Symbol calleeSymbol = null;

if (node.getCallee() instanceof Identifier) {
    functionName = ((Identifier) node.getCallee()).getName();
    calleeSymbol = symbolTable.lookup(functionName);
} else {
    // e.g. calling the result of an attribute/index access -
    // just walk it for undefined-variable checks.
    node.getCallee().accept(this);
}

if (functionName != null && calleeSymbol == null) {
    errors.add(new SemanticError(
        node.getLine(),
        SemanticError.ErrorType.UNDEFINED_FUNCTION,
        "Call to undefined function '" + functionName + "'"
    ));
} else if (calleeSymbol != null && calleeSymbol.getInferredType() != Type.FUNCTION) {
    errors.add(new SemanticError(
        node.getLine(),
        SemanticError.ErrorType.NOT_CALLABLE,
        "'" + functionName + "' is not callable"
    ));
}

for (Argument arg : node.getArguments()) {
    arg.accept(this);
}

return null;
}

@Override
public Void visitAttributeAccess(AttributeAccess node) {

    node.getTarget().accept(this);

    Type targetType = inferType(node.getTarget());

    if (targetType == Type.UNKNOWN) {

        errors.add(new SemanticError(
            node.getLine(),
            SemanticError.ErrorType.TYPE_MISMATCH,
            "Cannot access attribute '" +
                node.getAttribute() +
                "' of unknown type"));
    }

    return null;
}

@Override
public Void visitIndexAccess(IndexAccess node) {

    node.getTarget().accept(this);
    node.getIndex().accept(this);

    Type target = inferType(node.getTarget());

    if (target != Type.LIST &&
        target != Type.STRING &&
        target != Type.DICTIONARY &&
        target != Type.UNKNOWN) {

        errors.add(new SemanticError(

```



```

        node.getLine(),
        SemanticError.ErrorType.TYPE_MISMATCH,
        "Cannot index value of type " + target));
    }

    return null;
}

@Override
public Void visitPositionalArgument(PositionalArgument node) {
    node.getValue().accept(this);
    return null;
}

@Override
public Void visitKeywordArgument(KeywordArgument node) {
    // Not produced by the Jinja grammar (argList has no keyword
    // arguments), but implemented for interface completeness.
    node.getValue().accept(this);
    return null;
}

@Override
public Void visitIntegerLiteral(IntegerLiteral node) {
    return null;
}

@Override
public Void visitFloatLiteral(FloatLiteral node) {
    return null;
}

@Override
public Void visitStringLiteral(StringLiteral node) {
    return null;
}

@Override
public Void visitBooleanLiteral(BooleanLiteral node) {
    return null;
}

@Override
public Void visitNoneLiteral(NoneLiteral node) {
    return null;
}

/**
 * Infer the type of an expression for type checking (mirrors
 * SemanticAnalyzer#inferType on the Python side).
 */
private Type inferType(Expression expr) {
    if (expr instanceof IntegerLiteral) {
        return Type.INTEGER;
    } else if (expr instanceof FloatLiteral) {
        return Type.FLOAT;
    } else if (expr instanceof StringLiteral) {
        return Type.STRING;
    } else if (expr instanceof BooleanLiteral) {
        return Type.BOOLEAN;
    } else if (expr instanceof NoneLiteral) {
        return Type.NONE;
    } else if (expr instanceof Identifier) {
        Symbol symbol = symbolTable.lookup(((Identifier) expr).getName());
        if (symbol != null) {
            return symbol.getInferredType();
        }
    }
}

```

```

    }
} else if (expr instanceof BinaryExpr binary) {

    Type left = inferType(binary.getLeft());
    Type right = inferType(binary.getRight());

    switch (binary.getOperator()) {

        case ADD:

            if (left == Type.STRING &&
                right == Type.STRING)
                return Type.STRING;

            if (left == Type.FLOAT ||
                right == Type.FLOAT)
                return Type.FLOAT;

            if (left == Type.INTEGER &&
                right == Type.INTEGER)
                return Type.INTEGER;

            return Type.UNKNOWN;

        case SUBTRACT:
        case MULTIPLY:
        case DIVIDE:
        case MODULO:

            if (left == Type.FLOAT ||
                right == Type.FLOAT)
                return Type.FLOAT;

            if (left == Type.INTEGER &&
                right == Type.INTEGER)
                return Type.INTEGER;

            return Type.UNKNOWN;

        case EQ:
        case NE:
        case LT:
        case GT:
        case LE:
        case GE:
        case AND:
        case OR:

            return Type.BOOLEAN;

        default:
            return Type.UNKNOWN;
    }
} else if (expr instanceof UnaryExpr) {
    return inferType(((UnaryExpr) expr).getExpr());
} else if (expr instanceof AttributeAccess attribute) {

    Type target = inferType(attribute.getTarget());

    if (target == Type.STRING) {
        return Type.STRING;
    }

    return Type.UNKNOWN;
}
else if (expr instanceof IndexAccess index) {

```

```

        Type target = inferType(index.getTarget());

        switch (target) {

            case LIST:
                return Type.UNKNOWN;

            case STRING:
                return Type.STRING;

            case DICTIONARY:
                return Type.UNKNOWN;

            default:
                return Type.UNKNOWN;

        }
    }else if (expr instanceof CallExpr call) {

        if (call.getCallee() instanceof Identifier id) {

            Symbol symbol = symbolTable.lookup(id.getName());

            if (symbol != null) {
                return symbol.getInferredType();
            }

        }

        return Type.UNKNOWN;
    }
    return Type.UNKNOWN;
}

/* -----
Python-tree nodes
-----

Unreachable from a Jinja Template (see the mirror-image comment in
compiler.semantic.python.SemanticAnalyzer). Implemented only to
satisfy the shared AstVisitor<T> interface. */

@Override
public Void visitProgram(Program node) {
    return null;
}

@Override
public Void visitListExpr(ListExpr node) {
    return null;
}

@Override
public Void visitDictExpr(DictExpr node) {
    return null;
}

@Override
public Void visitDictEntry(DictEntry node) {
    return null;
}

@Override
public Void visitForStmt(ForStmt node) {
    return null;
}

@Override

```

```

    public Void visitAssign(Assign node) {
        return null;
    }

    @Override
    public Void visitExprStmt(ExprStmt node) {
        return null;
    }

    @Override
    public Void visitElifClause(ElifClause node) {
        return null;
    }

    @Override
    public Void visitReturnStmt(ReturnStmt node) {
        return null;
    }

    @Override
    public Void visitFunctionParameter(FunctionParameter node) {
        return null;
    }

    @Override
    public Void visitDecorator(Decorator node) {
        return null;
    }

    @Override
    public Void visitIfStmt(IfStmt node) {
        return null;
    }

    @Override
    public Void visitElseClause(ElseClause node) {
        return null;
    }

    @Override
    public Void visitWhileStmt(WhileStmt node) {
        return null;
    }

    @Override
    public Void visitFunctionDef(FunctionDef node) {
        return null;
    }
}

```

```

}

```

.

?? File: PythonSemanticAnalyzer.java

```

package compiler.semantic.python;

```

```

import compiler.ast.python.; import compiler.ast.common.AstVisitor; import compiler.ast.common.; import compiler.semantic.common.*;

```

```

import java.util.ArrayList; import java.util.HashMap; import java.util.List; import java.util.Map;

```

```

/**

```

- Semantic analyzer for Python AST.
- Performs symbol table construction and semantic checks. */ public class PythonSemanticAnalyzer implements AstVisitor {
 private final SymbolTable symbolTable; private final List errors; private final Map<String, Integer> functionParameterCounts;

```

public PythonSemanticAnalyzer() { this.symbolTable = new SymbolTable(); this.errors = new ArrayList<>(); this.functionParameterCounts = new
HashMap<>(); }

public List analyze(Program program) { program.accept(this); return new ArrayList<>(errors); }

public SymbolTable getSymbolTable() { return symbolTable; }

@Override public Void visitProgram(Program node) { for (Statement stmt : node.getStatements()) { stmt.accept(this); } return null; }

@Override public Void visitFunctionDef(FunctionDef node) { // Check for duplicate function definition Symbol existing =
symbolTable.lookup(node.getName()); if (existing != null && existing.getKind() == Symbol.Kind.FUNCTION) { errors.add(new SemanticError(
node.getLine(), SemanticError.ErrorType.DUPLICATE_FUNCTION_DEFINITION, "Duplicate function definition " + node.getName() + "" )); }
else { Symbol functionSymbol = new Symbol( node.getName(), Symbol.Kind.FUNCTION, node.getLine() );
functionSymbol.setInferredType(Type.FUNCTION); symbolTable.addSymbol(functionSymbol); }

```

```

// Store parameter count for call checking
functionParameterCounts.put(node.getName(), node.getParameters().size());

// Check for duplicate parameter names
for (int i = 0; i < node.getParameters().size(); i++) {
    FunctionParameter param = node.getParameters().get(i);
    for (int j = i + 1; j < node.getParameters().size(); j++) {
        if (param.getName().equals(node.getParameters().get(j).getName())) {
            errors.add(new SemanticError(
                node.getLine(),
                SemanticError.ErrorType.DUPLICATE_PARAMETER,
                "Duplicate parameter name '" + param.getName() + "' in function '" + node.getName()
            ));
        }
    }
}

// Enter function scope
symbolTable.enterScope(Scope.ScopeKind.FUNCTION);

// Add parameters to scope
for (FunctionParameter param : node.getParameters()) {
    Symbol paramSymbol = new Symbol(
        param.getName(),
        Symbol.Kind.PARAMETER,
        param.getLine()
    );
    symbolTable.addSymbol(paramSymbol);
}

// Visit decorators
for (Decorator decorator : node.getDecorators()) {
    decorator.accept(this);
}

// Visit body
for (Statement stmt : node.getBody()) {
    stmt.accept(this);
}

// Exit function scope
symbolTable.exitScope();

return null;

```

```

}

```

```

@Override public Void visitAssign(Assign node) { // First, visit the value expression (this triggers type checking in visitBinaryExpr)
node.getValue().accept(this);

```

```

// Then, infer its type for symbol table
Type valueType = inferType(node.getValue());

// Then, add targets to symbol table
for (Expression target : node.getTargets()) {
    if (target instanceof Identifier) {
        Identifier ident = (Identifier) target;
        Symbol existing = symbolTable.getCurrentScope().lookupLocal(ident.getName());
        if (existing == null) {
            Symbol varSymbol = new Symbol(
                ident.getName(),
                Symbol.Kind.VARIABLE,
                ident.getLine()
            );
            varSymbol.setInferredType(valueType);
            symbolTable.addSymbol(varSymbol);
        } else {
            // Variable already exists in this scope, update type
            existing.setInferredType(valueType);
        }
    }

    // For more complex targets (attributes, indices), we'd need more handling
    target.accept(this);
}

return null;

```

}

@Override public Void visitIdentifier(Identifier node) { // Check if identifier is defined Symbol symbol = symbolTable.lookup(node.getName()); if (symbol == null) { errors.add(new SemanticError(node.getLine(), SemanticError.ErrorType.UNDEFINED_VARIABLE, "Undefined variable " + node.getName() + "")); } return null; }

@Override public Void visitForStmt(ForStmt node) { // Check if iterable is actually iterable Type iterableType = inferType(node.getIterable()); if (iterableType != Type.UNKNOWN && iterableType != Type.LIST && iterableType != Type.STRING) { errors.add(new SemanticError(node.getLine(), SemanticError.ErrorType.NON_ITERABLE_IN_FOR, "Non-iterable type " + iterableType + " used in for loop")); }

```

// Add loop variable to scope
Symbol loopVar = new Symbol(
    node.getVariable().getName(),
    Symbol.Kind.VARIABLE,
    node.getLine()
);
symbolTable.addSymbol(loopVar);

// Visit iterable expression
node.getIterable().accept(this);

// Visit body
for (Statement stmt : node.getBody()) {
    stmt.accept(this);
}

return null;

```

}

@Override public Void visitCallExpr(CallExpr node) { // Resolve callee String functionName = null; Symbol calleeSymbol = null;

```

        if (node.getCallee() instanceof Identifier) {
            functionName = ((Identifier) node.getCallee()).getName();
            calleeSymbol = symbolTable.lookup(functionName);
        }

        // Check if callee exists
        if (calleeSymbol == null && functionName != null) {
            errors.add(new SemanticError(
                node.getLine(),
                SemanticError.ErrorType.UNDEFINED_FUNCTION,
                "Undefined function '" + functionName + "'"
            ));
            // Continue analysis - visit arguments but don't check argument count
            for (Argument arg : node.getArguments()) {
                arg.accept(this);
            }
            return null;
        }

        // Check if callee is callable (must be a function)
        if (calleeSymbol != null && calleeSymbol.getKind() != Symbol.Kind.FUNCTION) {
            errors.add(new SemanticError(
                node.getLine(),
                SemanticError.ErrorType.NOT_CALLABLE,
                "Variable '" + functionName + "' is not callable"
            ));
            // Continue analysis - visit arguments but don't check argument count
            for (Argument arg : node.getArguments()) {
                arg.accept(this);
            }
            return null;
        }

        // Check argument count (only if function exists and is actually a function)
        if (functionName != null && functionParameterCounts.containsKey(functionName)) {
            int expectedParams = functionParameterCounts.get(functionName);
            int actualArgs = node.getArguments().size();

            if (expectedParams != actualArgs) {
                errors.add(new SemanticError(
                    node.getLine(),
                    SemanticError.ErrorType.WRONG_ARGUMENT_COUNT,
                    "Wrong number of arguments for function '" + functionName +
                        "': expected " + expectedParams + ", got " + actualArgs
                ));
            }
        }

        // Visit arguments
        for (Argument arg : node.getArguments()) {
            arg.accept(this);
        }

        return null;
    }

    @Override public Void visitBinaryExpr(BinaryExpr node) { // Visit both sides node.getLeft().accept(this); node.getRight().accept(this);

```

```

// Infer types
Type leftType = inferType(node.getLeft());
Type rightType = inferType(node.getRight());

// Check for type mismatch in operators
if (leftType != Type.UNKNOWN && rightType != Type.UNKNOWN && leftType != rightType) {
    // Only allow certain numeric combinations (int + float)
    boolean isNumericMix = (leftType == Type.INTEGER && rightType == Type.FLOAT) ||
        (leftType == Type.FLOAT && rightType == Type.INTEGER);

    if (!isNumericMix) {
        errors.add(new SemanticError(
            node.getLine(),
            SemanticError.ErrorType.TYPE_MISMATCH,
            "Type mismatch in binary expression: '" + leftType +
                "' " + node.getOperator() + " '" + rightType + "'"
        ));
    }
}

return null;

```

}

@Override public Void visitUnaryExpr(UnaryExpr node) { node.getExpr().accept(this); return null; }

@Override public Void visitIfStmt(IfStmt node) { node.getCondition().accept(this);

```

symbolTable.enterScope(Scope.ScopeKind.BLOCK);
for (Statement stmt : node.getThenBody()) {
    stmt.accept(this);
}
symbolTable.exitScope();

for (ElifClause elif : node.getElifClauses()) {
    elif.accept(this);
}

if (node.getElseClause() != null) {
    node.getElseClause().accept(this);
}

return null;

```

}

@Override public Void visitElseClause(ElseClause node) { symbolTable.enterScope(Scope.ScopeKind.BLOCK); for (Statement stmt : node.getBody()) { stmt.accept(this); } symbolTable.exitScope();

```

return null;

```

}

@Override public Void visitElifClause(ElifClause node) { node.getCondition().accept(this);

```

symbolTable.enterScope(Scope.ScopeKind.BLOCK);
for (Statement stmt : node.getBody()) {
    stmt.accept(this);
}
symbolTable.exitScope();

return null;

```

}


```
@Override public Void visitWhileStmt(WhileStmt node) { node.getCondition().accept(this);
```

```
symbolTable.enterScope(Scope.ScopeKind.BLOCK);  
for (Statement stmt : node.getBody()) {  
    stmt.accept(this);  
}  
symbolTable.exitScope();  
  
return null;
```

```
}
```

```
@Override public Void visitExprStmt(ExprStmt node) { node.getExpression().accept(this); return null; }
```

```
@Override public Void visitReturnStmt(ReturnStmt node) { node.getValue().ifPresent(value -> value.accept(this)); return null; }
```

```
// Expression visitors - mostly for type inference
```

```
@Override public Void visitIntegerLiteral(IntegerLiteral node) { return null; }
```

```
@Override public Void visitFloatLiteral(FloatLiteral node) { return null; }
```

```
@Override public Void visitStringLiteral(StringLiteral node) { return null; }
```

```
@Override public Void visitBooleanLiteral(BooleanLiteral node) { return null; }
```

```
@Override public Void visitNoneLiteral(NoneLiteral node) { return null; }
```

```
@Override public Void visitListExpr(ListExpr node) { for (Expression elem : node.getElements()) { elem.accept(this); } return null; }
```

```
@Override public Void visitDictExpr(DictExpr node) { for (DictEntry entry : node.getEntries()) { entry.accept(this); } return null; }
```

```
@Override public Void visitDictEntry(DictEntry node) { node.getKey().accept(this); node.getValue().accept(this); return null; }
```

```
@Override public Void visitAttributeAccess(AttributeAccess node) { node.getTarget().accept(this); return null; }
```

```
@Override public Void visitIndexAccess(IndexAccess node) { node.getTarget().accept(this); node.getIndex().accept(this); return null; }
```

```
@Override public Void visitPositionalArgument(PositionalArgument node) { node.getValue().accept(this); return null; }
```

```
@Override public Void visitKeywordArgument(KeywordArgument node) { node.getValue().accept(this); return null; }
```

```
@Override public Void visitFunctionParameter(FunctionParameter node) { return null; }
```

```
@Override public Void visitDecorator(Decorator node) { node.getTarget().accept(this); for (Argument arg : node.getArguments()) {  
arg.accept(this); } return null; }
```

```
/**
```

- . Infer the type of an expression for type checking. / *private Type inferType(Expression expr) { if (expr instanceof IntegerLiteral) { return Type.INTEGER; } else if (expr instanceof FloatLiteral) { return Type.FLOAT; } else if (expr instanceof StringLiteral) { return Type.STRING; } else if (expr instanceof BooleanLiteral) { return Type.BOOLEAN; } else if (expr instanceof NoneLiteral) { return Type.NONE; } else if (expr instanceof ListExpr) { return Type.LIST; } else if*

```

(expr instanceof DictExpr) { return
Type.DICTIONARY; } else if (expr instanceof
Identifier) { Symbol symbol =
symbolTable.lookup(((Identifier)
expr).getName()); if (symbol != null) { return
symbol.getInferredType(); } } else if (expr
instanceof BinaryExpr) { BinaryExpr binary =
(BinaryExpr) expr; Type left =
inferType(binary.getLeft()); Type right =
inferType(binary.getRight()); // For arithmetic,
result is numeric if (left == Type.FLOAT || right
== Type.FLOAT) { return Type.FLOAT; } else if
(left == Type.INTEGER && right ==
Type.INTEGER) { return Type.INTEGER; } }
return Type.UNKNOWN; } / -----

```

Jinja-tree nodes

These are part of the shared AstVisitor interface because the Python AST and Jinja AST both flow through the same visitor contract (see AstVisitor.java). This analyzer only ever walks a Python Program, so it will never actually be handed a Jinja node -- a dedicated JinjaSemanticAnalyzer (implementing this same interface) is responsible for those. These overrides exist purely to satisfy the interface and are unreachable in practice. */

```

@Override public Void visitTemplate(compiler.ast.jinja.Template node) { return null; }

@Override public Void visitHtmlText(compiler.ast.jinja.HtmlText node) { return null; }

@Override public Void visitExpressionOutput(compiler.ast.jinja.ExpressionOutput node) { return null; }

@Override public Void visitJinjaIfStmt(compiler.ast.jinja.JinjaIfStmt node) { return null; }

@Override public Void visitJinjaElifClause(compiler.ast.jinja.JinjaElifClause node) { return null; }

@Override public Void visitJinjaElseClause(compiler.ast.jinja.JinjaElseClause node) { return null; }

@Override public Void visitJinjaForStmt(compiler.ast.jinja.JinjaForStmt node) { return null; }

```

?? File: ProductStore.java

```

package compiler.webapp;

import java.util.ArrayList; import java.util.LinkedHashMap; import java.util.List; import java.util.Map; import java.util.concurrent.atomic.AtomicLong;

/**

```

- In-memory product catalog backing the demo CRUD web app. Each product
- is a Map<String, Object> (id/name/price/description) - the exact shape
- ExpressionEvaluator/Renderer already expect, since it's the same shape

- Generator produces from a Python `products = [...], ...` literal. `*/ public final class ProductStore {`

```
private final Map<Long, Map<String, Object>> products = new LinkedHashMap<>(); private final AtomicLong nextId = new AtomicLong(1);

public ProductStore() { add("Phone", 300.0, "A smartphone with a large display."); add("Tablet", 400.0, "A lightweight tablet, great for reading.");
add("Laptop", 900.0, "A powerful laptop for development work."); }

public synchronized Map<String, Object> add(String name, double price, String description) { long id = nextId.getAndIncrement(); Map<String,
Object> product = new LinkedHashMap<>(); product.put("id", id); product.put("name", name); product.put("price", price);
product.put("description", description); products.put(id, product); return product; }

public synchronized List all() { return new ArrayList<>(products.values()); }

public synchronized Map<String, Object> get(long id) { return products.get(id); }

public synchronized boolean update(long id, String name, double price, String description) { Map<String, Object> product = products.get(id); if
(product == null) { return false; } product.put("name", name); product.put("price", price); product.put("description", description); return true; }

public synchronized boolean delete(long id) { return products.remove(id) != null; } }
```

?? File: TemplateEngine.java

```
package compiler.webapp;
```

```
import compiler.ast.jinja.Template; import compiler.frontend.jinja.AstBuilder; import compiler.generated.jinja.JinjaLexer; import
compiler.generated.jinja.JinjaParser; import compiler.generator.Renderer;
```

```
import org.antlr.v4.runtime.CharStream; import org.antlr.v4.runtime.CharStreams; import org.antlr.v4.runtime.CommonTokenStream; import
org.antlr.v4.runtime.tree.ParseTree;
```

```
import java.io.IOException; import java.nio.file.Files; import java.nio.file.Path; import java.util.Map; import
java.util.concurrent.ConcurrentHashMap;
```

```
/**
```

- Loads a .html template from disk, runs it through the project's own
- JinjaLexer -> JinjaParser -> compiler.frontend.jinja.AstBuilder chain
- to get a Template AST (parsed once, then cached), and renders it with
- Renderer. This is the same pipeline PIPELINE_USAGE.md walks through for
- the Python-driven case - here the context comes straight from
- ProductStore instead of from Generator, since a live web request isn't
- re-parsing a Python source file per click. `*/ public final class TemplateEngine {`

```
private final Path templatesDir; private final Map<String, Template> cache = new ConcurrentHashMap<>(); private final Renderer
renderer = new Renderer();

public TemplateEngine(Path templatesDir)

public String render(String templateName, Map<String, Object> context) { Template template = cache.computeIfAbsent(templateName,
this::parse); return renderer.render(template, context); }

private Template parse(String templateName) { try { String source = Files.readString(templatesDir.resolve(templateName)); CharStream
input = CharStreams.fromString(source); JinjaLexer lexer = new JinjaLexer(input); CommonTokenStream tokens = new
CommonTokenStream(lexer); JinjaParser parser = new JinjaParser(tokens); ParseTree tree = parser.template(); return (Template) new
AstBuilder().visit(tree); } catch (IOException e) { throw new RuntimeException("Failed to load template: " + templateName, e); } }
```

?? File: WebServer.java

```
package compiler.webapp;
```

```
import com.sun.net.httpserver.HttpExchange; import com.sun.net.httpserver.HttpServer;
```

```
import java.io.IOException; import java.io.OutputStream; import java.net.InetSocketAddress; import java.net.URLDecoder; import
java.nio.charset.StandardCharsets; import java.nio.file.Path; import java.util.HashMap; import java.util.LinkedHashMap; import java.util.Map;
import java.util.regex.Matcher; import java.util.regex.Pattern;
```

```
/**
```

- Requirement 6: integrated web interfaces (list / add / view details /
- edit / delete) with smooth navigation between them.
-
- Deliberately dependency-free - no Flask, no servlet container, just
- the JDK's built-in `HttpServer` - so it runs with nothing more than the
- project's existing ANTLR runtime on the classpath. The actual HTML for
- every page comes from your own `compiler.frontend.jinja.AstBuilder` +
- `compiler.generator.Renderer`, via `TemplateEngine` - this app exists to
- exercise that pipeline against real requests, not to reimplement Flask.
-
- Routes:
- `GET /products -> list.jinja`
- `GET /products/new -> form.jinja (mode = "new")`
- `POST /products (create, then redirect to /products)`
- `GET /products/ -> detail.jinja`
- `GET /products/edit -> form.jinja (mode = "edit")`
- `POST /products/ (update, then redirect to /products/)`
- `POST /products/delete (delete, then redirect to /products) */` public final class `WebServer {`

```

private static final String NAV_HTML =
    "<nav><a href=\"/products\">Products</a> | " +
    "<a href=\"/products/new\">Add Product</a></nav><hr>";

private static final Pattern PRODUCT_PATH = Pattern.compile("^/products/(\\d+)$");
private static final Pattern PRODUCT_EDIT_PATH = Pattern.compile("^/products/(\\d+)/edit$");
private static final Pattern PRODUCT_DELETE_PATH = Pattern.compile("^/products/(\\d+)/delete$");

private final ProductStore store = new ProductStore();
private final TemplateEngine templates;

public WebServer(Path templatesDir) {
    this.templates = new TemplateEngine(templatesDir);
}

public static void main(String[] args) throws IOException {
    Path templatesDir = Path.of(args.length > 0 ? args[0] : "templates");
    WebServer server = new WebServer(templatesDir);
    server.start(8080);
}

public void start(int port) throws IOException {
    HttpServer httpServer = HttpServer.create(new InetSocketAddress(port), 0);
    httpServer.createContext("/", this::handle);
    httpServer.setExecutor(null);
    httpServer.start();
    System.out.println("Listening on http://localhost:" + port + "/products");
}

private void handle(HttpExchange exchange) {
    try {
        route(exchange);
    } catch (Exception e) {
        respond(exchange, 500, "Internal error: " + e.getMessage());
    }
}

private void route(HttpExchange exchange) throws IOException {

    String method = exchange.getRequestMethod();
    String path = exchange.getRequestURI().getPath();

    if (method.equals("GET") && path.equals("/products")) {
        listProducts(exchange);
    } else if (method.equals("GET") && path.equals("/products/new")) {
        newProductForm(exchange);
    } else if (method.equals("POST") && path.equals("/products")) {
        createProduct(exchange);
    } else if (method.equals("GET") && PRODUCT_EDIT_PATH.matcher(path).matches()) {
        editProductForm(exchange, idFrom(PRODUCT_EDIT_PATH, path));
    } else if (method.equals("GET") && PRODUCT_PATH.matcher(path).matches()) {
        viewProduct(exchange, idFrom(PRODUCT_PATH, path));
    } else if (method.equals("POST") && PRODUCT_PATH.matcher(path).matches()) {
        updateProduct(exchange, idFrom(PRODUCT_PATH, path));
    } else if (method.equals("POST") && PRODUCT_DELETE_PATH.matcher(path).matches()) {
        deleteProduct(exchange, idFrom(PRODUCT_DELETE_PATH, path));
    } else if (method.equals("GET") && path.equals("/")) {
        redirect(exchange, "/products");
    }
}

```

```

    } else {
        respond(exchange, 404, "Not found: " + method + " " + path);
    }
}

private long idFrom(Pattern pattern, String path) {
    Matcher matcher = pattern.matcher(path);
    matcher.matches();
    return Long.parseLong(matcher.group(1));
}

/* -----
   Handlers
   ----- */

private void listProducts(HttpExchange exchange) {
    Map<String, Object> context = baseContext();
    context.put("products", store.all());
    renderResponse(exchange, "list.jinja", context);
}

private void newProductForm(HttpExchange exchange) {
    Map<String, Object> context = baseContext();
    context.put("mode", "new");
    context.put("product", emptyProduct());
    renderResponse(exchange, "form.jinja", context);
}

private void editProductForm(HttpExchange exchange, long id) {
    Map<String, Object> product = store.get(id);
    if (product == null) {
        respond(exchange, 404, "Product not found");
        return;
    }
    Map<String, Object> context = baseContext();
    context.put("mode", "edit");
    context.put("product", product);
    renderResponse(exchange, "form.jinja", context);
}

private void viewProduct(HttpExchange exchange, long id) {
    Map<String, Object> product = store.get(id);
    if (product == null) {
        respond(exchange, 404, "Product not found");
        return;
    }
    Map<String, Object> context = baseContext();
    context.put("product", product);
    renderResponse(exchange, "detail.jinja", context);
}

private void createProduct(HttpExchange exchange) throws IOException {
    Map<String, String> form = readForm(exchange);
    store.add(
        form.getDefault("name", ""),
        parseDouble(form.get("price")),
        form.getDefault("description", ""));
    redirect(exchange, "/products");
}

private void updateProduct(HttpExchange exchange, long id) throws IOException {
    Map<String, String> form = readForm(exchange);
    boolean updated = store.update(
        id,
        form.getDefault("name", ""),

```

```

        parseDouble(form.get("price")),
        form.getDefault("description", "");
    }
    redirect(exchange, updated ? "/products/" + id : "/products");
}

private void deleteProduct(HttpExchange exchange, long id) {
    store.delete(id);
    redirect(exchange, "/products");
}

/* -----
    Helpers
    ----- */

private Map<String, Object> baseContext() {
    Map<String, Object> context = new HashMap<>();
    context.put("nav", NAV_HTML);
    return context;
}

private Map<String, Object> emptyProduct() {
    Map<String, Object> product = new LinkedHashMap<>();
    product.put("id", "");
    product.put("name", "");
    product.put("price", "");
    product.put("description", "");
    return product;
}

private double parseDouble(String s) {
    try {
        return s == null || s.isEmpty() ? 0.0 : Double.parseDouble(s);
    } catch (NumberFormatException e) {
        return 0.0;
    }
}

private void renderResponse(HttpExchange exchange, String template, Map<String, Object> context) {
    try {
        String html = templates.render(template, context);
        byte[] bytes = html.getBytes(StandardCharsets.UTF_8);
        exchange.getResponseHeaders().add("Content-Type", "text/html; charset=utf-8");
        exchange.sendResponseHeaders(200, bytes.length);
        try (OutputStream os = exchange.getResponseBody()) {
            os.write(bytes);
        }
    } catch (IOException e) {
        respond(exchange, 500, "Failed to render " + template + ": " + e.getMessage());
    }
}

private void redirect(HttpExchange exchange, String location) {
    try {
        exchange.getResponseHeaders().add("Location", location);
        exchange.sendResponseHeaders(302, -1);
    } catch (IOException e) {
        respond(exchange, 500, "Redirect failed: " + e.getMessage());
    } finally {
        exchange.close();
    }
}

private void respond(HttpExchange exchange, int status, String message) {
    try {
        byte[] bytes = message.getBytes(StandardCharsets.UTF_8);
        exchange.sendResponseHeaders(status, bytes.length);
    }
}

```

```

        try (OutputStream os = exchange.getResponseBody()) {
            os.write(bytes);
        }
    } catch (IOException ignored) {
        // Best-effort error response - nothing more to do if this fails too.
    }
}

private Map<String, String> readForm(HttpExchange exchange) throws IOException {
    String body = new String(exchange.getRequestBody().readAllBytes(), StandardCharsets.UTF_8);
    Map<String, String> result = new HashMap<>();
    for (String pair : body.split("&")) {
        if (pair.isEmpty()) continue;
        int eq = pair.indexOf('=');
        String key = eq >= 0 ? pair.substring(0, eq) : pair;
        String value = eq >= 0 ? pair.substring(eq + 1) : "";
        result.put(
            URLDecoder.decode(key, StandardCharsets.UTF_8),
            URLDecoder.decode(value, StandardCharsets.UTF_8));
    }
    return result;
}

```

}

.

?? File: JinjaLexer.g4

lexer grammar JinjaLexer;

// Jinja delimiters must come before HTML_TEXT to ensure they're matched first JINJA_EXPR_START : '{{' -> pushMode(JINJA_MODE);
JINJA_STMT_START : '%{' -> pushMode(JINJA_MODE);

// Default mode - outside Jinja tags HTML_TEXT : (~'{' | '{' ~['{%']+ ;

// Jinja mode - inside {{ }} or {% %} mode JINJA_MODE; JINJA_EXPR_END : '}}' -> popMode; JINJA_STMT_END : '%}' -> popMode;

IF : 'if'; ELIF : 'elif'; ELSE : 'else'; ENDIF : 'endif'; FOR : 'for'; IN : 'in'; ENDFOR : 'endfor';

AND : 'and'; OR : 'or'; NOT : 'not';

TRUE : 'true'; FALSE : 'false'; NONE : 'none';

EQ : '='; NE : '!='; LT : '<'; GT : '>'; LE : '<='; GE : '>=';

PLUS : '+'; MINUS : '-'; STAR : '*'; SLASH : '/'; MOD : '%';

DOT : '.'; LPAREN : '('; RPAREN : ')'; LBRACK : '['; RBRACK : ']'; COMMA : ',';

STRING : '"' (~["\"] | '\') * '"'; INT : [0-9]+; FLOAT : [0-9]+ '.' [0-9]+;

IDENTIFIER : [_a-zA-Z] [_a-zA-Z0-9]*;

WS : [\t\r\n]+ -> skip;

.

?? File: JinjaParser.g4

parser grammar JinjaParser;

options

// ===== // TEMPLATE ROOT // ===== template : body=templateBody EOF
;

// ===== // TEMPLATE BODY // ===== templateBody : element* ;

// ===== // ELEMENTS // ===== element : HTML_TEXT #htmlElement |
expressionOutput #expressionElement | ifStatement #ifElement | forStatement #forElement ;


```
// ===== // {{ expression }} // =====

expressionOutput : JINJA_EXPR_START expression=expr JINJA_EXPR_END ;

// ===== // {% if %} ... {% elif %} ... {% else %} ... {% endif %} // =====
ifStatement : JINJA_STMT_START IF condition=expr JINJA_STMT_END thenBody=templateBody elifClause* elseClause?
JINJA_STMT_START ENDIF JINJA_STMT_END ;

elifClause : JINJA_STMT_START ELIF condition=expr JINJA_STMT_END body=templateBody ;

elseClause : JINJA_STMT_START ELSE JINJA_STMT_END body=templateBody ;

// ===== // {% for %} ... {% endfor %} // ===== forStatement :
JINJA_STMT_START FOR variable=IDENTIFIER IN iterable=expr JINJA_STMT_END body=templateBody JINJA_STMT_START ENDFOR
JINJA_STMT_END ;

// ===== // EXPRESSIONS (helper rules for operator precedence) // =====
expr : logicalOr ;

logicalOr : left=logicalOr op=OR right=logicalAnd #orExpr | logicalAnd #logicalAndExpr ;

logicalAnd : left=logicalAnd op=AND right=comparison #andExpr | comparison #comparisonExpr ;

comparison : left=comparison op=(EQ|NE|LT|GT|LE|GE) right=additive #comparisonOpExpr | additive #additiveExpr ;

additive : left=additive op=(PLUS|MINUS) right=multiplicative #addExpr | multiplicative #multiplicativeExpr ;

multiplicative : left=multiplicative op=(STAR|SLASH|MOD) right=unary #mulExpr | unary #unaryExpr ;

unary : op=NOT operand=unary #notExpr | op=MINUS operand=unary #unaryMinus | primary #primaryExpr ;

// ===== // PRIMARY EXPRESSIONS // ===== primary : LPAREN
expression=expr RPAREN #parenExpr | literal #literalExpr | identifier=IDENTIFIER trailers+=trailer* #identifierExpr ;

// ===== // LITERALS // ===== literal : STRING #stringLiteral | INT
#intLiteral | FLOAT #floatLiteral | TRUE #trueLiteral | FALSE #falseLiteral | NONE #noneLiteral ;

// ===== // TRAILERS (mirroring Python's atomTrailer) // ===== trailer :
DOT attribute=IDENTIFIER #attributeAccess | LBRACK index=expr RBRACK #indexAccess | LPAREN arguments=argList? RPAREN
#callTrailer ;

// ===== // ARGUMENT LIST // ===== argList : expr (COMMA expr)*
(COMMA)? ;

.
```

?? File: PythonLexer.g4

```
lexer grammar PythonLexer;

DEF : 'def'; RETURN : 'return';

IF : 'if'; ELIF : 'elif'; ELSE : 'else';

FOR : 'for'; IN : 'in'; WHILE : 'while';

TRUE : 'True'; FALSE : 'False'; NONE : 'None';

AND : 'and'; OR : 'or'; NOT : 'not';

EQ : '=='; NE : '!='; LE : '<='; GE : '>=';

ASSIGN : '=';

LT : '<'; GT : '>';

PLUS : '+'; MINUS : '-'; STAR : '*'; SLASH : '/'; MOD : '%';

LPAREN : '('; RPAREN : ')';

LBRACK : '['; RBRACK : ']';

LBRACE : '{'; RBRACE : '}';

COLON : ':'; COMMA : ','; DOT : '.'; AT : '@';
```

```

FLOAT_LITERAL : DIGIT+ '.' DIGIT+ ; INTEGER_LITERAL : DIGIT+ ;

STRING_LITERAL : '"' (ESC_SEQ | ~["\r\n])* '"' | "'" (ESC_SEQ | ~["\r\n])* "'" ;

IDENTIFIER : [_a-zA-Z] [_a-zA-Z0-9]* ;

NEWLINE : '\r'? '\n' | '\r' ;

COMMENT : '#' ~[\r\n]* -> skip ;

WS : [ \t]+ -> skip ;

fragment DIGIT : [0-9]; fragment ESC_SEQ : '\\' . ;

```

?? File: PythonParser.g4

```

parser grammar PythonParser;

options

tokens { INDENT, DEDENT }

program : (statement | NEWLINE)* EOF ;

statement : RETURN expr (COMMA expr)* NEWLINE? #returnStatement | expr (ASSIGN expr)* NEWLINE? #assignOrExprStatement |
decorator* NEWLINE? DEF IDENTIFIER LPAREN paramList? RPAREN COLON suite #functionDef | IF expr COLON suite (ELIF expr COLON
suite)* (ELSE COLON suite)? #ifStatement | WHILE expr COLON suite #whileStatement | FOR IDENTIFIER IN expr COLON suite
#forStatement ;

decorator : AT IDENTIFIER (DOT IDENTIFIER)* LPAREN argList? RPAREN ;

paramList : IDENTIFIER (COMMA IDENTIFIER)* ;

suite : simpleLine #singleSuite | NEWLINE INDENT statement+ DEDENT #indentedSuite ;

simpleLine : expr (ASSIGN expr)* NEWLINE?;

expr : expr OR expr #orExpr | expr AND expr #andExpr | NOT expr #notExpr | expr (EQ|NE|LT|GT|LE|GE) expr #comparisonExpr | expr
(PLUS|MINUS) expr #addExpr | expr (STAR|SLASH|MOD) expr #mulExpr | MINUS expr #unaryMinus | atom #atomExpr ;

listItems : expr (COMMA expr)* (COMMA)? ;

atom : LPAREN expr RPAREN #parenExpr | LBRACK listItems? RBRACK #listExpr | LBRACE dictItems? RBRACE #dictExpr |
STRING_LITERAL #stringLiteral | INTEGER_LITERAL #intLiteral | FLOAT_LITERAL #floatLiteral | TRUE #trueLiteral | FALSE #falseLiteral |
NONE #noneLiteral | IDENTIFIER atomTrailer* #identifierAtom ;

atomTrailer : DOT IDENTIFIER #attributeAccess | LBRACK expr RBRACK #indexAccess | LPAREN argList? RPAREN #callTrailer ;

argList : argument (COMMA argument)* (COMMA)? ;

argument : expr #positionalArg | IDENTIFIER ASSIGN expr #keywordArg ;

dictItems : dictItem (COMMA dictItem)* (COMMA)? ;

dictItem : (STRING_LITERAL | IDENTIFIER) COLON expr ;

```

?? File: test.g4

```

parser grammar PythonParser;

options

//===== // Program
//=====

program : programElement* EOF #Program ;

programElement : functionDefinition NEWLINE* #FunctionElement

```

```

| statement NEWLINE*
  #StatementElement

| NEWLINE
  #BlankLine

;

```

```

//===== // Statements
//=====

```

statement : assignmentStatement #AssignmentStmt

```

| returnStatement
  #ReturnStmt

| expressionStatement
  #ExpressionStmt

| ifStatement
  #IfStmt

| whileStatement
  #WhileStmt

| forStatement
  #ForStmt

| passStatement
  #PassStmt

;

```

block : NEWLINE INDENT statement+ DEDENT #Block ;

```

//===== // Function Definition
//=====

```

functionDefinition : routeDecorator* DEF IDENTIFIER LPAREN parameterList? RPAREN COLON block #FunctionDefinition ;

routeDecorator : AT IDENTIFIER DOT IDENTIFIER LPAREN STRING_LITERAL RPAREN NEWLINE #RouteDecorator ;

parameterList : parameter (COMMA parameter)* COMMA? #ParameterList ;

parameter : IDENTIFIER #Parameter ;

```

//===== // Assignment
//=====

```

assignmentStatement : assignmentTarget ASSIGN expression #AssignmentStatement ;

assignmentTarget : IDENTIFIER #VariableAssignment ;

```

//===== // Return
//=====

```

returnStatement : RETURN expression? #ReturnStatement ;

```

//===== // Expression Statement
//=====

```

expressionStatement : expression #ExpressionStatement ;

```

//===== // Pass
//=====

```

passStatement : IDENTIFIER #PassStatement ;

```

//===== // If Statement
//=====

```

ifStatement : IF expression COLON block elifClause* elseClause? #IfStatement ;

elifClause : ELIF expression COLON block #ElifClause ;

elseClause : ELSE COLON block #ElseClause ;

//===== // While Statement
//=====

whileStatement : WHILE expression COLON block #WhileStatement ;

//===== // For Statement
//=====

forStatement : FOR IDENTIFIER IN expression COLON block #ForStatement ;

//===== // Expression
//=====

expression : logicalOrExpression #Expression ;

//===== // Logical OR
//=====

logicalOrExpression : logicalOrExpression OR logicalAndExpression #OrExpression

```
| logicalAndExpression
  #LogicalAndRoot
;
```

//===== // Logical AND
//=====

logicalAndExpression : logicalAndExpression AND equalityExpression #AndExpression

```
| equalityExpression
  #EqualityRoot
;
```

//===== // Equality
//=====

equalityExpression : equalityExpression EQ comparisonExpression #EqualExpression

```
| equalityExpression
  NE
  comparisonExpression
  #NotEqualExpression

| comparisonExpression
  #ComparisonRoot
;
```

//===== // Comparison
//=====

comparisonExpression : comparisonExpression LT additiveExpression #LessThanExpression

```

| comparisonExpression
  LE
  additiveExpression
  #LessEqualExpression

| comparisonExpression
  GT
  additiveExpression
  #GreaterThanExpression

| comparisonExpression
  GE
  additiveExpression
  #GreaterEqualExpression

| additiveExpression
  #AdditiveRoot
;

```

?? File: CompilerPipelineTest.java

package test;

import compiler.ast.jinja.Template; import compiler.ast.python.Program;

import compiler.generated.python.BasePythonLexer; import compiler.generated.python.PythonParser; import compiler.generated.jinja.JinjaLexer; import compiler.generated.jinja.JinjaParser;

import compiler.semantic.common.SemanticError; import compiler.semantic.common.SymbolTable; import compiler.semantic.jinja.JinjaSemanticAnalyzer; import compiler.semantic.python.PythonSemanticAnalyzer;

import compiler.generator.Generator; import compiler.generator.Renderer;

import org.antlr.v4.runtime.BaseErrorListener; import org.antlr.v4.runtime.CharStream; import org.antlr.v4.runtime.CharStreams; import org.antlr.v4.runtime.CommonTokenStream; import org.antlr.v4.runtime.Recognizer; import org.antlr.v4.runtime.RecognitionException; import org.antlr.v4.runtime.tree.ParseTree;

import java.io.IOException; import java.nio.file.Files; import java.nio.file.Path; import java.util.List;

/**

- End-to-end integration test that executes the complete compiler pipeline
- starting from a Python source file and ending with generated HTML.
-
- The test automatically discovers Jinja template names from render_template()
- calls in the Python source, so only the Python file and templates directory
- need to be specified. */ public class CompilerPipelineTest {

/**

- Run the complete compiler pipeline.
-
- @param pythonFile Path to the Python source file
- @param templatesDirectory Path to the directory containing Jinja templates
- @throws IOException If file operations fail */ public void test(String pythonFile, String templatesDirectory) throws IOException {
 banner("COMPILER PIPELINE TEST", "Python: " + pythonFile + " | Templates: " + templatesDirectory);

 // Validate Python file exists Path pythonPath = Path.of(pythonFile); if (!Files.exists(pythonPath)) { System.err.println("Python
 source file not found: " + pythonFile); return; }
 }
}

```

// Validate templates directory exists Path templatesPath = Path.of(templatesDirectory); if (!Files.exists(templatesPath)) {
System.err.println("Templates directory not found: " + templatesDirectory); return; }

String pythonSource = Files.readString(pythonPath);

/* ===== PHASE 1: Python Compilation
===== */ section("PYTHON COMPILATION");

// Step 1: Python Lexer BasePythonLexer pythonLexer = new BasePythonLexer(CharStreams.fromString(pythonSource));

// Step 2: Python Parser CommonTokenStream pythonTokens = new CommonTokenStream(pythonLexer); PythonParser
pythonParser = new PythonParser(pythonTokens); attachErrorListener(pythonLexer, pythonParser, "Python");

ParseTree pythonParseTree = pythonParser.program(); if (pythonParseTree == null || pythonParser.getNumberOfSyntaxErrors()
> 0) { System.err.println("Python parsing failed."); return; }

// Step 3: Python AST Builder Program program = (Program) new compiler.frontend.python.AstBuilder().visit(pythonParseTree); if
(program == null) { System.err.println("Python AST building failed."); return; }

// Step 4: Python Semantic Analysis PythonSemanticAnalyzer pythonAnalyzer = new PythonSemanticAnalyzer(); List
pythonErrors = pythonAnalyzer.analyze(program); SymbolTable pythonSymbols = pythonAnalyzer.getSymbolTable();

printSectionHeader("Python Semantic Analysis"); printErrors(pythonErrors);

printSectionHeader("Python Symbol Table"); pythonSymbols.prettyPrint();

if (!pythonErrors.isEmpty()) { System.err.println("Stopping pipeline due to Python semantic errors."); return; }

/* ===== PHASE 2: Generator - Discover
render_template calls ===== */ section("GENERATOR
PHASE");

Generator generator = new Generator(); List<Generator.RenderCall> renderCalls = generator.generate(program,
pythonSymbols);

if (renderCalls.isEmpty()) { System.err.println("No render_template calls found."); return; }

System.out.println("Found " + renderCalls.size() + " render_template call(s).");

/* ===== PHASE 3: Process each render call
===== */ for (int i = 0; i < renderCalls.size(); i++) {
Generator.RenderCall renderCall = renderCalls.get(i); processRenderCall(renderCall, i + 1, templatesPath); }

banner("PIPELINE COMPLETE", "Processed " + renderCalls.size() + " render call(s)"); }

/**
    ■ Process a single render call through Jinja compilation and rendering. */ private void processRenderCall(Generator.RenderCall
renderCall, int callNumber, Path templatesDirectory) { String templateName = renderCall.getTemplateName();
System.out.println("\n" + "=".repeat(72)); System.out.println("Render Call #" + callNumber); System.out.println("Template: " +
templateName); System.out.println("Line: " + renderCall.getLine()); System.out.println("=".repeat(72));

// Resolve template path Path templatePath = templatesDirectory.resolve(templateName); if (!Files.exists(templatePath)) {
System.err.println("Jinja template not found: " + templatePath); return; }

try { String jinjaSource = Files.readString(templatePath);

```

```

/* =====
    Jinja Compilation
    ===== */
section("JINJA COMPILATION");

// Jinja Lexer
CharStream jinjaInput = CharStreams.fromString(jinjaSource);
JinjaLexer jinjaLexer = new JinjaLexer(jinjaInput);

// Jinja Parser
CommonTokenStream jinjaTokens = new CommonTokenStream(jinjaLexer);
JinjaParser jinjaParser = new JinjaParser(jinjaTokens);
attachErrorListener(jinjaLexer, jinjaParser, "Jinja");

ParseTree jinjaParseTree = jinjaParser.template();
if (jinjaParseTree == null || jinjaParser.getNumberOfSyntaxErrors() > 0) {
    System.err.println("Jinja parsing failed for template: " + templateName);
    return;
}

// Jinja AST Builder
Template template = (Template) new compiler.frontend.jinja.AstBuilder().visit(jinjaParseTree);
if (template == null) {
    System.err.println("Jinja AST building failed for template: " + templateName);
    return;
}

/* =====
    Jinja Semantic Analysis
    ===== */
printSectionHeader("Jinja Semantic Analysis");

JinjaSemanticAnalyzer jinjaAnalyzer = new JinjaSemanticAnalyzer(renderCall.getContextSymbols());
List<SemanticError> jinjaErrors = jinjaAnalyzer.analyze(template);
printErrors(jinjaErrors);

if (!jinjaErrors.isEmpty()) {
    System.err.println("Stopping rendering due to Jinja semantic errors.");
    return;
}

/* =====
    Rendering Phase
    ===== */
printSectionHeader("Generated HTML");

Renderer renderer = new Renderer();
String html = renderer.render(template, renderCall.getContextValues());
System.out.println(html);

System.out.println("\nGenerated " + html.length() + " characters of HTML.");

} catch (IOException e) { System.err.println("Error reading template file: " + templatePath); e.printStackTrace(); }}

/* ----- Helper Methods ----- */

private static void banner(String title, String subtitle) { String line = "=".repeat(72); System.out.println("\n" + line); System.out.println(title);
System.out.println(subtitle); System.out.println(line); }

private static void section(String title) { String line = "-".repeat(72); System.out.println("\n" + line); System.out.println(title);
System.out.println(line); }

private static void printSectionHeader(String title) { System.out.println("\n" + "=".repeat(72)); System.out.println(title);
System.out.println("=".repeat(72)); }

```

```

private static void printErrors(List errors) { if (errors.isEmpty()) { System.out.println("No semantic errors."); } else {
System.out.println(errors.size() + " semantic error(s):"); for (SemanticError error : errors) { System.out.println(" " + error); } }
System.out.println(); }

private static void attachErrorListener(org.antlr.v4.runtime.Lexer lexer, org.antlr.v4.runtime.Parser parser, String label) {
lexer.removeErrorListeners(); parser.removeErrorListeners(); BaseErrorListener listener = new BaseErrorListener() { @Override public
void syntaxError(Recognizer recognizer, Object offendingSymbol, int line, int charPositionInLine, String msg, RecognitionException e) {
System.err.printf("[%s SYNTAX ERROR] %d:%d - %s%n", label, line, charPositionInLine, msg); } }; lexer.addErrorListener(listener);
parser.addErrorListener(listener); } }

```

?? File: CompilerTest.java

```

package test;

public interface CompilerTest { public void test(String filePath) throws Exception; }

```

?? File: EndToEndTest.java

```

package test;

import compiler.ast.jinja.Template; import compiler.ast.python.Program;

import compiler.generated.python.BasePythonLexer; import compiler.generated.python.PythonParser; import
compiler.generated.jinja.JinjaLexer; import compiler.generated.jinja.JinjaParser;

import compiler.printer.ASTPrinter; import compiler.printer.ParseTreePrinter; import compiler.printer.TokenPrinter;

import compiler.semantic.common.SemanticError; import compiler.semantic.common.SymbolTable; import
compiler.semantic.jinja.JinjaSemanticAnalyzer; import compiler.semantic.python.PythonSemanticAnalyzer;

import compiler.generator.Generator; import compiler.generator.Renderer;

import org.antlr.v4.runtime.BaseErrorListener; import org.antlr.v4.runtime.CharStream; import org.antlr.v4.runtime.CharStreams; import
org.antlr.v4.runtime.CommonTokenStream; import org.antlr.v4.runtime.Recognizer; import org.antlr.v4.runtime.RecognitionException; import
org.antlr.v4.runtime.tree.ParseTree;

import java.nio.file.Files; import java.nio.file.Path; import java.util.LinkedHashMap; import java.util.List; import java.util.Map;

```

```

/**

```

- Full pipeline test: runs a Python source file and a Jinja template file
- through every stage of the compiler, end to end -
-
- Python lexer -> Python parser -> Python AST -> Python semantic
- analysis -> Generator -> Jinja lexer -> Jinja parser -> Jinja AST ->
- Jinja semantic analysis (checked against the Generator's context) ->
- Renderer -> final rendered HTML.
-
- Every stage's output is printed with its own labeled section, reusing
- the project's existing printer utilities (TokenPrinter, ParseTreePrinter,
- ASTPrinter) rather than duplicating that logic.
-
- =====
- HOW TO USE
- =====
- The two file paths right below are the only thing you normally need
- to touch - point them at whatever .py / .html (Jinja) pair you want
- to test and re-run:
-

```

public static String PYTHON_FILE = "input/python/test01.py";

```

```

public static String JINJA_FILE = "input/jinja/index.html";

```

```

■

```


- Run it directly:

- ```
java test.EndToEndTest
```

- 

- Or override both paths from the command line without editing anything:

- ```
java test.EndToEndTest input/python/test02.py input/jinja/other.html
```

-

- Or call it from your own Main.java / another test:

- ```
test.EndToEndTest.run("input/python/test01.py", "input/jinja/index.html");
```

```
*/ public final class EndToEndTest {
```

```

// =====
// EDIT THESE TWO PATHS TO TEST A DIFFERENT PYTHON / JINJA FILE PAIR
// =====
public static String PYTHON_FILE = "input/python/test01.py";
public static String JINJA_FILE = "input/jinja/index.html";
// =====

public static void main(String[] args) throws Exception {
 String pythonFile = args.length > 0 ? args[0] : PYTHON_FILE;
 String jinjaFile = args.length > 1 ? args[1] : JINJA_FILE;
 run(pythonFile, jinjaFile);
}

public static void run(String pythonFile, String jinjaFile) throws Exception {

 banner("PIPELINE TEST", "Python: " + pythonFile + " | Jinja: " + jinjaFile);

 String pythonSource = Files.readString(Path.of(pythonFile));
 String jinjaSource = Files.readString(Path.of(jinjaFile));

 /* =====
 STEP 1 - Python Lexer
 ===== */
 section(1, "PYTHON LEXER", "Token stream produced from " + pythonFile);
 TokenPrinter.printText(newPythonLexer(pythonSource));

 /* =====
 STEP 2 - Python Parser
 ===== */
 section(2, "PYTHON PARSER", "Parse tree produced by PythonParser.program()");

 BasePythonLexer pythonLexer = newPythonLexer(pythonSource);
 CommonTokenStream pythonTokens = new CommonTokenStream(pythonLexer);
 PythonParser pythonParser = new PythonParser(pythonTokens);
 attachErrorListener(pythonLexer, pythonParser, "Python");

 ParseTree pythonParseTree = pythonParser.program();
 ParseTreePrinter.printText(pythonParseTree, pythonParser);

 /* =====
 STEP 3 - Python AST
 ===== */
 section(3, "PYTHON AST", "Built by compiler.frontend.python.AstBuilder");

 Program program = (Program) new compiler.frontend.python.AstBuilder().visit(pythonParseTree);
 ASTPrinter.printText(program);

 /* =====
 STEP 4 - Python Semantic Analysis
 ===== */
 section(4, "PYTHON SEMANTIC ANALYSIS", "PythonSemanticAnalyzer + SymbolTable");

 PythonSemanticAnalyzer pythonAnalyzer = new PythonSemanticAnalyzer();
 List<SemanticError> pythonErrors = pythonAnalyzer.analyze(program);
 SymbolTable pythonSymbols = pythonAnalyzer.getSymbolTable();

 printErrors("Python", pythonErrors);
 System.out.println("--- Python symbol table ---");
 pythonSymbols.prettyPrint();

 /* =====
 STEP 5 - Generator (Python data -> Jinja context)
 ===== */
 section(5, "GENERATOR", "Resolving render_template(...) and extracting its context");

```

```

Generator generator = new Generator();
List<Generator.RenderCall> renderCalls = generator.generate(program, pythonSymbols);

Generator.RenderCall renderCall;
if (renderCalls.isEmpty()) {
 System.out.println("No render_template(...) call found in " + pythonFile + ".");
 System.out.println("Falling back to an empty context - only names the template");
 System.out.println("doesn't reference (or Python module-level constants) will render correctly.");
 renderCall = new Generator.RenderCall(jinjaFile, -1, new LinkedHashMap<>(), new SymbolTable());
} else {
 if (renderCalls.size() > 1) {
 System.out.println(renderCalls.size() + " render_template(...) calls found - using the first
 }
 renderCall = renderCalls.get(0);
 System.out.println("render_template(\"" + renderCall.getTemplateName() + "\", ...) found at line
}

System.out.println("\n--- Context values (what the template will actually see) ---");
if (renderCall.getContextValues().isEmpty()) {
 System.out.println("(empty)");
} else {
 for (Map.Entry<String, Object> entry : renderCall.getContextValues().entrySet()) {
 System.out.println(" " + entry.getKey() + " = " + describe(entry.getValue()));
 }
}

System.out.println("\n--- Context symbol table (handed to JinjaSemanticAnalyzer) ---");
renderCall.getContextSymbolTable().prettyPrint();

/* =====
STEP 6 - Jinja Lexer
===== */
section(6, "JINJA LEXER", "Token stream produced from " + jinjaFile);
TokenPrinter.printText(new JinjaLexer(CharStreams.fromString(jinjaSource)));

/* =====
STEP 7 - Jinja Parser
===== */
section(7, "JINJA PARSER", "Parse tree produced by JinjaParser.template()");

CharStream jinjaInput = CharStreams.fromString(jinjaSource);
JinjaLexer jinjaLexer = new JinjaLexer(jinjaInput);
CommonTokenStream jinjaTokens = new CommonTokenStream(jinjaLexer);
JinjaParser jinjaParser = new JinjaParser(jinjaTokens);
attachErrorListener(jinjaLexer, jinjaParser, "Jinja");

ParseTree jinjaParseTree = jinjaParser.template();
ParseTreePrinter.printText(jinjaParseTree, jinjaParser);

/* =====
STEP 8 - Jinja AST
===== */
section(8, "JINJA AST", "Built by compiler.frontend.jinja.AstBuilder");

Template template = (Template) new compiler.frontend.jinja.AstBuilder().visit(jinjaParseTree);
ASTPrinter.printText(template);

/* =====
STEP 9 - Jinja Semantic Analysis
===== */
section(9, "JINJA SEMANTIC ANALYSIS", "Checked against the Generator's context");

JinjaSemanticAnalyzer jinjaAnalyzer = new JinjaSemanticAnalyzer(renderCall.getContextSymbolTable());
List<SemanticError> jinjaErrors = jinjaAnalyzer.analyze(template);
printErrors("Jinja", jinjaErrors);

```

```

/* =====
STEP 10 - Render (code generation)
===== */
section(10, "RENDER", "Final HTML produced by Renderer.render(template, context)");

if (!jinjaErrors.isEmpty()) {
 System.out.println("Rendering anyway despite the semantic errors above, so you can");
 System.out.println(" see the output - fix the template/context before trusting it.\n");
}

Renderer renderer = new Renderer();
String html = renderer.render(template, renderCall.getContextValues());
System.out.println(html);

 banner("PIPELINE COMPLETE", pythonFile + " " + jinjaFile + " -> " + html.length() + " characters of
}

/* -----
Helpers
----- */

private static BasePythonLexer newPythonLexer(String source) {
 return new BasePythonLexer(CharStreams.fromString(source));
}

private static void printErrors(String label, List<SemanticError> errors) {
 if (errors.isEmpty()) {
 System.out.println(label + ": no semantic errors.");
 } else {
 System.out.println(label + ": " + errors.size() + " semantic error(s):");
 for (SemanticError error : errors) {
 System.out.println(" " + error);
 }
 }
 System.out.println();
}

private static String describe(Object value) {
 if (value == null) return "None";
 if (value instanceof List) return "List of " + ((List<?>) value).size() + " item(s) -> " + value;
 if (value instanceof Map) return "Dict with keys " + ((Map<?, ?>) value).keySet();
 return String.valueOf(value) + " (" + value.getClass().getSimpleName() + ")";
}

private static void attachErrorListener(org.antlr.v4.runtime.Lexer lexer, org.antlr.v4.runtime.Parser parser) {
 lexer.removeErrorListeners();
 parser.removeErrorListeners();
 BaseErrorListener listener = new BaseErrorListener() {
 @Override
 public void syntaxError(Recognizer<?, ?> recognizer, Object offendingSymbol,
 int line, int charPositionInLine, String msg, RecognitionException e) {
 System.err.printf("[%s SYNTAX ERROR] %d:%d - %s\n", label, line, charPositionInLine, msg);
 }
 };
 lexer.addErrorListener(listener);
 parser.addErrorListener(listener);
}

private static void banner(String title, String subtitle) {
 String line = "=".repeat(72);
 System.out.println("\n" + line);
 System.out.println(title);
 System.out.println(subtitle);
 System.out.println(line);
}

```

```
private static void section(int step, String title, String subtitle) {
 String line = "-".repeat(72);
 System.out.println("\n" + line);
 System.out.println("STEP " + step + ": " + title);
 System.out.println(subtitle);
 System.out.println(line);
}
```

```
}
```

## ?? File: JinjaAstBuilderTest.java

```
package test;
```

```
import compiler.frontend.jinja.AstBuilder; import compiler.ast.jinja.Template; import compiler.generated.jinja.JinjaLexer; import
compiler.generated.jinja.JinjaParser; import org.antlr.v4.runtime.*;
```

```
import java.nio.file.Path; import java.nio.file.Paths;
```

```
/**
```

- Simple non-JUnit test harness that parses a Jinja file, builds the AST using
- AstBuilder and prints the parse tree and pretty-printed AST to stdout.
- 
- Usage:
- test.JinjaAstBuilderTest.Test("input/jinja/example.jinja"); \*/ public final class JinjaAstBuilderTest implements CompilerTest {

```

@Override
public void test(String file) throws Exception {
 Path path = Paths.get(file);
 CharStream input = CharStreams.fromPath(path);

 // Lexer / parser
 JinjaLexer lexer = new JinjaLexer(input);
 CommonTokenStream tokens = new CommonTokenStream(lexer);
 JinjaParser parser = new JinjaParser(tokens);

 // Optional: attach a simple error listener to surface syntax errors
 lexer.removeErrorListeners();
 parser.removeErrorListeners();
 VerboseErrorListener err = new VerboseErrorListener();
 lexer.addErrorListener(err);
 parser.addErrorListener(err);

 // Parse
 JinjaParser.TemplateContext templateCtx = parser.template();

 // Print parse tree (text form)
 System.out.println("=== Parse tree (toStringTree) ===");
 System.out.println(templateCtx.toStringTree(parser));
 System.out.println("=== End parse tree ===\n");

 // Build AST
 AstBuilder builder = new AstBuilder();
 Template template = builder.build(templateCtx);

 // Print pretty-printed AST
 System.out.println("=== AST prettyPrint ===");
 System.out.println(template.prettyPrint());
 System.out.println("=== End AST prettyPrint ===");
}

// Simple verbose error listener to surface syntax errors
private static final class VerboseErrorListener extends BaseErrorListener {
 @Override
 public void syntaxError(Recognizer, ? recognizer,
 Object offendingSymbol,
 int line, int charPositionInLine,
 String msg, RecognitionException e) {
 System.err.printf("SYNTAX ERROR at %d:%d - %s\n", line, charPositionInLine, msg);
 }
}

```

```

}

```

## ?? File: JinjaLexerTest.java

```

package test;

```

```

import compiler.generated.jinja.; import compiler.printer.TokenPrinter; import org.antlr.v4.runtime.;

```

```

import java.io.IOException;

```

```

public class JinjaLexerTest implements CompilerTest{

```

```

@Override
public void test(String file) throws IOException {
 CharStream input = CharStreams.fromFileName(file);
 JinjaLexer lexer = new JinjaLexer(input);

 TokenPrinter.printText(lexer);
}

```

```

}

```

## ?? File: JinjaParserTest.java

```

package test;

```

```

import compiler.generated.jinja.; import compiler.printer.ParseTreePrinter; import org.antlr.v4.runtime.; import org.antlr.v4.runtime.tree.*;

```

```

import java.io.IOException; import java.nio.file.Files; import java.nio.file.Paths;

```

```

public class JinjaParserTest implements CompilerTest { @Override public void test(String filePath) throws IOException { CharStream input =
CharStreams.fromFileName(filePath);

```

```

 JinjaLexer lexer = new JinjaLexer(input);
 CommonTokenStream tokens = new CommonTokenStream(lexer);
 JinjaParser parser = new JinjaParser(tokens);

 ParseTree tree = parser.template();

 ParseTreePrinter.printText(tree, parser);
}

private static void printTree(ParseTree tree, int level) {
 if (tree == null) return;

 String indent = " ".repeat(level);
 System.out.println(indent + tree.getClass().getSimpleName() + ": " + tree.getText());

 for (int i = 0; i < tree.getChildCount(); i++) {
 printTree(tree.getChild(i), level + 1);
 }
}
}

```

```

}

```

## ?? File: JinjaSemanticAnalyzerTest.java

```

package test;

```

```

import compiler.ast.jinja.Template; import compiler.ast.python.Program; import compiler.frontend.python.AstBuilder; import
compiler.generated.jinja.JinjaLexer; import compiler.generated.jinja.JinjaParser; import compiler.generated.python.BasePythonLexer; import
compiler.generated.python.PythonLexer; import compiler.generated.python.PythonParser; import compiler.semantic.common.SemanticError;
import compiler.semantic.common.SymbolTable; import compiler.semantic.jinja.JinjaSemanticAnalyzer; import
compiler.semantic.python.PythonSemanticAnalyzer; import org.antlr.v4.runtime.CharStream; import org.antlr.v4.runtime.CharStreams; import
org.antlr.v4.runtime.CommonTokenStream; import org.antlr.v4.runtime.tree.ParseTree;

```

```

import java.io.IOException; import java.nio.file.*; import java.util.List;

```

```

public class JinjaSemanticAnalyzerTest { public void test(String pythonFile, String jinjaFile) throws IOException { String pythonSource =
Files.readString(Path.of(pythonFile));

```

```

CharStream pythonInput = CharStreams.fromString(pythonSource);

BasePythonLexer pythonLexer = new BasePythonLexer(pythonInput);

CommonTokenStream pythonTokens = new CommonTokenStream(pythonLexer);

PythonParser pythonParser = new PythonParser(pythonTokens);

ParseTree pythonParseTree = pythonParser.program();

compiler.frontend.python.AstBuilder pythonAstBuilder = new compiler.frontend.python.AstBuilder();

Program program = (Program) pythonAstBuilder.visit(pythonParseTree);

PythonSemanticAnalyzer pythonAnalyzer = new PythonSemanticAnalyzer();
pythonAnalyzer.analyze(program);
SymbolTable symbolTable = pythonAnalyzer.getSymbolTable();

String jinjaSource = Files.readString(Path.of(jinjaFile));

CharStream jinjaInput = CharStreams.fromString(jinjaSource);

JinjaLexer jinjaLexer = new JinjaLexer(jinjaInput);

CommonTokenStream jinjaTokens = new CommonTokenStream(jinjaLexer);

JinjaParser jinjaParser = new JinjaParser(jinjaTokens);

ParseTree jinjaParseTree = jinjaParser.template();

compiler.frontend.jinja.AstBuilder jinjaAstBuilder = new compiler.frontend.jinja.AstBuilder();

Template template = (Template) jinjaAstBuilder.visit(jinjaParseTree);

JinjaSemanticAnalyzer analyzer =
 new JinjaSemanticAnalyzer(symbolTable);

List<SemanticError> errors =
 analyzer.analyze(template);

System.out.println();
System.out.println("=====");
System.out.println("Semantic Analysis");
System.out.println("=====");

if (errors.isEmpty()) {

 System.out.println("No semantic errors.");

} else {

 for (SemanticError error : errors) {
 System.out.println(error);
 }

 System.out.println();
 System.out.println(errors.size() + " semantic error(s).");
}

System.out.println();
System.out.println("=====");
System.out.println("Symbol Table");
System.out.println("=====");

```



```

 analyzer.getSymbolTable().prettyPrint();
 }
}

```

## ?? File: PythonAnalyzerTest.java

```
package test;
```

```
import compiler.ast.python.Program; import compiler.frontend.python.AstBuilder; import compiler.generated.python.BasePythonLexer; import
compiler.generated.python.PythonParser; import compiler.semantic.python.PythonSemanticAnalyzer; import
compiler.semantic.common.SemanticError;
```

```
import org.antlr.v4.runtime.*; import org.antlr.v4.runtime.tree.ParseTree;
```

```
import java.io.IOException; import java.nio.file.Files; import java.nio.file.Path; import java.util.List;
```

```
public final class PythonAnalyzerTest implements CompilerTest {
```

```

 public PythonAnalyzerTest() {
 }

 @Override
 public void test(String fileName) throws IOException {

 String source = Files.readString(Path.of(fileName));

 CharStream input = CharStreams.fromString(source);

 BasePythonLexer lexer = new BasePythonLexer(input);

 CommonTokenStream tokens = new CommonTokenStream(lexer);

 PythonParser parser = new PythonParser(tokens);

 ParseTree tree = parser.program();

 AstBuilder builder = new AstBuilder();

 Program program = (Program) builder.visit(tree);
 }
}

```

```
// System.out.println(""); // System.out.println("AST"); // System.out.println(""); // System.out.println(program.prettyPrint(""));
```

```

PythonSemanticAnalyzer analyzer = new PythonSemanticAnalyzer();

List<SemanticError> errors = analyzer.analyze(program);

System.out.println();
System.out.println("=====");
System.out.println("Semantic Analysis");
System.out.println("=====");

if (errors.isEmpty()) {

 System.out.println("No semantic errors.");

} else {

 for (SemanticError error : errors) {
 System.out.println(error);
 }

 System.out.println();
 System.out.println(errors.size() + " semantic error(s).");
}

System.out.println();
System.out.println("=====");
System.out.println("Symbol Table");
System.out.println("=====");

analyzer.getSymbolTable().prettyPrint();
}

```

}.

## ?? File: PythonAstBuilderTest.java

package test;

import compiler.frontend.python.AstBuilder; import compiler.ast.python.Program; import compiler.generated.python.BasePythonLexer; import compiler.generated.python.PythonParser; import org.antlr.v4.runtime.\*;

import java.nio.file.Path; import java.nio.file.Paths;

/\*\*

- Simple non-JUnit test harness that parses a Python file, builds the AST using
- AstBuilder and prints the parse tree and pretty-printed AST to stdout.
- 
- Usage:
- test.AstBuilderTest.Test("input/python/exercise\_supported\_syntax\_fixed.py"); \*/ public final class PythonAstBuilderTest implements CompilerTest {

```

@Override
public void test(String file) throws Exception {
 Path path = Paths.get(file);
 CharStream input = CharStreams.fromPath(path);

 // Lexer / parser
 BasePythonLexer lexer = new BasePythonLexer(input);
 CommonTokenStream tokens = new CommonTokenStream(lexer);
 PythonParser parser = new PythonParser(tokens);

 // Optional: attach a simple error listener to surface syntax errors
 lexer.removeErrorListeners();
 parser.removeErrorListeners();
 VerboseErrorListener err = new VerboseErrorListener();
 lexer.addErrorListener(err);
 parser.addErrorListener(err);

 // Parse
 PythonParser.ProgramContext programCtx = parser.program();

 // Print parse tree (text form)
 System.out.println("=== Parse tree (toStringTree) ===");
 System.out.println(programCtx.toStringTree(parser));
 System.out.println("=== End parse tree ===\n");

 // Build AST
 AstBuilder builder = new AstBuilder();
 Program program = builder.build(programCtx);

 // Print pretty-printed AST
 System.out.println("=== AST prettyPrint ===");
 System.out.println(program.prettyPrint());
 System.out.println("=== End AST prettyPrint ===");
}

// Simple verbose error listener to surface syntax errors
private static final class VerboseErrorListener extends BaseErrorListener {
 @Override
 public void syntaxError(Recognizer<?, ?> recognizer,
 Object offendingSymbol,
 int line, int charPositionInLine,
 String msg, RecognitionException e) {
 System.err.printf("SYNTAX ERROR at %d:%d - %s\n", line, charPositionInLine, msg);
 }
}

```

```

}

```

## ?? File: PythonLexerTest.java

```

package test;

```

```

import compiler.generated.python.BasePythonLexer; import compiler.generated.python.PythonLexer; import compiler.printer.TokenPrinter;
import org.antlr.v4.runtime.*;

```

```

import java.io.IOException;

```

```

public class PythonLexerTest implements CompilerTest {

```

```

@Override
public void test(String file) throws IOException {
 CharStream input = CharStreams.fromFileName(file);
 BasePythonLexer lexer = new BasePythonLexer(input);

 TokenPrinter.printText(lexer);
}

```

```

}

```

## ?? File: PythonParserTest.java

```

package test;

```

```

import compiler.generated.python.BasePythonLexer; import compiler.generated.python.PythonParser; import compiler.printer.ParseTreePrinter;
import compiler.printer.TreeVisualizer; import org.antlr.v4.runtime.*; import org.antlr.v4.runtime.tree.ParseTree;

```

```

public class PythonParserTest implements CompilerTest{

```

```

 public void test(String file) throws Exception {
 CharStream input = CharStreams.fromFileName(file);

 BasePythonLexer lexer = new BasePythonLexer(input);
 CommonTokenStream tokens = new CommonTokenStream(lexer);
 PythonParser parser = new PythonParser(tokens);

 ParseTree tree = parser.program();
 ParseTreePrinter.printText(tree, parser);
 }

```

```

}

```

## ?? File: TestAll.java

```

package test;

```

```

import compiler.ast.jinja.Template; import compiler.ast.python.Program; import compiler.generated.jinja.JinjaLexer; import
compiler.generated.jinja.JinjaParser; import compiler.generated.python.BasePythonLexer; import compiler.generated.python.PythonParser;
import compiler.generator.Generator; import compiler.generator.Renderer; import compiler.printer.ASTPrinter; import
compiler.printer.ParseTreePrinter; import compiler.printer.TokenPrinter; import compiler.semantic.common.SemanticError; import
compiler.semantic.common.SymbolTable; import compiler.semantic.jinja.JinjaSemanticAnalyzer; import
compiler.semantic.python.PythonSemanticAnalyzer; import org.antlr.v4.runtime.*; import org.antlr.v4.runtime.tree.ParseTree;

```

```

import java.io.IOException; import java.nio.file.Files; import java.nio.file.Path; import java.util.LinkedHashMap; import java.util.List; import
java.util.Map;

```

```

public class TestAll { public static void test(String pythonFile , String jinjaFile) throws IOException { banner("PIPELINE TEST", "Python: " +
pythonFile + " | Jinja: " + jinjaFile);

```

```

String pythonSource = Files.readString(Path.of(pythonFile));
String jinjaSource = Files.readString(Path.of(jinjaFile));

/* =====
STEP 1 - Python Lexer
===== */
section(1, "PYTHON LEXER", "Token stream produced from " + pythonFile);
TokenPrinter.printText(newPythonLexer(pythonSource));

/* =====
STEP 2 - Python Parser
===== */
section(2, "PYTHON PARSER", "Parse tree produced by PythonParser.program()");

BasePythonLexer pythonLexer = newPythonLexer(pythonSource);
CommonTokenStream pythonTokens = new CommonTokenStream(pythonLexer);
PythonParser pythonParser = new PythonParser(pythonTokens);
attachErrorListener(pythonLexer, pythonParser, "Python");

ParseTree pythonParseTree = pythonParser.program();
ParseTreePrinter.printText(pythonParseTree, pythonParser);

/* =====
STEP 3 - Python AST
===== */
section(3, "PYTHON AST", "Built by compiler.frontend.python.AstBuilder");

Program program = (Program) new compiler.frontend.python.AstBuilder().visit(pythonParseTree);
ASTPrinter.printText(program);

/* =====
STEP 4 - Python Semantic Analysis
===== */
section(4, "PYTHON SEMANTIC ANALYSIS", "PythonSemanticAnalyzer + SymbolTable");

PythonSemanticAnalyzer pythonAnalyzer = new PythonSemanticAnalyzer();
List<SemanticError> pythonErrors = pythonAnalyzer.analyze(program);
SymbolTable pythonSymbols = pythonAnalyzer.getSymbolTable();

printErrors("Python", pythonErrors);
System.out.println("--- Python symbol table ---");
pythonSymbols.prettyPrint();

/* =====
STEP 5 - Generator (Python data -> Jinja context)
===== */
section(5, "GENERATOR", "Resolving render_template(...) and extracting its context");

Generator generator = new Generator();
List<Generator.RenderCall> renderCalls = generator.generate(program, pythonSymbols);

Generator.RenderCall renderCall;
if (renderCalls.isEmpty()) {
 System.out.println("No render_template(...) call found in " + pythonFile + ".");
 System.out.println("Falling back to an empty context - only names the template");
 System.out.println("doesn't reference (or Python module-level constants) will render correctly.");
 renderCall = new Generator.RenderCall(jinjaFile, -1, new LinkedHashMap<>(), new SymbolTable());
} else {
 if (renderCalls.size() > 1) {
 System.out.println(renderCalls.size() + " render_template(...) calls found - using the first
 }
 renderCall = renderCalls.get(0);
 System.out.println("render_template(\"" + renderCall.getTemplateName() + "\", ...) found at line
}

```

```

System.out.println("\n--- Context values (what the template will actually see) ---");
if (renderCall.getContextValues().isEmpty()) {
 System.out.println("empty");
} else {
 for (Map.Entry<String, Object> entry : renderCall.getContextValues().entrySet()) {
 System.out.println(" " + entry.getKey() + " = " + describe(entry.getValue()));
 }
}

System.out.println("\n--- Context symbol table (handed to JinjaSemanticAnalyzer) ---");
renderCall.getContextSymbolTable().prettyPrint();

/* =====
STEP 6 - Jinja Lexer
===== */
section(6, "JINJA LEXER", "Token stream produced from " + jinjaFile);
TokenPrinter.printText(new JinjaLexer(CharStreams.fromString(jinjaSource)));

/* =====
STEP 7 - Jinja Parser
===== */
section(7, "JINJA PARSER", "Parse tree produced by JinjaParser.template()");

CharStream jinjaInput = CharStreams.fromString(jinjaSource);
JinjaLexer jinjaLexer = new JinjaLexer(jinjaInput);
CommonTokenStream jinjaTokens = new CommonTokenStream(jinjaLexer);
JinjaParser jinjaParser = new JinjaParser(jinjaTokens);
attachErrorListener(jinjaLexer, jinjaParser, "Jinja");

ParseTree jinjaParseTree = jinjaParser.template();
ParseTreePrinter.printText(jinjaParseTree, jinjaParser);

/* =====
STEP 8 - Jinja AST
===== */
section(8, "JINJA AST", "Built by compiler.frontend.jinja.AstBuilder");

Template template = (Template) new compiler.frontend.jinja.AstBuilder().visit(jinjaParseTree);
ASTPrinter.printText(template);

/* =====
STEP 9 - Jinja Semantic Analysis
===== */
section(9, "JINJA SEMANTIC ANALYSIS", "Checked against the Generator's context");

JinjaSemanticAnalyzer jinjaAnalyzer = new JinjaSemanticAnalyzer(renderCall.getContextSymbolTable());
List<SemanticError> jinjaErrors = jinjaAnalyzer.analyze(template);
printErrors("Jinja", jinjaErrors);

/* =====
STEP 10 - Render (code generation)
===== */
section(10, "RENDER", "Final HTML produced by Renderer.render(template, context)");

if (!jinjaErrors.isEmpty()) {
 System.out.println("Rendering anyway despite the semantic errors above, so you can");
 System.out.println(" see the output - fix the template/context before trusting it.\n");
}

Renderer renderer = new Renderer();
String html = renderer.render(template, renderCall.getContextValues());
System.out.println(html);

banner("PIPELINE COMPLETE", pythonFile + " + " + jinjaFile + " -> " + html.length() + " characters of
}

```

```

/* -----
 Helpers
 ----- */

private static BasePythonLexer newPythonLexer(String source) {
 return new BasePythonLexer(CharStreams.fromString(source));
}

private static void printErrors(String label, List<SemanticError> errors) {
 if (errors.isEmpty()) {
 System.out.println(label + ": no semantic errors.");
 } else {
 System.out.println(label + ": " + errors.size() + " semantic error(s):");
 for (SemanticError error : errors) {
 System.out.println(" " + error);
 }
 }
 System.out.println();
}

private static String describe(Object value) {
 if (value == null) return "None";
 if (value instanceof List) return "List of " + ((List<?>) value).size() + " item(s) -> " + value;
 if (value instanceof Map) return "Dict with keys " + ((Map<?, ?>) value).keySet();
 return String.valueOf(value) + " (" + value.getClass().getSimpleName() + ")";
}

private static void attachErrorListener(org.antlr.v4.runtime.Lexer lexer, org.antlr.v4.runtime.Parser parser) {
 lexer.removeErrorListeners();
 parser.removeErrorListeners();
 BaseErrorListener listener = new BaseErrorListener() {
 @Override
 public void syntaxError(Recognizer<?, ?> recognizer, Object offendingSymbol,
 int line, int charPositionInLine, String msg, RecognitionException e) {
 System.err.printf("[%s SYNTAX ERROR] %d:%d - %s%n", label, line, charPositionInLine, msg);
 }
 };
 lexer.addErrorListener(listener);
 parser.addErrorListener(listener);
}

private static void banner(String title, String subtitle) {
 String line = "-".repeat(72);
 System.out.println("\n" + line);
 System.out.println(title);
 System.out.println(subtitle);
 System.out.println(line);
}

private static void section(int step, String title, String subtitle) {
 String line = "-".repeat(72);
 System.out.println("\n" + line);
 System.out.println("STEP " + step + ": " + title);
 System.out.println(subtitle);
 System.out.println(line);
}
}

```

## ?? File: Tester.java

package test;

```
import java.io.IOException; import java.nio.file.Files; import java.nio.file.Path; import java.nio.file.Paths; import java.util.List; import
java.util.concurrent.atomic.AtomicInteger; import java.util.stream.Collectors; import java.util.stream.Stream;
```

```
public class Tester {
```



```

/**
 * ðŸš€ NEW METHOD: Instantly executes a test strategy on ONE specific file path.
 * Bypasses folder loops entirely while reusing your polymorphic test engine.
 *
 * @param filePath The exact path to the file (e.g., "input/jinja/variables.html")
 * @param testStrategy The specific testing module instance (e.g., new JinjaParserTest())
 */
public static void testFile(String filePath, CompilerTest testStrategy) {
 Path targetFile = Paths.get(filePath);

 // Fail-fast safeguard check if file path points to a ghost target location
 if (!Files.exists(targetFile) || !Files.isRegularFile(targetFile)) {
 System.err.println("ðŸš€ Error: Target file does not exist or is invalid: " + filePath);
 return;
 }

 System.out.println("=====");
 System.out.println("ðŸš€ TARGETED SINGLE FILE TEST | Strategy: " + testStrategy.getClass().getSimpleName());
 System.out.println("ðŸš€ Path: " + filePath);
 System.out.println("=====");

 try {
 // Polymorphically execute the signature method matching your exact design contract
 testStrategy.test(filePath);
 System.out.println("ðŸš€ Single file test executed successfully.");
 } catch (Exception e) {
 System.err.println("ðŸš€ Critical failure parsing file [" + targetFile.getFileName() + "]: " + e);
 e.printStackTrace(); // Output detail tracking matrix logs
 }

 System.out.println("=====\\n");
}

/**
 * Dynamically scans a folder and executes the passed CompilerTest strategy on matching files.
 */
public static void testFolder(String folderPath, String extension, CompilerTest testStrategy) {
 Path targetDir = Paths.get(folderPath);

 if (!Files.exists(targetDir) || !Files.isDirectory(targetDir)) {
 System.err.println("Error: Target directory does not exist or is invalid: " + folderPath);
 return;
 }

 System.out.println("=====");
 System.out.println("ðŸš€ BATCH TESTING IN: " + folderPath + " | Strategy: " + testStrategy.getClass().getSimpleName());
 System.out.println("=====");

 AtomicInteger fileCounter = new AtomicInteger(1);

 try (Stream<Path> stream = Files.list(targetDir)) {

 List<Path> targetFiles = stream
 .filter(Files::isRegularFile)
 .filter(path -> path.toString().toLowerCase().endsWith(extension.toLowerCase()))
 .sorted()
 .collect(Collectors.toList());

 for (Path filePath : targetFiles) {
 System.out.println("\\n-----");
 System.out.println("ðŸš€ File " + fileCounter.getAndIncrement() + " : " + filePath.getFileName());
 System.out.println("-----");

 try {
 // ðŸš€ POLYMORPHIC CALL: Runs the specific implementation passed to the method
 testStrategy.test(filePath.toString());
 }

```

```
 } catch (Exception e) {
 System.err.println("âE Failure in [" + filePath.getFileName() + "]: " + e.getMessage())
 }
 }

 } catch (IOException e) {
 System.err.println("Failed to read files from directory: " + e.getMessage());
 }

 System.out.println("\n=====");
 System.out.println("â... COMPLETED. Processed " + (fileCounter.get() - 1) + " files.");
 System.out.println("=====");
}

}
```

?? File: detail.jinja

```
{{ nav }}

{{ product.name }}

Price: ${{ product.price }}

{{ product.description }}

Edit \(/products/{{ product.id }}/edit\) |

Delete

Back to list \(/products\)
```

. ### ?? File: form.jinja {{ nav }} {% if mode == "edit" %}

## Edit Product

```
{% else %}

Add Product

{% endif %}

Name:

Price:

Description:

Save

Cancel \(/products\)
```

. ### ?? File: list.jinja {{ nav }}

## Products

```
{% for p in products %} {% endfor %}

Name	Price	Actions
{{ p.name }}	\${{ p.price }}	View (/products/{{ p.id }}) Edit (/products/{{ p.id }}/edit) Delete

Add New Product \(/products/new\)
```